

SQLD 완벽 이론 정리

무단 전재 및 재배포를 금지합니다.



이 책의 목차

1과목 : 데이터 모델링의 이해

PART1. 데이터 모델링의 이해

1-1. 데이터 모델의 이해	5	1-4. 관계	20
1-2 엔터티(Entity)	11	1-5. 식별자	25
1-3. 속성	15		

PART2. 데이터 모델과 SQL

1-6. 정규화	33	1-9. Null 속성의 이해	48
1-7. 관계와 조인의 이해	40	1-10. 본질식별자 vs 인조식별자	53
1-8. 모델이 표현하는 트랜잭션의 이해	45		
단원 종합 문제	57		

2과목 : SQL 기본 및 활용

PART1. SQL 기본

2-1. 관계형 데이터베이스 개요	61	2-5. GROUP BY, HAVING 절	117
2-2. SELECT 문	67	2-6. ORDER BY 절	127
2-3. 함수	76	2-7. 조인	135
2-4. WHERE 절	106	2-8. 표준 조인	146
단원 종합 문제	157		

PART2. SQL 활용

2-9. 서브 쿼리	165	2-13. Top N 쿼리	225
2-10. 집합 연산자	185	2-14. 계층형 질의와 셀프 조인	236
2-11. 그룹 함수	192	2-15. PIVOT 절과 UNPIVOT 절	246
2-12. 윈도우 함수	204	2-16. 정규 표현식	258
단원 종합 문제	274		

PART3. 관리 구문

2-17. DML	287	2-19. DDL	301
2-18. TCL	296	2-20. DCL	315
단원 종합 문제	321	기출 문제	324

들어가기 전

1. SQLD 자격증이란

● SQLD의 역할과 목적

- SQLD(SQL Developer)는 데이터를 불러오고, 가공하고, 분석할 수 있는 능력을 평가하는 국가 공인 자격증

● 시험 구조

- 총 2 과목 구성

과목명	문항수	배점	과락기준	검정시간
1 과목: 데이터 모델링의 이해	10	20 점	8 점 미만	90 분
2 과목: SQL 기본 및 활용	40	80 점	32 점 미만	
계	50	100		

- 4 지선다 객관식
- 필기시험 50 문항, 시험시간 총 90 분

● 합격 기준

- 1 과목 + 2 과목 합산 60 점 이상
- 과목별 최소 점수(과락) 조건 있음
 - 1 과목: 최소 8 점 이상 취득 필요
 - 2 과목: 최소 32 점 이상 취득 필요

● 공부 방법

- SQL 문장 해석이 핵심임 → 요구하는 데이터 정확히 파악
- 조건에 맞는 데이터 식별 능력 필요
- 반복 출제 유형 정리 필수(COUNT, 조인 결과, NULL 연산, 함수 결과 등)
- 문제별 시간 배분 및 우선순위 설정 필요
- 제한 시간 내 문제풀이 연습 필요

2. 데이터와 데이터베이스 이해

● 데이터(Data)의 개념

- 데이터는 현실에서 발생하는 사실 또는 기록을 의미함
- 숫자, 문자, 날짜, 사진, 영상 등 형태와 상관없이 '어떤 현상을 기록한 모든 것'을 데이터라고 함

● 데이터의 유형

: 데이터는 표현 방식과 구조에 따라 크게 다음과 같이 구분한다.

1. 정형 데이터(Structured Data)

- 엑셀과 같이 **행(row)과 열(column)** 형태로 구조가 명확하게 정해진 데이터
- 구조가 일정하기 때문에 검색, 정렬, 필터링과 같은 작업을 매우 빠르게 수행할 수 있음
- 예: 고객정보(고객번호, 이름, 연락처 등), 주문내역 등

2. 비정형 데이터(Unstructured Data)

- 비정형 데이터는 **정해진 구조가 없는 데이터**를 의미함
- 표 형태로 정리되어 있지 않기 때문에 컴퓨터가 즉시 이해하기 어려움
- 예: 사진, 동영상, 음성 파일, 메신저 대화 내용 등

3. 반정형 데이터(Semi-structured Data)

- 구조가 있기는 하지만, 정형 데이터처럼 행과 열로 고정되어 있지 않은 형태의 데이터
- 웹 서비스나 모바일 앱에서 자주 등장하는 데이터 구조
- 예: HTML, XML, JSON

HTML	XML	JSON
<pre><html> <body> <h1>학생 정보</h1> <p>이름: 홍길동</p> <p>학번: 2024001</p> </body> </html></pre>	<pre><Student> <Name>홍길동</Name> <StudentID>2024001</StudentID> </Student></pre>	<pre>{ "name": "홍길동", "studentId": 2024001 }</pre>

● DB 와 DBMS 의 차이

1. DB(DataBase)

- DB 는 데이터가 실제로 저장되어 있는 공간 즉, 정리된 상태로 보관된 데이터 그 자체를 의미함

2. DBMS(DataBase Management System)

- DBMS 는 데이터베이스를 생성·관리·제어하는 소프트웨어를 말함
- DBMS 를 통해 사용자는 데이터를 검색하고, 삽입하고, 수정하고, 삭제할 수 있음

● DBMS 가 필요한 이유

- 대규모 데이터의 빠른 조회
- 사용자 동시 접속 처리
- 데이터 무결성 보장
- 백업 및 복구 기능
- 권한 관리

● DBMS 종류

1. 관계형 데이터베이스(RDBMS)

- 정형 데이터를 저장하기 위한 DBMS
- 데이터를 행(Row)과 열(Column)로 이루어진 **테이블(Table)** 형태로 저장
- 예: Oracle, SQL Server, MySQL, PostgreSQL, MariaDB 등

2. NoSQL

- 기존 관계형 모델로 처리하기 어려운 **비정형·반정형 데이터**를 저장하고 처리하기 위해 등장한 DBMS 유형
- 정해진 테이블 구조 없이, 데이터 특성에 맞춘 유연한 저장 방식을 제공함
- 예: MongoDB, Redis, Cassandra, Neo4j 등

● SQLD 학습 전 반드시 알아야 할 핵심 용어 정리

1. 테이블(Table)

- RDBMS 에서 데이터를 저장하는 기본 단위로 **행(Row)과 열(Column)로 구성된 2차원 구조**를 가짐

2. 행(Row) / 레코드(Record)

- 테이블에서 **가로 방향**으로 저장된 데이터 단위를 의미함
- RDBMS 에서는 행 단위로 데이터가 입력되며 출력 및 연산도 행 단위로 처리함

3. 열(Column) / 속성(Attribute)

- 테이블에서 **세로 방향**으로 정의된 데이터 항목
- 각 열은 해당 테이블이 어떤 정보를 저장하는지를 결정함
예) 고객번호, 이름, 연락처, 가입날짜 컬럼 등

4. 스키마(Schema)

- 테이블 구조를 정의한 설계 정보로 어떤 열이 있고, 데이터 타입은 무엇이며, 제약조건은 어떻게 되어 있는지를 모두 포함하는 개념

5. 기본키(Primary Key, PK)

- 테이블에서 각 행을 **유일하게 식별하기** 위해 사용하는 한 개의 열 또는 둘 이상의 열들의 조합
- 값이 중복될 수 없으며 NULL(아직 정의되지 않은 값으로 공백과는 다른 의미) 삽입 불가
- **반드시 하나의 테이블에 하나만 존재함**
예) 사번, 학번, 주문번호 등

6. 외래키(Foreign Key, FK)

- 다른 테이블의 기본키를 참조하는 열로 두 테이블 간의 관계를 표현하는 데 사용됨
예) 주문 테이블의 고객번호는 반드시 고객 테이블의 고객번호를 참조하도록 설정

7. 데이터 타입(Data Type)

- 엑셀과 다르게 테이블의 각 컬럼(속성)은 정해진 데이터 타입을 가짐
- 숫자형(Number), 문자형(Char, Varchar2), 날짜형(Date)

8. 제약조건(Constraint)

- 데이터의 정확성과 무결성을 보장하기 위해 정보를 제한하는 규칙

구분	설명
PRIMARY KEY(기본키)	• 각 행을 유일하게 식별할 수 있도록 하는 제약조건(중복불가, NULL 불가)
UNIQUE	• 중복된 값을 금지하는 제약조건
CHECK	• 특정 조건만 허용하는 제약조건
FOREIGN KEY(외래키)	• 관계를 맺는 다른 테이블의 데이터를 참조하도록 하는 제약조건
NOT NULL	• 빈 데이터의 삽입 및 수정이 불가하도록 하는 제약조건

9. 관계(Relationship)

- **테이블 간의 연결**을 의미함
- RDBMS의 핵심으로 모든 테이블은 최소 한 개 이상의 다른 테이블과의 관계(연관성)를 맺고 있음

10. 뷰(View)

- 실제 데이터를 저장하지 않고, 기존 테이블에서 원하는 부분만 **가상의 테이블처럼 보여주는 객체**

11. 인덱스(Index)

- 검색 속도를 향상시키기 위해 사용하는 자료구조

12. 트랜잭션(Transaction)

- 데이터베이스에서 **하나의 작업 단위**를 의미함
- 하나의 트랜잭션은 **반드시 모두 성공하거나 모두 실패**해야 정상적인 상태를 유지할 수 있음(All-or-Nothing)
예) 계좌이체: A 계좌 -> B 계좌로 10만원 송금 시 A 계좌 -10만원, B 계좌 +10만원이 함께 성공해야 함

13. SQL(Structured Query Language)

- 관계형 데이터베이스(RDBMS)에서 데이터를 **정의하고, 조작하고, 제어하기 위해 사용하는 표준 언어**
- SQL 문장은 명령의 종료를 명확히 구분하기 위해 세미콜론(;)으로 마무리함

1과목.데이터 모델링의 이해

엔티티(Entity), 속성(Attribute), 인스턴스(Instance)에 대해서는 아래와 같이 생각하면 쉽습니다.

<학생테이블>

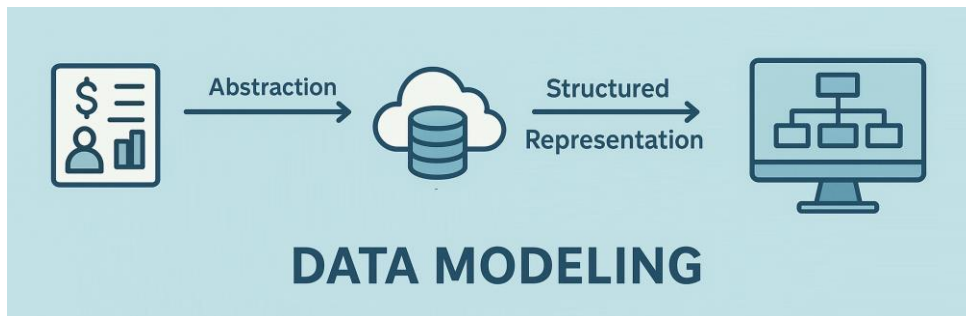
학번	이름	전화번호	주소	학과번호	지도교수번호	컬럼(속성)
1001	홍길동	010-0000-0000	서울시 ...	101	10000	행(인스턴스)
1001	박길동	010-1111-1111	경기도 ...	102	10001	
...	...	...	...	...	...	

테이블(엔티티)

1-1. 데이터 모델의 이해

● 모델링의 개념

- 현실 세계의 비즈니스 프로세스와 데이터 요구 사항을 추상화·구조화하여 표현하는 과정
- 데이터의 구조, 속성, 제약 및 관계를 정의하여 저장·활용 방식에 대한 명확한 기준을 마련



● 모델링의 특징

1. 단순화(Simplification)

- 현실의 복잡한 내용을 불필요한 세부사항은 제거하고 핵심만 남기고 정리하는 것
- 중요한 것만 남기면 전체를 이해하거나 설명하기가 쉬워짐

2. 추상화(Abstraction)

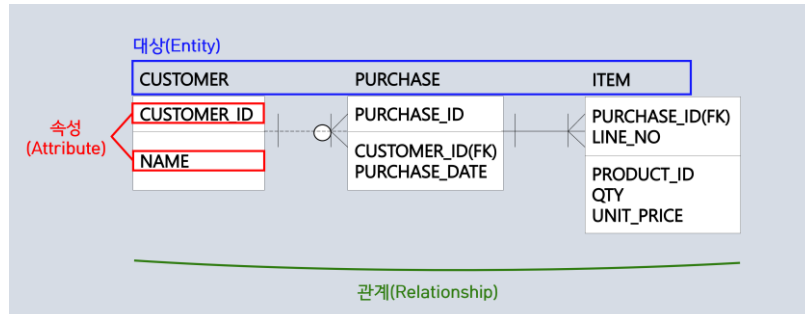
- 현실세계를 일정한 규칙이나 표기법으로 간략하고 대략적으로 표현하는 과정
- 같은 방식으로 표현하면 누가 봐도 비슷한 의미로 이해할 수 있음

3. 명확화(Clarity)

- 애매한 표현을 줄이고 정확하게 표현하는 것
- 정확히 표현하면 사람들끼리 오해 없이 의사소통할 수 있음

● 데이터 모델링 3 가지 요소

- 대상(Entity): 업무에서 관리해야 하는 사물, 사람, 개념 등의 객체
- 속성(Attribute): 대상이 가지는 특징이나 정보(값으로 표현될 수 있는 특성)
- 관계(Relationship): 대상들 간의 연결 또는 연관성



<ERD 예시>

● 데이터 모델링 3 가지 관점

1. 데이터 관점

- 데이터가 어떻게 저장·구조화·관리되는지를 정의하는 관점

2. 프로세스 관점

- 시스템이 무슨 일을 수행하고, 그 작업들이 어떻게 조직되는지를 정의하는 관점
- 업무 단계마다 어떤 데이터가 들어오고, 어떤 결과가 나오는지 보여줌
- 데이터가 시스템 내에서 어떻게 흐르고 처리되는지를 표현

3. 데이터와 프로세스 관점

- 데이터와 프로세스의 관점을 통합하여 시스템 전체를 이해하는 관점
- 각 프로세스가 어떤 데이터를 사용·생성·변경하는지를 명확히 정의

● 데이터 모델링 유의점

1. 중복(Duplication)

- 동일한 정보를 한 곳에서만 관리하도록 설계해야 함
- 여러 테이블에 같은 데이터를 저장하면 관리 비용 증가·오류 발생 가능

2. 비유연성(Inflexibility)

- 작은 업무 변화에도 테이블 구조를 계속 수정해야 하는 상황을 피해야 함
- 데이터 구조는 업무 절차(프로세스)와 최대한 분리하여 설계

3. 비일관성(Inconsistency)

- 비일관성: 데이터 간에 모순되거나 다른 결과가 나타나는 상태
- 데이터 간의 연관성과 규칙을 명확히 정의해야 함
- 데이터 품질 관리의 핵심
- 데이터 중복이 없더라도 업데이트 오류, 규칙 미정의 등으로 비일관성 발생 가능

● 데이터 모델링의 3 단계

1. 개념적 모델링

- 업무 중심적이며 전사적인 관점의 고수준 모델링
- 업무를 분석하여 **핵심 엔터티(Entity)**를 식별하는 단계
- **추상화 수준이 가장 높음**
- 엔터티 간의 관계를 표현하기 위해 ERD 초안 작성

2. 논리적 모델링

- 개념적 모델링의 결과를 바탕으로 **세부 속성, 식별자, 관계 등을 구체적으로 정의하는 단계**
- 이 단계에서 데이터의 구조가 명확히 정의되므로, 동일하거나 유사한 업무·프로젝트에서 동일한 논리 모델을 재사용할 수 있음
- 논리 모델을 재사용하면, 쿼리 역시 동일한 구조를 전제로 작성될 수 있어 재사용과 표준화가 가능함
- 이 과정에서 **정규화를 수행**하여 데이터 중복을 줄이고 데이터 품질을 높임
- 재사용성이 높은 논리적 모델은, 다양한 프로젝트에서 일관된 데이터 구조를 제공하기 때문에 유지보수 부담이 크게 낮아짐

3. 물리적 모델링

- 논리 모델을 **실제 데이터베이스 구조로 구현하는 단계**
- 성능, 저장 구조, 보안, 가용성 등 물리적 요소 고려
- **가장 구체적인 모델링이며, 추상화 수준이 가장 낮음**

● 스키마와 3 단계 구조

1. 개념

- 데이터 모델링을 통해 설계된 데이터 구조는 실제 데이터베이스에서 스키마로 표현됨
- 스키마란 데이터베이스에 저장되는 데이터의 구조와 제약 조건을 논리적으로 정의한 메타데이터
- 데이터베이스는 스키마를 **사용자·논리·물리 관점**으로 구분하여 관리하기 위해 **외부·개념·내부 스키마의 3 단계 구조를 사용함**

2. 스키마의 3 단계

구분	설명
외부 스키마	• 개별 사용자 또는 응용 프로그램 관점에서의 데이터베이스 스키마 • 사용자나 프로그램이 필요로 하는 데이터만을 정의(View가 대표적인 예)
개념 스키마	• 여러 외부 스키마를 통합하여 데이터베이스 전체의 논리적 구조를 정의 • 전체 데이터베이스의 개체, 속성, 관계, 데이터 타입, 제약 조건 등을 정의
내부 스키마	• 데이터가 물리적으로 어떻게 저장되는지를 정의 • 저장 구조, 파일 구조, 인덱스 등 물리적 구현 방법을 기술

3. 스키마의 독립성

- 독립성: 물리적, 논리적 구조를 변경하더라도 사용자가 사용하는 응용 프로그램에 영향을 주지 말아야 함

구분	설명
논리적 독립성	• 개념 스키마가 변경되어도 외부 스키마와 응용 프로그램에 영향이 없도록 하는 특성
물리적 독립성	• 내부 스키마(물리 구조)가 변경되어도 개념/외부 스키마에 영향을 주지 않는 특성

● 데이터 모델의 표기법(ERD: Entity Relationship Diagram)

- 엔터티(Entity)와 엔터티 간의 관계(Relationship)를 시각적으로 표현한 다이어그램
- 1976년 피터 첸(Peter Chen)이 제안한 표기법으로, 데이터 모델링의 표준 방식으로 사용됨
- 실무에서는 IE 표기법, Barker 표기법 등이 주로 사용됨

● ERD 작성 절차 (6 단계)

① 엔터티 도출

업무 분석을 통해 엔터티 후보를 식별하고 목록화

② 엔터티 배치

엔터티 간 관계 파악이 용이하도록 배치 구조 설정

③ 엔터티 간 관계 설정

엔터티 간의 연관성을 식별하고 관계 선으로 연결

④ 관계명 서술

관계의 의미를 명확히 표현

⑤ 관계의 참여도 기술(카디널리티)

1:1, 1:N, N:M 등 관계의 참여도를 정의

⑥ 관계의 필수 여부 확인(옵셔널/맨더토리)

관계가 반드시 존재해야 하는지 여부를 결정

<< 단원 정리 문제 >>

1. 데이터 모델링의 관점 중, 다음 설명에 가장 적합한 것은?

- 데이터가 처리되는 과정에 초점을 맞추고, 데이터가 어떻게 변형되고 이동하는지를 정의한다.
- 시스템이 어떻게 데이터를 처리하고, 어떻게 변환되는지에 대한 과정과 동작을 정의한다.

- ① 데이터 관점
- ② 프로그램 관점
- ③ 프로세스 관점
- ④ 데이터와 프로세스 관점

2. 다음 중 모델링의 특징이 아닌 것은?

- ① 단순화
- ② 구체화
- ③ 추상화
- ④ 명확화

3. 다음 설명에 해당하는 데이터 모델링 단계는 무엇인가?

엔티티를 기반으로 속성, 식별자, 관계를 구체적으로 정의하고 데이터 구조의 재사용성과 표준화를 높이는 단계이다.

- ① 개념적 모델링
- ② 논리적 모델링
- ③ 물리적 모델링
- ④ 구조적 모델링

4. 다음 중 데이터 모델링에 대한 설명으로 가장 적합하지 않은 것은?

- ① 현실 세계의 데이터를 분석하여 핵심 개념만 추출해 구조적으로 표현하는 활동이다.
- ② 데이터 간의 관계와 제약을 정의하여 이해관계자 간 의사소통을 돕는다.
- ③ 데이터 중복과 비일관성을 줄이기 위해 정규화 등의 기법을 적용할 수 있다.
- ④ 데이터베이스를 자동으로 생성하고 튜닝하는 기술적 기능을 포함한다.

1. ③

프로세스 관점은 고객 주문을 처리하는 절차나 주문에 대한 결제를 처리하는 과정을 모델링하는 것을 의미한다. 따라서 데이터가 처리되는 과정, 프로세스 흐름, 작업의 순서에 중점을 두고 모델링하는 과정을 말한다.

2. ②

모델링의 특징은 단순화, 추상화, 명확화이다.

3. ②

논리적 모델링은 개념적 모델링에서 도출한 엔티티를 바탕으로 속성, 식별자, 엔티티 간 관계를 구체적으로 정의하는 단계이다. 이 단계에서 데이터 구조를 표준화·정형화하면 유사한 업무에서 동일한 구조를 재사용할 수 있고, 그 구조를 전제로 작성된 쿼리 역시 재사용 및 표준화가 가능해진다.

4. ④

데이터 모델링은 현실 세계를 구조화하여 표현하고 데이터의 관계·제약을 정의하는 설계 활동이며, 데이터 품질 향상과 중복 감소를 위해 정규화 등의 기법을 활용할 수 있다. 반면, 데이터베이스를 자동으로 생성하거나 성능을 튜닝하는 기술적 기능을 포함하지 않으므로, ④번은 모델링의 개념과 맞지 않는 설명이다.

1-2. 엔터티

● 엔터티(Entity)의 개념

- 현실 세계에서 독립적으로 식별 가능한 객체나 사물을 의미함
- 엔터티는 **업무적으로 분석해야 하는 대상(객체)들의 집합**
- 인스턴스(Instance)는 **엔터티를 구성하는 개별 사례**로, 각 인스턴스는 엔터티의 속성 값들로 표현되며 엔터티를 현실에서 구체화한 것임

예) 엔터티, 속성, 인스턴스

- 엔터티(Entity): 학생
- 속성(Attribute): 학번, 이름, 학과 등
- 식별자(Identifier): 학번
- 인스턴스(Instance): 특정 학생의 데이터(학번: 2021001, 이름: 홍길동, 학과: 컴퓨터 공학)

● 엔터티(Entity)의 특징

1. 유일한 식별자에 의해 식별 가능

- 각 인스턴스는 식별자를 통해 서로 구분될 수 있어야 함
- 유일한 식별자는 각 인스턴스만의 고유한 이름 역할을 함
- 예: 이름은 중복될 수 있으므로, 사번·학번 등과 같이 중복되지 않는 값이 식별자가 됨

2. 해당 업무에 필요하고 관리하고자 하는 정보

- 엔터티는 설계하려는 업무 시스템에 필요한 정보여야 함
- 예: 학교 시스템에서는 학생 정보가 필요하지만, 다른 업무에서는 필요하지 않을 수 있음

3. 인스턴스들의 집합

- 엔터티는 2 개 이상의 인스턴스가 지속적으로 존재하는 집합이어야 함
- 인스턴스가 1 개뿐이라면, 집합으로 보기 어려워 엔터티로 적합하지 않음

4. 엔터티는 반드시 속성을 가짐

- 각 엔터티는 2 개 이상의 속성(Attribute)을 가져야 함
- 하나의 인스턴스는 각 속성에 대해 하나의 값만 가짐
- 예: 학생 엔터티에서 한 학생(인스턴스)의 '이름' 속성에는 한 명의 이름 값만 저장됨

5. 엔터티는 업무 프로세스에 의해 이용됨

- 업무적으로 필요하다고 선정했지만 실제로 사용되지 않는 엔터티는 잘못 설계된 것
- 모델링 과정에서 발견이 어려울 수 있으므로, 검증 과정이나 프로세스 점검을 통해 확인 필요
- 누락된 프로세스가 있으면 추가, 사용되지 않는 고립된 엔터티는 제거하는 것이 바람직함

6. 다른 엔터티와 최소 1 개 이상의 관계 성립

- 엔터티는 업무상 다른 엔터티와 의미 있는 연관 관계를 가져야 함
- 관계가 전혀 없다면, 부적절한 엔터티이거나 관계 설정이 잘못된 것일 수 있음

● 엔터티의 분류

1. 유형과 무형에 따른 분류

구분	설명	예시
유형 엔터티	- 물리적인 형태가 있는 실체적 대상 - 비교적 안정적이고 지속적으로 관리되는 정보	사원, 학생, 물품, 지점 등
개념 엔터티	물리적 형태는 없지만, 관리해야 하는 개념적 정보를 의미	부서, 요금제, 대출상품 등
사건 엔터티	- 업무 수행 과정에서 발생하는 사건이나 활동 - 발생량이 많고, 통계나 분석 자료로 활용됨	주문, 청구, 가입, 신청 등

2. 발생 시점에 따른 분류

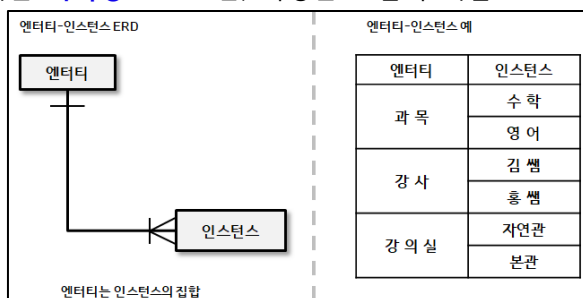
구분	설명	예시
기본 엔터티	- 해당 업무에서 원래부터 존재하는 핵심 정보 - 다른 엔터티와의 관계에 의해 생성되지 않고 독립적으로 생성됨 - 다른 엔터티의 부모 역할을 하는 상위 엔터티 - 다른 엔터티로부터 주식별자를 상속받지 않고 고유한 주식별자를 가짐	고객, 사원, 부서, 상품 등
중심 엔터티	- 기본 엔터티를 기반으로 생성되며, 업무에서 핵심적인 역할을 수행 - 많은 데이터가 발생하고, 다른 엔터티와의 관계를 통해 여러 행위 엔터티를 만들어냄	계약, 주문, 청구, 매출, 대출 등
행위 엔터티	- 중심 엔터티의 발생/변화/흐름을 기록하는 엔터티 2 개 이상의 부모 엔터티로부터 발생 - 데이터가 자주 변경되거나 증가하는 특성을 가짐 - 주로 분석 초기보다는 상세 설계 단계나 프로세스/상관 모델링 과정에서 도출	주문이력, 결제이력, 계좌거래, 인사이력 등

● 엔터티의 명명 규칙

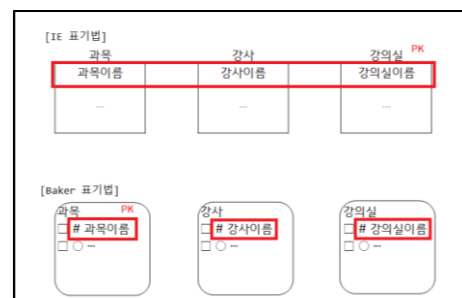
- ① 현업에서 사용하는 용어 사용
- ② 약어(줄임말) 사용 최소화
- ③ 단수 명사 사용
- ④ 엔터티마다 고유한 이름 부여
- ⑤ 엔터티의 의미를 반영하여 명명

● 엔터티와 인스턴스 표기법

- 엔터티는 **사각형**으로 표현, 속성은 조금씩 다름



< 엔터티와 인스턴스 >



< 엔터티 표기법 >

<< 단원 정리 문제 >>

1. 다음 중 엔터티(Entity)에 대한 설명으로 가장 적절하지 않은 것은?

- ① 엔터티는 실제 세계에서 객체나 개념을 표현하는 데이터베이스 구조이다.
- ② 엔터티는 데이터베이스에서 저장될 수 있는 정보를 나타내며, 속성과 관계를 가질 수 있다.
- ③ 엔터티는 오직 한 개체만을 대표하며, 여러 개체가 결합되어 하나의 엔터티가 형성된다.
- ④ 엔터티는 데이터 모델링에서 중요한 요소로, 데이터베이스 설계에서 관계를 정의하는 데 사용된다.

2. 다음 중 엔터티의 분류중 유형과 무형에 따른 분류에 속하지 않는 것은?

- ① 유형엔터티
- ② 사건엔터티
- ③ 개념엔터티
- ④ 중심엔터티

3. 다음 중 다른 유형의 엔터티에 해당하는 것은?

- ① 고객
- ② 조직
- ③ 회원등급
- ④ 부서

4. 다음 중 엔터티의 특징에 대한 설명으로 가장 적절하지 않은 것은?

- ① 엔터티는 각 인스턴스를 유일하게 구분할 수 있는 식별자를 가져야 한다.
- ② 엔터티는 해당 업무에서 관리할 필요가 있는 정보를 나타낸다.
- ③ 엔터티는 하나의 인스턴스만 존재하더라도 엔터티로 인정될 수 있다.
- ④ 엔터티는 반드시 2개 이상의 속성을 가진다.

1. ③

엔터티는 하나의 개체를 나타내며 여러 개체의 결합으로 형성되지 않는다.

즉, 여러 엔터티가 관계를 맺을 수 있지만, 각 엔터티는 독립적인 개체나 개념을 의미한다.

2. ④

중심엔터티는 발생 시점에 따른 분류에 속한다.

3. ①

조직, 부서, 회원등급은 물리적 실체가 없는 개념 엔터티이다. 고객은 실제 사람이므로 유형 엔터티에 속한다.

4. ③

엔터티는 여러 인스턴스(2개 이상)가 지속적으로 존재하는 집합이어야 하므로 인스턴스가 1개뿐이면 엔터티로 보기 어렵다.

1-3. 속성

● 속성(Attribute)의 개념

- 업무에서 필요로 하는 고유한 성질이나 특징을 의미하며, 데이터 모델에서 컬럼으로 표현됨
- 업무상 관리하고자 하는 정보 중 더 이상 분리되지 않는 최소 단위의 데이터임
- 속성은 인스턴스를 구성하는 요소로, 각 인스턴스는 여러 속성 값들로 표현됨
- 예: 학생 엔터티에서 이름, 학번, 학과번호 등은 각각 속성이 될 수 있음

● 엔터티, 인스턴스, 속성, 속성값의 관계

- 하나의 엔터티는 2 개 이상의 인스턴스로 구성된 집합이어야 함
→ 즉, 하나의 테이블은 2 개 이상의 행을 가져야 함
- 하나의 엔터티는 2 개 이상의 속성(Attribute)을 가져야 함
→ 즉, 하나의 테이블은 2 개 이상의 컬럼으로 구성됨
- 하나의 속성은 각 인스턴스마다 1 개의 속성값(Value)을 가짐
→ 즉, 한 컬럼의 데이터는 각 행마다 1 개의 값만 저장됨
- 속성은 엔터티에 대한 구체적이고 상세한 정보를 표현하며, 각 속성은 해당 정보를 나타내는 구체적인 값을 가지고 있음

● 속성의 특징

- 반드시 업무상 필요하고 관리할 가치가 있는 정보여야 함
- 해당 엔터티의 **주식별자에 함수적으로 종속**되어야 함
- 속성은 **단일값**이어야 함(Atomic)
- 속성이 여러 값을 가지는 구조(다중값)일 경우, 별도의 엔터티로 분리하여 관리해야 한다.
- 하나의 인스턴스는 각 속성에 대해 반드시 하나의 속성값을 가짐(속성의 원자성)



원자성(Atomicity)



하나의 속성은 더 이상 분해되지 않는 하나의 값만을 가져야 한다는 특성을 의미한다.

● 도메인(Domain)

- 각 속성이 가질 수 있는 값의 범위를 의미
- 데이터 타입, 길이, 허용 값, 제약조건 등을 정의하여 어떤 값이 입력될 수 있는지를 제한하는 것

● 함수적 종속성(FD: Functional Dependency)

1. 개념

- 어떤 속성의 값이 다른 속성에 의해 유일하게 결정되는 관계
- 즉, 어떤 속성 A의 값에 의해 다른 속성 B가 유일하게 결정된다면 B는 A에 함수적으로 종속되었다 하고, 이를 수식으로 나타내면 $A \rightarrow B$ 라고 표현함
- 예: 학번 $\rightarrow$ 학생이름(학번을 알면 학생 이름은 하나로 결정됨)
주민번호 $\rightarrow$ 생년월일(주민번호를 알면 생년월일은 하나로 결정됨)

2. 복합키의 함수적 종속

1) 완전 함수적 종속(Full FD)

- 어떤 속성이 복합키 전체에 의해 결정됨
- 예: 주문(PK: 주문번호 + 제품번호)에서 주문번호와 제품번호에 의해 수량 속성의 값이 결정
 $\rightarrow$ 완전 함수적 종속 관계

주문 PK		
주문번호	제품번호	수량
1	100	4
1	101	1
2	100	3
2	101	2

2) 부분 함수적 종속(Partial FD)

- 어떤 속성이 복합키의 일부에만 의해 결정됨
- 예: 수강상세(PK: 학번 + 과목)에서 강사 속성은 주식별자의 일부인 과목 속성에 의해 값이 결정
 $\rightarrow$ 부분 함수적 종속 관계
(단, 과목에 대한 강사가 유일하다고 가정)

수강상세 PK		
학번	과목	강사
1001	수학	홍쌤
1002	수학	홍쌤
1001	과학	최쌤
1003	영어	김쌤



키(Key)



테이블에서 각 행(인스턴스)을 유일하게 식별하기 위해 사용되는 속성 또는 속성들의 집합이며, 그중 각 행을 유일하게 구분하는 키를 주식별자(Primary Key)라고 한다.



복합키(Composite Key)



두 개 이상의 속성을 결합하여 하나의 튜플(행)을 유일하게 식별하는 키를 의미한다.

● 속성의 분류

1. 속성의 특성에 따른 분류

구분	설명	예시
기본 속성	- 업무에서 원래부터 존재하는 데이터 - 주로 사용자가 직접 입력하거나 관리하는 값으로, 다른 속성의 값을 이용해 산출(계산)해내는 값의 형태가 아님 - 엔터티에 가장 일반적으로 많이 존재하는 속성	사번, 이름, 생년월일, 원금, 예치기간, 계좌번호, 상품코드, 통화코드, 이자율 등
설계 속성	- 업무에는 원래 필요하지 않았지만 시스템 설계나 운영을 편하게 하기 위해 새로 만든 데이터 - 현실 세계에는 존재하지 않는 가상의 데이터 - 대체로 사람이 직접 입력하기보다는 시스템에서 자동 생성	거래상태코드, 응답코드, 삭제여부, 주문번호, 예금분류 등
파생 속성	- 다른 속성 값을 기반으로 계산하거나 추론하여 도출되는 속성 - 주로 저장, 입력하지 않아도 자동으로 계산 가능한 값의 형태 - 원래 데이터가 존재해야 산출됨 - 조회 시 반복적인 연산을 줄여 성능을 향상시키기 위해 사용됨	나이, 이자(원금 X 이자율), 총판매액 등

2. 엔터티 구성방식에 따른 분류

구분	설명	예시
기본키 속성(PK)	인스턴스를 유일하게 식별하는 속성	학생(학번), 고객(고객번호), 상품(상품번호) 등
외래키 속성(FK)	다른 엔터티의 키를 참조하여 관계를 표현하는 속성	주문(고객번호, 상품번호) 등
일반 속성	PK/FK를 제외한 일반적인 정보 속성	주소, 전화번호, 가격 등

3. 분해 여부에 따른 속성

구분	설명	예시
단일 속성	하나의 값만 저장	고객번호, 이름, 가격 등
복합 속성	여러 하위 속성으로 분해 가능	주소(도/시/구/동) 등
다중값 속성	- 하나의 속성이 여러 값을 가짐 - 별도 엔터티로 분리 필요	여러 개의 전화번호, 여러 개의 자격증 등

● 속성의 명명규칙

- ① 업무에서 실제로 사용하는 용어를 사용
- ② 서술형(문장형) 이름은 사용하지 않음
- ③ 약어는 가급적 사용을 제한
- ④ 전체 데이터 모델에서 중복되지 않는 고유한 이름을 사용

<< 단위 정리 문제 >>

1. 다음 중 엔터티, 인스턴스, 속성, 속성값의 관계에 대한 설명으로 가장 적절하지 않은 것은?
 - ① 엔터티는 실제 세계의 객체나 개념을 나타내며, 속성은 엔터티가 가진 특성을 나타낸다.
 - ② 인스턴스는 엔터티의 구체적인 실체로, 각 인스턴스는 엔터티의 속성값을 가질 수 있다.
 - ③ **한 개의 엔터티는 한 개 이상의 인스턴스의 집합이어야 한다.**
 - ④ 속성은 엔터티의 특성을 설명하는 항목으로, 엔터티에 속하며 여러 개의 인스턴스를 가질 수 있다.

2. 다음 중 분해 여부에 따른 속성 분류에 속하지 않는 것은?
 - ① 단일 속성
 - ② 복합 속성
 - ③ 다중값 속성
 - ④ **일반 속성**

3. 다음 중 유형이 다른 속성 하나를 고르시오.
 - ① 단가
 - ② 수량
 - ③ 상품명
 - ④ **매출금액**

4. 다음 중 설계 속성이 아닌 것은?
 - ① **신용점수**
 - ② 삭제여부
 - ③ 정렬순서코드
 - ④ **생성일시**

1. ③

한 개의 엔터티는 두 개 이상의 인스턴스의 집합이어야 한다.

2. ④

일반 속성은 분해 여부에 따른 속성 분류에는 포함되지 않는다.

3. ④

매출금액은 단가와 수량 등 다른 속성으로부터 계산되는 파생 속성이다. 나머지는 모두 기본 속성이다.

4. ①

설계 속성은 시스템 구현이나 데이터 관리 목적을 위해 추가되는 속성으로, 업무 자체의 의미보다는 관리·통제 용도로 사용된다. 삭제여부, 정렬순서코드, 생성일시는 데이터 관리 및 시스템 설계를 위해 부여되는 전형적인 설계 속성이다. 반면 신용점수는 업무에서 실제로 의미를 가지며, 다른 값들에 의해 자동으로 생성되는 값이므로 파생 속성에 해당한다.

1-4. 관계

● 관계(Relationship)의 개념

- 두 엔터티 간에 존재하는 업무적 연관성을 의미
- 관계를 정의할 때는 엔터티 수준이 아닌, 인스턴스(행) 수준에서 실제로 연관이 발생하는지를 기준으로 판단함
- 엔터티의 범위나 정의가 변경되면, 그에 따라 **관계의 종류나 방향도 달라질 수 있음**

● 관계의 종류

1. 존재적 관계

- 한 엔터티의 존재 여부가 다른 엔터티의 존재에 영향을 미치는 관계
- 즉, 어떤 엔터티는 다른 엔터티가 있어야만 존재할 수 있는 경우
- 주로 부모-자식 구조에서 많이 나타남
- 예: 사원-부서(부서가 삭제되면, 해당 부서를 참조하고 있는 사원 엔터티의 존재에 영향을 미침)

2. 행위적 관계

- 한 엔터티의 행동(행위)에 의해 다른 엔터티가 발생하는 관계
- 즉, 업무 활동이 새로운 엔터티를 생성하거나 변경시키는 경우
- 예: 고객-주문(고객의 행동이 주문이라는 데이터를 만들고, 양쪽 엔터티가 인스턴스 수준에서 연결됨)

※ ERD에서는 존재관계와 행위관계를 구분하지 않는다.

● 관계의 구성

1. 관계명(Relationship Name)

- 엔터티 간의 관계가 무엇을 의미하는지 명시하는 이름
- 보통 동사형으로 표현
- 예:
 - 부서 (배치한다) 사원
 - 고객 (주문한다) 주문
 - 학생 (수강한다) 강의

2. 차수(Cardinality)

- 두 엔터티가 얼마나 연결되는지(수량)를 나타냄
- 두 엔터티의 한 행과 다른 행이 어떻게 연결되어 있는지 보여주는 것

1) 1:1(일대일)

- 하나의 엔터티의 인스턴스가 다른 엔터티의 인스턴스와 오직 하나씩만 연결되는 관계
- 예:
 - 사원-사원증(한 명의 사원은 하나의 사원증만 가짐)
 - 국가-수도(각 나라는 하나의 수도를 가짐)

2) 1:N(일대다)

- 한 엔터티의 하나의 인스턴스가 다른 엔터티의 여러 인스턴스와 연결될 수 있는 관계
- 예:
 - 고객-계좌(한 고객은 여러 개의 계좌를 소유할 수 있음)
 - 부서-사원(한 부서는 여러 명의 사원을 가질 수 있음)

3) N:M(다대다)

- 두 엔터티의 인스턴스가 서로 여러 개씩 연결될 수 있는 관계
(A의 한 인스턴스가 여러 개의 B와 연결되고, B의 한 인스턴스도 여러 개의 A와 연결되는 구조)
- **조인 시 의미 없는 Cartesian product 위험 존재**
- 두 엔터티 사이에 연결 엔터티를 추가하여 두 개의 1:N 관계로 분해하는 것이 일반적
- 예:
 - 학생-강의(한 학생은 여러 강의를 수강할 수 있고, 한 강의도 여러 학생이 수강할 수 있음)
 - 직원-프로젝트(한 프로젝트에는 여러 명의 직원 참여 가능, 한 직원은 여러 프로젝트에 참여할 수 있음)

3. 선택성(Optionality)

- 두 엔터티 사이의 관계가 반드시 존재해야 하는지, 아니면 존재하지 않아도 되는지를 나타내는 제약 조건
- **필수(Mandatory), 선택(Optional) 관계**로 구분
- 양쪽 엔터티가 각각 독립적으로 정의될 수 있음
 - 두 엔터티 간의 **관계 방향에 따라 한쪽은 필수, 다른 쪽은 선택이 될 수 있음**

1) 필수 관계

- 한 엔터티의 인스턴스가 반드시 다른 엔터티와 연결되어 있어야 하는 관계
- 예:
 - 사원 → 부서(사원은 부서 없이 정의 불가)
 - 계좌 → 고객(계좌가 존재하면 반드시 계좌의 주인(고객)이 있어야 함)

2) 선택 관계

- 한 엔터티의 인스턴스가 다른 엔터티와 연결되지 않아도 존재할 수 있는 관계
- 예:
 - 부서 → 사원(부서는 사원이 없어도 존재할 수 있음)
 - 고객 → 계좌(고객은 계좌를 갖지 않아도 존재할 수 있음)

● ERD의 표현

- ERD에서는 두 엔터티 간의 관계를 시각적으로 표현하기 위해 **차수와 선택성을 표시함**
- 관계명은 일반적으로 생략함

1. 차수

IE 표기법)

- 관계선 끝에 다음 기호를 사용

1	{(Bar)	N	<(삼발이)
---	--------	---	--------

Barker 표기법)

- IE 표기법과 동일하게 표현되지만 **Bar(|) 기호를 사용하지 않는다**
(| 기호는 Barker에서 식별/비식별 관계를 나타내는 용도로 사용되기 때문)

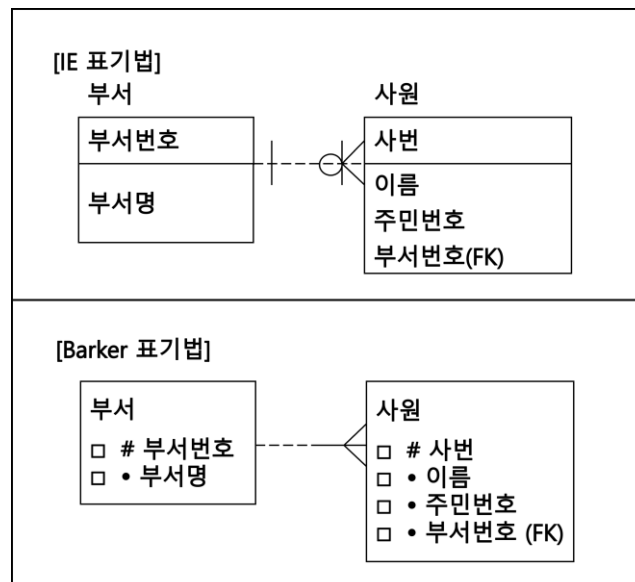
2. 필수/선택 관계

IE 표기법)

- **원을 사용하여** 필수적 관계와 선택적 관계를 구분
- 필수 관계에는 관계선 끝에 **원이 없음**
- 선택 관계에는 관계선 끝에 **원을 그림**

바커 표기법)

- **실선과 점선으로 구분**
- 필수 관계는 관계선을 **실선**으로 표기
- 선택 관계는 관계선을 **점선**으로 표기



<< 단원 정리 문제 >>

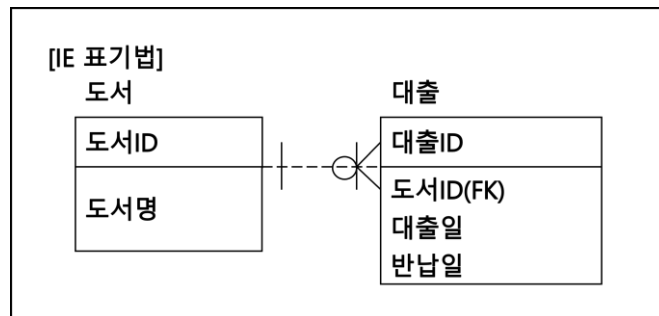
1. 다음 중 관계에 대한 설명으로 가장 적절하지 않은 것은?

- ① 관계는 두 개 이상의 엔터티 간의 연관성을 나타낸다.
- ② 관계는 테이블 내에서 한 컬럼을 다른 컬럼과 연결하는 방식으로 설정된다.
- ③ 관계는 주로 1:N, N:M, 1:1의 형태로 나타낼 수 있다.
- ④ 관계를 설정하면 데이터베이스에서 각 엔터티 간의 상호작용을 정의할 수 있다.

2. 다음 중 관계에 대한 설명으로 가장 적절하지 않은 것은?

- ① 존재적 관계란 한 엔터티의 존재가 다른 엔터티의 존재에 영향을 미치는 관계를 의미한다.
- ② 행위적 관계란 엔터티 간의 어떤 행위가 있는 것을 의미한다.
- ③ 고객과 주문 엔터티는 행위적 관계를 갖는다.
- ④ 선택적 관계란 하나의 엔터티의 고유키가 다른 엔터티의 고유값으로 갖지 않는 경우의 관계를 말한다.

3. 다음 ERD를 보고 관계를 해석한 내용으로 옳지 않은 것은?



- ① 대출은 반드시 한 개의 도서를 소속해야 한다.
- ② 도서는 대출되지 않아도 존재할 수 있다.
- ③ 한 도서는 여러 번 대출될 수 있다.
- ④ 한 번의 대출 기록이 여러 권의 도서 대여를 처리할 수 있다.

1. ②

관계는 인스턴스간의 논리적인 연관성을 연결하는 방식으로 정의된다.

2. ④

선택적 관계란 하나의 엔터티에 관계되는 엔터티가 하나이거나 없을 수 있는 경우의 관계를 말한다.

3. ④

대출 1건(하나의 대출ID)이 여러 도서를 포함할 수 없으므로 틀린 설명이다.

1-5. 식별자

● 식별자 개념

- 하나의 엔터티 안에서 각 인스턴스를 유일하게 식별할 수 있도록 하는 속성을 나타냄
- 엔터티는 여러 인스턴스의 집합이므로, 이들을 구분하기 위해 반드시 유일한 식별자 후보가 존재해야 함
- 여러 식별자 후보 중에서 엔터티를 대표하는 식별자를 주식별자라고 하며, 논리 모델의 주식별자는 데이터베이스에서 기본키(Primary Key, PK)로 구현됨
- 예: 학생 엔터티의 주식별자는 학생번호 속성 => 학생 테이블의 기본키는 학생번호 컬럼
(논리 모델링) (물리 모델링)

● 식별자 분류

1. 대표성 여부에 따른 식별자의 종류

구분	설명
주식별자	- 엔터티 내에서 각 인스턴스를 유일하게 구분할 수 있는 식별자 - 유일성과 최소성을 만족하는 식별자 - 다른 엔터티와의 참조관계 연결 기준으로 사용되는 대표 식별자 - 예: 학생(학생번호), 도서(ISBN)
보조식별자	- 엔터티 내에서 인스턴스를 구분할 수는 있지만, 대표성이 없어 참조관계 연결에 사용되지 않는 식별자 - 유일성과 최소성은 만족하지만, 대표성을 만족하지 못하는 식별자 - 예: 학생(주민번호), 고객(이메일)

2. 생성 여부(발생 시점)에 따른 식별자의 종류

구분	설명
내부식별자	- 다른 엔터티를 참조하지 않고, 엔터티 내부에서 자체적으로 생성되는 식별자 - 예: 주문(주문번호), 상품(상품번호), 고객(고객번호)
외부식별자	- 다른 엔터티의 식별자를 상속받아 사용하는 식별자 - 관계 설정을 통해 생성되며 외래키(FK)로 구현됨 - 예: 주문(고객번호, 상품번호)

3. 속성 수에 따른 식별자 종류

구분	설명
단일식별자	- 하나의 속성으로 구성된 식별자 - 예: 학생(학생번호), 사원(사번)
복합식별자	- 두 개 이상의 속성으로 구성된 식별자 - 예: 주문(주문번호 + 상품번호)

4. 대체 여부에 따른 식별자의 종류

구분	설명
본질식별자 (원조식별자)	- 인스턴스가 탄생할 때 업무적으로 자연스럽게 부여되는 속성 - 예: 학생(학번, 주민번호), 도서(ISBN), 차량(차량등록번호)
인조식별자	- 업무적으로 의미가 없으나 시스템에서 고유성을 보장하기 위해 인위적으로 생성한 식별자 - 주로 자동 증가하는 일련번호 같은 형태 - 예: 주문(주문번호: 날짜 + 일련번호 등의 속성을 결합하여 생성), 송장(송장번호) 등

● 주식별자 특징

1. 유일성

- 주식별자는 모든 인스턴스를 유일하게 구분할 수 있어야 함
- 예: **학생 엔터티**
 - **이름**: 동명이인이 존재할 수 있으므로 유일성을 만족하지 못함
 - **학생번호**: 각 인스턴스를 유일하게 구분할 수 있으므로 주식별자로 사용함

2. 최소성

- 주식별자는 유일성을 만족하기 위해 필요한 최소한의 속성만으로 구성되어야 함
- 예: **직원 엔터티**
 - **사번**: 사번만으로 유일성이 확보되므로, 사번과 이름을 함께 주식별자로 사용할 필요가 없음

3. 불변성

- 주식별자의 값은 시간이 지나도 변경되지 않아야 함(값이 바뀌면 데이터의 연속성과 무결성이 깨짐)
- 예: **고객 엔터티**
 - **고객번호**: 정보 수정 여부와 무관하게 항상 동일해야 함

4. 존재성

- 주식별자는 NULL 이 될 수 없으며 반드시 값이 존재해야 함
- 예: **상품 엔터티**
 - **상품번호**: 상품번호는 각 상품을 식별하므로 NULL 일 수 없음

● 주식별자 도출 기준 (Primary Key 선정 기준)

1. 업무에서 자주 사용되는 속성을 우선적으로 선택

- 동일한 조건을 만족하더라도, 업무적으로 활용 빈도가 높은 속성을 주식별자로 지정하는 것이 바람직함
- 예: **학생 엔터티**
 - **학생번호**: 주식별자
 - **주민번호**: 보조식별자

2. 명칭이나 설명(이름) 기반 속성은 피하기

- 사람 이름, 부서명 등 텍스트 기반 명칭을 주식별자로 사용하는 것은 부적절
- 가능한 경우 코드형 속성을 만들거나 사용하는 것이 좋음
- 예: **부서 엔터티**(부서코드가 정의되지 않은 경우)
 - 부서명 보다는 **부서코드**를 인위적으로 생성하여 부서코드를 주식별자로 사용

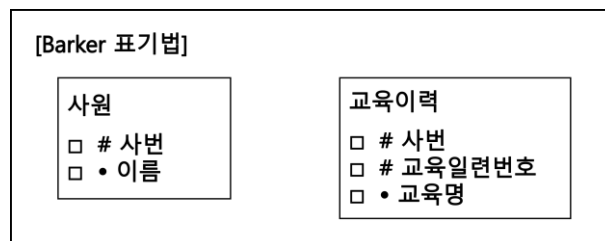
3. 속성 수는 최소화하여 단순하게 구성

- 주식별자를 너무 많은 속성으로 구성하면 조인 성능 저하 발생 가능
- 일반적으로 7~8 개 이상의 속성이 필요하다면, 새로운 인조 식별자를 생성하는 것이 바람직함
- 예: **주문 엔터티**
 - 주문일자 + 주문상품코드 + 고객번호 등 여러 속성의 조합 대신 **주문번호** 속성을 추가하여 단일식별자로 사용

● 엔터티 간 관계 유형: 식별 관계와 비식별관계

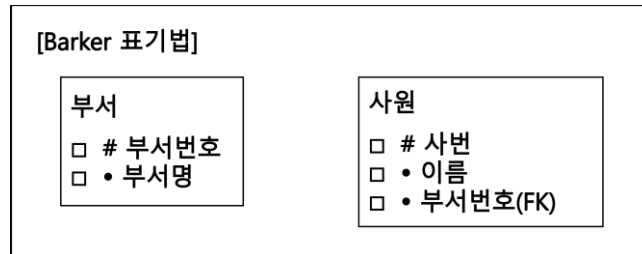
1. 식별 관계(Identifying Relationship)

- 부모 엔터티의 주식별자(Primary Key)가 자식 엔터티의 주식별자에도 포함되는 관계
- 부모의 키가 자식의 식별자 역할까지 수행하는 구조
- 부모 엔터티 없이는 **자식 엔터티가 독립적으로 식별될 수 없음**
- 예: **사원 엔터티**(주식별자: 사번) - **교육이력 엔터티**(주식별자: 사번 + 교육일련번호)



2. 비식별관계(Non-Identifying Relationship)

- 부모 엔터티의 주식별자가 자식 엔터티의 주식별자에 포함되지 않는 관계
- 부모의 식별자는 자식에서 일반 속성(FK)로만 존재
- 자식 엔터티는 독립적으로 식별자(PK)를 가짐
- 예: 부서 엔터티(주식별자: 부서번호) - 사원 엔터티(주식별자: 사번, 일반 속성: 부서번호(FK))



● 관계에 따른 엔터티 성격(강/약 개체)

1. 강한 개체(Strong Entity)

- 비식별관계로도 존재할 수 있는 독립 엔터티
- 자체 PK로 식별 가능
- 부모 엔터티 없이도 존재 가능
- 기본 엔터티가 대부분 이에 해당
- 예: 고객, 사원, 상품

2. 약한 개체(Weak Entity)

- 식별관계를 통해 정의되는 엔터티
- 자식이 부모의 PK 없이는 식별 불가
- 독립적 존재 불가능
- 대부분 복합 키 구성이 요구됨

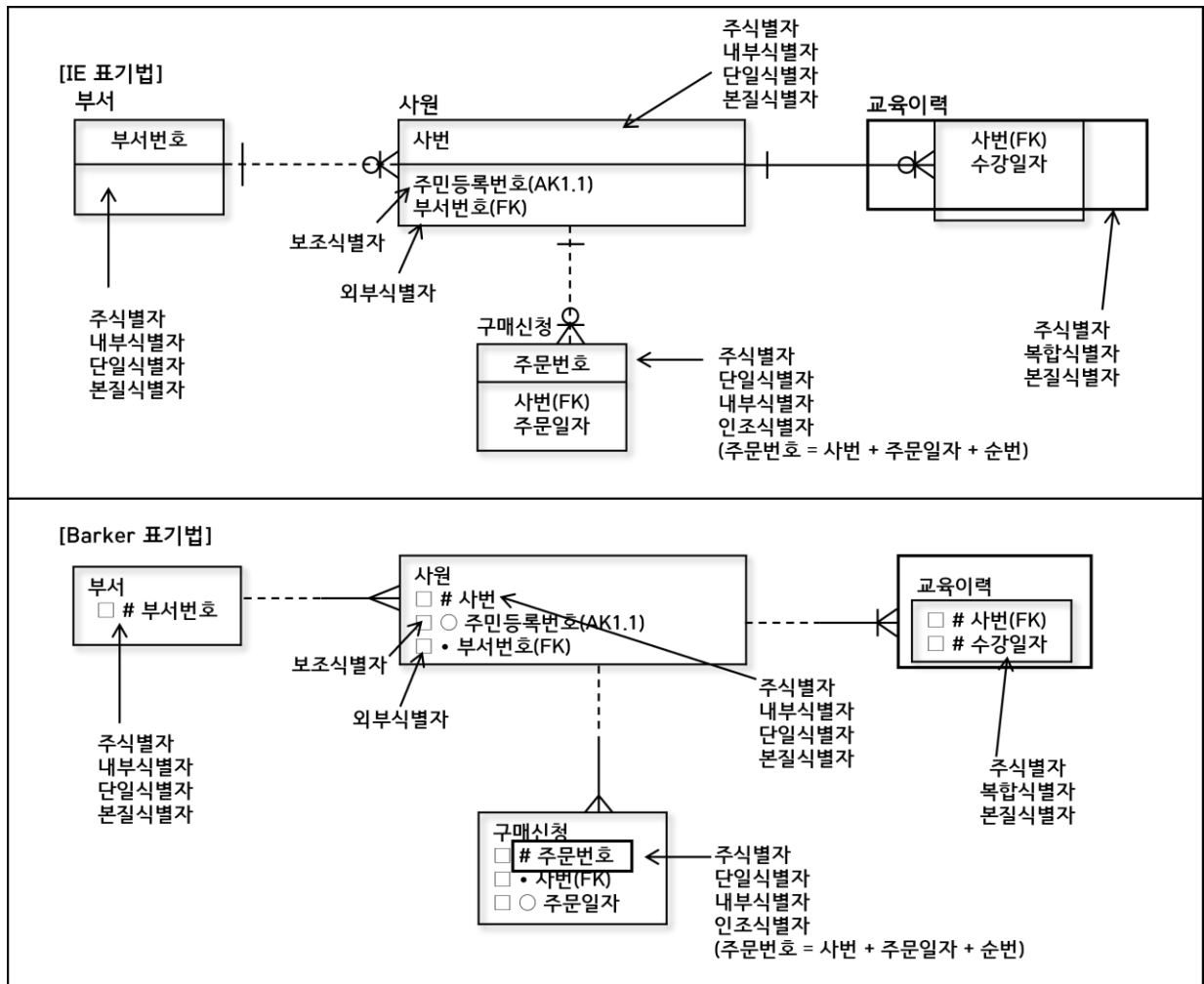
● Key 의 종류

: 논리 모델링에서의 식별자는 물리 모델링에서 Key 로 구현되며, Key 는 데이터의 식별, 무결성, 관계 설정에 중요한 역할을 함

구분	설명
기본키 (Primary Key, PK)	• 각 레코드를 고유하게 식별할 수 있는 대표 키 • NULL 불가, 중복 불가 • 속성이 하나일 수도 있고 여러 개(복합키)일 수도 있음 • 예: 학생(학번), 사원(사번)
후보키 (Candidate Key)	• 기본키가 될 수 있는 후보 키 집합 • 모든 후보키는 유일성 + 최소성을 만족해야 함 • 후보키 중 하나가 기본키로 선택되고, 나머지 후보키는 대체키가 됨 • 예: 학생(학번, 주민번호)
슈퍼키 (Super Key)	• 레코드를 유일하게 식별할 수 있는 속성 또는 속성 집합 • 유일성은 만족하지만, 최소성은 만족하지 않을 수 있음
대체키 (Alternate Key)	• 후보키 중에서 기본키로 선택되지 않은 키 • 여전히 유일성과 최소성을 만족 • 예: 학생(주민번호)
외래키 (Foreign Key, FK)	• 다른 테이블의 기본키(PK)를 참조하는 키 • 참조 테이블에 존재하는 값만 입력 가능 • 두 테이블 간 관계를 표현하고 무결성을 유지 • 하나의 FK 는 하나 이상의 테이블을 참조할 수 있음 • 예: 사원(부서번호), 수강(학번)

● 식별자 표기법

구분	IE 표기법	Barker 표기법
주식별자와 일반 속성	- 주식별자(PK)는 최상단에 배치 - 일반 속성은 그 아래에 배치	- 주식별자(PK)는 속성명 앞에 # 기호 붙임 - 일반 속성은 앞에 • (점) 기호 붙임
식별/ 비식별 관계	- 식별관계: 실선 - 비식별관계: 점선	- 식별관계: 약한 개체쪽에 Bar () 표시 - 비식별관계: 표시 생략



<< 단원 정리 문제 >>

1. 다음 중 주식별자 특징에 포함되지 않은 것은?

- ① 유일성
- ② 최소성
- ③ 불변성
- ④ **중복성**

2. 다음 중 Key에 대한 설명으로 가장 적절하지 않은 것은?

- ① 주식별자(Primary Key)는 테이블 내에서 각 레코드를 고유하게 식별하는데 사용된다.
- ② 외래키(Foreign Key)는 다른 테이블의 주식별자 값을 참조하여 두 테이블 간의 관계를 정의한다.
- ③ 대체키(Alternate Key)는 주식별자 외에 유일성을 가진 다른 모든 속성을 의미한다.
- ④ 슈퍼키(Super Key)는 최소성은 만족하지만 유일성을 만족하지 않는 키를 의미한다.

3. 다음 중 식별자 특성이 다른 것은?

- ① 계좌 테이블에서 계좌번호
- ② **주문 테이블에서 시스템에서 자동으로 부여하는 주문번호**
- ③ 상품 테이블에서 제품의 바코드 값을 PK로 사용하는 경우
- ④ 사원 테이블에서 순차적으로 발급되는 사원번호

1. ④

주식별자는 중복값을 가질 수 없다.

2. ④

슈퍼키는 유일성은 만족하지만 최소성은 만족하지 않는 키

3. ②

① 계좌번호, ③ 바코드번호, ④ 직원번호는 시스템이나 업무에서 이미 존재하는 단일 속성을 그대로 식별자로 사용하는 본질식별자이다. 반면 ② 주문번호는 시스템이 인위적으로 생성하는 인조식별자로, 기존 속성을 조합하거나 새로운 번호 체계를 만들어 식별자로 사용한다.

1-6. 정규화

● 정규화(Normalization)

1. 개념

- 데이터베이스 설계 과정에서 중복을 제거하고, 데이터 이상(Anomaly)을 방지하기 위해 테이블을 구조적으로 분해해 데이터 무결성을 높이는 과정
- 하나의 엔터티에 많은 속성을 포함하면, 해당 엔터티를 조회할 때마다 불필요하게 많은 데이터가 함께 조회됨 → 필요한 최소한의 데이터만을 하나의 엔터티에 포함하도록 데이터를 분해 필요(정규화)
- 데이터의 **일관성 확보, 최소한의 데이터 중복, 최대한의 데이터 유연성을 확보하기 위한 과정**
- 이는 엔터티를 상세화하는 과정으로, **논리 데이터 모델링 단계에서 고려됨**
- 정규화는 제 1 정규화부터 제 5 정규화까지 존재하지만, 실무에서는 일반적으로 제 3 정규화까지만 수행함

2. 목적

- ① 데이터 중복(Multiplicity) 최소화
- ② 삽입/수정/삭제 이상(anomaly) 제거
- ③ 데이터 일관성 및 무결성 유지
- ④ 저장 공간 효율성 향상
- ⑤ 관계형 모델을 논리적으로 균형 있게 구성

● 이상현상(Abnormality)

- 정규화를 하지 않아 발생하는 현상(삽입이상, 갱신이상, 삭제이상)

1. 삽입 이상 (Insertion Anomaly)

- 필요한 데이터를 삽입하려는데 불필요한 데이터를 임시로 넣어야 하는 문제
- 예: 학생 엔터티에 수강과목 정보가 함께 저장되는 경우
 - 한 학생이 등록될 때 수강 정보가 아직 없다면, 존재하지 않는 데이터를 억지로 입력해야 하는 상황이 발생할 수 있음

2. 갱신 이상 (Update Anomaly)

- 중복된 데이터가 여러 곳에 있어 한 곳만 수정하면 불일치 발생
- 예: 학생에 대한 정보(전화번호)가 학생 엔터티, 수강 엔터티에 중복 존재하는 경우

3. 삭제 이상 (Deletion Anomaly)

- 데이터를 삭제했는데 의도치 않은 정보까지 함께 삭제
- 예: 학생 엔터티에 수강과목 정보가 함께 저장되는 경우
 - 특정 수강 과목 정보를 삭제하려고 하면, 그 과목을 수강한 학생 정보까지 함께 삭제되는 문제가 발생할 수 있음

● 정규화 단계

1. 제 1 정규화(1NF)

- 엔터티의 속성이 원자성(하나의 속성이 하나의 값만을 갖는 특성)을 만족하도록 데이터를 분해하는 단계
- 즉, 하나의 인스턴스와 속성에는 반드시 단일 값만 존재하도록 속성을 분리하거나 구조를 재조정하는 과정

예) 구매 테이블의 제 1 정규화(구매상품)

- 구매상품 속성에 여러 값이 존재 → 하나의 구매상품만 저장하도록 분해

구매		구매	
고객번호	구매상품	고객번호	구매상품
1000	샴푸, 린스	1000	샴푸
1001	우유, 치즈	1000	린스
1002	세제	1001	우유
1003	맥주	1001	치즈
		1002	세제
		1003	맥주

예) 사원 테이블의 제 1 정규화(전화번호)

- 하나의 전화번호만 갖도록 별도의 엔터티로 분해

사원			사원		전화번호	
사번	이름	전화번호	사번	이름	사번	전화번호
1000	홍길동	010-1111-1111, 010-2222-2222	1000	홍길동	1000	010-1111-1111
1001	박길동	010-3333-3333	1001	박길동	1000	010-2222-2222
1002	김길동	010-4444-4444	1002	김길동	1001	010-3333-3333
1003	최길동	010-5555-5555	1003	최길동	1002	010-4444-4444
					1003	010-5555-5555

예) 동일한 의미의 속성이 반복적으로 존재하는 경우 단일 속성으로 구조화

☐ ◦ 구매품목1 ☐ ◦ 구매품목2 ☐ ◦ 구매품목3 ☐ ◦ 구매품목가격1 ☐ ◦ 구매품목가격2	→	☐ ◦ 구매품목 ☐ ◦ 구매품목가격
--	---	--

2. 제 2 정규화(2NF)

- 이미 1NF 는 만족한 대상에 대해 부분 함수 종속을 제거하는 작업
- **부분 함수 종속은 기본키가 복합키일 때만 발생**

예) 수강이력 엔터티

- 강의실 속성은 PK(학생번호+강의명) 중 강의명에 의해 결정됨
→ 완전 함수 종속 위배(부분 함수 종속) → 강의실은 별도의 엔터티로 분리하여 관리해야 함

수강이력 PK				수강이력			강의실	
학번	강의명	강의실	성적	학번	강의명	성적	강의명	강의실
1000	컴퓨터공학	자연관	A	1000	컴퓨터공학	A	컴퓨터공학	자연관
1001	컴퓨터공학	자연관	B	1001	컴퓨터공학	B	기초통계	본관
1002	기초통계	본관	A+	1002	기초통계	A+	데이터베이스	자연관
1000	데이터베이스	자연관	B+	1000	데이터베이스	B+	경영학개론	본관
1001	경영학개론	본관	C	1001	경영학개론	C		
1002	경영학개론	본관	A	1002	경영학개론	A		

3. 제 3 정규화(3NF)

- 2NF 를 만족하면서, 이행 함수 종속을 제거하는 과정
- 즉, 기본키가 아닌 속성이 **기본키가 아닌 다른 속성에 종속되지** 않도록 하는 것
- **이행 함수 종속**: A 가 B 를 결정하고(B 는 기본키 아님) B 가 C 를 결정하는 경우, A 가 C 를 결정하는 관계
→ (A + B)와 (B + C)로 분리

예) 구매 테이블의 제 3 정규화

- 구매 엔터티에서 고객번호(기본키) → 상품명, 상품명 → 가격이 결정되는 경우
가격은 기본키(고객번호)가 아닌 상품명에 종속되므로 이는 이행 함수 종속에 해당함
→ 구매(고객번호+상품명), 상품(상품명+가격)으로 분리 필요

구매			구매		상품	
고객번호	상품명	가격	고객번호	상품명	상품명	가격
1001	우유	2500	1001	우유	우유	2500
1002	치즈	3500	1002	치즈	치즈	3500
1003	소시즈	4000	1003	소시즈	소시지	4000
1004	우유	2500	1004	우유		

- ☞ 테이블을 분리하지 않으면 상품명 변경 시 구매 엔터티에서도 가격을 반복 수정해야 하므로 비효율적임
제 3 정규화를 통해 상품 정보를 분리하면 가격은 한 곳에서만 수정하면 되어 갱신 효율이 높아짐

예) 학생 테이블의 제 3 정규화

- 학번에 의해 전공이 결정되고, 전공에 의해 담당교수가 결정되는 경우

담당교수는 학번이 아닌 전공(PK가 아닌 속성)에 종속, 학번 또한 교수의 간접적인 결정자가 됨

→ 학생(학번+이름+전공명), 과목(전공명+담당교수)로 분리 필요

학생

학번	이름	전공명	담당교수
1001	홍길동	통계학	홍교수
1002	김길동	수학	김교수
1003	최길동	경제학	박교수
1004	홍길동	경영학	최교수



학생

학번	이름	전공명
1001	홍길동	통계학
1002	김길동	수학
1003	최길동	경제학
1004	홍길동	경영학



과목

전공명	담당교수
통계학	홍교수
수학	김교수
경제학	박교수
경영학	최교수

예) 계좌번호 제 3 정규화

- 계좌번호에 의해 관리점코드가 결정되고, 관리점코드에 의해 관리점이 결정되는 경우

관리점은 계좌번호가 아닌 관리점코드(PK가 아닌 속성)에 종속, 계좌번호 또한 관리점의 간접적인 결정자임

→ 계좌(계좌번호+예수금+관리점코드), 관리점(관리점코드+관리점)으로 분리 필요

계좌

계좌번호	예수금	관리점코드	관리점
100111	100	1000	서울점
100222	200	1001	경기점
100333	300	1002	인천점
100444	400	1003	제주점



계좌

계좌번호	예수금	관리점코드
100111	100	1000
100222	200	1001
100333	300	1002
100444	400	1003



관리점

관리점코드	관리점
1000	서울점
1001	경기점
1002	인천점
1003	제주점

4. BCNF(Boyce-Codd Normal Form) 정규화

- 제 3 정규화를 강화한 개념으로, 모든 함수적 종속 $X \rightarrow Y$ 에서 결정자 X 는 반드시 후보키여야 한다.
- BCNF 위반이 발견되면, 해당 종속 $X \rightarrow Y$ 를 이용하여 릴레이션을 두 개 이상으로 분해하며, 원래 릴레이션과 새로운 릴레이션에 X 가 포함되도록 함

5. 제 4 정규화

- 하나의 엔터티에 두 개 이상의 독립적인 다중값 속성(예: 고객별 여러 전화번호, 여러 취미)이 동시에 존재할 때, 이들을 각각의 별도 엔터티로 분리하는 단계
- 다중값 종속성을 제거하여 하나의 엔터티가 여러 독립적 반복 정보를 동시에 포함하지 않도록 구조화함으로써 중복과 이상을 방지함

6. 제 5 정규화

- 하나의 엔터티가 여러 엔터티 간의 조인 관계에 의해 생성된 경우, 이 조인 관계가 불필요한 중복을 야기할 때, 엔터티를 더 작은 엔터티로 분해하여 조인 종속성을 제거하는 단계

<< 단원 정리 문제 >>

1. 다음 중 정규화에 대한 설명으로 가장 적절하지 않은 것은?

- ① 정규화는 데이터베이스의 중복을 줄이고 무결성을 유지하기 위해 수행된다.
- ② 정규화는 테이블을 여러 개의 하위 테이블로 분리하여 데이터 중복을 최소화한다.
- ③ 정규화는 관계형 데이터베이스의 쿼리 성능을 항상 향상시킨다.
- ④ 정규화는 이상 현상(삽입, 삭제, 수정)을 방지하는 데 중요한 역할을 한다.

2. 다음 중 제 2 정규화(2NF)에 대한 설명으로 가장 적절하지 않은 것은?

- ① 제 2 정규화는 제 1 정규화(1NF)를 만족한 후 진행한다.
- ② 제 2 정규화에서는 기본 키의 일부에 의존하는 속성을 제거하여 완전 함수 종속을 만족시킨다.
- ③ 제 2 정규화는 모든 속성이 기본 키에 완전히 종속되도록 하는 과정이다.
- ④ 제 2 정규화는 각 속성이 원자성을 갖도록 테이블을 분해하는 단계이다.

3. 다음은 어떤 정규화를 적용한 사례인가?

수강(PK: 학번+과목코드)

학번	과목코드	과목명	학점
1000	100	데이터베이스	A+
1001	200	통계학	B-
1002	300	경영학개론	B+
1000	400	심리학	C

수강

학번	과목코드	학점
1000	100	A+
1001	200	B-
1002	300	B+
1003	400	C

과목

과목코드	과목명
100	데이터베이스
200	통계학
300	경영학개론
400	심리학

- ① 제 1 정규화
- ② 제 2 정규화
- ③ 제 3 정규화
- ④ BCNF

4. 다음은 어떤 정규화를 적용한 사례인가?

주문

주문ID	고객ID	주소	주문금액
1000	100	서울시 강남구	1000
1001	200	경기도 수원시	2500
1002	300	부산시 중구	40000
1003	100	서울시 강남구	200

주문ID

주문ID	고객ID	주문금액
1000	100	1000
1001	200	2500
1002	300	40000
1003	100	200

고객

고객ID	주소
100	서울시 강남구
200	경기도 수원시
300	부산시 중구
400	서울시 동대문구

- ① 제 1 정규화
- ② 제 2 정규화
- ③ 제 3 정규화
- ④ BCNF

5. 다음은 구매 정보를 하나의 엔터티에 모두 담은 것이다. 이 구매엔터티에 대해 제 1 정규화부터 제 3 정규화까지 순차적으로 수행하면, 최종적으로 몇 개의 엔터티로 분해되는가?

(한 번의 주문에서 최대 2 개의 상품을 구매할 수 있으며, 상품코드가 같으면 상품명과 가격도 동일하다고 가정한다. 주문번호는 한 주문을 식별하는 값이다.)

구매

- ☐ # 주문번호
☐ ◦ 고객번호
☐ ◦ 고객명
☐ ◦ 고객주소
☐ ◦ 상품코드1
☐ ◦ 상품코드2
☐ ◦ 상품명1
☐ ◦ 상품명2
☐ ◦ 가격1
☐ ◦ 가격2

- ① 2 개
- ② 3 개
- ③ 4 개
- ④ 5 개

1. ③

정규화는 관계형 데이터베이스의 쿼리 성능을 항상 향상시키지 않을 수 있기 때문에, 모델링 시 반정규화도 함께 고려가 되어야 한다.

2. ④

제 1 정규화에 대한 설명이다.

3. ②

과목명과 학점은 과목코드에만 종속(부분 함수 종속) 되어 있어, 이를 제거하기 위해 별도 엔터티로 분리하였으므로 제 2 정규화이다.

4. ③

고객주소는 주문 ID 가 아니라 고객 ID 에 종속되므로, 이행 함수 종속을 제거하기 위해 고객 정보를 별도 엔터티로 분리한 것은 제 3 정규화에 해당한다.

5. ③

① 제 1 정규화: 반복 속성인 상품코드 1, 상품코드 2, 상품명 1, 상품명 2, 가격 1, 가격 2 를 상품코드, 상품명, 가격으로 통합하고, 주문번호별로 행을 분리한다

→ 주문엔터티(주문번호, 고객번호, 고객명, 고객주소), 주문상세엔터티(주문번호, 상품코드, 상품명, 가격)

② 제 2 정규화: 주문상세엔터티에서 기본키는 (주문번호+상품코드)이고, 상품명과 가격은 주문번호가 아니라 상품코드에만 종속된다. 이는 부분 함수 종속이므로, 상품 정보를 별도 엔터티로 분리한다.

→ 주문엔터티(주문번호, 고객번호, 고객명, 고객주소), 주문상세엔터티(주문번호, 상품코드), 상품엔터티(상품코드, 상품명, 가격)

③ 제 3 정규화: 주문엔터티에서 고객번호→고객명, 고객주소가 성립하므로 고객명과 고객주소는 주문번호가 아닌 고객번호에 종속되는 이행 함수 종속이다. 이를 제거하기 위해 고객 정보를 별도 엔터티로 분리한다.

→ 주문엔터티(주문번호, 고객번호), 주문상세엔터티(주문번호, 상품코드), 상품엔터티(상품코드, 상품명, 가격), 고객엔터티(고객번호, 고객명, 고객주소)

따라서 제 1 정규화부터 제 3 정규화까지 진행하면 총 4 개의 엔터티로 분해되므로 정답은 3 번이다.

1-7. 관계와 조인의 이해

● 관계(Relationship)의 개념

- 엔터티의 인스턴스들 사이에 존재하는 논리적 연관성을 의미함
- 엔터티의 정의, 속성 구성, 업무 규칙에 따라 관계의 형태와 의미는 다양하게 달라질 수 있다.
- 관계를 맺는다는 것은 **부모 엔터티의 식별자를 자식 엔터티가 상속받아**, 해당 속성을 매핑키(조인키)로 활용한다는 의미이며, 이를 통해 부모-자식 엔터티가 연결됨

사원 테이블			부서 테이블		
사번	이름	부서번호	부서번호	부서명	위치
1000	홍길동	100	100	영업부	서울
1001	최길동	101	101	관리부	서울
1002	김길동	201	201	생산부	부산

● 조인

1. 개념

- 두 개 이상의 엔터티(또는 테이블)가 맺고 있는 관계를 기반으로 서로의 데이터를 연결하여 하나의 결과로 결합하는 연산
- 관계에서 상속한 **매핑키(조인키)**를 이용해 부모-자식 엔터티 간 또는 서로 관련된 엔터티의 데이터를 연관 지어 조회하는 과정

2. 조인의 필요성

- 엔터티는 **정규화**로 인해 여러 개로 분리되어 저장됨
- 분리된 엔터티는 관계(Relationship)를 통해 서로 연결됨
- 실제 조회 시에는 이 분리된 엔터티의 정보를 하나처럼 결합해야 함
→ 이때 사용하는 연산이 JOIN

3. 조인 예시

- 계좌 테이블은 제 3 정규화 과정에서 계좌와 관리점 테이블로 분리됨
- 이때 두 테이블은 **관리점코드**를 공통으로 가짐

계좌				계좌			관리점	
계좌번호	예수금	관리점코드	관리점	계좌번호	예수금	관리점코드	관리점코드	관리점
100111	100	1000	서울점	100111	100	1000	1000	서울점
100222	200	1001	경기점	100222	200	1001	1001	경기점
100333	300	1002	인천점	100333	300	1002	1002	인천점
100444	400	1003	제주점	100444	400	1003	1003	제주점

예) 계좌번호 100111의 관리점이 어딘지를 찾으려면?

- ① 계좌 테이블에서 계좌번호가 100111 데이터 확인
- ② 계좌 테이블에서 계좌번호가 100111 데이터의 관리점코드(1000)를 확인
- ③ 해당 관리점코드(1000)를 관리점 테이블에 전달하여 그에 매칭되는 관리점 확인(서울점)

● 식별 관계와 비식별 관계

1. 식별 관계

- 부모 엔터티의 주식별자(Primary Key)가 자식 엔터티의 주식별자에도 포함되는 관계
- 부모 엔터티 없이는 자식 엔터티가 독립적으로 식별될 수 없음
- SQL 문의 조인 관계를 최소화하는 경우 식별 관계로 연결해야 함

2. 비식별 관계

- 부모 엔터티의 주식별자가 자식 엔터티의 주식별자에 포함되지 않는 관계
- 자식 엔터티는 독립적으로 식별자(PK)를 가짐

● 계층형 데이터 모델

- 하나의 엔터티 내부에서 인스턴스끼리 관계가 발생하여 계층 구조를 가지는 경우를 말함
- 같은 엔터티(테이블) 내에서 상위-하위 인스턴스를 연결해야 하므로, 이러한 계층 구조를 조회할 때는 동일한 테이블을 여러 번 조인하는 셀프 조인(Self Join)을 사용함

예) 아래 EMP 테이블을 사용하여 각 직원의 이름과 해당 직원의 상위 관리자의 이름을 함께 출력
(MGR 컬럼의 값은 각 직원의 상위 관리자의 사번을 의미함)

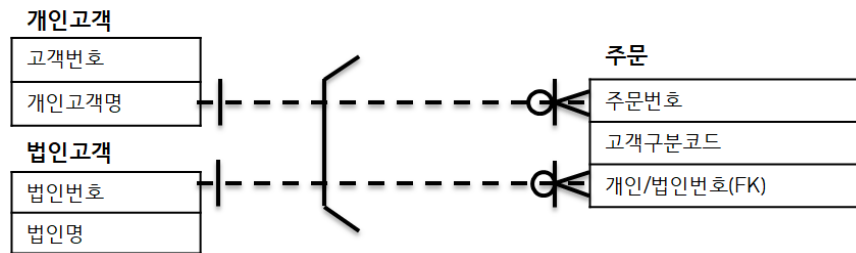
	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	1980/12/17 00:00:00	800		20
2	7499	ALLEN	SALESMAN	7698	1981/02/20 00:00:00	1600	300	30
3	7521	WARD	SALESMAN	7698	1981/02/22 00:00:00	1250	500	30
4	7566	JONES	MANAGER	7839	1981/04/02 00:00:00	2975		20
5	7654	MARTIN	SALESMAN	7698	1981/09/28 00:00:00	1250	1400	30
6	7698	BLAKE	MANAGER	7839	1981/05/01 00:00:00	2850		30
7	7782	CLARK	MANAGER	7839	1981/06/09 00:00:00	2450		10
8	7788	SCOTT	ANALYST	7566	1987/04/19 00:00:00	3000		20
9	7839	KING	PRESI...		1981/11/17 00:00:00	5000		10
10	7844	TURNER	SALESMAN	7698	1981/09/08 00:00:00	1500	0	30
11	7876	ADAMS	CLERK	7788	1987/05/23 00:00:00	1100		20
12	7900	JAMES	CLERK	7698	1981/12/03 00:00:00	950		30
13	7902	FORD	ANALYST	7566	1981/12/03 00:00:00	3000		20
14	7934	MILLER	CLERK	7782	1982/01/23 00:00:00	1300		10

● 상호배타적 관계(Mutually Exclusive Relationship)

- 두 엔터티(또는 속성) 중 하나만 선택적으로 관계를 맺을 수 있는 구조를 말함
- 특정 엔터티가 둘 중 하나의 엔터티와만 관계를 가질 수 있고, 동시에 두 곳과 연결될 수 없는 관계

예) 주문 엔터티에는 개인고객번호 또는 법인고객번호 중 단 하나만 상속될 수 있다.

즉, 주문은 개인 고객의 주문이거나 법인 고객의 주문 중 하나이며, 동시에 두 유형의 고객과 관계를 맺을 수 없음



< 상호 배타적 관계 ERD 표기법 >

<< 단원 정리 문제 >>

1. 다음 중 존재적 관계에 속하지 않는 것은?

- ① 학생과 교수 간의 관계
- ② 주문과 주문상세 간의 관계
- ③ 부서와 직원 간의 관계
- ④ 학생과 수강 간의 관계

2. 다음 중 조인과 관계에 대한 설명으로 가장 적절하지 않은 것은?

- ① 조인은 여러 테이블을 연결하여 하나의 결과 집합으로 만드는 연산이다.
- ② 조인은 기본 키와 외래 키를 이용하여 테이블 간의 관계를 설정한다.
- ③ 조인은 관계형 데이터베이스에서 데이터 무결성을 유지하는 데 중요한 역할을 한다.
- ④ 조인은 관계형 데이터베이스에서 성능을 향상시키는 주된 방법이다.

1. ④

학생과 수강 간의 관계는 행위적 관계에 속한다.

2. ④

조인은 여러 테이블을 결합하는 과정에서 성능에 부정적인 영향을 미칠 수 있기 때문에, 성능 향상을 위한 방법으로는 반정규화를 고려할 수 있다.

1-8. 모델이 표현하는 트랜잭션의 이해

● 트랜잭션이란

1. 개념

- 데이터베이스에서 하나의 논리적 기능을 수행하기 위한 업무 단위를 말함
- 여러 SQL 문이 하나의 작업 단위로 묶여 **전부 수행되거나, 전부 취소되어야 하는 처리 단위(All or Nothing)**
- 트랜잭션에 의한 관계는 **필수적인 관계 형태**를 가짐
- 하나의 트랜잭션에는 여러 SELECT, INSERT, DELETE, UPDATE 등이 포함될 수 있음

2. 필요성

- 데이터 수정 도중 오류가 발생할 경우, 수정되기 전 상태로 되돌릴 수 있도록 보장
- 여러 사용자가 동시에 데이터베이스를 사용할 때 일관성과 정확성을 유지
- 금융 시스템 등 중간 결과가 존재하면 안 되는 업무에서 필수적

3. 특징(ACID 속성)

트랜잭션이 충족해야 하는 네 가지 핵심 성질은 다음과 같다.

구분	설명
원자성 (Atomicity)	• 트랜잭션의 모든 연산은 전부 수행되거나 전부 취소(ROLLBACK) 되어야 함 • 부분 완료는 없음
일관성 (Consistency)	• 트랜잭션 수행 전 후 데이터는 일관된 상태를 유지해야 함 • 제약조건, 규칙 등이 항상 만족되는 상태를 유지해야 함
격리성 (Isolation)	• 동시에 수행되는 트랜잭션이 서로의 작업에 간섭하지 않도록 분리해야 함 • 서로 영향을 주지 않는 독립된 실행 환경을 보장
지속성 (Durability)	• 트랜잭션이 정상적으로 완료(COMMIT)되면 결과는 영구적으로 저장 • 시스템 장애가 발생해도 결과는 보존됨

4. 트랜잭션의 주요 명령어

- COMMIT: 모든 작업을 확정(영구 반영)
- ROLLBACK: 모든 작업을 취소하고 트랜잭션 시작 직전 상태로 복구

5. 예시

계좌이체를 예를 들어 A 고객이 B 고객에게 100 만원을 이체하려고 한다고 가정하자.

STEP1) A 고객 잔액을 -100 UPDATE

STEP2) B 고객 잔액에 +100 UPDATE

이 때, 1, 2 과정이 동시에 수행되어야 한다. 즉, **모두 성공하거나 모두 취소되어야 함**

☞ 이런 특성을 갖는 연속적인 업무 단위를 **트랜잭션**이라고 한다.

※ 주의

1. A 잔액 차감과 B 잔액 가산이 **서로 독립적으로 발생하면 안됨** → 각각의 INSERT 문으로 개발되면 안됨

2. 부분 COMMIT 불가

→ 동시 COMMIT 또는 ROLLBACK 처리

<< 단위 정리 문제 >>

1. 다음 중 트랜잭션에 대한 설명으로 가장 적절하지 않은 것은?

- ① 트랜잭션은 데이터베이스의 여러 작업을 하나의 논리적인 단위로 묶은 것이다.
- ② 트랜잭션은 ACID 속성, 즉 원자성, 일관성, 고립성, 지속성을 만족해야 한다.
- ③ 트랜잭션이 성공적으로 완료되면 그 작업은 영구적으로 반영된다.
- ④ 하나의 트랜잭션에 포함되는 작업들은 서로 독립적으로 발생할 수 있다.

2. 아래 트랜잭션에 대한 설명으로 가장 적절한 것은?

트랜잭션의 중요한 특성 중 하나로, 트랜잭션 내의 모든 작업이 하나의 단위로 처리되어야 한다는 의미를 갖는다. 즉, 트랜잭션에 포함된 작업이 모두 성공적으로 완료되거나, 하나라도 실패하면 전체 트랜잭션이 취소되어야 하는 특징을 말한다.

- ① 원자성
- ② 일관성
- ③ 고립성
- ④ 지속성

3. 다음 중 트랜잭션의 특징(ACID 성질)이 아닌 것은?

- ① 원자성(Atomicity)
- ② 일관성(Consistency)
- ③ 독립성(Independence)
- ④ 지속성(Durability)

1. ④

하나의 트랜잭션에 포함되는 작업들은 서로 독립적으로 발생해서는 안 된다.

2. ①

원자성에 대한 설명이다.

3. ③

트랜잭션의 ACID 특징에는 원자성(Atomicity), 일관성(Consistency), 격리성(Isolation), 지속성(Durability)이 있으며, '독립성(Independence)'은 트랜잭션의 공식적인 성질이 아니다.

1-9. NULL 속성의 이해

● NULL 이란

- DBMS 에서 아직 값이 정해지지 않았음을 의미하는 특수한 값을 나타내는 표현 방식
- 0 이나 한 칸의 공백(' ') 과는 전혀 다른 개념이며, “값의 미정(Unknown)”을 나타낸다.
- 모델 설계 시 각 컬럼에 대해 NULL 을 허용할지 여부(Nullable 여부)를 반드시 결정해야 한다.

● NULL 의 특성

1. NULL 을 포함한 산술 연산의 결과는 항상 NULL 이 리턴됨

예) $NULL + 300 = NULL$

예제) NULL 값을 갖는 컬럼과의 산술 연산 결과

SQL1 *

1 ▶

SELECT ENAME, SAL, COMM, SAL + COMM

2

FROM EMP;

Result

 Grid Result

 Server Output

 Text Output

 Explain Plan

 Statistics

	ENAME	SAL	COMM	SAL+COMM
1	SMITH	800		
2	ALLEN	1600	300	1900
3	WARD	1250	500	1750
4	JONES	2975		
5	MARTIN	1250	1400	2650
6	BLAKE	2850		
7	CLARK	2450		
8	SCOTT	3000		
9	KING	5000		
10	TURNER	1500	0	1500
11	ADAMS	1100		
12	JAMES	950		
13	FORD	3000		
14	MILLER	1300		

☞ COMM 컬럼에서 공백처럼 보이는 값들은 실제로는 NULL 값이다.

(물론 빈 문자열일 가능성도 있으나, 해당 데이터에서는 NULL 로 저장되어 있음)

이 상황에서 NULL 을 포함한 COMM 과 SAL 의 연산 결과는 모두 NULL 로 반환된다.

따라서 연산을 수행하기 위해서는 반드시 NULL 을 적절한 값으로 치환한 후 계산해야 한다.

※ NULL 치환 후 연산 결과

SQL1 *

1 ▶

SELECT ENAME, SAL, COMM, SAL + NVL(COMM, 0)

2

FROM EMP;

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	ENAME	SAL	COMM	SAL+NVL (COMM,0)
1	SMITH	800		800
2	ALLEN	1600	300	1900
3	WARD	1250	500	1750
4	JONES	2975		2975
5	MARTIN	1250	1400	2650
6	BLAKE	2850		2850

이하 생략

이하 생략

2. 집계 함수는 NULL 값을 제외하고 연산을 수행한다.

- SUM, AVG, MIN, MAX 등의 집계 함수는 **항상 NULL 을 무시하고 연산함**
- **COUNT(컬럼명) 은 NULL 이 아닌 값의 개수만 계산하지만, COUNT(*)는 NULL 여부와 상관없이 테이블의 전체 행 수를 반환함**

예제) NULL 을 포함한 컬럼의 집계함수 결과 1

SQL1 *

1

2

SELECT COUNT(*), COUNT(EMPNO), COUNT(SAL), COUNT(COMM)

FROM EMP;

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	COUNT(*)	COUNT(EMPNO)	COUNT(SAL)	COUNT(COMM)
1	14	14	14	4

☞ COUNT 는 행의 수를 세는 함수이지만, 일반적으로 NULL 은 세지 않으며 NULL 이 아닌 값이 있는 행만 계산한다. 반면 COUNT(*)는 컬럼의 값과 무관하게, NULL 여부와 상관없이 테이블의 모든 행 수를 반환한다.

예제) NULL 을 포함한 컬럼의 집계함수 결과 2

```
SQL1 *
1 SELECT SUM(COMM), MIN(COMM), MAX(COMM)
2 FROM EMP;
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	SUM(COMM)	MIN(COMM)	MAX(COMM)
1	2200	0	1400

☞ SUM, MIN, MAX 역시 NULL 값을 모두 무시하고 연산 결과를 계산한다.

예제) NULL 을 포함한 컬럼의 평균 연산

SQL1 *	
1	SELECT AVG(COMM), SUM(COMM)/COUNT(*)
2	FROM EMP;

Result					
 Grid Result  Server Output  Text Output  Explain Plan  Statistics					
	<table><tr><th>AVG(COMM)</th><th>SUM(COMM)/COUNT(*)</th></tr><tr><td>1</td><td>550 157.142857142857142857142857142857</td></tr></table>	AVG(COMM)	SUM(COMM)/COUNT(*)	1	550 157.142857142857142857142857142857
AVG(COMM)	SUM(COMM)/COUNT(*)				
1	550 157.142857142857142857142857142857				

☞ AVG 함수는 NULL 을 제외한 값들만을 대상으로 평균을 계산하므로, NULL 이 아닌 4 개의 데이터만을 사용하여 평균을 반환한다.

반면 두 번째 수식은 COMM 의 총합을 전체 행 수(14 명)로 직접 나눈 값이므로, 전체 인원을 기준으로 계산한 평균이 된다.

따라서 COMM 컬럼에 NULL 이 포함되어 있는 경우, AVG 함수의 결과와 직접 계산한 평균은 항상 다르게 반환된다.

3. 비교 연산이 불가능하다.

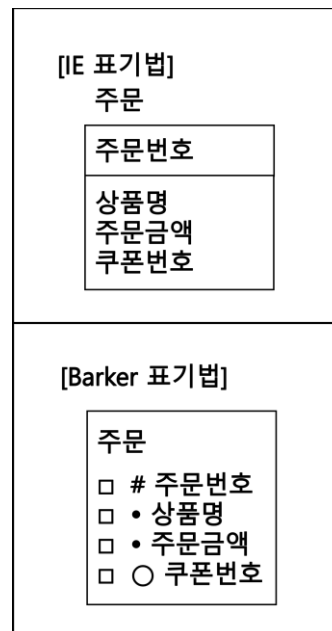
- NULL 은 =, <>, > 등의 비교 연산으로 비교할 수 없음
- 비교할 때는 반드시 **IS NULL / IS NOT NULL** 사용

4. 정렬 시 맨 앞 또는 맨 뒤로 배치

- DBMS 마다 다르지만 일반적으로 오름차순 정렬 시 NULL 이 상단에 배치됨
- **Oracle** 의 경우 오름차순 정렬 시 NULL 이 **가장 마지막**에 배치됨
- **SQL Server** 의 경우 오름차순 정렬 시 NULL 이 **가장 처음**에 배치됨

● NULL 의 ERD 표기법

- IE 표기법은 속성의 NULL 허용 여부를 표기하지 않음
- Barker 표기법에서는 속성명 앞의 동그라미(○)가 NULL 허용을 의미



<< 단원 정리 문제 >>

1. 다음 중 NULL에 대한 설명으로 가장 적절하지 않은 것은?

- ① NULL은 아직 정해지지 않은 값을 의미하며, 0과 빈 문자열과는 다른 개념이다.
- ② NULL을 포함한 산술 연산 결과는 항상 NULL이 리턴된다.
- ③ 집계 함수인 SUM, AVG는 NULL 값을 무시하고 연산을 수행한다.
- ④ COUNT(*)는 NULL을 제외한 행의 수를 리턴한다.

2. Key와 NULL에 대한 설명으로 가장 적절하지 않은 것은?

- ① 기본키는 NULL 값을 허용하지 않는다.
- ② 외래키는 NULL 값을 가질 수 있다.
- ③ 고유키는 NULL 값을 허용하지 않는다.
- ④ 각 속성에 대해 NULL 허용 여부를 개별적으로 설정할 수 있다.

1. ④

COUNT(*)는 항상 모든 행의 수를 리턴한다.

2. ③

고유키는 중복된 값을 허용하지 않지만 NULL 값은 삽입될 수 있다.

1-10. 본질식별자와 인조식별자

● 식별자 구분(대체 여부에 따른)

1. 본질식별자

- 업무에서 원래 필요하여 생성되는 식별자
- 실제 업무 처리 과정에서 자연스럽게 부여되는 값으로, 엔터티의 인스턴스를 식별하기 위해 반드시 필요한 식별자

2. 인조식별자

- 업무적으로 꼭 필요하지는 않지만, 관리의 편의성, 단순화 등의 이유로 인위적으로 생성된 식별자
- 본질식별자가 지나치게 복잡하거나 여러 속성의 조합으로 이루어져 있을 경우 이를 대체하기 위해 생성
- 보통 자동 증가 번호 등 단일 속성으로 구성된 기본키(PK) 형태로 많이 사용됨

● 인조식별자 생성 과정

예) 수강이력 엔터티에서 본질식별자로 사용할 수 있는 후보는 다음과 같다.

- 학생명(또는 학번)
- 강의명(또는 강의번호)
- 수강일자

☞ 그러나 이 값들만으로 식별하기에는 문제가 있음

- 같은 학생이 여러 강의를 들을 수 있음
- 강의명이 동일할 수 있음
- 강의일자는 중복 가능

STEP1) 본질식별자 조합(PK) 형태로 PK 생성: (학생명 + 강의명 + 수강일자)

☞ 단점

- 조인 시에도 불편
- 변경 가능성이 있는 속성이 포함됨
- 관리하기 어려움

STEP2) 단순한 신규 식별자 필요: (수강 ID or 등록번호)

☞ 복잡한 본질식별자 조합을 대체하기 위해 인위적으로 만들어짐

STEP3) 인조식별자를 PK 로 적용

수강이력

수강ID
학번 강의번호 수강일자

● 인조식별자의 단점

1. 업무 의미를 잃어버림

- 인조식별자는 숫자(1,2,3...)처럼 업무적 의미가 없는 값이라 데이터만 보고는 어떤 엔터티인지 파악하기 어려움
- 예: 주문번호만 봐서는 어떤 주문인지 명확히 파악하기 어려움

2. 본질식별자를 제대로 검증하지 않고 도입하는 경우가 많음

- 인조식별자를 너무 쉽게 도입하면 사실상 본질식별자(FK 후보)를 검증하지 않고 설계가 진행되는 문제가 생김
- 예: 본질적으로 수강이력(학번 + 강의번호) 만으로 유일한데 굳이 수강 ID 를 넣어버리면,
 - 실제 업무상 유일성을 보장하는 키가 무엇인지 파악이 어려워짐
 - 데이터 품질 관리가 느슨해질 수 있음

3. 중복 데이터 발생 가능성

- 인조식별자를 PK 로 쓰면, 본질값 조합이 중복되어도 데이터가 입력되는 것을 막을 수 없음
- 예: 수강 ID 가 PK 일 때,

수강이력

수강ID	학번	강의번호
1	A1000	C001
2	A1000	C001

☞ 두 개의 레코드가 중복 입력돼도 PK 제약상 문제없음

따라서 중복을 피하기 위해 본질식별자를 UNIQUE 제약조건으로 따로 관리해야 함

4. 불필요한 인덱스 생성

- 인조식별자를 기본키(PK)로 설정하면 DBMS 가 자동으로 인덱스를 생성함
- 하지만 인조식별자는 업무적으로 조회에 거의 사용되지 않는 경우가 많아, 실제로 쓰이지 않는 인덱스가 존재하게 됨
- 이로 인해 불필요한 인덱스가 저장공간을 차지하고, 데이터 변경 시 인덱스까지 관리해야 하므로 성능 부담이 발생할 수 있음

<< 단원 정리 문제 >>

1. 다음 중 본질식별자와 인조식별자에 대한 설명으로 가장 적절하지 않은 것은?

- ① 본질식별자는 엔터티의 자연적인 속성으로, 해당 엔터티의 고유한 특성을 기준으로 정의된다.
- ② 인조식별자는 본질식별자가 아닌, 시스템에서 생성한 고유한 값으로 엔터티를 구별하는 데 사용된다.
- ③ **본질식별자는 항상 유일성을 보장하지만, 인조식별자는 유일성을 보장하지 않는다.**
- ④ 본질식별자는 엔터티의 속성 중 하나일 수 있으며, 인조식별자는 외부에서 생성된 값으로 엔터티를 식별한다.

2. 다음 중 인조식별자에 속하지 않는 것은?

- ① 자동 생성된 게시글 번호
- ② **주민등록번호**
- ③ 시스템에서 생성된 제품 번호
- ④ 주문 번호

1. ③

본질 식별자는 자연적인 속성으로, 반드시 유일성을 보장하지 않을 수 있지만, 인조식별자는 시스템에서 생성한 값이기 때문에 유일성을 보장한다.

2. ②

주민등록번호는 의미를 갖고 만들어진 값이므로 인조식별자라고 볼 수 없다.

단원 종합 문제

- 다음 중 데이터 모델링의 3 요소에 속하지 않는 것은?
 - Entity
 - Key**
 - Attribute
 - Relationship
- 개념적 모델링에 대한 설명 중 가장 적절하지 않은 것은?
 - 업무 중심적인 모델링이다.
 - 전사적인 수준의 모델링을 의미한다.
 - 추상화 수준이 가장 낮다.**
 - 도출된 핵심 엔터티(Entity)들과의 관계를 표현하기 위해 ERD 작성하는 단계이다.
- 유형과 무형에 따른 엔터티의 분류에 속하지 않는 것은?
 - 유형엔터티
 - 개념엔터티
 - 중심엔터티**
 - 사건엔터티
- 엔터티의 명명규칙에 대한 설명으로 가장 적절하지 않은 것은?
 - 현업에서 사용하는 용어를 사용해야 한다.
 - 엔터티 생성 목적에 맞는 이름을 부여해야 한다.
 - 가능하면 약자는 사용하지 않도록 해야 한다.
 - 엔터티의 이름은 중복될 수 있다.**
- 다음 중 속성의 특성에 따른 분류에 속하지 않는 것은?
 - 단일 속성**
 - 기본 속성
 - 설계 속성
 - 파생 속성
- 속성에 대한 설명 중 가장 적절하지 않은 것은?
 - 단일 속성은 하나의 의미로 구성된 속성을 의미한다.
 - 복합 속성은 속성에 여러 개의 값을 가질 수 있는 경우를 말한다.**
 - 다중값 속성은 정규화 과정을 거쳐 별도의 엔터티로 분해될 수 있다.
 - 이자나 평균 같은 속성은 파생 속성에 포함된다.

7. 다음 중 관계에 대한 설명으로 가장 적절하지 않은 것은?

- ① 1:1 관계에서는 두 엔터티 간에 하나의 레코드가 각각 다른 엔터티의 하나의 레코드와만 관계를 맺는다.
- ② 1:N 관계에서는 하나의 엔터티가 다른 엔터티의 여러 레코드와 관계를 맺을 수 있으며, 반대는 성립하지 않는다.
- ③ N:M 관계는 중간 테이블을 사용하여 구현할 수 있으며, 두 엔터티가 서로 여러 레코드 간의 관계를 맺는다.
- ④ 1:N 관계에서는 양쪽 모두에서 다수의 레코드가 서로 관계를 맺을 수 있다.

8. 다음 중 행위적 관계에 속하지 않는 것은?

- ① 고객과 구매 간의 관계
- ② 학생과 수강 간의 관계
- ③ 부서와 직원 간의 관계
- ④ 제품과 주문 간의 관계

9. 다음 중 주식별자에 대한 설명으로 가장 적절하지 않은 것은?

- ① 주식별자는 각 레코드를 고유하게 식별할 수 있어야 한다.
- ② 주식별자는 속성의 수를 가능한 적게 구성하는 것이 좋다.
- ③ 주식별자의 값은 자주 변경되는 값이 적합하다.
- ④ 주식별자는 NULL 값을 가질 수 없다.

10. 다음이 설명하는 Key 로 가장 적절한 것은?

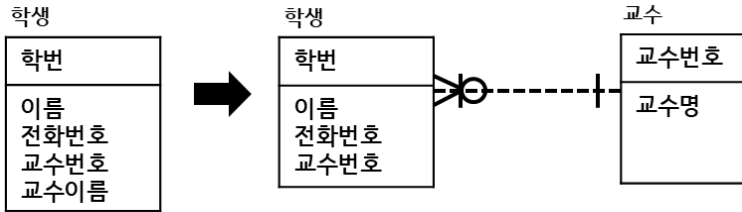
여러 후보키 중에서 기본키가 아닌 키를 의미한다.

- ① 대체키
- ② 후보키
- ③ 슈퍼키
- ④ 외래키

11. 다음 중 제 3 정규화에 대한 설명으로 가장 적절한 것은?

- ① 제 3 정규화는 제 2 정규화를 만족한 후, 이행적 종속(transitive dependency)을 제거하는 과정이다.
- ② 제 3 정규화에서는 모든 속성이 기본 키에 부분적으로 의존하도록 구성한다.
- ③ 제 3 정규화는 다중 값 종속(Multi-valued dependency)을 제거하는 과정이다.
- ④ 제 3 정규화는 기본 키가 아닌 속성들이 서로 간에 의존하도록 만든다.

12. 아래 정규화 과정을 가장 잘 설명한 단계는?



- ① 제 1 정규화 (1NF)
- ② 제 2 정규화 (2NF)
- ③ 제 3 정규화 (3NF)
- ④ 제 4 정규화 (4NF)

13. 다음 중 조인과 관계에 대한 설명으로 가장 적절하지 않은 것은?

- ① 조인은 두 개 이상의 테이블을 결합하여 하나의 결과 집합을 반환하는 연산이다.
- ② 계층 구조를 갖는 인스턴스끼리 연결하는 조인을 셀프 조인이라고 한다.
- ③ 조인은 기본 키와 외래 키를 사용하여 테이블 간의 관계를 설정하고 데이터를 연결한다.
- ④ 조인은 테이블 간의 관계를 정의하고 데이터를 결합하는 방식으로만 사용된다.

14. 다음 중 조인과 관계에 대한 설명으로 가장 적절하지 않은 것은?

- ① 계층형 데이터 모델은 한 엔터티 내의 인스턴스끼리 부모-자식 관계를 갖는다.
- ② 계층형 데이터 모델에서는 하나의 부모가 여러 자식을 가질 수 있지만, 하나의 자식은 여러 부모를 가질 수 없다.
- ③ 계층 구조를 갖는 인스턴스끼리 연결하는 조인을 내부조인이라고 한다.
- ④ 계층형 데이터 모델은 데이터의 중복을 최소화하고, 트리 구조를 통해 빠르게 검색할 수 있다.

15. 다음 중 트랜잭션의 특징으로 적절하지 않은 것은?

- ① 원자성
- ② 일관성
- ③ 고립성
- ④ 휘발성

16. 아래 트랜잭션에 대한 설명으로 가장 적절한 것은?

트랜잭션이 성공적으로 완료되었을 때, 그 결과가 영구적으로 반영되며, 시스템 장애나 오류가 발생하더라도 데이터가 손실되지 않도록 보장하는 특성이다.

- ① 원자성
- ② 일관성
- ③ 고립성
- ④ 지속성

17. 다음 중 NULL 에 대한 설명으로 가장 적절하지 않은 것은?

- ① NULL 은 아직 정의되지 않은 값을 의미하며, 0 이나 빈 문자열과는 다른 개념이다.
- ② NULL 값을 포함한 연산 결과는 항상 NULL 로 리턴된다.
- ③ 집계 함수인 COUNT 는 항상 NULL 값을 포함하여 행의 수를 센다.
- ④ NULL 값을 포함한 컬럼에서 평균을 계산할 때, NULL 은 무시하고 계산된다.

18. 다음 중 속성의 분류가 다른 하나는?

- ① 계좌번호
- ② **이자**
- ③ 이자율
- ④ 계좌개설일자

19. 다음 중 본질식별자에 포함되지 않는 것은?

- ① 주민등록번호
- ② 이메일 주소
- ③ **제품의 일련번호**
- ④ ISBN 번호

20. 다음 중 식별자에 대한 설명으로 가장 적절하지 않은 것은?

- ① 기본키는 각 레코드를 고유하게 식별하는데 사용된다.
- ② 고유키는 기본키와 달리 NULL 값을 허용할 수 있다.
- ③ 인조식별자는 시스템에서 생성된 값을 사용하여 엔터티를 고유하게 식별한다.
- ④ **본질식별자는 엔터티의 자연적인 속성에 의해 정의되는 식별자이지만, 항상 필수적인 식별자는 아니다.**

2과목.SQL 기본 및 활용

2-1. 관계형 데이터베이스 개요

● DATABASE(데이터베이스)

- 데이터베이스는 여러 데이터의 집합을 의미
- 데이터베이스를 보다 쉽게 관리할 수 있도록 도와주는 소프트웨어를 DBMS 라고 함

● DBMS(Database Management System)

- 데이터베이스의 생성, 관리, 유지 보수 및 접근을 지원하는 소프트웨어 시스템
- DBMS 를 통해 데이터베이스를 보다 효율적으로 관리하고 조작할 수 있음
- **SQL 언어를 사용**하여 데이터베이스를 생성하고 관리함

● SQL(Structured Query Language)

- RDBMS 에서 데이터를 정의, 조작, 조회, 수정, 삭제하는 데 사용되는 표준 언어
- 기능에 따라 DDL, DML, DCL, DQL, TCL 등으로 구분
- 문법은 **대소문자를 구분하지 않음**
- **SQL 문장은 세미콜론(;)으로 마침**

● ERD(Entity Relationship Diagram)

- 엔터티 간의 상관관계를 도식화하여 표현한 다이어그램
- ERD 의 구성 요소는 **엔터티(Entity), 관계(Relationship), 속성(Attribute)**으로 이루어짐

● DBMS 종류

1. 관계형 DBMS(RDBMS)

- 데이터를 2 차원의 행과 열로 구성된 구조화된 테이블 형태로 표현
- 테이블 간의 **관계 정의가 중요** → **조인 연산 필요**
- **정형 데이터를 저장**
- 예) Oracle, MySQL, PostgreSQL, SQL Server

2. 비관계형 DBMS(NoSQL)

- 관계형 DB 의 엄격한 테이블 구조와 달리 데이터 구조 및 저장 방식에 유연성을 부여
- **비정형, 반정형 데이터를 주로 저장**
- 예) MongoDB, Cassandra, Redis

● 관계형 데이터베이스 구성 요소

1. 계정

- 데이터 접근을 제한하고 관리하기 위해 업무별 또는 시스템별로 생성된 사용자 계정을 의미
- 계정별로 권한을 부여하여 데이터베이스 보안과 접근 제어를 수행

2. 테이블

- **데이터가 실제로 저장되는 기본 단위**
- 행(Row)과 열(Column)로 구성된 구조화된 형태로 데이터를 관리

3. 스키마

- 데이터베이스 내에서 데이터와 관련된 구조적 정의를 의미
- **테이블, 뷰, 인덱스, 관계 등 데이터베이스 객체들의 논리적인 집합**
- 테이블이 어떤 구조로 구성되어 있는지, 어떤 정보를 저장하는지에 대한 기본적인 틀을 제공

● 관계형 데이터베이스 장단점

구분	설명
장점	• 데이터의 분류, 정렬, 탐색 속도가 빠름 • 정형화된 구조로 인해 데이터 관리가 용이함 • 데이터 무결성과 일관성이 높아 신뢰성이 높음 • 기본키, 외래키, 제약조건 등을 통해 데이터 무결성을 보장함 • 표준화된 SQL 을 사용하여 데이터 조회 및 관리가 용이
단점	• 스키마가 사전에 정의되어 있어 구조 변경(컬럼 추가, 수정 등)이 어려움 • 시스템 부하에 따른 성능 병목을 사전에 분석하고 대응하기 어려움 • 시스템 규모가 커질수록 확장성이 제한적일 수 있음 • 안정적인 운영을 위해 전문적인 데이터베이스 관리자(DBA)가 필요 • 장애 발생 시 트랜잭션 처리 및 복구 과정이 복잡하여 시스템 전체에 영향이 커질 수 있음

● 데이터 무결성(integrity)

1. 개념

- 데이터의 정확성과 일관성을 유지하고, 데이터에 결손이나 부정합이 없음을 보장하는 개념
- 데이터베이스에 저장된 값이 현실 세계의 비즈니스 모델에서 의미하는 값과 일치하는 정확성을 의미함
- 데이터 무결성을 유지하는 것은 데이터베이스 관리 시스템(DBMS)의 핵심적인 기능 중 하나임

2. 종류

구분	설명
개체 무결성	테이블의 기본키를 구성하는 컬럼(속성)은 NULL 값을 가질 수 없으며, 중복될 수 없음
참조 무결성	외래키 값은 NULL 이거나, 참조하고 있는 테이블의 기본키 값과 동일해야 함
도메인 무결성	속성에 저장되는 값은 사전에 정의된 도메인에 속한 값이어야 함
NULL 무결성	특정 속성에 대해 NULL 값을 허용하지 않는 제약 조건의 특징
고유 무결성	특정 속성의 값이 테이블 내에서 중복되지 않도록 보장하는 특징
키 무결성	하나의 릴레이션(관계)에는 적어도 하나 이상의 키가 존재해야 함

● 테이블

1. 개념

- DBMS 에서 데이터를 입력하여 저장하는 최소 단위
- 행(Row)과 열(Column)로 구성된 정형화된 2 차원 구조
- 논리 모델링 단계에서 정의된 엔터티(Entity)는 물리 모델링 단계에서 테이블(Table)로 변환됨
- 하나의 테이블은 동일한 성격의 데이터 집합을 저장함

2. 특징

- 하나의 테이블은 하나의 스키마에 속속됨
(Oracle 에서는 스키마 = 소유자 개념, SQL Server 에서는 스키마 단위 관리)
- 테이블 간에는 일대일(1:1), 일대다(1:N), 다대다(N:N)의 관계를 가질 수 있음
- 테이블명은 동일한 스키마 내에서는 중복될 수 없으나, **스키마가 다를 경우 동일한 이름으로 생성 가능**
- 각 테이블은 기본키(PK)를 통해 각 행을 유일하게 식별함
- 외래키(FK)를 통해 다른 테이블과의 관계를 표현함
- 테이블을 구성하는 각 컬럼은 데이터 타입, 길이, 제약조건 등을 정의하여 데이터의 무결성을 관리함

예) 소유자(스키마)가 다른 경우, 동일한 이름의 테이블 생성

Oracle 에서 SCOTT 소유의 EMP 테이블이 이미 존재할 때, SYSTEM 소유의 EMP 테이블 생성 가능

SQL1 *

1

2

3

SELECT *

FROM DBA_OBJECTS

WHERE OBJECT_NAME = 'EMP';

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	OWNER	OBJECT_NAME	SUBOBJECT_NAME	OBJECT_ID	DATA_OBJECT_ID	OBJECT_TYPE
1	SYSTEM	EMP		79986	79986	TABLE
2	SCOTT	EMP		73196	73196	TABLE

예) EMP 테이블의 각 컬럼별 데이터 타입 확인

SQL1 *

1

2

3

DESC EMP;

Result

Grid Result

Server Output

Text Output

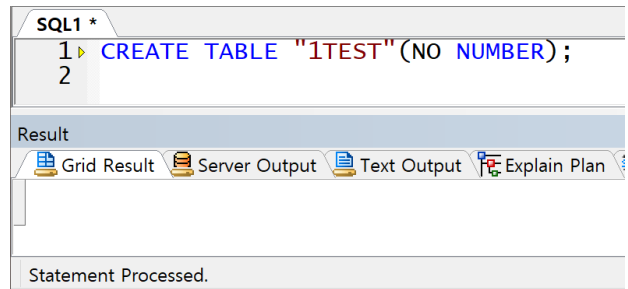
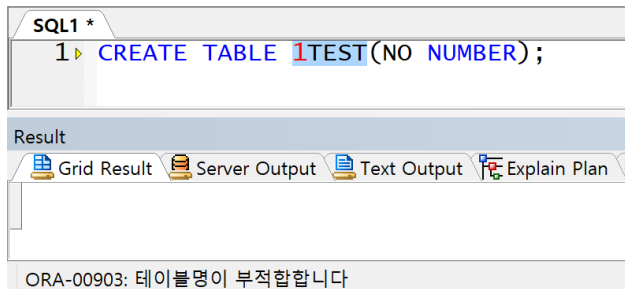
Explain Plan

Statistics

	Column	Nullable	Type	Comment
1	EMPNO		NUMBER(4)	
2	ENAME		VARCHAR2(10)	
3	JOB		VARCHAR2(9)	
4	MGR		NUMBER(4)	
5	HIREDATE		DATE	
6	SAL		NUMBER(7,2)	
7	COMM		NUMBER(7,2)	
8	DEPTNO		NUMBER(2)	

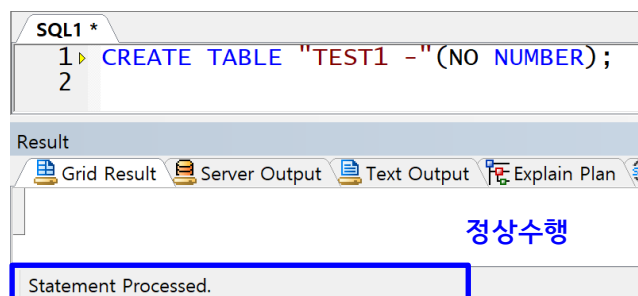
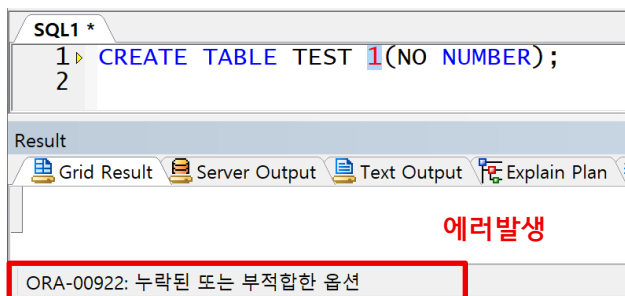
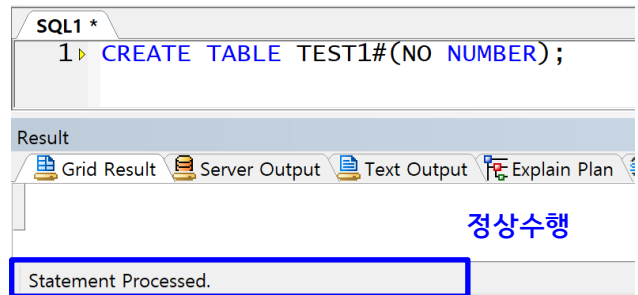
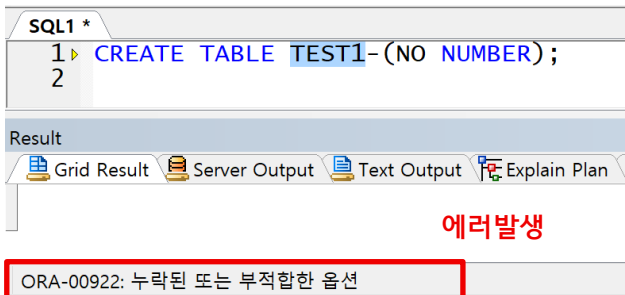
● 테이블 명명 규칙

1. 테이블 이름은 최대 30 자까지 지정할 수 있음
2. 대소문자를 구분하지 않지만, **쌍따옴표(" ")를 사용하면 대소문자를 구분함**
 - 예) TABLE1, Table1, table1 은 모두 동일한 이름의 테이블로 인식됨
3. 영문자와 숫자를 포함할 수 있으나, **숫자로 시작할 수는 없음**
 - 예) 1TABLE 이라는 이름의 테이블은 생성할 수 없음



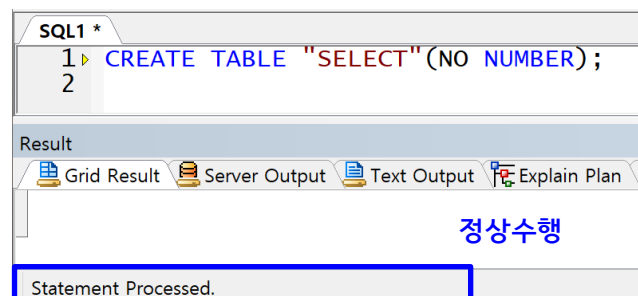
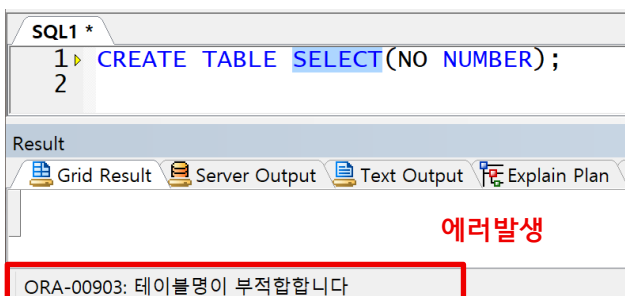
4. 특수기호 및 공백 사용 제한

- 공백 및 특수기호 사용 불가(사용할 수 있는 특수기호는 `_`, `#`, `$` 만 가능)



5. 예약어 사용 금지

- 이미 존재하는 SQL 문법이나 키워드는 테이블 이름으로 사용할 수 없음
- 예) SELECT, DROP, USER, TABLE 등의 이름을 갖는 테이블은 생성 불가



<< 단원 정리 문제 >>

1. 관계형 데이터베이스에 대한 설명 중 가장 적절하지 않은 것은?

- ① 데이터에 대한 신뢰성이 높다.
- ② 데이터에 대한 검색, 정렬, 탐색 속도가 빠르다.
- ③ 기존에 작성된 스키마에 대한 정의를 변경하기 어렵다.
- ④ 구조화된 데이터베이스는 부하를 분석하기 편리하다.

2. 다음이 설명하는 무결성 종류로 가장 적절한 것은?

관계형 데이터베이스에서 각 테이블의 기본키가 고유하고, NULL 값을 가질 수 없다는 규칙을 의미한다.

- ① 개체 무결성
- ② 참조 무결성
- ③ 도메인 무결성
- ④ 고유 무결성

3. 다음 중 테이블 이름으로 사용할 수 없는 것은?

- ① CUSTOMER
- ② ORDER
- ③ EMP_2025
- ④ SALES_DETAIL

1. ④

구조화된 데이터베이스는 부하를 분석하기 어렵다.

2. ①

개체 무결성에 대한 설명이다.

3. ②

ORDER 는 SQL 에서 정렬을 위해 사용되는 예약어이므로 테이블 이름으로 사용할 수 없다.

2-2. SELECT 문

● SQL 종류

- SQL 은 그 기능에 따라 다음과 같이 구분함
- SELECT 문은 데이터 조작이나 정의가 아닌 **조회 전용**이므로, 이를 구분하기 위해 DQL 개념이 등장함

구분	종류
DDL (Data Definition Language)	CREATE, ALTER, DROP, TRUNCATE
DML (Data Manipulation Language)	INSERT, DELETE, UPDATE, MERGE
DCL (Data Control Language)	GRANT, REVOKE
TCL (Transaction Control Language)	COMMIT, ROLLBACK
DQL (Data Query Language)	SELECT

● SELECT 문 구조

- SELECT 문은 총 6 개의 절로 구성됨
- **각 절은 정해진 순서대로 작성해야 함**
- GROUP BY 절과 HAVING 절은 논리적으로 함께 사용되므로 순서를 바꿀 수는 있으나, 권장하지 않음
- 작성 순서와 실제 실행(해석) 순서는 서로 다름
- **SELECT 문의 실행 순서(해석)**는 다음과 같음

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

** 문법

SQL1 *	
1	SELECT * 컬럼명 표현식
2	FROM 테이블명 또는 뷰명
3	WHERE 조회할 데이터 조건
4	GROUP BY 그룹핑컬럼명
5	HAVING 그룹핑 필터링 조건
6	ORDER BY 정렬컬럼명;



표현식(Expression)



테이블에 실제로 존재하는 컬럼명이 아닌, 연산·함수·상수 등을 사용하여 계산되거나 가공된 결과를 반환하는 모든 대상을 의미한다.
(산술 연산식, 함수 적용 결과, 문자열 결합, 상수 값 등 포함)

● SELECT 절

- SELECT 절은 테이블에서 출력하고자 하는 대상을 선택하는 절
- * 또는 컬럼명, 표현식 등을 지정할 수 있음
- *와 컬럼명 또는 표현식은 함께 사용할 수 없음
- 출력 대상이 여러 개인 경우, 원하는 컬럼 및 표현식을 콤마(,)로 나열하여 작성하며 작성된 순서대로 출력됨

** 문법

SQL1 *			
1	SELECT	*	컬럼명 표현식
2	FROM	테이블명 또는 뷰명;	

** 특징

- SELECT 절에서 표시할 대상 컬럼에 Alias(별칭)을 지정할 수 있음

예제) EMP 테이블의 전체 컬럼 조회(14 개 행, 8 개 컬럼으로 구성)

SQL1 *			
1	SELECT	*	
2	FROM	EMP;	

Result									
	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	
1	7369	SMITH	CLERK	7902	1980/12/17 00:00:00	800		20	
2	7499	ALLEN	SALESMAN	7698	1981/02/20 00:00:00	1600	300	30	
3	7521	WARD	SALESMAN	7698	1981/02/22 00:00:00	1250	500	30	
4	7566	JONES	MANAGER	7839	1981/04/02 00:00:00	2975		20	
5	7654	MARTIN	SALESMAN	7698	1981/09/28 00:00:00	1250	1400	30	
6	7698	BLAKE	MANAGER	7839	1981/05/01 00:00:00	2850		30	
7	7782	CLARK	MANAGER	7839	1981/06/09 00:00:00	2450		10	
8	7788	SCOTT	ANALYST	7566	1987/04/19 00:00:00	3000		20	
9	7839	KING	PRESIDENT		1981/11/17 00:00:00	5000		10	
10	7844	TURNER	SALESMAN	7698	1981/09/08 00:00:00	1500	0	30	
11	7876	ADAMS	CLERK	7788	1987/05/23 00:00:00	1100		20	
12	7900	JAMES	CLERK	7698	1981/12/03 00:00:00	950		30	
13	7902	FORD	ANALYST	7566	1981/12/03 00:00:00	3000		20	
14	7934	MILLER	CLERK	7782	1982/01/23 00:00:00	1300		10	

예제) EMP 테이블에서 특정 컬럼을 조회

SQL1 *			
1	SELECT EMPNO, ENAME, SAL		
2	FROM EMP;		

Result			
	EMPNO	ENAME	SAL
1	7369	SMITH	800
2	7499	ALLEN	1600
3	7521	WARD	1250
4	7566	JONES	2975
5	7654	MARTIN	1250
6	7698	BLAKE	2850

이하 생략

예제) 표현식을 통해 원본 데이터 값을 가공하여 새로운 형태로 출력

SQL1 *			
1	SELECT EMPNO, ENAME, SAL * 1.1		
2	FROM EMP;		

Result			
	EMPNO	ENAME	SAL * 1.1
1	7369	SMITH	880
2	7499	ALLEN	1760
3	7521	WARD	1375
4	7566	JONES	3272.5
5	7654	MARTIN	1375
6	7698	BLAKE	3135

이하 생략

☞ SAL * 1.1 은 물리적으로 존재하는 컬럼이 아니라, 기존 컬럼을 활용한 연산식으로 SELECT 절에서 일시적으로 정의되어 출력되는 표현식임

예제) 연결 연산자(II)를 사용한 문자열 결합

SQL1 *			
1	SELECT EMPNO '-' ENAME		
2	FROM EMP;		
3			

Result			
	EMPNO '-' ENAME		
1	7369-SMITH		
2	7499-ALLEN		
3	7521-WARD		

이하 생략

☞ 연결 연산자를 사용하면 분리된 두 대상(문자열 또는 컬럼 등)을 하나의 값으로 결합할 수 있음

예제) 잘못 작성된 SELECT 절

SQL1 *			
1	SELECT *, ENAME		
2	FROM EMP;		

Result			
ORA-00923: FROM 키워드가 필요한 위치에 없습니다. Col : 1 Ln 2, Col 12			

☞ 일반적으로 *와 특정 컬럼명을 함께 지정하여 사용할 수 없음

● 컬럼 Alias(별칭)

- 컬럼명 대신 출력할 임시 이름을 의미함
- SELECT 절에서만 정의 가능하며, 테이블의 실제 컬럼명은 변경되지 않음
- 컬럼명 뒤에 **AS 와 함께 별칭을 지정하며, AS 는 생략 가능함**

** 특징 및 주의사항

- SELECT 절보다 늦게 수행되는 ORDER BY 절에서만 컬럼 별칭 사용 가능
(그 외 절에서 사용 시 에러 발생)
- 한글 사용 가능(한글을 지원하는 캐릭터셋 설정 시)
- 문자로 시작해야 하며, 예약어나 공백, 특수기호를 포함할 수 없음(., #, \$ 제외)
- 이미 존재하는 예약어는 별칭으로 사용 불가(SELECT, FROM 등)

** 별칭 사용 시 쌍따옴표가 필요한 경우

1. 별칭에 공백을 포함하는 경우
2. 별칭에 특수문자를 포함하는 경우(., #, \$ 제외)
3. 별칭을 입력한 형태 그대로 전달해야 하는 경우
 - ※ 대소문자를 그대로 유지하거나, 일반적으로 사용할 수 없는 예약어 등을 표현하는 경우

예제) 별칭 사용 예(AS 생략 가능)

SQL1 *			
1	▶	SELECT EMPNO AS 사번,	
2		ENAME 사원이름,	
3		SAL * 1.1 AS NEW_SAL	
4		FROM EMP;	

Result			
	사번	사원이름	NEW_SAL
1	7369	SMITH	880
2	7499	ALLEN	1760
3	7521	WARD	1375
4	7566	JONES	3272.5
5	7654	MARTIN	1375
6	7698	BLAKE	3135
7	7782	CLARK	2695
8	7788	SCOTT	3300

이하 생략

예제) 별칭 선언 시 쌍따옴표가 필요한 경우 1

SQL1 *			
1	▶	SELECT EMPNO, ENAME, SAL * 1.1 AS NEW SAL	
2		FROM EMP;	

Result			
ORA-00923: FROM 키워드가 필요한 위치에 없습니다. Ln 2, Col 12			

SQL1 *			
1	▶	SELECT EMPNO, ENAME, SAL * 1.1 AS "NEW SAL"	
2		FROM EMP;	

Result			
	EMPNO	ENAME	NEW SAL
1	7369	SMITH	880
2	7499	ALLEN	1760
3	7521	WARD	1375
4	7566	JONES	3272.5

이하 생략

☞ 별칭에 공백이 포함된 경우 반드시 쌍따옴표와 함께 전달해야 함

예제) 별칭 선언 시 쌍따옴표가 필요한 경우 2

SQL1 *	
1	SELECT ENAME AS SELECT
2	FROM EMP;

Result	
ORA-00923: FROM 키워드가 필요한 위치에 없습니다.	

SQL1 *	
1	SELECT ENAME AS "SELECT"
2	FROM EMP;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

SELECT	
1	SMITH
2	ALLEN
3	WARD
4	JONES

이하 생략

☞ 예약어를 별칭으로 사용할 경우 쌍따옴표 필수

● FROM 절

- 출력할 데이터가 저장된 테이블명 또는 뷰(View)명을 전달하는 절
- 조인 시 여러 테이블을 전달할 수 있으며, 이 경우 콤마(,)로 구분함(Oracle 표준)
(여러 테이블을 FROM 절에 나열하여 조인 가능)
- **테이블 별칭 선언 가능**
(Oracle에서는 AS 사용 불가, SQL Server에서는 AS 사용 또는 생략 가능)
- **테이블 별칭을 선언한 경우 컬럼 구분자는 반드시 테이블 별칭으로만 전달**해야 하며, 테이블명을 사용할 경우 에러가 발생함
- Oracle에서는 FROM 절을 생략할 수 없음(의미상 참조할 테이블이 없는 경우 DUAL 테이블을 사용함)
- SQL Server에서는 FROM 절이 필요 없는 경우 생략 가능(예: 현재 날짜 조회)

※ DUAL TABLE

- Oracle에서 제공하는 특수한 테이블임
- 주로 단일 행을 반환하는 쿼리에서 사용됨
- 계산식이나 함수 호출 등에서 실제 테이블을 참조할 필요가 없을 때 사용함
- 기본적으로 한 행과 한 컬럼을 가지고 있음

SQL1 *	
1	SELECT *
2	FROM DUAL;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

DUMMY	
1	X

 뷰(View)

 실제 데이터를 저장하지 않고, 하나 이상의 테이블에 대한 SELECT 문의 결과를 이름으로 저장하여 가상 테이블처럼 조회할 수 있도록 해주는 객체이다.

예제) Oracle 에서 FROM 절을 생략할 경우 에러가 발생하는 경우

SQL1 *	
1 ▶	SELECT 24 * 123

Result	
ORA-00923: FROM 키워드가 필요한 위치에 없습니다. Ln 1, Col 16	

☞ Oracle 에서 FROM 절을 생략할 경우 에러가 발생함

※ DUAL 테이블 사용 필수

SQL1 *	
1 ▶	SELECT 24 * 123
2	FROM DUAL;

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
	24*123
1	2952

☞ 의미상 FROM 절이 필요 없는 경우 DUAL 테이블을 사용함

예제) 테이블 별칭 사용

※ 잘못된 문법

SQL1 *	
1 ▶	SELECT E.ENAME, EMP.SAL, E.DEPTNO
2	FROM EMP E
3	WHERE E.DEPTNO = 10;

Result	
ORA-00904: "EMP"."SAL": 부적합한 식별자 Col : 1 Ln 3, Col 22 0.00 sec.	

※ 올바른 문법

SQL1 *	
1 ▶	SELECT E.ENAME, E.SAL, E.DEPTNO
2	FROM EMP E
3	WHERE E.DEPTNO = 10;

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
	ENAME SAL DEPTNO
1	CLARK 2450 10
2	KING 5000 10
3	MILLER 1300 10

☞ 테이블 별칭을 선언한 경우, 컬럼 구분자로 테이블명을 사용할 수 없음



컬럼 구분자



테이블명 또는 테이블 별칭과 컬럼명을 점(.)으로 연결하여 특정 컬럼을 구분하기 위해 사용하는 표기 방식이다.

● DISTINCT 를 사용한 중복 제거

- 조회 결과에서 중복된 행을 제거하고 유일한 값만 출력하기 위해 사용하는 키워드
- SELECT 절 바로 뒤에 위치하여, 선택한 모든 컬럼 조합을 기준으로 중복 여부를 판단함
- DISTINCT 는 SELECT 절에서 단 한 번만 지정 가능하며, 중복 적용은 불가능함

예제) DISTINCT 를 사용하여 EMP 테이블의 부서 번호 중복 제거

SQL1 *	
1	SELECT DISTINCT DEPTNO
2	FROM EMP;
3	

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
DEPTNO	
1	20
2	30
3	10

EMP 테이블에 저장된 여러 직원의 부서 번호 중에서 중복을 제거하고 부서 번호의 종류만 조회하기 위해 DISTINCT 를 사용함

예제) DISTINCT 에 여러 컬럼을 지정한 경우의 결과

SQL1 *	
1	SELECT DISTINCT DEPTNO, JOB
2	FROM EMP
3	ORDER BY DEPTNO, JOB;
4	

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
DEPTNO	JOB
1	10 CLERK
2	10 MANAGER
3	10 PRESIDENT
4	20 ANALYST
5	20 CLERK
6	20 MANAGER
7	30 CLERK
8	30 MANAGER
9	30 SALESMAN

DISTINCT 에 여러 컬럼을 지정한 경우, 각 컬럼을 개별적으로 비교하는 것이 아니라 지정된 컬럼들의 조합이 모두 동일한 행만 중복으로 판단하여 제거한다. 따라서 동일한 부서번호라도 직무가 다르면 서로 다른 행으로 인식되어 결과에 함께 출력된다.

<< 단위 정리 문제 >>

1. 다음 중 실행이 불가능한 문장을 고르시오

①

```
SELECT *
  FROM TAB1 T;
```

②

```
SELECT TAB1.COL1, COL2
  FROM TAB1;
```

③

```
SELECT T.COL1, TAB1.COL2
  FROM TAB1 T;
```

④

```
SELECT 100 * 250
  FROM DUAL;
```

2. SQL 의 분류 중 DDL 에 속하지 않는 문장을 고르시오

① DROP

② ALTER

③ TRUNCATE

④ DELETE

3. 다음 중 SELECT 문에 대한 설명으로 가장 적절하지 않은 것은? (단, DBMS 는 Oracle)

① SELECT 문은 테이블에 저장된 데이터를 조회하기 위해 사용한다.

② SELECT 문에서 FROM 절은 생략할 수 있다.

③ SELECT 문에서 테이블 별칭을 선언한 경우, 컬럼 구분자로 테이블명을 사용할 수 없다.

④ SELECT 문에서 DUAL 테이블은 의미상 FROM 절이 필요 없는 경우 사용한다.

4. 다음 중 Oracle DBMS 의 DUAL 테이블에 대한 설명으로 가장 적절하지 않은 것은?

① DUAL 테이블은 실제 데이터를 저장하지 않으며, SELECT 문의 결과를 반환하기 위한 용도로 사용된다.

② DUAL 테이블은 의미상 테이블 참조가 필요 없는 경우 FROM 절에서 사용된다.

③ DUAL 테이블은 기본적으로 하나의 행을 가지며, 컬럼의 개수는 고정되어 있지 않다.

④ DUAL 테이블은 계산식이나 단일 행을 반환하는 함수 호출 시 사용된다.

5. 다음 중 실행 시 에러가 발생하는 SELECT 문장은?

① SELECT COLUMN1 COLUMN FROM TABLE1;

② SELECT COLUMN1 AS "SELECT" FROM TABLE1;

③ SELECT COLUMN1 AS "COL*12" FROM TABLE1;

④ SELECT COLUMN1 COLUMN# FROM TABLE1;

1. ③

FROM 절에 테이블 별칭을 정의한 경우, 컬럼 구분자로 테이블명을 사용할 수 없다.

2. ④

DELETE 는 수정 언어로, DML 에 속한다.

3. ②

Oracle DBMS 에서는 계산식이나 함수 호출 등 의미상 테이블 참조가 필요 없는 경우라도 DUAL 테이블을 사용하여 FROM 절을 반드시 명시해야 한다.

4. ③

DUAL 테이블은 Oracle DBMS 에서 제공하는 특수한 테이블로, 실제 데이터를 저장하지 않으며 의미상 테이블 참조가 필요 없는 경우 SELECT 문에서 사용된다. DUAL 테이블은 기본적으로 한 행과 한 컬럼을 가지는 구조로 고정되어 있으므로, 컬럼의 개수가 고정되어 있지 않다는 설명은 적절하지 않다.

5. ①

컬럼 별칭은 문자로 시작해야 하며, 예약어나 특수문자를 포함하는 경우 쌍따옴표로 감싸서 지정할 수 있다.

①번은 예약어인 COLUMN 을 쌍따옴표 없이 컬럼 별칭으로 사용하였으므로 실행 시 에러가 발생한다.

②번은 예약어를 쌍따옴표로 감싸 별칭으로 지정하였으므로 정상 실행된다.

③번은 특수문자를 포함하고 있으나 쌍따옴표로 감싸 별칭으로 지정하였으므로 정상 실행된다.

④번은 특수문자 #를 포함하였지만 쌍따옴표 없이도 허용되는 식별자 규칙에 해당하므로 정상 실행된다.

2-3. 함수

● 함수

1. 정의

- 입력값(Input Value)이 있을 경우, 그에 대응하는 출력값(Output Value)을 반환해주는 객체
- 입력값과 출력값 간의 관계를 정의한 객체임
- FROM 절을 제외한 SELECT, WHERE, GROUP BY, HAVING, ORDER BY 절 등에서 사용 가능함

2. 기능

- 기본적인 쿼리문을 더욱 강력하게 만들어 줌
- 데이터에 대한 계산을 수행함
- 개별 데이터 항목의 값을 가공하거나 변환함
- 출력 시 날짜 및 숫자 형식을 지정함
- 열 데이터의 유형(Data Type)을 변환함

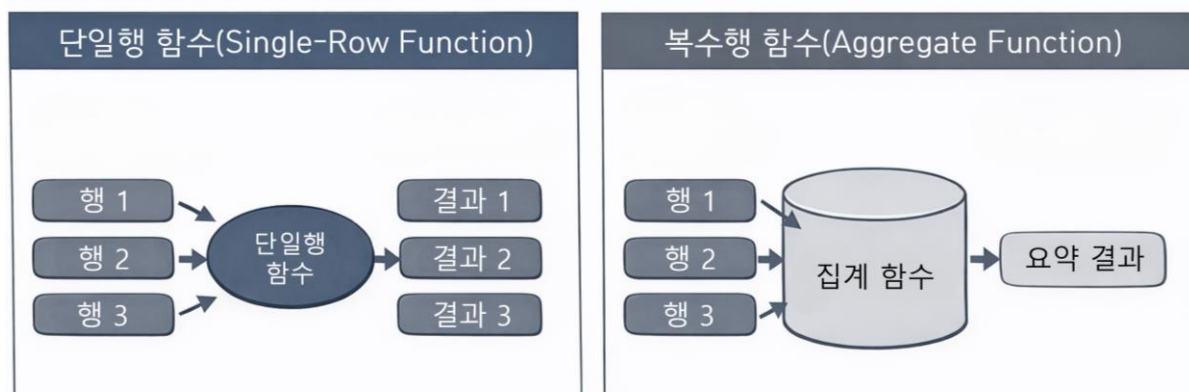
● 함수의 종류

1. 입력값의 수에 따른 분류

구분	설명
단일행 함수	• 입력값과 출력값의 관계가 1:1 인 함수
복수행 함수(집계 함수)	• 여러 건의 데이터를 동시에 입력 받아 하나의 요약값을 반환하는 함수

2. 입·출력값의 타입에 따른 함수 분류

구분	설명
문자 함수	• 문자열의 결합, 추출, 삭제 등의 처리를 수행하는 함수 • 함수의 처리 결과는 반드시 문자 타입으로 반환됨(INSTR 제외)
숫자 함수	• 숫자 데이터에 대한 연산을 수행하기 위한 함수임
날짜 함수	• 날짜 데이터의 출력 및 연산을 수행하기 위한 함수
변환 함수	• 데이터 타입(형)을 변환하기 위한 함수
일반 함수	• 위 분류에 포함되지 않는 기타 기능을 수행하는 함수



● 입·출력값의 타입에 따른 함수 분류

1. 문자함수

1) UPPER / LOWER / INITCAP

- 각각 문자열을 대문자, 소문자, 첫 글자만 대문자(카멜 표기법)로 변환하는 함수

** 문법

SQL1 *
1 UPPER(대상) LOWER(대상) INITCAP(대상)

함수 결과)

SQL1 *
1 SELECT UPPER('hdata1ab') AS 대문자치환,
2 LOWER('HDATATAB') AS 소문자치환,
3 INITCAP('HDATATAB') AS 앞글자만대문자
4 FROM DUAL;

Result
Grid Result Server Output Text Output Explain Plan Statistics
대문자치환 소문자치환 앞글자만대문자
1 HDATALAB hdatatab Hdatatab

2) LENGTH / LENGTHB

- 각각 문자열의 길이(문자 수) 와 총 크기(Bytes)를 반환하는 함수
- 영문자와 숫자의 경우 한 글자는 1Byte 임
- 한글의 경우 2~4Byte 로 처리되며, 이는 캐릭터 셋에 따라 달라짐

** 문법

SQL1 *
1 LENGTH(대상)

함수 결과)

SQL1 *
1 SELECT LENGTH('제주도') AS RESULT1,
2 LENGTHB('제주도') AS RESULT2
3 FROM DUAL;

Result
Grid Result Server Output Text Output Explain Plan Statistics
RESULT1 RESULT2
1 3 6

☞ 한글의 경우 캐릭터셋 설정에 따라 한 글자의 크기가 달라지며, 주로 2, 3, 4Byte 로 저장됨

3) CONCAT

- 문자열을 결합하는 함수
- 두 개의 문자열만 결합 가능(세 개 이상의 문자열 결합은 불가능)

** 문법

```
SQL1 *
1 CONCAT(대상1, 대상2)
```

함수 결과)

```
SQL1 *
1 SELECT CONCAT('AB', 'CD')
2 FROM DUAL;
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	CONCAT('AB', 'CD')
1	ABCD

4) SUBSTR

- 문자열에서 일부분을 추출하는 함수

** 문법

```
SQL1 *
1 SUBSTR(대상, 추출위치 [, 추출개수])
```

** 특징

- '추출위치'가 0 보다 작은 경우, 문자열의 뒤에서부터 추출함
- '추출개수'는 생략 가능하며, 생략 시 기본값은 -1 로 설정됨
- 함수 실행 결과는 문자 타입으로 반환됨

함수 결과)

```
SQL1 *
1 SELECT SUBSTR('서울시 강남구 역삼동', 1, 3) AS RESULT
2 FROM DUAL;
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	RESULT
1	서울시

☞ '서울시 강남구 역삼동' 문자열에서 첫 번째 글자부터 세 글자를 추출하여 출력함

함수 결과)

SQL1 *	
1	SELECT SUBSTR('010-1111-2222', -4) AS RESULT
2	FROM DUAL;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

RESULT	
1	2222

☞ '010-1111-2222' 문자열에서 뒤에서부터 네 번째 위치에 있는 문자(2)부터 문자열 끝까지를 추출함

예제) 주민등록번호에서 생년월일 추출

SQL1 *	
1	SELECT NAME, JUMIN,
2	SUBSTR(JUMIN,1,6) AS 생년월일
3	FROM STUDENT;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

	NAME	JUMIN	생년월일
1	강성근	9810071305173	981007
2	고대영	9809121384567	980912
3	권정현	9805291644489	980529
4	김건일	98111111004241	981111
5	김재현	9812211826123	981221
6	나세희	9907252327711	990725

이하 생략

☞ 주민등록번호 문자열의 앞 6 자리를 추출하여 생년월일 정보만 출력할 수 있음

5) INSTR

- 문자열 내에서 **특정 문자열의 위치를 반환하는 함수**

** 문법

SQL1 *	
1	INSTR(대상, 문자열 [, 시작위치] [, 발견횟수])

** 특징

- '시작위치'는 생략 가능하며, 생략 시 기본값은 1 임
- '시작위치'가 0 보다 작은 경우, 문자열의 뒤에서부터 검색함
- '발견횟수'는 생략 가능하며, 생략 시 기본값은 1 로 설정되어 첫 번째로 발견되는 문자열의 위치를 반환함
- 찾는 문자열이 없는 경우 0 을 반환함

함수 결과)

SQL1 *	
1	SELECT INSTR('123#45#6#789#0', '#') AS RESULT
2	FROM DUAL;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

RESULT	
1	4

☞ '123#45#6#789#0' 문자열에서 첫 번째 위치부터 검색하여 가장 처음 발견되는 '#'의 위치를 숫자값으로 출력함

함수 결과)

SQL1 *	
1	SELECT INSTR('123#45#6#789#0', '#', 5) AS RESULT
2	FROM DUAL;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

RESULT	
1	7

☞ '123#45#6#789#0' 문자열에서 다섯 번째 위치부터 검색하여 가장 처음 발견되는 '#'의 위치를 숫자값으로 출력함

함수 결과)

SQL1 *	
1	SELECT INSTR('123#45#6#789#0', '#', 5, 2) AS RESULT
2	FROM DUAL;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

RESULT	
1	9

☞ '123#45#6#789#0' 문자열에서 다섯 번째 위치부터 검색하여 두 번째로 발견되는 '#'의 위치를 숫자값으로 출력함

함수 결과)

SQL1 *	
1	SELECT INSTR('123#45#6#789#0', '#', -3, 2) AS RESULT
2	FROM DUAL;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

RESULT	
1	7

☞ '123#45#6#789#0' 문자열에서 뒤에서부터 세 번째 위치(9)부터 왼쪽 방향으로 검색하여 두 번째로 발견되는 '#'의 위치를 숫자값으로 출력함

6) LPAD / RPAD

- 문자열의 왼쪽 또는 오른쪽에 **특정 문자열을 삽입하는 함수**

**** 문법**

SQL1 *	
1	LPAD(대상, 총크기 [, 삽입문자열])

**** 특징**

- '삽입문자열'은 생략 가능하며, **생략 시 기본값으로 공백이 삽입됨**
- '총크기'는 Byte 단위로 계산됨

함수 결과)

SQL1 *	
1	SELECT LPAD('abcd', 10, '*') AS RESULT1,
2	LPAD('abcd', 10) AS RESULT2
3	FROM DUAL;

Result	
RESULT1	RESULT2
*****abcd	abcd

☞ 'abcd' 문자열의 왼쪽에 '*'를 삽입하여 전체 크기가 10Byte 가 되도록 출력함

함수 결과)

SQL1 *	
1	SELECT RPAD(' ', 5, '#') AS RESULT1,
2	RPAD('#', 5, '#') AS RESULT2
3	FROM DUAL;

Result	
RESULT1	RESULT2
	#####

☞ 첫 번째 인수에 빈 문자열을 전달할 경우 결과 역시 빈 문자열이 반환되며, RESULT2와 같이 첫 번째 인수와 세 번째 인수의 값이 같으면 문자열의 반복 출력이 가능함

함수 결과)

SQL1 *	
1	SELECT RPAD('가', 4, '나') AS RESULT1
2	FROM DUAL;

Result	
RESULT1	
가나	

☞ '가' 문자열의 오른쪽에 '나'를 삽입하여 전체 크기가 4Byte 가 되도록 출력하며, 한글 한 글자를 2Byte 로 가정할 경우 '나'는 한 번만 삽입됨

7) LTRIM / RTRIM

- 문자열의 왼쪽 또는 오른쪽에 위치한 **특정 문자를 제거하는 함수**

**** 문법**

SQL1 *	
1	LTRIM(대상 [, 제거문자열])

**** 특징**

- '제거문자열'은 생략 가능하며, 생략 시 기본값으로 공백이 제거됨
- 문자열의 왼쪽(또는 오른쪽)에 **연속으로 위치한 '제거문자열'만 삭제됨**

함수 결과)

SQL1 *	
1	SELECT LTRIM('###AB#C', '#') AS RESULT
2	FROM DUAL;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	
RESULT	
1	AB#C

☞ '###AB#C' 문자열에서 왼쪽부터 연속된 '#' 문자만 제거하여 출력함

함수 결과)

SQL1 *	
1	SELECT LTRIM(' AB#C', '') AS RESULT1,
2	LTRIM(' AB#C') AS RESULT2
3	FROM DUAL;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	
RESULT1	RESULT2
1	AB#C

☞ 두 번째 인수에 빈 문자열을 전달하면 빈 문자열이 반환되며, 두 번째 인수를 생략하면 기본값으로 한 칸의 공백이 삭제됨

기출) 다음 중 출력 결과가 잘못된 것은?

- ① SUBSTR('SQL SERVER', 2): 'QL SERVER'
- ② SUBSTR('SQL SERVER', -2): 'ER'
- ③ INSTR('SQL SERVER', 'R'): 7
- ④ INSTR('SQL SERVER', 'E', -1, 2): 0

정답 ④

해설: INSTR('SQL SERVER', 'E', -1, 2)는 문자열의 뒤에서부터 검색하여 두 번째로 발견되는 'E'의 위치를 반환해야 하므로 0이 아니라 정상적인 위치 값이 반환되어야 한다. 따라서 출력 결과가 잘못된 것은 ④번이다.

8) TRIM

- 문자열의 왼쪽과 오른쪽에 위치한 공백을 제거하는 함수

** 문법

SQL1 *
1 TRIM(대상)

** 특징

- 문자열의 양 끝에 위치한 공백만 제거 가능하며, 제거할 문자는 별도로 지정할 수 없음

함수 결과)

SQL1 *
1 SELECT TRIM(' AB ') AS RESULT1,
2 LENGTH(TRIM(' AB ')) AS RESULT2
3 FROM DUAL;

Result
Grid Result Server Output Text Output Explain Plan Statistics
RESULT1 RESULT2
1 AB 2

☞ 양쪽에서 공백이 제거된 후 총 2 자 길이의 문자열이 반환됨

9) REPLACE

- 문자열을 치환하거나 삭제하는 함수

** 문법

SQL1 *
1 REPLACE(대상, 찾을문자열 [, 바꿀문자열])

** 특징

- '바꿀문자열'은 생략 가능함(생략할 경우 찾을 문자열이 삭제됨)

함수 결과)

SQL1 *
1 SELECT REPLACE('ABCBA', 'AB', 'ab') AS RESULT1,
2 REPLACE('ABCBA', 'AB', '') AS RESULT2,
3 REPLACE('ABCBA', 'AB') AS RESULT3
4 FROM DUAL;

Result
Grid Result Server Output Text Output Explain Plan Statistics
RESULT1 RESULT2 RESULT3
1 abCBA CBA CBA

☞ '바꿀문자열'을 생략하거나 빈 문자열을 전달하면, '찾을문자열'이 삭제됨

10) TRANSLATE

- 문자열 내 **문자를 치환하거나 삭제하는 함수**

**** 문법**

SQL1 *	
1	TRANSLATE(대상, 찾을문자열, 바꿀문자열)

**** 특징**

- '찾을문자열'과 '바꿀문자열'은 **1 대 1로 문자 단위로 치환**
- '찾을문자열'의 길이가 '바꿀문자열'보다 큰 경우, 짝이 없는 문자는 삭제
- '바꿀문자열'의 길이가 '찾을문자열'보다 큰 경우, 짝이 없는 문자는 무시

함수 결과)

SQL1 *	
1	SELECT TRANSLATE('ABCBA', 'AB', 'ab') AS RESULT
2	FROM DUAL;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	
RESULT	
1	abCba

☞ 'A'와 'B'가 각각 대응되는 문자 'a', 'b'로 치환되어 출력됨

함수 결과)

SQL1 *

1

2

3

4

SELECT

TRANSLATE('ABCBA', 'AB', 'a')

AS

RESULT1,

TRANSLATE('ABCBA', 'AB', 'abC')

AS

RESULT2,

TRANSLATE('ABCBA', 'AB', '')

AS

RESULT3

FROM

DUAL;

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

RESULT1

RESULT2

RESULT3

1

aCa

abCba

☞ RESULT1: '찾을문자열'에서 매칭되는 문자가 없는 'B'는 삭제되고, 'A'는 'a'로 치환되어 출력됨

RESULT2: '바꿀문자열'에서 짝이 없는 'c'는 무시되며, 'A'와 'B'는 각각 대응되는 문자 'a', 'b'로 치환되어 출력됨

RESULT3: '바꿀문자열'에 빈 문자열을 전달하면 NULL 이 출력됨

11) CHR

- 숫자 값(ASCII 코드)에 해당하는 문자를 반환하는 함수

** 문법

SQL1 *
1 CHR(n)

** 특징

- ASCII 코드 10 은 줄바꿈 문자를 의미함

함수 결과)

SQL1 *
1 SELECT 'AB' CHR(10) 'CD' AS RESULT
2 FROM DUAL;

Result
Grid Result Server Output Text Output Explain Plan Statistics
RESULT
1 ABCD

☞ 실제로는

AB
CD

 형태의 문자열이 반환되지만, SQL 출력 화면에서는 줄바꿈이 적용되지 않아 'ABCD'로 출력됨

2. 숫자함수

- 숫자 데이터에 대한 연산을 수행하기 위한 함수

1) ROUND

- 숫자를 반올림하는 함수

** 문법

SQL1 *
1 ROUND(숫자 [, 자리수])

** 특징

- '자리수'는 생략 가능하며, 생략 시 소수점 첫 번째 자리에서 반올림됨
- '자리수'가 0 보다 작은 경우 정수 자리에서 반올림
(예: -1 은 일의 자리에서 반올림을 의미함)

함수 결과)

SQL1 *		
1	SELECT ROUND(123.4567, 2) AS RESULT1,	
2	ROUND(123.4567, -2) AS RESULT2	
3	FROM DUAL;	

Result		
	Grid Result	Server Output
	Text Output	Explain Plan
	Statistics	
RESULT1	RESULT2	
1	123.46	100

RESULT1: 소수점 세 번째 자리에서 반올림하여 두 번째 자리로 맞추므로 123.46 이 반환됨

RESULT2: 자리수가 음수(-2)이므로 십의 자리에 해당하는 정수 자리에서 반올림이 수행되어 100 이 반환됨

2) TRUNC

- 숫자를 버림(내림) 처리하는 함수

** 문법

SQL1 *	
1	TRUNC(숫자 [, 자리수])

** 특징

- '자리수'는 생략 가능하며, 생략 시 소수점 첫 번째 자리에서 버림
- '자리수'가 0 보다 작은 경우 정수 자리에서 버림

함수 결과)

SQL1 *		
1	SELECT TRUNC(123.4567, 2) AS RESULT1,	
2	TRUNC(123.4567, -2) AS RESULT2	
3	FROM DUAL;	

Result		
	Grid Result	Server Output
	Text Output	Explain Plan
	Statistics	
RESULT1	RESULT2	
1	123.45	100

RESULT1: 소수점 세 번째 자리를 버려 123.45 가 반환됨

RESULT2: 십의 자리에서 버림이 수행되어 값이 100 이 됨

3) CEIL

- 주어진 값보다 크거나 같은 값 중에서 가장 작은 정수를 반환하는 함수임(올림)

** 문법

SQL1 *
1 CEIL(대상)

함수 결과)

SQL1 *
1 SELECT CEIL(3.3) AS RESULT1,
2 CEIL(-3.3) AS RESULT2
3 FROM DUAL;

Result
Grid Result Server Output Text Output Explain Plan Statistics
RESULT1 RESULT2
1 4 -3

- 3.3 보다 큰 값 중 가장 작은 정수는 4 이며,
- 3.3 보다 큰 값은 -3, -2, -1 ... 이고, 이 중 가장 작은 정수는 -3 임

4) FLOOR

- 주어진 값보다 작거나 같은 값 중에서 가장 큰 정수를 반환하는 함수(버림)

** 문법

SQL1 *
1 FLOOR(대상)

함수 결과)

SQL1 *
1 SELECT FLOOR(3.3) AS RESULT1,
2 FLOOR(-3.3) AS RESULT2
3 FROM DUAL;

Result
Grid Result Server Output Text Output Explain Plan Statistics
RESULT1 RESULT2
1 3 -4

- 3.3 보다 작으면서 가장 큰 정수는 3 이며, -3.3 보다 작으면서 가장 큰 정수는 -4 임

5) ABS

- 숫자의 절대값을 반환하는 함수임
- 양수와 음수 모두 양수 값으로 반환됨

** 문법

SQL1 *
1 ABS(대상)

함수 결과)

SQL1 *
1 SELECT ABS(5.4) AS RESULT1,
2 ABS(-5.4) AS RESULT2,
3 ABS(0) AS RESULT3
4 FROM DUAL;

Result
Grid Result Server Output Text Output Explain Plan Statistics
RESULT1 RESULT2 RESULT3
1 5.4 5.4 0

☞ 양수와 음수는 모두 양수로 반환되며, 0 은 0 으로 출력됨

6) MOD

- 특정 값으로 나눈 나머지를 반환하는 함수

** 문법

SQL1 *
1 MOD(숫자1, 숫자2)

** 특징

- '숫자 1'을 '숫자 2'로 나눈 나머지 출력
- '숫자 2'가 0 이면 첫 번째 인수 값이 그대로 반환됨

함수 결과)

SQL1 *
1 SELECT MOD(9,2) AS RESULT1,
2 MOD(9,0) AS RESULT2
3 FROM DUAL;

Result
Grid Result Server Output Text Output Explain Plan Statistics
RESULT1 RESULT2
1 1 9

☞ RESULT1: 9 를 2 로 나눈 나머지인 1 이 반환됨

RESULT2: 9 를 0 으로 나누는 연산은 실제로는 불가능하지만, MOD 함수의 경우 두 번째 인수(숫자 2)가 0 이면 에러가 발생하지 않고 첫 번째 인수 값이 그대로 반환됨

7) SIGN

- 숫자의 부호를 판별하는 함수
- 양수이면 1, 음수이면 -1, 0 이면 0 이 반환됨

** 문법

SQL1 *	
1	SIGN(대상)

함수 결과)

SQL1 *	
1	SELECT SIGN(5) AS RESULT1,
2	SIGN(-5) AS RESULT2,
3	SIGN(0) AS RESULT3
4	FROM DUAL;

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	RESULT1	RESULT2	RESULT3
1	1	-1	0

☞ 양수이면 1, 음수이면 -1, 0 이면 0 이 출력

8) POWER

- 숫자의 거듭제곱 값을 반환하는 함수

** 문법

SQL1 *	
1	POWER(숫자1, 숫자2)

** 특징

- '숫자 1'의 '숫자 2' 거듭제곱이 반환됨

함수 결과)

SQL1 *	
1	SELECT POWER(2,4) AS RESULT1,
2	POWER(2,-2) AS RESULT2,
3	POWER(2,0) AS RESULT3
4	FROM DUAL;

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	RESULT1	RESULT2	RESULT3
1	16	0.25	1

☞ 2의 4 제곱은 16 이며, 2의 -2 제곱은 1/4 이므로 0.25 가 반환됨
또한 2의 0 제곱은 1 이 반환됨

기출) 다음 중 출력 결과가 잘못된 것은?

- ① ROUND(12.345, 2): 12.35
- ② MOD(17, 3): 2
- ③ CEIL(-3.3): -4
- ④ POWER(2,4): 16

정답 ③

해설: CEIL(-3.3)은 -3.3 보다 크거나 같은 값 중 가장 작은 정수를 반환하므로 -3 이 반환된다.

3. 날짜함수

- 날짜 출력 및 연산과 관련된 함수

1) SYSDATE / SYSTIMESTAMP

- 현재 날짜와 시간을 반환하는 함수임
- 입력값(Input Value) 없이 호출 가능함

** 문법

SQL1 *
1 SYSDATE

** 특징

- DBMS 의 날짜 포맷 설정에 따라 출력 형식이 달라짐
- SYSDATE 는 DATE 타입으로 반환됨(초 단위까지만 표현 가능)
- SYSTIMESTAMP 는 TIMESTAMP 타입으로 반환됨(초 단위 + 소수점 이하 초(밀리초)까지 표현 가능)

함수 결과)

SQL1 *
1 SELECT SYSDATE, SYSTIMESTAMP FROM DUAL;
2
3

Result		Grid Result		Server Output	Text Output	Explain Plan	Statistics
		SYSDATE	SYSTIMESTAMP				
1		2025/12/25 13:32:14	2025/12/25 13:32:14.823000 +09:00				

2) EXTRACT

- 날짜 또는 타임스탬프 값에서 연, 월, 일 등 특정 요소를 추출하는 함수임
- 출력 결과는 숫자 타입으로 반환됨

** 문법

SQL1 *
1 EXTRACT(추출대상 FROM 날짜)

** 특징

- 추출 대상은 다음과 같음
: YEAR, MONTH, DAY, HOUR, MINUTE, SECOND
- SYSDATE 와 같은 DATE 타입에서는 YEAR, MONTH, DAY 만 추출 가능함
- SYSTIMESTAMP 와 같은 TIMESTAMP 타입에서는 HOUR, MINUTE, SECOND 까지 추출 가능함

함수 결과)

SQL1 *
1 SELECT EXTRACT(DAY FROM SYSDATE) AS TODAY,
2 EXTRACT(HOUR FROM SYSTIMESTAMP) AS TOHOUR
3 FROM DUAL;

Result
Grid Result Server Output Text Output Explain Plan Statistics
TODAY TOHOUR
1 26 11

☞ DATE 타입에서는 연·월·일만 추출 가능하며, 시·분·초 추출은 TIMESTAMP 타입에서만 가능함

3) ROUND

- 날짜를 지정한 기준 단위로 반올림하는 함수

** 문법

SQL1 *
1 ROUND(날짜 [, 자리수])

** 특징

- 자리수는 YEAR, MONTH, DD, HH, MI 등을 지정할 수 있음
- 지정한 자리수에서 직접 반올림하는 것이 아니라,
해당 자리수보다 더 작은 단위에서 반올림을 수행하여 지정된 자리수로 맞춤
- '자리수'는 생략 가능하며, 생략 시 기본값은 'DD'(일) 임
→ 시(HOUR) 단위를 기준으로 반올림이 수행됨

함수 결과)

SQL1 *			
1	SELECT ROUND(TO_DATE('2025/10/31 09:10:10')) AS RESULT1,		
2	ROUND(TO_DATE('2025/10/31 14:10:10'), 'MONTH') AS RESULT2,		
3	ROUND(TO_DATE('2025/10/31 14:10:10'), 'HH') AS RESULT3		
4	FROM DUAL;		

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	RESULT1	RESULT2	RESULT3
1	2025/10/31 00:00:00	2025/11/01 00:00:00	2025/10/31 14:00:00

RESULT1: '자리수'가 생략되어 시(HOUR) 단위를 기준으로 반올림이 수행됨

RESULT1: '자리수'가 'MONTH' 이므로, 그보다 하위 단위인 'DD'(일)을 기준으로 반올림이 수행됨

RESULT1: '자리수'가 'HH' 이므로, 그보다 하위 단위인 'MI'(분)를 기준으로 반올림이 수행됨

4) TRUNC

- 날짜를 지정한 기준 단위로 버리는 함수

** 문법

SQL1 *	
1	TRUNC(날짜 [, 자리수])

** 특징

- 자리수는 YEAR, MONTH, DD, HH, MI 등을 지정할 수 있음
- 지정한 자리수에서 직접 버리는 것이 아니라,
해당 자리수보다 더 작은 단위에서 버림을 수행하여 지정된 자리수로 맞춤
- '자리수'는 생략 가능하며, 생략 시 기본값은 'DD'(일) 임
→ 시(HOUR) 단위를 기준으로 버림이 수행됨

함수 결과)

SQL1 *			
1	SELECT TRUNC(TO_DATE('2025/10/31 09:10:10')) AS RESULT1,		
2	TRUNC(TO_DATE('2025/10/31 14:10:10'), 'MONTH') AS RESULT2,		
3	TRUNC(TO_DATE('2025/10/31 14:10:10'), 'HH') AS RESULT3		
4	FROM DUAL;		

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	RESULT1	RESULT2	RESULT3
1	2025/10/31 00:00:00	2025/10/01 00:00:00	2025/10/31 14:00:00

RESULT1: '자리수'가 생략되어 시(HOUR) 단위를 기준으로 버림이 수행됨

RESULT1: '자리수'가 'MONTH' 이므로, 그보다 하위 단위인 'DD'(일)을 기준으로 버림이 수행됨

RESULT1: '자리수'가 'HH' 이므로, 그보다 하위 단위인 'MI'(분)를 기준으로 버림이 수행됨

4. 변환함수

- 데이터 타입을 **다른 타입으로 변환하는 함수**
(문자 타입을 숫자 타입으로, 날짜 타입을 문자 타입으로 변환하는 등)

1) TO_NUMBER

- 문자 타입 데이터를 숫자 타입으로 변환하는 함수

** 문법

```
SQL1 *
1  TO_NUMBER(대상)
```

** 특징

- 숫자 형태의 문자와 숫자 데이터 간 연산 시, **묵시적 형 변환이 발생하여 연산이 정상적으로 수행됨**

함수 결과)

SQL1 *	
1	SELECT TO_NUMBER('1') AS RESULT
2	FROM DUAL;

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
RESULT	
1	1

☞ 문자 타입의 '1'이 묵시적으로 숫자 타입으로 변환되어 1로 출력됨

예제) 묵시적 형 변환

SQL1 *	
1	SELECT '1' + 1
2	FROM DUAL;

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
'1'+1	
1	2

☞ 숫자로 변환 가능한 문자값('1')과 숫자 연산 시, 문자값에 TO_NUMBER 함수가 자동으로 적용되어 TO_NUMBER('1') + 1 연산이 수행되며, 이를 묵시적 형 변환이라고 함

2) TO_CHAR

- 숫자 또는 날짜 데이터를 문자 타입으로 변환하는 함수
- 반환 결과는 항상 문자 타입으로 반환됨

** 문법

SQL1 *

1 TO_CHAR(대상 [, FORMAT])

** 특징

- 변환 대상이 숫자일 경우, 숫자 변환에 필요한 FORMAT 을 전달함
- 변환 대상이 날짜일 경우, 날짜 변환에 필요한 FORMAT 을 전달함
- FORMAT 을 생략하면, 단순히 문자 타입으로만 변환됨

📌 숫자 FORMAT

FORMAT	설명	예
9	• 숫자를 자릿수만큼 출력함 (자릿수가 부족한 경우 공백으로 채워짐)	TO_CHAR(100, '9999'): ' 100'
0	• 숫자를 자릿수만큼 출력하며, 부족한 자리는 0 으로 채움	TO_CHAR(100, '0000'): '0100'
9,999	• 천 단위 구분자를 표시함	TO_CHAR(1000, '9,999'): '1,000'
\$9,999	• 달러(\$) 기호를 출력함	TO_CHAR(1000, '\$9999'): '\$1000'
99.99	• 소수점 위치를 지정함	TO_CHAR(10, '99.99'): '10.00'

📌 날짜 FORMAT

- MONTH, DAY, DY, MON 등 문자 출력 포맷은 대소문자를 구분하므로 포맷 입력 형태에 따라 출력 결과가 달라짐(포맷을 소문자로 입력하면 소문자로, 대문자로 입력하면 대문자로 출력됨)

FORMAT	설명	예
YYYY / RRRR	• 연도를 4 자리로 출력함	TO_CHAR(SYSDATE, 'YYYY'): '2026' TO_CHAR(SYSDATE, 'RRRR'): '2026'
YY / RR	• 연도를 2 자리로 출력함	TO_CHAR(SYSDATE, 'YY'): '26' TO_CHAR(SYSDATE, 'RR'): '26'
MM	• 월을 2 자리 숫자로 출력함	TO_CHAR(SYSDATE, 'MM'): '01'
MON	• 월을 문자(약어)로 출력함	TO_CHAR(SYSDATE, 'MON'): 'JAN' TO_CHAR(SYSDATE, 'MON'): '1 월'
MONTH	• 월을 문자(전체)로 출력함	TO_CHAR(SYSDATE, 'MONTH'): 'JANUARY' TO_CHAR(SYSDATE, 'MONTH'): '1 월'
DD	• 일을 2 자리 숫자로 출력함	TO_CHAR(SYSDATE, 'DD'): '01'
DAY	• 요일을 문자(전체)로 출력함	TO_CHAR(SYSDATE, 'DAY'): 'THURSDAY' TO_CHAR(SYSDATE, 'DAY'): '목요일'
DY	• 요일을 문자(약어)로 출력함	TO_CHAR(SYSDATE, 'DAY'): 'THU' TO_CHAR(SYSDATE, 'DAY'): '목'
HH / HH12	• 시간을 12 시간 기준으로 출력함	TO_CHAR(SYSDATE, 'HH'): '02'
HH24	• 시간을 24 시간 기준으로 출력함	TO_CHAR(SYSDATE, 'HH24'): '14'
MI	• 분(Minute)을 출력함	TO_CHAR(SYSDATE, 'MI'): '30'
SS	• 초(Second)를 출력함	TO_CHAR(SYSDATE, 'SS'): '10'
Q	• 날짜가 속한 분기(Quarter)를 숫자로 반환함	TO_CHAR(SYSDATE, 'SS'): '1'

예제) 숫자 형식 변환

SQL1 *					
1	SELECT ENAME, SAL,				
2	TO_CHAR(SAL, '9999') AS RESULT1,				
3	TO_CHAR(SAL, '0999') AS RESULT2,				
4	TO_CHAR(SAL, '9,999') AS RESULT3				
5	FROM EMP;				
6					

Result					
	ENAME	SAL	RESULT1	RESULT2	RESULT3
1	SMITH	800	800	0800	800
2	ALLEN	1600	1600	1600	1,600
3	WARD	1250	1250	1250	1,250
4	JONES	2975	2975	2975	2,975
5	MARTIN	1250	1250	1250	1,250
6	BLAKE	2850	2850	2850	2,850

이하 생략

예제) 날짜 형식 변환

SQL1 *			
1	SELECT ENAME, HIREDATE,		
2	TO_CHAR(HIREDATE, 'YYYY-MM') AS RESULT		
3	FROM EMP;		

Result			
	ENAME	HIREDATE	RESULT
1	SMITH	1980/12/17 00:00:00	1980-12
2	ALLEN	1981/02/20 00:00:00	1981-02
3	WARD	1981/02/22 00:00:00	1981-02
4	JONES	1981/04/02 00:00:00	1981-04
5	MARTIN	1981/09/28 00:00:00	1981-09
6	BLAKE	1981/05/01 00:00:00	1981-05

이하 생략

3) TO_DATE

- 문자 타입 데이터를 날짜 타입으로 변환하는 함수임
- 날짜처럼 생긴 문자열을 날짜 데이터로 인식시키기 위해 사용함

** 문법

SQL1 *	
1	TO_DATE(날짜문자열 [, FORMAT])

** 특징

- 날짜처럼 생긴 '날짜문자열'을 지정한 FORMAT에 맞게 해석하여 날짜 타입으로 변환함
- FORMAT을 생략할 경우, 세션의 기본 날짜 형식(NLS_DATE_FORMAT)을 기준으로 해석함

함수 결과)

SQL1 *	
1	SELECT TO_DATE('12/10/14', 'DD/MM/YY')
2	FROM DUAL;

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
TO_DATE('12/10/14', 'DD/MM/YY')	
1	2014/10/12 00:00:00

☞ '12/10/14'를 DD/MM/YY 형식으로 해석하여 날짜 타입으로 변환하고 기본 날짜 형식으로 출력함

☞ 고득점 포인트) 두 자리 연도 YY / RR 의 서로 다른 해석

구분	설명
YY	- 입력된 두 자리 연도를 현재 세기 기준으로 해석함 - 예: '49' → 2049 년, '50' → 2050 년
RR	- 입력된 두 자리 연도를 현재 연도를 기준으로 과거 또는 미래로 해석함 - 현재 연도의 앞뒤 50 년 범위를 기준으로 연도를 판단함 - 예: '00~49' → 20XX 년, '50~99' → 19XX 년

☞ 고득점 포인트) TO_DATE 에서 지정되지 않은 날짜 요소의 처리 방식

- 지정되지 않은 연도(YEAR)와 월(MONTH)은 현재 날짜 기준으로 설정됨
- 지정되지 않은 일(DAY), 시(HOUR), 분(MINUTE), 초(SECOND)는 초기값으로 설정됨

예시	결과(현재시간은 2025/12/25 14:30:50 이라고 가정)
TO_DATE('2025','YYYY')	'2025/12/01 00:00:00'
TO_DATE('12','MM')	'2025/12/01 00:00:00'
TO_DATE('31','DD')	'2025/12/31 00:00:00'
TO_DATE('14','HH24')	'2025/12/01 14:00:00'
TO_DATE('50','MI')	'2025/12/01 00:50:00'
TO_DATE('30','SS')	'2025/12/01 00:00:30'

기출) 다음 중 SQL 실행 결과로 적절하지 않은 것은?

BIRTHDAY(Date)
1993/02/28 00:00:00

- ① TO_CHAR(BIRTHDAY, 'RRRR'): '1993'
- ② TO_CHAR(BIRTHDAY, 'MM'): 2
- ③ EXTRACT(MONTH FROM BIRTHDAY): 2
- ④ TO_CHAR(BIRTHDAY, 'Q'): '1'

정답 ②

해설: TO_CHAR(BIRTHDAY, 'MM') 역시 문자 타입으로 월을 반환하므로 결과는 '02'가 되어야 하며, 숫자 2 로 출력된다는 설명은 적절하지 않다.

● 날짜 연산

- 날짜(DATE) 타입의 값과 숫자 간 더하기·빼기 연산이 가능함
- 숫자 1 은 하루(DAY)를 의미함
- 숫자 1/24 는 한 시간(HOUR)을 의미함
- 날짜 - 날짜 연산의 결과는 두 날짜 간의 일(DAY)수 차이를 의미하는 숫자 값으로 반환됨

함수 결과) 날짜 연산

SQL1 *	
1	SELECT TO_DATE('25/12/31 14:30:00', 'YY/MM/DD HH24:MI:SS') + 1 DATE1,
2	TO_DATE('25/12/31 14:30:00', 'YY/MM/DD HH24:MI:SS') + 1/24 DATE2,
3	TO_DATE('25/12/31 14:30:00', 'YY/MM/DD HH24:MI:SS') + 1/24/60 DATE3,
4	TO_DATE('25/12/31 14:30:00', 'YY/MM/DD HH24:MI:SS') + 10/24/60 DATE4
5	FROM DUAL;
6	

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
DATE1	DATE2	DATE3	DATE4	
1 2026/01/01 14:30:00	2025/12/31 15:30:00	2025/12/31 14:31:00	2025/12/31 14:40:00	

1/24 는 한 시간을 의미하고, 1/24/60 은 1 분을 의미함
따라서 10/24/60 은 $10 \times (1/24/60)$ 이므로 10 분을 의미함

5. 일반함수

1) NVL

- NULL 값을 다른 값으로 치환하는 함수임
- 대상 값과 치환 값은 서로 동일한 데이터 타입이어야 함

** 문법

SQL1 *	
1	NVL(대상, 치환값)

예제) NVL 을 사용한 NULL 치환

SQL1 *	
1	SELECT ENAME, COMM, NVL(COMM, 100) AS NEW_COMM
2	FROM EMP
3	WHERE DEPTNO = 30;

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	ENAME	COMM	NEW_COMM
1	ALLEN	300	300
2	WARD	500	500
3	MARTIN	1400	1400
4	BLAKE		100
5	TURNER	0	0
6	JAMES		100

1/24 는 한 시간을 의미하고, 1/24/60 은 1 분을 의미함
따라서 10/24/60 은 $10 \times (1/24/60)$ 이므로 10 분을 의미함

2) NVL2

- 대상 값이 NULL 인지 여부에 따라 서로 다른 값을 반환하는 함수임

** 문법

```
SQL1 *
1 NVL2(대상, 그외치환값, 널치환값)
```

** 특징

- 대상 값이 NULL 이 아니면 첫 번째 인수인 '그외치환값'이 반환됨
- 대상 값이 NULL 이면 두 번째 인수인 '널치환값'이 반환됨
- 반환되는 두 값은 서로 동일한 데이터 타입이어야 함

예제) NVL2 을 사용한 NULL 치환

```
SQL1 *
1 SELECT ENAME, COMM,
2       NVL2(COMM, '있음', '없음') AS 치환결과
3 FROM EMP
4 WHERE DEPTNO = 30;
5
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	ENAME	COMM	치환결과
1	ALLEN	300	있음
2	WARD	500	있음
3	MARTIN	1400	있음
4	BLAKE		없음
5	TURNER	0	있음
6	JAMES		없음

☞ COMM 값이 NULL 인 경우에는 '없음'으로, NULL 이 아닌 경우에는 '있음'으로 출력함

3) COALESCE

- NULL 값을 치환하기 위한 함수
- NVL과 달리 두 개 이상의 인수를 지정할 수 있음

** 문법

```
SQL1 *
1 COALESCE(대상1, 대상2 [, 대상3] [, ...])
```

** 특징

- 나열된 값들 중에서 가장 먼저 등장하는 NULL 이 아닌 값을 반환함
- 모든 인수는 서로 호환 가능한 데이터 타입이어야 함

예제) COALESCE 을 사용한 NULL 치환

SQL1 *			
1	▶	SELECT ENAME, COMM, COALESCE(COMM, 100) AS NEW_COMM	
2		FROM EMP	
3		WHERE DEPTNO = 30;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	ENAME	COMM	NEW_COMM
1	ALLEN	300	300
2	WARD	500	500
3	MARTIN	1400	1400
4	BLAKE		100
5	TURNER	0	0
6	JAMES		100

☞ COMM의 값이 NULL인 경우 100으로 치환

예제) COALESCE를 사용하여 NULL이 아닌 값 출력하기

SQL1 *					
1	▶	SELECT STUDNO, SCORE1, SCORE2, SCORE3,			
2		COALESCE(SCORE1, SCORE2, SCORE3) AS RESULT			
3		FROM T_SCORE;			

Result					
Grid Result Server Output Text Output Explain Plan Statistics					
	STUDNO	SCORE1	SCORE2	SCORE3	RESULT
1	1	90		80	90
2	2		98	90	98
3	3	70	85	100	70
4	4			88	88

☞ 나열된 값 중 가장 먼저 발견되는 NULL이 아닌 값을 출력함

4) NULLIF

- 두 값을 비교하여 동일한 경우 NULL을 반환하는 함수임

** 문법

SQL1 *	
1	NULLIF(대상, 값)

특징

- 두 값이 서로 같으면 NULL을 반환함
- 두 값이 서로 다르면 첫 번째 값을 반환함
- 반환되는 값의 데이터 타입은 첫 번째 인수의 타입을 따름

예제) NULLIF 사용

SQL1 *					
1	▶	SELECT	ENAME, SAL, COMM,		
2			NULLIF(COMM, 100) AS 결과1,		
3			NULLIF(COMM, 500) AS 결과2		
4		FROM	EMP		
5		WHERE	DEPTNO = 30;		

Result					
Grid Result Server Output Text Output Explain Plan Statistics					
	ENAME	SAL	COMM	결과1	결과2
1	ALLEN	1600	300	300	300
2	WARD	1250	500	500	
3	MARTIN	1250	1400	1400	1400
4	BLAKE	2850			
5	TURNER	1500	0	0	0
6	JAMES	950			

- 결과 1의 경우 COMM 값이 100과 일치하는 행이 없으므로 모든 행에서 원래 값이 유지됨
 결과 2의 경우 COMM 값이 500인 WARD의 COMM만 NULL로 변경되어 출력됨

5) ISNULL

- NULL 값을 다른 값으로 치환하는 함수임
- SQL Server에서만 제공되는 함수임

** 문법

SQL1 *	
1	ISNULL(대상, 치환값)

특징

- 대상 값이 NULL이면 '치환값'을, NULL이 아니면 원래 값을 반환함
- 반환되는 데이터 타입은 첫 번째 인수의 데이터 타입을 따름

예제) ISNULL 사용(SQL Server에서 실행)

SQL1 *				
1	▶	SELECT	ENAME, SAL, COMM,	
2			ISNULL(COMM, 100) AS 결과	
3		FROM	EMP	
4		WHERE	DEPTNO = 30;	

Result				
Grid Result Text Output				
	ENAME	SAL	COMM	결과
1	ALLEN	1600	300	300
2	WARD	1250	500	500
3	MARTIN	1250	1400	1400
4	BLAKE	2850		100

- COMM 값이 NULL인 경우에는 100으로 반환됨

기출) 다음 중 출력 결과가 다른 하나는?

TAB

COL
200
NULL

- ① SELECT NVL(COL, 0) FROM TAB;
- ② SELECT COALESCE(COL, 0) FROM TAB;
- ③ SELECT NULLIF(COL, 0) FROM TAB;
- ④ SELECT ISNULL(COL, 0) FROM TAB;

정답 ③

해설: NULLIF(COL, 0)은 COL 값이 0 과 같을 경우에만 NULL 을 반환하고, 그렇지 않으면 원래 값을 반환한다.

6) DECODE

- Oracle에서 제공하는 **조건에 따라 서로 다른 값을 반환 함수**

** 문법

SQL1 *

```
1 DECODE(대상, 값1, 결과1 [, 값2] [, 결과2] [, ... 기본값])
```

** 특징

- 대상 값이 '값1'과 일치하면 '결과1'을 반환함
- 대상 값이 '값2'와 일치하면 '결과2'를 반환함
- '기본값'은 모든 조건에 일치하지 않을 경우 반환되는 값임(**생략하면 NULL 반환**)
- '결과1', '결과2' 위치에 DECODE 함수를 중첩하여 조건에 따른 이중 분기를 수행할 수 있음

예제) DECODE 사용1

SQL1 *

```
1 SELECT DEPTNO,
2        DECODE(DEPTNO, 10, '인사부', 20, '재무부') AS DNAME
3 FROM EMP;
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	DEPTNO	DNAME
1	20	재무부
2	30	
3	30	
4	20	재무부
5	30	
6	30	

이하 생략

☞ 부서번호가 10인 경우 인사부, 20인 경우 재무부를 반환하며, 그 외의 경우에는 NULL을 반환함

예제) DECODE를 사용한 이중 분기

SQL1 *			
1	▶	SELECT	DEPTNO, JOB,
2			DECODE(DEPTNO,
3			10,
4			DECODE(JOB, 'CLERK', 'A', 'B'), 'C') AS RESULT
5		FROM	EMP;

Result			
	DEPTNO	JOB	RESULT
1	10	CLERK	A
2	10	MANAGER	B
3	10	PRESIDENT	B
4	20	MANAGER	C
5	30	SALESMAN	C

이하 생략

☞ DEPTNO가 10인 경우 바로 출력값을 반환하지 않고, DECODE를 추가로 사용하여 JOB이 'CLERK'인지 여부를 한 번 더 확인함
 그 결과 DEPTNO가 10이면서 JOB이 'CLERK'이면 'A', DEPTNO가 10이면서 'CLERK'가 아니면 'B', DEPTNO가 10이 아닌 경우에는 'C'가 반환됨

7) CASE문

- 조건에 따라 서로 다른 결과를 반환하는 조건문
- 반드시 END 키워드로 마무리해야 함
- DECODE와 유사하게, **조건과 그에 대응하는 치환 값을 여러 개 전달할 수 있음**

** 문법1

SQL1 *			
1	▶	CASE	WHEN 조건1 THEN 결과1
2			[WHEN 조건2 THEN 결과2]
3			[....]
4			[ELSE 기본값]
5		END	[AS 컬럼별칭]

** 문법2(축약형)

SQL1 *			
1	▶	CASE	대상 WHEN 값1 THEN 결과1
2			[WHEN 값2 THEN 결과2]
3			[....]
4			[ELSE 기본값]
5		END	[AS 컬럼별칭]

** 특징

- ELSE 절을 생략하면 NULL이 반환됨
- 모든 조건의 대상이 같고, 항상 동등 비교(=) 조건인 경우 축약형 문법(문법2)을 사용할 수 있음
- 축약형 문법에서는 대상이 CASE와 WHEN 사이에 위치하며, WHEN과 THEN 사이에는 상수 값만 배치할 수 있음
- 축약형 문법에서 '값'과 '대상'의 데이터 타입이 일치하지 않을 경우 예러가 발생

예제) CASE문 사용1

SQL1 *		
1	SELECT SAL,	
2	CASE WHEN SAL < 2000 THEN 'C'	
3	WHEN SAL < 3000 THEN 'B'	
4	ELSE 'A'	
5	END AS SAL_GRADE	
6	FROM EMP;	

Result		
Grid Result Server Output Text Output Explain Plan Statistics		
	SAL	SAL_GRADE
1	800	C
2	1600	C
3	1250	C
4	2975	B
5	1250	C
6	2850	B
7	2450	B
8	3000	A

이하 생략

☞ CASE문을 사용하여 여러 조건(대소비교 가능)에 대한 치환 및 연산 가능!

예제) CASE문 사용2

SQL1 *		
1	SELECT DEPTNO,	
2	CASE DEPTNO WHEN 10 THEN '인사부'	
3	WHEN 20 THEN '총무부'	
4	WHEN 30 THEN '재무부'	
5	ELSE '기타'	
6	END AS DNAME1,	
7	CASE WHEN DEPTNO = 10 THEN '인사부'	
8	WHEN DEPTNO = 20 THEN '총무부'	
9	WHEN DEPTNO = 30 THEN '재무부'	
10	ELSE '기타'	
11	END AS DNAME2	
12	FROM EMP;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	DEPTNO	DNAME1	DNAME2
1	20	총무부	총무부
2	30	재무부	재무부
3	30	재무부	재무부
4	20	총무부	총무부
5	30	재무부	재무부
6	30	재무부	재무부

이하 생략

☞ 동등 비교 조건의 경우, 비교 대상(DEPTNO)을 CASE와 WHEN 사이에 한 번만 배치하여 WHEN 절마다 반복해서 작성하지 않아도 됨
단, 이 경우 DEPTNO의 데이터 타입과 WHEN 절에 명시된 비교 대상의 데이터 타입은 반드시 일치해야 함

<< 단원 정리 문제 >>

1. 다음 함수 실행 결과로 적절하지 않은 것을 고르시오

- ① SELECT ROUND(3.3) FROM DUAL: 3
- ② SELECT FLOOR(-3.3) FROM DUAL: -3
- ③ SELECT MOD(9,5) FROM DUAL: 4
- ④ SELECT SIGN(5) FROM DUAL: 1

2. 다음 함수 실행 결과로 적절하지 않은 것을 고르시오

- ① SELECT INSTR('SQL Server', 'e', -3) FROM DUAL: '9'
- ② SELECT SUBSTR('SQL Server', -3) FROM DUAL: 'ver'
- ③ SELECT LTRIM('AABABA', 'A') FROM DUAL: 'BABA'
- ④ SELECT LPAD('SQLD',5,'X') FROM DUAL: 'XSQLD'

3. 다음 SQL 결과로 가장 적절한 것은? (단, 현재시간은 2026/01/14 14:30:50 으로 가정함)

SELECT TO_DATE('26/01', 'RR/MM') FROM DUAL;

- ① 1926/01/01 00:00:00
- ② 1926/01/14 00:00:00
- ③ 2026/01/01 00:00:00
- ④ 2026/01/14 00:00:00

4. 다음 중 출력 결과가 다른 하나는?

TAB

JUMIN(CHAR)
9802031111111
9908092222222

- ① SELECT DECODE(SUBSTR(JUMIN,7,1), 1, '남', '여') FROM TAB;
- ② SELECT DECODE(SUBSTR(JUMIN,7,1), 1, '남', 2, '여') FROM TAB;
- ③ SELECT CASE WHEN SUBSTR(JUMIN,7,1) = 1 THEN '남' ELSE '여' END FROM TAB;
- ④ SELECT CASE SUBSTR(JUMIN,7,1) WHEN 1 THEN '남' ELSE '여' END FROM TAB;

5. 다음 SQL 결과로 가장 적절한 것은? (단, 현재시간은 2026/01/14 14:30:50 으로 가정함)

SELECT TO_DATE('25/12/31 14:30:00', 'YY/MM/DD HH24:MI:SS') - 3/24/60 FROM DUAL;

- ① 2025/12/28 14:30:00
- ② 2025/12/31 11:30:00
- ③ 2025/12/31 14:27:00
- ④ 2025/12/31 14:29:30

1. ②

FLOOR(-3.3)은 -3.3 보다 작은값 중 최대 정수를 리턴하므로 -4 가 출력된다.

2. ①

INSTR('SQL Server', 'e', -3)은 뒤에서 3 번째 자리인 v 부터 왼쪽으로 e 를 찾아 첫 번째로 발견되는 위치를 리턴하므로 6 이 리턴된다.

3. ③

- RR 은 두 자리 연도를 현재 연도(2026 년)를 기준으로 해석하므로 2026 년으로 변환된다.
- MM 은 01 월로 설정된다.
- FORMAT 에 일(DAY)과 시간 정보가 지정되지 않았으므로,
일은 1 일, 시간은 00:00:00 으로 기본 설정된다.

따라서 SQL 의 출력 결과는 2026/01/01 00:00:00 이므로 정답은 ③번이다.

4. ④

④는 축약형 CASE 문법으로, CASE 와 WHEN 사이의 비교 대상과 WHEN 절의 비교 값은 데이터 타입이 반드시 일치해야 한다. SUBSTR(JUMIN, 7, 1)은 문자 타입인데 WHEN 1 은 숫자 타입이므로 타입이 일치하지 않아 실행 시 에러가 발생한다.

5. ③

TO_DATE('25/12/31 14:30:00', 'YY/MM/DD HH24:MI:SS')는 2025/12/31 14:30:00 으로 변환된다.

날짜 연산에서

- 1 은 1 일,
- 1/24 는 1 시간,
- 1/24/60 은 1 분을 의미하므로,
3/24/60 은 3 분을 의미한다.

따라서 2025/12/31 14:30:00 - 3 분 = 2025/12/31 14:27:00 이 된다.

그러므로 가장 적절한 결과는 ③번이다.

2-4. WHERE 절

● WHERE 절

- 테이블에서 원하는 조건에 맞는 데이터(행)만 조회할 때 사용하는 절
- 여러 조건을 동시에 지정할 수 있으며, AND 와 OR 연산자로 조건을 연결할 수 있음
- NULL 값을 조회할 때는 IS NULL 또는 IS NOT NULL 연산자를 사용해야 하며, '=' 연산자로 NULL 비교가 불가능함
- 비교 연산자, 논리 연산자 등 다양한 연산자를 사용하여 조건을 표현할 수 있음
- 집계 함수(SUM, AVG, COUNT 등)는 WHERE 절에서 사용할 수 없음
- 컬럼 별칭은 WHERE 절에서 사용할 수 없음

연산자 종류	설명
=	같은 조건을 검색
!=, <>	같지 않은 조건을 검색
>	큰 조건을 검색
>=	크거나 같은 조건을 검색
<	작은 조건을 검색
<=	작거나 같은 조건을 검색
BETWEEN a AND b	A 와 B 사이에 있는 범위 값을 모두 검색
IN(a, b, c)	A 이거나 B 이거나 C 인 조건을 검색
LIKE	특정 패턴을 가지고 있는 조건을 검색
Is Null / Is Not Null	Null 값을 검색 / Null 이 아닌 값을 검색
A AND B	A 조건과 B 조건을 모두 만족하는 값만 검색
A OR B	A 조건이나 B 조건 중 한가지라도 만족하는 값을 검색
NOT A	A가 아닌 모든 조건을 검색

** 문법

SQL1 *			
1	SELECT	*	컬럼명 표현식
2	FROM	테이블명 또는 뷰명	
3	WHERE	조회할 데이터 조건;	

** 주의사항

- 문자 또는 날짜 상수를 표현할 때는 반드시 홑따옴표(' ')를 사용해야 하며, (WHERE 절 외에도 다른 절에서도 동일하게 적용됨)
- Oracle에서는 문자 상수의 대소문자를 구분함
- SQL Server에서는 기본 설정 기준으로 문자 상수의 대소문자를 구분하지 않음

예제) 이름이 SMITH 인 직원 조회(조건절에 문자 상수 사용)

SQL1 *			
1	▶	SELECT EMPNO, ENAME, SAL	
2		FROM EMP	
3		WHERE ENAME = 'SMITH';	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
EMPNO	ENAME	SAL	
1	7369	SMITH	800

이름이 'SMITH'인 직원의 사번, 이름, 급여를 출력함

Oracle 의 경우 문자 데이터를 기본적으로 대소문자를 구분하여 저장하므로
ENAME 이 'smith'로 저장되어 있는 경우, 데이터가 출력되지 않음

예제) SAL 이 1500 이상인 사원 정보 조회(숫자 상수 전달)

SQL1 *			
1	▶	SELECT EMPNO, ENAME, SAL	
2		FROM EMP	
3		WHERE SAL >= 1500;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
EMPNO	ENAME	SAL	
1	7499	ALLEN	1600
2	7566	JONES	2975
3	7698	BLAKE	2850
4	7782	CLARK	2450

이하 생략

급여가 1500 이상인 직원의 사번, 이름, 급여를 출력함

예제) COMM 값이 NULL 인 직원 정보 출력

SQL1 *			
1	▶	SELECT EMPNO, ENAME, SAL, COMM	
2		FROM EMP	
3		WHERE COMM IS NULL;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
EMPNO	ENAME	SAL	COMM
1	7369	SMITH	800
2	7566	JONES	2975
3	7698	BLAKE	2850
4	7782	CLARK	2450
5	7788	SCOTT	3000

이하 생략

COMM 값이 NULL 인 데이터를 조회하려면 IS NULL 연산자를 사용해야 함

COMM = NULL 조건은 항상 거짓으로 평가되므로 아무 데이터도 출력되지 않음

● 논리 연산자

1. AND: 모든 조건을 동시에 만족하는 교집합을 반환함
2. OR: 조건 중 하나라도 만족하면 포함하는 합집합을 반환함
3. NOT: 조건 결과의 반대 집합, 즉 여집합을 출력하는 연산자임
(NOT IN, NOT BETWEEN A AND B, NOT LIKE, IS NOT NULL 형태로 사용됨)

예제) AND 연산자를 사용한 여러 조건 전달

SQL1 *					
1	▶	SELECT	EMPNO, ENAME, SAL, COMM, DEPTNO		
2		FROM	EMP		
3		WHERE	DEPTNO = 10		
4		AND	SAL >= 2000;		

Result					
Grid Result Server Output Text Output Explain Plan Statistics					
	EMPNO	ENAME	SAL	COMM	DEPTNO
1	7782	CLARK	2450		10
2	7839	KING	5000		10

☞ DEPTNO 가 10 번이면서 SAL 이 2000 이상인 직원 출력

예제) OR 연산자를 사용한 여러 조건 전달

SQL1 *					
1	▶	SELECT	EMPNO, ENAME, SAL, COMM, DEPTNO		
2		FROM	EMP		
3		WHERE	DEPTNO = 10		
4		OR	SAL >= 2000;		

Result					
Grid Result Server Output Text Output Explain Plan Statistics					
	EMPNO	ENAME	SAL	COMM	DEPTNO
1	7566	JONES	2975		20
2	7698	BLAKE	2850		30
3	7782	CLARK	2450		10
4	7788	SCOTT	3000		20
5	7839	KING	5000		10

이하 생략

☞ DEPTNO 가 10 번 이거나 SAL 이 2000 이상인 직원 정보 출력

예제) NOT 연산자의 사용

SQL1 *					
1	▶	SELECT	EMPNO, ENAME, SAL		
2		FROM	EMP		
3		WHERE	NOT(SAL < 3000);		
4					

Result					
Grid Result Server Output Text Output Explain Plan Statistics					
	EMPNO	ENAME	SAL		
1	7788	SCOTT	3000		
2	7839	KING	5000		
3	7902	FORD	3000		

☞ 이 경우 '3000 보다 작거나 같은' 조건으로 직접 표현할 수 있으므로, NOT 연산자를 사용하지 않는다.

● IN 연산자

- 포함 연산자로, 여러 상수 중 하나와 일치하는 조건을 전달할 때 사용함
- 비교할 상수들을 괄호로 묶어 동시에 전달함

예제) SMITH 와 SCOTT 의 직원 정보 출력

SQL1 *					
1	▶	SELECT	EMPNO, ENAME, SAL, COMM, DEPTNO		
2		FROM	EMP		
3		WHERE	ENAME = 'SMITH'		
4		OR	ENAME = 'SCOTT';		

Result					
EMPNO	ENAME	SAL	COMM	DEPTNO	
1	7369	SMITH	800		20
2	7788	SCOTT	3000		20

SQL1 *					
1	▶	SELECT	EMPNO, ENAME, SAL, COMM, DEPTNO		
2		FROM	EMP		
3		WHERE	ENAME IN ('SMITH', 'SCOTT');		

Result					
EMPNO	ENAME	SAL	COMM	DEPTNO	
1	7369	SMITH	800		20
2	7788	SCOTT	3000		20

☞ 이름이 SMITH 이면서 동시에 SCOTT 일 수는 없으므로, 각 조건을 만족하는 결과의 합집합을 출력함

● BETWEEN A AND B 연산자

- A 이상(A 포함)이고 B 이하(B 포함) 인 조건을 만족하는 대상을 선택할 때 사용함
- A 와 B 에는 문자, 숫자, 날짜 값 모두 사용 가능함
- A 는 반드시 B 보다 작아야 하며, 반대로 작성할 경우 조건을 만족하는 데이터가 존재하지 않아 아무것도 출력되지 않음(공집합)

예제) SAL 이 2000 이상 3000 이하인 직원 정보 출력

SQL1 *					
1	▶	SELECT	EMPNO, ENAME, SAL, COMM, DEPTNO		
2		FROM	EMP		
3		WHERE	SAL >= 2000		
4		AND	SAL <= 3000;		

Result					
EMPNO	ENAME	SAL	COMM	DEPTNO	
1	7566	JONES	2975		20
2	7698	BLAKE	2850		30
3	7782	CLARK	2450		10
4	7788	SCOTT	3000		20
5	7902	FORD	3000		20

SQL1 *					
1	▶	SELECT	EMPNO, ENAME, SAL, COMM, DEPTNO		
2		FROM	EMP		
3		WHERE	SAL BETWEEN 2000 AND 3000;		

Result					
EMPNO	ENAME	SAL	COMM	DEPTNO	
1	7566	JONES	2975		20
2	7698	BLAKE	2850		30
3	7782	CLARK	2450		10
4	7788	SCOTT	3000		20
5	7902	FORD	3000		20

☞ SAL 이 2000 이상이면서 3000 이하인 조건을 모두 만족하는 교집합을 출력함

예제) BETWEEN 연산자 주의사항

SQL1 *					
1	▶	SELECT	EMPNO, ENAME, SAL, COMM, DEPTNO		
2		FROM	EMP		
3		WHERE	SAL BETWEEN 3000 AND 2000;		

Result					
EMPNO	ENAME	SAL	COMM	DEPTNO	

☞ A 가 B 보다 큰 값일 경우, 조건을 만족하는 데이터가 없어 아무것도 조회되지 않음

예제) NOT BETWEEN 사용

SQL1 *					
1	▶	SELECT EMPNO, ENAME, SAL, COMM, DEPTNO			
2		FROM EMP			
3		WHERE SAL NOT BETWEEN 1000 AND 3000;			

Result					
Grid Result Server Output Text Output Explain Plan Statistics					
	EMPNO	ENAME	SAL	COMM	DEPTNO
1	7369	SMITH	800		20
2	7839	KING	5000		10
3	7900	JAMES	950		30

☞ 1000 이상 3000 이하 조건의 반대 집합은 1000 미만 또는 3000 초과인 집합을 의미함

기출) 아래 SQL 출력 결과로 가장 적절한 것은?

TAB

NAME	GRADE	SCORE
홍길동	1	90
박길동	1	88
김길동	2	86
최길동	2	70
유길동	3	68
오길동	4	85

```
SELECT COUNT(*)
  FROM TAB
 WHERE GRADE IN (3,4)
        OR SCORE BETWEEN 80 AND 90;
```

- ① 2
- ② 3
- ③ 4
- ④ 5

정답 ④

해설: WHERE 절은 OR 조건이므로 두 조건 중 하나라도 만족하면 행이 포함된다.

GRADE IN (3,4)에 해당하는 행은 유길동(3), 오길동(4) → 2 건이다.

SCORE BETWEEN 80 AND 90 조건은 경계값을 포함하므로

홍길동(90), 박길동(88), 김길동(86), 오길동(85) → 4 건이다.

이 중 오길동은 두 조건을 모두 만족하지만 OR 조건에서는 중복 제거 없이 행 단위로 한 번만 카운트된다.

따라서 최종적으로 포함되는 행은

홍길동, 박길동, 김길동, 유길동, 오길동으로 총 5 건이며, COUNT(*) 결과는 5 이다.

● LIKE 연산자

- 정확히 일치하지 않아도 되는 패턴 조건을 전달할 때 사용하는 연산자
- 와일드카드 문자(% , _)와 함께 사용됨
 - %: 자리수 제한 없이 모든 문자열을 의미함
 - _: 한 글자를 의미하며, _ 하나당 한 자리의 모든 값을 표현함

예시) LIKE 연산자의 활용

- ENAME LIKE 'S%'
 - 이름이 'S'로 시작하는 조건
- ENAME LIKE '%S%'
 - 이름에 'S'가 포함된 조건
- ENAME LIKE '%S'
 - 이름이 'S'로 끝나는 조건
- ENAME LIKE '_S%'
 - 이름의 두 번째 글자가 'S'인 조건
- ENAME LIKE '__S__'
 - 이름의 가운데 글자가 'S'이며, 전체 길이가 5 글자인 조건

👉 고득점 포인트) ESCAPE CHARACTER

- % 또는 _를 와일드카드가 아닌 일반 문자로 조회하고 싶을 때 사용함
- 특정 문자를 ESCAPE 문자로 지정하여, 그 뒤에 오는 % 또는 _를 문자 그대로 인식하게 함

예시) ESCAPE 사용

- WHERE ENAME LIKE 'A_%' ESCAPE '\'
 - 이름이 'A_'로 시작하는 조건 (_를 문자로 인식)
- ENAME LIKE '%!%%' ESCAPE '!'
 - 이름에 '%' 문자가 포함된 조건

예제) ESCAPE 를 사용한 LIKE 연산자

출력 1) 이름이 'A'로 시작하며 두 글자 이상인 데이터를 조회함

원본 테이블)

SQL1 *	
1	SELECT * FROM TAB_ESCAPE;
2	
Result	
Grid Result Server Output Text Output Explain Plan	
NAME	
1 A_TEST	
2 A1TEST	
3 100%SAFE	
4 NOSIGN	

결과)

SQL1 *	
1	SELECT *
2	FROM TAB_ESCAPE
3	WHERE NAME LIKE 'A_%';
4	
Result	
Grid Result Server Output Text Output Explain Plan	
NAME	
1 A_TEST	
2 A1TEST	

👉 '_'는 한 글자의 모든 값을 의미하는 와일드카드이므로, 'A' 뒤에 반드시 한 글자의 문자가 포함되어야 함

출력 2) 'A_'로 시작하는 글자 찾기

SQL1 *	
1	SELECT *
2	FROM TAB_ESCAPE
3	WHERE NAME LIKE 'A!_%' ESCAPE '!';
4	

Result	
Grid Result	Server Output
Text Output	Explain Plan

NAME	
1	A_TEST

☞ ESCAPE '!'로 인해 '!' 뒤에 위치한 '_'는 와일드카드가 아닌 일반 문자로 인식됨
따라서 이름이 'A'로 시작하고, 두 번째 글자가 반드시 '_'인 경우만 출력됨

기출) 아래 SQL 출력 결과로 가장 적절한 것은?

TAB

NAME
1AA11
AA11
ABABC
BBAAB
BA%A1

```
SELECT COUNT(*)
FROM TAB
WHERE COL LIKE '_A%%A%';
```

- ① 0
- ② 1
- ③ 2
- ④ 3

정답 ③

해설: LIKE '_A%%A%' 조건은 두 번째 글자가 'A'이면서, 이후 문자열 중 어딘가에 'A'가 한 번 더 포함된 데이터를 의미한다. 해당 조건을 만족하는 데이터는 2 건이므로 정답은 ③번이다.

● 논리 연산자 우선순위

- NOT > AND > OR 순으로 연산 우선순위가 적용됨

예제) 잘못된 논리 연산자 사용

출력 1) 부서번호가 10 또는 20 이 아니면서, 동시에 급여가 2000 이상인 직원 출력 실패

SQL1 *			
1	▶	SELECT	ENAME, DEPTNO, SAL
2		FROM	EMP
3		WHERE	NOT DEPTNO = 10 OR DEPTNO = 20
4		AND	SAL >= 2000
5		ORDER BY	DEPTNO, SAL;

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	ENAME	DEPTNO	SAL
1	SMITH	20	800
2	ADAMS	20	1100
3	JONES	20	2975
4	SCOTT	20	3000
5	FORD	20	3000
6	JAMES	30	950
7	WARD	30	1250
8	MARTIN	30	1250
9	TURNER	30	1500
10	ALLEN	30	1600
11	BLAKE	30	2850

부서번호가 30 번이면서 급여가 2000 이상인 직원이 출력되어야 하지만 조건에 맞지 않는 데이터도 출력됨

출력 2) 연산자 우선순위에 의해 다음과 같이 해석됨

SQL1 *			
1	▶	SELECT	ENAME, DEPTNO, SAL
2		FROM	EMP
3		WHERE	(NOT DEPTNO = 10)
4		OR	(DEPTNO = 20 AND SAL >= 2000)
5		ORDER BY	DEPTNO, SAL;

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	ENAME	DEPTNO	SAL
1	SMITH	20	800
2	ADAMS	20	1100
3	JONES	20	2975
4	SCOTT	20	3000
5	FORD	20	3000
6	JAMES	30	950
7	WARD	30	1250
8	MARTIN	30	1250
9	TURNER	30	1500
10	ALLEN	30	1600
11	BLAKE	30	2850

DEPTNO 가 20, 30 번인 집합 전체가 출력됨

출력 3) 올바른 논리 연산자 순서 전달(부서번호가 10 또는 20 이 아니면서, 동시에 급여가 2000 이상인 직원)

SQL1 *

```

1 ▶ SELECT ENAME, DEPTNO, SAL
2   FROM EMP
3   WHERE NOT (DEPTNO = 10 OR DEPTNO = 20)
4     AND SAL >= 2000
5   ORDER BY DEPTNO, SAL;

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	ENAME	DEPTNO	SAL
1	BLAKE	30	2850

기출) 아래 SQL 출력 결과로 가장 적절한 것은?

사원

사번	이름	부서	실적
1	SMITH	10	100
2	ALLEN	10	98
3	FORD	10	85
4	KING	10	92
5	JAMES	20	98
6	ADAMS	20	88
7	BLAKE	20	80
8	WARD	30	88
9	SCOTT	30	90

```

SELECT COUNT(*)
  FROM 사원
 WHERE 이름 NOT LIKE '%A%'
    OR 부서 = 10
    AND 실적 BETWEEN 90 AND 100;

```

- ① 4
- ② 5
- ③ 6
- ④ 7

정답 ②

해설: WHERE 절에서 AND 가 OR 보다 우선순위가 높다. 따라서 조건은 다음과 같이 해석된다.

이름 NOT LIKE '%A%' OR (부서 = 10 AND 실적 BETWEEN 90 AND 100)

먼저 이름에 A가 포함되지 않은 사원은 SMITH, FORD, KING, SCOTT → 4 건이다.

다음으로 부서가 10 이고 실적이 90 이상 100 이하인 사원은

SMITH(100), ALLEN(98), KING(92) → 3 건이다.

OR 조건이므로 두 조건 중 하나라도 만족하면 포함되며,

SMITH와 KING은 중복되므로 한 번만 계산된다.

최종적으로 포함되는 사원은

SMITH, FORD, KING, SCOTT, ALLEN 으로 총 5 건이며, COUNT(*) 결과는 5 이다.

<< 단원 정리 문제 >>

1. 다음 SQL 문 실행 결과의 총 건수는?

<STUDENT>

ID	NAME	AGE	GENDER	DEPARTMENT	EDATE
1	ALICE	22	F	COMPUTER SCIENCE	2020-03-01
2	BOB	21	M	MATHEMATICS	2021-09-10
3	CHARLIE	24	M	PHYSICS	2019-05-20
4	DAVID	23	M	CHEMISTRY	2020-11-15
5	EVE	20	F	BIOLOGY	2022-01-05
6	FRANK	25	M	COMPUTER SCIENCE	2018-02-11
7	GRACE	22	F	CHEMISTRY	2021-06-10

```
SELECT *
FROM STUDENT
WHERE DEPARTMENT IN ('COMPUTER SCIENCE', 'CHEMISTRY')
AND EDATE <= '2020-12-01'
OR AGE <= 22;
```

- ① 4
- ② 5
- ③ 6
- ④ 7

2. 다음 중 NULL 에 대한 조건 전달로 가장 적절한 것은?

- ① COMM = NULL
- ② COMM != 'NULL'
- ③ COMM IS NOT 'NULL'
- ④ COMM IS NULL

3. 다음 SQL 문을 실행했을 때 오류가 발생하는 부분은?

- ① SELECT ENAME NAME
- ② FROM EMP E
- ③ WHERE SUM(SAL) >= 1000
- ④ NOT (DEPTNO = 10);

4. 다음 SQL 문장 중 오류가 발생하는 것은?

(단, COL1 은 NUMBER, COL2 는 VARCHAR2 타입)

- ① SELECT * FROM TAB WHERE COL1 BETWEEN 2000 AND 1000;
- ② SELECT * FROM TAB WHERE COL2 LIKE 1%;
- ③ SELECT * FROM TAB WHERE COL1 IN ('1000','2000');
- ④ SELECT * FROM TAB WHERE COL2 = NULL;

1. ③

DEPARTMENT 와 EDATE 에 대한 조건을 만족하는 집합과 AGE 조건을 만족하는 집합의 합집합이 출력되며, ALICE, FRANK, DAVID, BOB, EVE, GRACE 총 6 개의 행이 출력된다.

2. ④

NULL 에 대한 비교는 IS NULL 또는 IS NOT NULL 연산자만 가능하다.

3. ③

③의 WHERE SUM(SAL) >= 1000 부분에서 그룹 함수(SUM)를 WHERE 절에서 사용하고 있으므로 오류가 발생한다.

4. ②

LIKE 1%에서 문자 상수(%)는 반드시 홑따옴표로 감싸야 하므로, LIKE '1%'가 되어야 하며 현재 문장은 문법 오류가 발생한다.

2-5. GROUP BY 절

● GROUP BY 절

- 행을 특정 기준에 따라 그룹화하고, 각 그룹에 대해 집계 연산을 수행하기 위해 사용하는 구문임
- 주로 집계 함수(COUNT, SUM, AVG, MAX, MIN 등) 와 함께 사용되어, 그룹별 요약 결과를 제공함
- GROUP BY 절에는 그룹 기준이 될 컬럼을 지정하며, 여러 개의 컬럼을 동시에 지정할 수 있음

** 주의

- GROUP BY 연산에서 제외할 행이 있는 경우, GROUP BY 이전 단계인 WHERE 절에서 미리 제외해야 함
- GROUP BY 절에 나열하지 않은 컬럼은 SELECT 절에서 사용할 수 없음
(단, 집계 함수에 포함된 컬럼은 예외)

** 문법

SQL1 *			
1	SELECT	*	컬럼명 표현식
2	FROM	테이블명 또는 뷰명	
3	WHERE	조회할 데이터 조건	
4	GROUP BY	그룹핑 컬럼명	
5	HAVING	그룹핑 대상 필터링 조건;	

● 집계 함수

- 여러 행의 값을 입력 받아 하나의 요약 값을 반환하는 다중 행 함수
- 수학·통계 연산을 수행하는 함수로, COUNT, SUM, AVG, MIN, MAX 등이 있음
- 전체 데이터에 대한 집계 또는 GROUP BY 절에 의해 그룹별 집계 결과를 반환함
- 하나의 인수만 허용함
- NULL 값은 연산에서 제외됨

1. COUNT

- 테이블의 행 수를 세는 함수임
- **조건을 만족하는 행이 없는 경우에도 COUNT 함수의 결과는 NULL 이 아니라 0 이 반환됨**
- NULL 값은 세지 않음
- 문자, 숫자, 날짜 타입 컬럼 중 하나의 인수만 전달 가능함
- 전달되는 인수에 따라 COUNT 결과가 달라질 수 있음(NULL 포함 여부에 따라)
- **COUNT(*)는 NULL 여부와 관계없이 항상 전체 행의 개수를 반환함**

** 문법

SQL1 *	
1	COUNT(대상)

예제) 각 컬럼의 COUNT 결과

SQL1 *			
1	SELECT COUNT(*),		
2	COUNT(EMPNO),		
3	COUNT(COMM)		
4	FROM EMP;		

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	COUNT(*)	COUNT(EMPNO)	COUNT(COMM)
1	14	14	4

☞ COMM 컬럼은 NULL 값을 포함하고 있으므로, 전체 행의 수와 다른 COUNT 결과가 출력됨

예제) NULL 로만 구성된 테이블의 COUNT 결과

<NULL_TAB1>

	NO	NAME
1		
2		

SQL1 *	
1	SELECT COUNT(*)
2	FROM NULL_TAB1;

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
	COUNT(*)
1	2

☞ COUNT(*)의 결과는 항상 전체 행의 건수를 반환함

2. SUM

- 숫자 값의 총합을 반환하는 집계 함수임
- 숫자 타입 컬럼만 인수로 전달할 수 있음
- 조건을 만족하는 행이 없거나, 모든 값이 NULL 인 경우 SUM의 결과는 NULL 이 반환됨

예제) 급여의 전체 총합

SQL1 *	
1	SELECT SUM(SAL)
2	FROM EMP;

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
	SUM(SAL)
1	29025

예제) NULL 로만 구성된 행들의 SUM 결과

SQL1 *	
1	SELECT COUNT(COMM), SUM(COMM)
2	FROM EMP
3	WHERE COMM IS NULL;
4	

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

COUNT(COMM)	SUM(COMM)
1	0

☞ NULL 로만 구성된 행들의 경우, COUNT 의 결과는 0 이지만 SUM 의 결과는 NULL 이 반환됨

예제) 조건을 만족하는 대상이 없는 경우의 SUM 함수 결과

SQL1 *	
1	SELECT COUNT(COMM), SUM(COMM)
2	FROM EMP
3	WHERE COMM < 0;
4	

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

COUNT(COMM)	SUM(COMM)
1	0

☞ 조건을 만족하는 대상이 없는 경우에도 SUM 은 NULL 을 반환하고 COUNT 는 0 을 반환함

3. AVG

- 평균값을 반환하는 집계 함수
- 숫자 타입 컬럼만 인수로 전달할 수 있음
- NULL 값을 제외한 값들에 대해서만 평균을 계산하므로, 전체 대상에 대한 평균을 구할 때는 주의가 필요함

** 문법

SQL1 *	
1	AVG(대상)

예제) 평균 계산 결과

SQL1 *	
1	SELECT AVG(COMM) AS AVG1,
2	ROUND(SUM(COMM) / COUNT(EMPNO)) AS AVG2
3	FROM EMP;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

AVG1	AVG2
1	550 157

SQL1 *	
1	SELECT ROUND(AVG(NVL(COMM, 0))) AS AVG3
2	FROM EMP;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

AVG3
1 157

☞ AVG1 은 NULL 을 제외한 값들에 대한 평균을 반환하고, AVG2 는 전체 14 명에 대한 평균을 출력함

☞ NVL 함수를 사용하여 NULL 을 0 으로 치환한 후 평균을 계산하면, 전체 14 명에 대한 평균과 동일한 결과가 반환됨

4. MIN / MAX

- 각각 최솟값과 최댓값을 반환하는 집계 함수임
- 날짜, 숫자, 문자 타입 모두에서 사용 가능함

** 문법

SQL1 *	
1	MIN(대상) / MAX(대상)

예제) 부서별 직원수와 급여 총합 출력

<EMP>

	ENAME	SAL	DEPTNO
1	CLARK	2450	10
2	KING	5000	10
3	MILLER	1300	10
4	JONES	2975	20
5	FORD	3000	20
6	ADAMS	1100	20
7	SMITH	800	20
8	SCOTT	3000	20
9	WARD	1250	30
10	TURNER	1500	30
11	ALLEN	1600	30
12	JAMES	950	30
13	BLAKE	2850	30
14	MARTIN	1250	30

SQL1 *	
1	SELECT DEPTNO,
2	COUNT(*) AS 직원수,
3	SUM(SAL) AS 급여총합
4	FROM EMP
5	GROUP BY DEPTNO;

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	DEPTNO	직원수	급여총합
1	30	6	9400
2	20	5	10875
3	10	3	8750

☞ DEPTNO 값이 동일한 행들이 하나의 그룹으로 묶여, 각 그룹별로 COUNT, SUM 결과가 반환됨

기출) 다음 SQL 의 출력 결과를 순서대로 가장 올바르게 나타낸 것은?

TAB

COL1	COL2	COL3
NULL	100	0
200	NULL	100
300	0	NULL
NULL	300	200

1. SELECT AVG(COL3) FROM TAB WHERE COL1 IS NULL;
2. SELECT AVG(COL2) FROM TAB WHERE COL2 > 0;
3. SELECT COUNT(COL1) FROM TAB WHERE COL3 = NULL;

- ① 100, 200, 0
- ② 100, 200, NULL
- ③ 100, 150, 0
- ④ 200, 100, 0

정답 ①

해설: 1. COL1 이 NULL 인 행은 1 행, 4 행이며 COL3 는 0 과 200 이므로 AVG 는 100 이 반환됨

2. COL2 > 0 인 값은 100 과 300 이므로 AVG 는 200 이 반환됨

3. COL3 = NULL 조건은 성립하지 않아 0 건이며, COUNT 는 대상이 없으면 NULL 이 아닌 0 이 반환됨

예제) 가전제품의 카테고리별 브랜드별 평균 제품 가격 출력

SQL1 *		
1	▶	SELECT CATEGORY, BRAND,
2		AVG(PRICE) AS 평균가격
3		FROM ELECTRONICS
4		GROUP BY CATEGORY, BRAND
5		ORDER BY CATEGORY, BRAND;

Result		
CATEGORY	BRAND	평균가격
1 LAPTOP	APPLE	1549000
2 LAPTOP	DELL	1689000
3 LAPTOP	MICROSOFT	1699000
4 SMARTPHONE	APPLE	1329000
5 SMARTPHONE	SAMSUNG	1699000

☞ CATEGORY와 BRAND가 모두 동일한 경우 하나의 그룹으로 묶어, PRICE에 대한 평균값을 출력함

예제) 성별과 학년별로 학생 수와 최대 키를 출력

<STUDENT>

	STUDNO	NAME	GRADE	JUMIN	HEIGHT
1	5001	강성근	4	9810071305173	184
2	5002	고대영	4	9809121384567	177
3	5003	권정현	4	9805291644489	168
4	5004	김건일	4	98111111004241	175
5	5005	김재현	4	9812211826123	181
6	6001	나세희	3	9907252327711	160
7	6002	박주현	3	9903262298371	155
8	6003	송원재	3	9909091228361	173
9	6004	임주은	3	9912232377122	168
10	6005	이민기	3	9910281196941	170
11	7001	이병훈	2	0012221366122	182
12	7002	채현정	2	0001312157684	171
13	7003	홍은희	2	0003142179111	159
14	7004	전영훈	2	0006191246561	173
15	7005	최슬기	2	0005112265861	185
16	8001	이유나	1	0109292179874	162
17	8002	이민정	1	0108132556316	162
18	8003	전영훈	1	0107011436851	178
19	8004	허민기	1	0101211131967	176
20	8005	최훈	1	0111201128794	183

SQL1 *

1

▶

SELECT GRADE,

2

DECODE(SUBSTR(JUMIN,7,1), '1', '남자', '여자') AS 성별,

3

COUNT(*) AS 학생수,

4

MAX(HEIGHT) AS 최대키

5

FROM STUDENT

6

GROUP BY GRADE, SUBSTR(JUMIN,7,1)

7

ORDER BY GRADE, SUBSTR(JUMIN,7,1);

Result

4

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	GRADE	성별	학생수	최대키
1	1	남자	3	183
2	1	여자	2	162
3	2	남자	2	182
4	2	여자	3	185
5	3	남자	2	173
6	3	여자	3	168
7	4	남자	5	184

☞ 각 학년별로 남학생과 여학생을 각각 하나의 그룹으로 구분하여 키의 최대값을 출력함

예제) GROUP BY 절의 잘못된 사용 예

SQL1 *		
1	▶	SELECT DEPTNO, MAX(SAL), ENAME
2		FROM EMP
3		GROUP BY DEPTNO;

Result		

☞ GROUP BY 절에 명시되지 않은 컬럼은 SELECT 절에 전달할 수 없음!

● HAVING 절

- GROUP BY 로 그룹화된 결과에 대해 조건을 설정할 때 사용
- WHERE 절은 행 단위 조건을 적용하는데 사용되지만, 그룹화된 결과에 대해서는 조건을 설정할 수 없기 때문에 HAVING 절을 사용해야 함
- HAVING 절은 GROUP BY 절 앞에 올 수 있지만, 일반적으로 GROUP BY 절 뒤에 사용하는 것을 권장

예제) GROUP BY 결과에 대한 조건 설정

부서번호별 급여총합)

SQL1 *

```
1 ▶ SELECT DEPARTMENT_ID,
2     SUM(SALARY) AS 급여총합
3     FROM EMPLOYEES
4     WHERE DEPARTMENT_ID IS NOT NULL
5     GROUP BY DEPARTMENT_ID
6     ORDER BY DEPARTMENT_ID;
7
```

Result

 Grid Result  Server Output  Text Output  Explain Plan  Statistics

	DEPARTMENT_ID	급여총합
1	10	4400
2	20	19000
3	30	24900
4	40	6500
5	50	156400
6	60	28800
7	70	10000
8	80	304500
9	90	58000
10	100	51600
11	110	20300

출력 1) 급여 총합이 100000 이상인 그룹만 출력

SQL1 *

1

▶

SELECT DEPARTMENT_ID,

2

SUM(SALARY) AS 급여총합

3

FROM EMPLOYEES

4

WHERE DEPARTMENT_ID IS NOT NULL

5

GROUP BY DEPARTMENT_ID

6

HAVING SUM(SALARY) >= 100000

7

ORDER BY DEPARTMENT_ID;

8

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	DEPARTMENT_ID	급여총합
1	50	156400
2	80	304500

GROUP BY 결과에 대해 집계 함수 조건을 적용할 경우, WHERE 절이 아닌 HAVING 절에 조건 전달!

출력 2) 잘못된 조건 전달(오류 발생)

SQL1 *

```

1 ▶ SELECT DEPARTMENT_ID,
2       SUM(SALARY) AS 급여총합
3       FROM EMPLOYEES
4       WHERE DEPARTMENT_ID IS NOT NULL
5       AND SUM(SALARY) >= 100000
6       GROUP BY DEPARTMENT_ID
7       ORDER BY DEPARTMENT_ID;

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

ORA-00934: 그룹 함수는 허가되지 않습니다

☞ WHERE 절에서는 집계 함수(SUM)를 사용할 수 없음

● 집계 함수와 NULL 정리

1. NULL 로만 구성된 행들에 대한 집계 결과

- COUNT 의 결과는 0 이 반환됨
- 그 외 집계 함수(SUM, AVG, MIN, MAX)의 결과는 NULL 이 반환됨

SQL1 *

```

1 ▶ SELECT COUNT(COMM)
2       FROM EMP
3       WHERE COMM IS NULL;

```

Result

Grid Result Server Output Text Output Explain Plan

	COUNT (COMM)
1	0

SQL1 *

```

1 ▶ SELECT SUM(COMM), AVG(COMM),
2       MIN(COMM), MAX(COMM)
3       FROM EMP
4       WHERE COMM IS NULL;

```

Result

Grid Result Server Output Text Output Explain Plan

	SUM (COMM)	AVG (COMM)	MIN (COMM)	MAX (COMM)
1				

2. 공집합(조건에 맞는 데이터가 없는 경우)에 대한 집계 결과

- COUNT 의 결과는 0 이 반환됨
- 그 외 집계 함수의 결과는 NULL 이 반환됨

SQL1 *

```

1 ▶ SELECT COUNT(EMPNO)
2       FROM EMP
3       WHERE 1=2;

```

Result

Grid Result Server Output Text Output Explain Plan

	COUNT (EMPNO)
1	0

SQL1 *

```

1 ▶ SELECT SUM(SAL)
2       FROM EMP
3       WHERE 1=2;

```

Result

Grid Result Server Output Text Output Explain Plan

	SUM (SAL)
1	

3. 존재하지 않는 그룹에 대한 집계 결과

- 해당 그룹 자체가 생성되지 않으므로 **결과는 공집합으로 반환됨**

SQL1 *	
1	SELECT COUNT(SAL), SUM(SAL)
2	FROM EMP
3	WHERE 1=2
4	GROUP BY DEPTNO;

Result	
Grid Result	Server Output
Text Output	Explain Plan
COUNT(SAL)	SUM(SAL)

SQL1 *	
1	SELECT COUNT(SAL), SUM(SAL)
2	FROM EMP
3	GROUP BY DEPTNO
4	HAVING DEPTNO = 40;

Result	
Grid Result	Server Output
Text Output	Explain Plan
COUNT(SAL)	SUM(SAL)

기출) 다음 SQL 의 출력 결과를 순서대로 가장 알맞게 작성한 것은?

TAB

COL1	COL2
10	ABC
20	AAA
20	BCD
NULL	BAC
NULL	NULL

```
SELECT AVG(COL1)
FROM TAB
WHERE COL2 LIKE 'A_B%';
```

```
SELECT AVG(COL1)
FROM TAB
GROUP BY COL1
HAVING COL1 = 40;
```

- ① 공집합, 공집합
- ② 공집합, NULL
- ③ NULL, 공집합
- ④ NULL, NULL

정답 ③

해설: 첫 번째 SQL 문장에서는 WHERE 절 조건을 만족하는 행이 없으므로 AVG 값은 NULL 이 반환된다. 두 번째 SQL 문장에서는 먼저 COL1 값이 같은 행끼리 그룹으로 묶은 뒤 HAVING 조건을 만족하는 그룹만 선택하는데, 조건을 만족하는 그룹이 존재하지 않는다. 이처럼 조건을 만족하는 그룹 자체가 없으면 AVG 값을 계산할 대상이 없으므로 NULL 이 아니라 공집합(출력 행 없음) 이 반환된다.

<< 단원 정리 문제 >>

1. 다음 SQL 실행 결과로 알맞은 것은?

<TAB1>

COL1	COL2
1	10
1	20
1	30
2	25
3	5
3	35
NULL	0
NULL	10

```
SELECT SUM(COUNT(COL1))
  FROM TAB1
 WHERE COL2 <= 25
  GROUP BY COL1
  HAVING COUNT(*) >= 2;
```

- ① NULL
- ② 0
- ③ 1
- ④ 2

2. 다음 SQL 중 항상 실행 오류가 발생하는 문장은?

①

```
SELECT COL1, COUNT(*)
  FROM TAB1
  GROUP BY COL1
  HAVING COUNT(*) > 5;
```

②

```
SELECT COL1, AVG(COL2)
  FROM TAB1
  GROUP BY COL1
  HAVING AVG(COL2) < 50000;
```

③

```
SELECT COL1, SUM(COL2)
  FROM TAB1
  GROUP BY COL1
  WHERE COL1 = 'SALES';
```

④

```
SELECT COL1, MAX(COL2)
  FROM TAB1
  GROUP BY COL1
  HAVING COL1 = 'SALES';
```

1. ④

COL2 <= 25 인 조건을 만족하는 행 중에서 COL1 별로 COUNT 가 2 이상인 그룹은 1 과 NULL 이며, 각 그룹의 COUNT(COL1)은 2, 0 이다.

2. ③

WHERE 절은 GROUP BY 절보다 앞에 위치해야 한다.

2-6. ORDER BY 절

● ORDER BY 절

- 출력되는 행의 순서를 변경하고자 할 때 ORDER BY 절을 사용함
(데이터는 기본적으로 입력된 순서대로 출력)
- ORDER BY 뒤에 명시한 컬럼을 기준으로 정렬되며, 여러 컬럼을 사용하여 1 차, 2 차 정렬을 수행할 수 있음
- 정렬 순서는 오름차순(ASC) 또는 내림차순(DISC)으로 지정할 수 있으며, 기본값은 오름차순(ASC) 임

**문법

SQL1 *	
1	SELECT * 컬럼명 표현식
2	FROM 테이블명 또는 뷰명
3	WHERE 조회할 데이터 조건
4	GROUP BY 그룹핑컬럼명
5	HAVING 그룹핑 대상 필터링 조건
6	ORDER BY 정렬컬럼명 [ASC DESC];

● 정렬 순서(오름차순)

- 한글: 가, 나, 다, 라...
- 영어: A, B, C, D...
- 숫자: 1, 2, 3, 4...
- 날짜: 과거 날짜부터 시작해서 최근 날짜로 정렬

● 문자 정렬 순서

- 문자 컬럼은 사전식(문자 코드) 순서로 정렬됨
- 기본적으로 숫자 → 영문 대문자 → 영문 소문자 → 한글 순서
- 문자 정렬은 **왼쪽부터 차례로 값을 비교하여 작은 값이 먼저 정렬**되며, 앞의 값이 동일한 경우에는 다음 위치의 값을 기준으로 정렬됨

예제) 문자 정렬(맨 왼쪽부터 비교)

```
SQL1 *
1 SELECT FIRST_NAME, EMPLOYEE_ID, DEPARTMENT_ID
2 FROM EMPLOYEES
3 WHERE DEPARTMENT_ID = 30
4 ORDER BY FIRST_NAME;
```

Result

 Grid Result  Server Output  Text Output  Explain Plan  Statistics

	FIRST_NAME	EMPLOYEE_ID	DEPARTMENT_ID
1	Alexander	115	30
2	Den	114	30
3	Guy	118	30
4	Karen	119	30
5	Shelli	116	30
6	Sigal	117	30

예제) 숫자와 문자의 정렬 순서 차이

SQL1 *	
1	SELECT SAL
2	FROM EMP
3	WHERE DEPTNO = 30
4	ORDER BY SAL;
5	
Result	
Grid Result Server Output Text Output Explain Plan	
SAL	
1	950
2	1250
3	1250
4	1500
5	1600
6	2850

SQL1 *	
1	SELECT TO_CHAR(SAL) AS SAL
2	FROM EMP
3	WHERE DEPTNO = 30
4	ORDER BY SAL;
5	
Result	
Grid Result Server Output Text Output Explain Plan	
SAL	
1	1250
2	1250
3	1500
4	1600
5	2850
6	950

- ☞ SAL 컬럼은 숫자 타입이므로 오름차순 정렬 시 950 → 1250 순으로 정렬됨
 반면 TO_CHAR를 사용하여 문자 타입으로 변환하면, 왼쪽 문자 값이 작은 순서대로 정렬되므로 1250 → 1500 순으로 정렬됨

예제) 날짜 정렬(오름차순: 오래된 순서대로)

SQL1 *	
1	SELECT FIRST_NAME, EMPLOYEE_ID,
2	DEPARTMENT_ID, HIRE_DATE
3	FROM EMPLOYEES
4	WHERE DEPARTMENT_ID = 30
5	ORDER BY HIRE_DATE;
Result	
Grid Result Server Output Text Output Explain Plan Statistics	
FIRST_NAME	EMPLOYEE_ID DEPARTMENT_ID HIRE_DATE
1 Den	114 30 2002/12/07 00:00:00
2 Alexander	115 30 2003/05/18 00:00:00
3 Sigal	117 30 2005/07/24 00:00:00
4 Shelli	116 30 2005/12/24 00:00:00
5 Guy	118 30 2006/11/15 00:00:00
6 Karen	119 30 2007/08/10 00:00:00

- ☞ 날짜 타입 컬럼인 HIRE_DATE 를 오름차순으로 정렬하면, 과거 날짜부터(오래된 날짜 순서대로) 출력됨

예제) SELECT 절에 나열된 컬럼의 순번을 사용한 정렬

SQL1 *			
1	▶	SELECT	EMPLOYEE_ID, FIRST_NAME, HIRE_DATE
2		FROM	EMPLOYEES
3		ORDER BY	3;

Result			
	EMPLOYEE_ID	FIRST_NAME	HIRE_DATE
1	102	Lex	2001/01/13 00:00:00
2	203	Susan	2002/06/07 00:00:00
3	204	Hermann	2002/06/07 00:00:00
4	205	Shelley	2002/06/07 00:00:00
5	206	William	2002/06/07 00:00:00
6	109	Daniel	2002/08/16 00:00:00
7	108	Nancy	2002/08/17 00:00:00

이하 생략

☞ ORDER BY 절에서는 정렬할 컬럼명 대신 SELECT 절에 나열된 컬럼의 순번을 사용하여 정렬할 수 있음

● 복합 정렬

- 1 차 정렬 결과가 동일한 경우, 2 차 정렬을 통해 행의 순서를 추가로 결정할 수 있음
- 즉, 1 차 정렬 값이 같은 행들에 대해, 2 차 정렬 컬럼 값을 기준으로 다시 정렬이 수행됨

예제) 복합 정렬 예

< 정렬 전 >

SQL1 *			
1	▶	SELECT	ENAME, SAL, HIREDATE
2		FROM	EMP
3		WHERE	DEPTNO = 20;
4			
5			

Result			
	ENAME	SAL	HIREDATE
1	SMITH	800	1980/12/17 00:00:00
2	JONES	2975	1981/04/02 00:00:00
3	SCOTT	3000	1987/04/19 00:00:00
4	ADAMS	1100	1987/05/23 00:00:00
5	FORD	3000	1981/12/03 00:00:00

< SAL 내림차순, HIREDATE 오름차순 정렬 후 >

SQL1 *			
1	▶	SELECT	ENAME, SAL, HIREDATE
2		FROM	EMP
3		WHERE	DEPTNO = 20
4		ORDER BY	SAL DESC, HIREDATE ASC;
5			

Result			
	ENAME	SAL	HIREDATE
1	FORD	3000	1981/12/03 00:00:00
2	SCOTT	3000	1987/04/19 00:00:00
3	JONES	2975	1981/04/02 00:00:00
4	ADAMS	1100	1987/05/23 00:00:00
5	SMITH	800	1980/12/17 00:00:00

☞ 급여를 내림차순으로 정렬하되, 급여가 동일한 FORD와 SCOTT의 경우에는 입사일이 빠른 순서대로 정렬

● ORDER BY 절과 컬럼 별칭

- ORDER BY 절에서는 SELECT 절에서 정의한 컬럼 별칭을 사용할 수 있음(ORDER BY 절에서만 허용)
- SELECT 절에 선언된 컬럼의 순번을 이용한 정렬이 가능함
- **컬럼명과 순번을 혼합하여 사용하는 것도 가능함**

예제) 컬럼 별칭을 사용한 정렬

SQL1 *			
1	▶	SELECT	EMPLOYEE_ID AS EID ,
2			SALARY,
3			DEPARTMENT_ID
4		FROM	EMPLOYEES E
5		WHERE	DEPARTMENT_ID = 100
6		ORDER BY	EID DESC;

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	EID	SALARY	DEPARTMENT_ID
1	113	6900	100
2	112	7800	100
3	111	7700	100
4	110	8200	100
5	109	9000	100
6	108	12008	100

- ☞ SELECT 절보다 늦게 수행되는 구문은 ORDER BY 절뿐이므로,
SELECT 절에서 정의한 컬럼 별칭은 ORDER BY 절에서만 사용할 수 있음

예제) 컬럼명(또는 컬럼 별칭)과 컬럼 순번을 함께 사용한 정렬

SQL1 *			
1	▶	SELECT	FIRST_NAME, EMPLOYEE_ID,
2			DEPARTMENT_ID, HIRE_DATE
3		FROM	EMPLOYEES
4		WHERE	DEPARTMENT_ID IN (10, 20, 30)
5		ORDER BY	DEPARTMENT_ID, 2;

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	FIRST_NAME	EMPLOYEE_ID	DEPARTMENT_ID HIRE_DATE
1	Jennifer	200	10 2003/09/17 00:00:00
2	Michael	201	20 2004/02/17 00:00:00
3	Pat	202	20 2005/08/17 00:00:00
4	Den	114	30 2002/12/07 00:00:00
5	Alexander	115	30 2003/05/18 00:00:00
6	Shelli	116	30 2005/12/24 00:00:00
7	Sigal	117	30 2005/07/24 00:00:00
8	Guy	118	30 2006/11/15 00:00:00
9	Karen	119	30 2007/08/10 00:00:00

- ☞ 컬럼명, 컬럼 별칭, 컬럼 순번을 혼합하여 사용할 수 있음

기출) 아래 SQL 출력 결과로 가장 적절한 것은?

TAB

GRADE	NAME	RDATE
1	홍길동	2025/12/10
1	김길동	2025/10/25
2	박길동	2025/11/28
3	김길동	2025/09/20
2	최길동	2025/12/25

```
SELECT GRADE, NAME, RDATE
FROM TAB
ORDER BY 1, 3 DESC;
```

①

GRADE	NAME	RDATE
1	김길동	2025/10/25
1	홍길동	2025/12/10
2	박길동	2025/11/28
2	최길동	2025/12/25
3	김길동	2025/09/20

②

GRADE	NAME	RDATE
1	홍길동	2025/12/10
1	김길동	2025/10/25
2	최길동	2025/12/25
2	박길동	2025/11/28
3	김길동	2025/09/20

③

GRADE	NAME	RDATE
3	김길동	2025/09/20
2	박길동	2025/11/28
2	최길동	2025/12/25
1	김길동	2025/10/25
1	홍길동	2025/12/10

④

GRADE	NAME	RDATE
3	김길동	2025/09/20
2	최길동	2025/12/25
2	박길동	2025/11/28
1	홍길동	2025/12/10
1	김길동	2025/10/25

정답 ②

해설: ORDER BY 1, 3 DESC 는 SELECT 절에 나열된 컬럼의 순번인 GRADE 와 RDATE 를 기준으로 한 복합 정렬이다. 따라서 GRADE 는 오름차순(1 → 2 → 3)으로 정렬되고, GRADE 값이 동일한 경우에는 RDATE 가 내림차순으로 정렬되어 최신 날짜가 먼저 출력되므로 ②번이 정답이다.

● NULL 의 정렬

- NULL 값을 포함한 데이터를 정렬할 경우, Oracle 은 기본적으로 NULL 을 마지막에 배치하고, SQL Server 는 기본적으로 NULL 을 처음에 배치함
- Oracle 에서는 ORDER BY 절에 NULLS FIRST 또는 NULLS LAST 를 명시하여, NULL 값의 정렬 위치를 직접 지정할 수 있음

예제) NULL 을 포함한 컬럼의 정렬 결과(Oracle)

SQL1 *			
1	SELECT ENAME, SAL, COMM		
2	FROM EMP		
3	WHERE DEPTNO = 30		
4	ORDER BY COMM;		

Result			
	ENAME	SAL	COMM
1	TURNER	1500	0
2	ALLEN	1600	300
3	WARD	1250	500
4	MARTIN	1250	1400
5	JAMES	950	
6	BLAKE	2850	

SQL1 *			
1	SELECT ENAME, SAL, COMM		
2	FROM EMP		
3	WHERE DEPTNO = 30		
4	ORDER BY COMM NULLS FIRST;		

Result			
	ENAME	SAL	COMM
1	JAMES	950	
2	BLAKE	2850	
3	TURNER	1500	0
4	ALLEN	1600	300
5	WARD	1250	500
6	MARTIN	1250	1400

<< 단원 정리 문제 >>

1. ORDER BY 절에 대한 설명 중 가장 적절하지 않은 것은?

- ① ORDER BY 절에는 컬럼 별칭과 컬럼명을 동시에 사용할 수 있다.
- ② 첫 번째 정렬 컬럼이 UNIQUE 하다면, 두 번째 정렬 컬럼은 의미가 없다.
- ③ SELECT 절에 명시되지 않은 컬럼도 ORDER BY 절에 사용 가능하다.
- ④ GROUP BY 절에 명시되지 않은 컬럼도 ORDER BY 절에 사용 가능하다.

2. 다음 문장 중 실행이 불가능 한 것은?

①

```
SELECT COL1, COL2 AS C2
FROM TAB1
ORDER BY 1, C2;
```

②

```
SELECT COL1 COLUMN, COL2
FROM TAB1
ORDER BY COLUMN, COL2;
```

③

```
SELECT COL1 AS "COL!", COL2
FROM TAB1
ORDER BY "COL!";
```

④

```
SELECT COL1 AS C1, COL2 AS C2
FROM TAB1
ORDER BY 1 NULLS LAST;
```

3. 아래 SQL 실행 결과로 가장 적절한 것은? (단, DBMS 는 Oracle)

<TAB>

COL
NULL
100
200
200
NULL
1000
1000

```
SELECT COL
FROM TAB
GROUP BY COL
HAVING COUNT(*) = 2
ORDER BY (CASE WHEN COL = 1000 THEN 0 ELSE COL END)
```

①

COL
200
1000

②

COL
1000
200

③

COL
200
1000
NULL

④

COL
1000
200
NULL

1. ④

GROUP BY 절에 명시되지 않은 컬럼은 ORDER BY 절에서 사용할 수 없습니다.

2. ②

예약어는 컬럼 별칭으로 사용할 수 없다.

3. ④

TAB 에서 값별 개수는 NULL 2 건, 100 1 건, 200 2 건, 1000 2 건이므로 HAVING COUNT(*) = 2 조건을 만족하는 그룹은 200, NULL, 1000 이다.

ORDER BY 절의 CASE WHEN COL = 1000 THEN 0 ELSE COL END 에 의해 1000 은 0 으로 치환되어 가장 먼저 정렬되고, 그 다음은 0 < 200 순서로 정렬된다. 단, Oracle 은 기본적으로 NULL 을 마지막에 배치하므로 최종 출력 순서는 1000 → 200 → NULL 이다.

2-7. 조인

● JOIN (조인)

- 여러 테이블의 데이터를 결합하여 동시에 조회하거나 참조할 때 사용하는 기법임
- 조인할 테이블들은 FROM 절에 나열하여 지정함

※ 조인 테이블 나열 방식

- Oracle: 테이블 나열 순서는 중요하지 않음
- SQL Server: OUTER JOIN 의 경우 테이블 나열 순서가 중요함

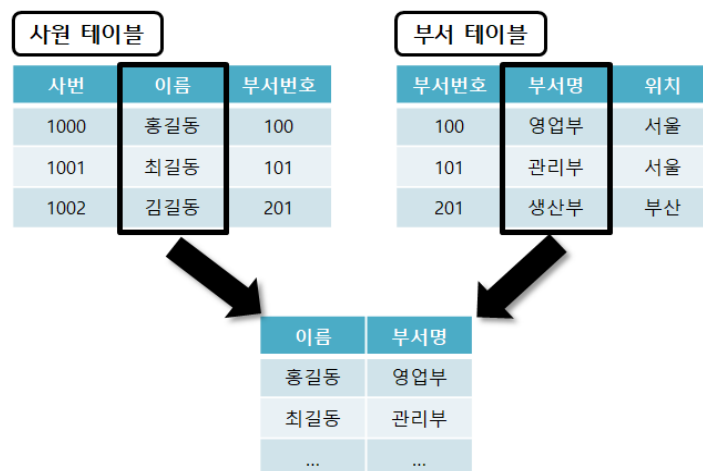
- N 개의 테이블을 조인하려면 최소 N-1 개의 조인 조건이 필요

※ 조인 조건 나열 방식

- **Oracle**: 조인 조건을 **WHERE 절**에 작성함
- **SQL Server**: 조인 조건을 **ON 절**에 작성함

- 여러 테이블에 동일한 이름의 컬럼이 존재할 경우, 컬럼명 앞에 테이블명 또는 테이블 별칭을 반드시 사용하여 구분해야 함

< 조인이 필요한 예시 >



● 조인 종류

1. 조인 조건의 형태에 따른 분류

구분	설명
EQUI JOIN (등가 조인)	- 조인 조건이 동등 비교(=) 연산자로 이루어진 조인 - 두 테이블의 특정 컬럼 값이 서로 일치하는 경우 데이터를 결합함
NON EQUI JOIN (비등가 조인)	- 조인 조건이 동등 비교가 아닌 비교 연산자로 이루어진 조인 - 두 테이블의 컬럼 값이 동일하지 않더라도, >, <, >=, <=, <> 등의 비교 연산자를 사용하여 데이터를 결합함

2. 조인 결과에 따른 분류

구분	설명
INNER JOIN	조인 조건을 만족하는 행만 결합하여 반환하는 조인
OUTER JOIN	- 조인 조건을 만족하지 않는 데이터도 함께 포함하여 반환하는 조인 - 기준 테이블의 위치에 따라 LEFT / RIGHT / FULL OUTER JOIN 으로 구분됨

3. 조인 방식에 따른 분류(= 문법적/구현적 특징 기준)

구분	설명
NATURAL JOIN	두 테이블에 공통으로 존재하는 동일한 이름의 컬럼으로 자동으로 조인하는 방식
CROSS JOIN	조인 조건 없이 두 테이블의 모든 행 조합(Cartesian Product)을 반환하는 조인
SELF JOIN	하나의 테이블을 두 번 이상 참조하여 서로 다른 테이블처럼 조인하는 방식

● EQUI JOIN (등가 JOIN)

- '='(equal) 비교 연산자를 사용하여, 두 테이블에서 동일한 값을 가지는 행을 연결하는 조인 방식임
(일치하지 않는 값은 결과에서 제외됨)
- 가장 일반적으로 사용되는 조인 형태
- FROM 절에 조인할 테이블을 모두 명시하며, 필요에 따라 테이블 별칭(Alias)을 정의할 수 있음

** 문법(Oracle 표준)

SQL1 *	
1	SELECT 테이블1.컬럼, 테이블2.컬럼
2	FROM 테이블1, 테이블2
3	WHERE 테이블1.컬럼 = 테이블2.컬럼;

** 조인 예

SELECT T1.NO, T2.NO, T1.NAME, T2.NAME
FROM T1, T2
WHERE T1.NO = T2.NO

T1	
NO	NAME
1	A
2	B
3	C
NULL	D

T2	
NO	NAME
1	A
1	B
2	C
NULL	D

EQUI JOIN(T1.NO = T2.NO)			
T1.NO	T2.NO	T1.NAME	T2.NAME
1	1	A	A
1	1	A	B
2	2	B	C

예제) EMP 테이블과 DEPT 테이블을 사용하여 각 직원의 이름과 부서명을 함께 출력

EMPNO	ENAME	SAL	DEPTNO
7369	SMITH	800	20
7499	ALLEN	1600	30
7521	WARD	1250	30
7566	JONES	2975	20
7654	MARTIN	1250	30
7698	BLAKE	2850	30
7782	CLARK	2450	10
7788	SCOTT	3000	20
7839	KING	5000	10
7844	TURNER	1500	30
7876	ADAMS	1100	20
7900	JAMES	950	30
7902	FORD	3000	20
7934	MILLER	1300	10

출력 대상

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

< 정답 >

SQL1 *	
1	SELECT EMP.ENAME, DEPT.DNAME
2	FROM EMP, DEPT
3	WHERE EMP.DEPTNO = DEPT.DEPTNO;

Result	
ENAME	DNAME
1 CLARK	ACCOUNTING
2 KING	ACCOUNTING
3 MILLER	ACCOUNTING
4 JONES	RESEARCH
5 FORD	RESEARCH
6 ADAMS	RESEARCH
7 SMITH	RESEARCH
8 SCOTT	RESEARCH
9 WARD	SALES
10 TURNER	SALES
11 ALLEN	SALES
12 JAMES	SALES
13 BLAKE	SALES
14 MARTIN	SALES

예제) NULL 과 조인

<직원>

EMPNO	ENAME	MGR
7369	SMITH	7902
7499	ALLEN	7698
7521	WARD	7934
7654	MARTIN	

<관리자>

MGRNO	NAME
7900	JAMES
7902	FORD
7934	MILLER
	KING

SQL1 *	
1	SELECT ENAME, NAME
2	FROM 직원, 관리자
3	WHERE MGR = MGRNO;

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
ENAME	NAME
1 SMITH	FORD
2 WARD	MILLER

☞ 양쪽 값이 모두 NULL 인 경우에도 '=' 비교 결과는 참이 아닌 결과로 평가되므로, 해당 행은 조인 결과에 포함되지 않음

● NON-EQUI JOIN

- 조인 조건에 '=' 외의 비교 연산자(예: <, >, BETWEEN A AND B 등)를 사용하여 테이블을 연결하는 방식
- EQUI JOIN 과 달리 값이 정확히 일치하지 않더라도, 지정한 조건을 만족하는 값들끼리 결합하여 결과를 반환함 (조건에 일치하지 않는 값은 결과에서 제외됨)

** 문법

SQL1 *	
1	SELECT 테이블1.컬럼, 테이블2.컬럼
2	FROM 테이블1, 테이블2
3	WHERE 테이블1.컬럼 비교조건 테이블2.컬럼;

예제) EMP 테이블의 급여를 확인하고 SALGRADE에 있는 급여 등급 기준에 따라 직원이름과 급여, 급여등급 출력

EMPNO	ENAME	SAL	DEPTNO
7369	SMITH	800	20
7499	ALLEN	1600	30
7521	WARD	1250	30
7566	JONES	2975	20
7654	MARTIN	1250	30
7698	BLAKE	2850	30
7782	CLARK	2450	10
7788	SCOTT	3000	20
7839	KING	5000	10
7844	TURNER	1500	30
7876	ADAMS	1100	20
7900	JAMES	950	30
7902	FORD	3000	20
7934	MILLER	1300	10

EMP TABLE

조인 컬럼

SALGRADE TABLE

GRADE	LOSAL	HISAL
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999

출력 대상

< 정답 >

SQL1 *

```

1 ▶ SELECT E.ENAME, E.SAL, S.GRADE
2   FROM EMP E, SALGRADE S
3  WHERE E.SAL BETWEEN S.LOSAL AND S.HISAL;

```

Result

4

Grid Result Server Output Text Output Explain Plan Statistics

	ENAME	SAL	GRADE
1	SMITH	800	1
2	JAMES	950	1
3	ADAMS	1100	1
4	WARD	1250	2
5	MARTIN	1250	2
6	MILLER	1300	2
7	TURNER	1500	3
8	ALLEN	1600	3

이하 생략

기출) 아래 조인 결과로 가장 적절한 것은?

TAB1

NO	NAME	RDATE
1	홍길동	2025/12/10
2	김길동	2025/10/25
3	박길동	2025/11/28
4	김길동	2025/09/20
NULL	최길동	2025/12/25

TAB2

NO	NAME	RDATE
1	김길동	2025/10/25
1	홍길동	2025/12/30
2	박길동	2025/11/28
2	최길동	2025/12/25
3	김길동	2025/09/20
NULL	NULL	NULL

```
SELECT COUNT(TAB2.NO)
  FROM TAB1, TAB2
 WHERE TAB1.NO = TAB2.NO
    AND TAB1.RDATE <= TAB2.RDATE;
```

- ① 0
- ② 1
- ③ 3
- ④ 5

정답 ③

해설: TAB1.NO = TAB2.NO 는 등가 조인이므로 NO 가 같은 행만 결합되며, NULL = NULL 은 거짓으로 평가되어 조인되지 않음. 또한 TAB1.RDATE <= TAB2.RDATE 조건을 만족하는 조합만 남는다.

NO=1 은 TAB2 에 2 건이 있으나 날짜 조건을 만족하는 것은 1 건이다(2025/12/10 ≤ 2025/12/30).

NO=2 는 날짜 조건을 만족하는 조합이 2 건이다(2025/10/25 ≤ 2025/11/28, 2025/12/25).

따라서 최종 조인 결과는 3 건이며, COUNT(TAB2.NO)는 NULL 이 아닌 값만 세므로 결과는 3 이다.

● 세 테이블 이상의 조인

- 여러 테이블을 조인할 경우, 각 테이블 간의 관계를 정확히 파악하고 모든 테이블이 서로 적절하게 연결되도록 조인 조건을 명시해야 함
- 세 개 이상의 테이블을 조인할 때는, N 개의 테이블에 대해 최소 N-1 개의 조인 조건이 필요함
- 조인 조건이 불완전하거나 누락되면, 의도하지 않은 결과가 출력되거나 잘못된 데이터가 생성될 수 있음

예제) EMPLOYEES, DEPARTMENTS, LOCATIONS 테이블 조인

EMPLOYEE_ID	FIRST_NAME	DEPARTMENT_ID	DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID	LOCATION_ID	CITY
198	Donald	50	10	Administration	1700	1000	Roma
199	Douglas	50	20	Marketing	1800	1100	Venice
200	Jennifer	10	30	Purchasing	1700	1200	Tokyo
201	Michael	20	40	Human Resources	2400	1300	Hiroshima
202	Pat	20	50	Shipping	1500	1400	Southlake
203	Susan	40	60	IT	1400	1500	South San Francisco
204	Hermann	70	70	Public Relations	2700	1600	South Brunswick
205	Shelley	110	80	Sales	2500	1700	Seattle
206	William	110	90	Executive	1700	1800	Toronto
100	Steven	90	100	Finance	1700	1900	Whitehorse
101	Neena	90	110	Accounting	1700	2000	Beijing
102	Lex	90	120	Treasury	1700	2100	Bombay

EMPLOYEES TABLE DEPARTMENTS TABLE LOCATIONS TABLE

< 정답 >

SQL1 *				
1	SELECT E.EMPLOYEE_ID, E.FIRST_NAME,			
2	D.DEPARTMENT_NAME, L.CITY			
3	FROM EMPLOYEES E, DEPARTMENTS D, LOCATIONS L			
4	WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID			
5	AND D.LOCATION_ID = L.LOCATION_ID;			

Result				
	EMPLOYEE_ID	FIRST_NAME	DEPARTMENT_NAME	CITY
1	198	Donald	Shipping	South San Francisco
2	199	Douglas	Shipping	South San Francisco
3	200	Jennifer	Administration	Seattle
4	201	Michael	Marketing	Toronto
5	202	Pat	Marketing	Toronto
6	203	Susan	Human Resources	London
7	204	Hermann	Public Relations	Munich
8	205	Shelley	Accounting	Seattle
9	206	William	Accounting	Seattle
10	100	Steven	Executive	Seattle
11	101	Neena	Executive	Seattle
12	102	Lex	Executive	Seattle

이하 생략

- 필수 조인 조건이 하나라도 누락될 경우 카티시안 곱(Cartesian Product)이 발생하며, 정상적인 조인 결과보다 훨씬 많은 행이 반환됨

● SELF JOIN

- 하나의 테이블 내에서 행과 행 간의 관계를 표현하기 위해 사용하는 조인 기법
- 동일한 테이블을 두 번 이상 참조하여 연결하는 방식임
- 테이블명이 중복되므로 각 참조를 구분하기 위해 반드시 **테이블 별칭(Alias) 및 컬럼 구분자를 사용해야 함**

예제) EMP 테이블에서 각 직원과 직원의 상위관리자 이름과 급여를 출력

** EMP SELF JOIN 과정

<직원 테이블>						<상위관리자 테이블>					
	EMPNO	ENAME	MGR	SAL	DEPTNO		EMPNO	ENAME	MGR	SAL	DEPTNO
1	7369	SMITH	7902	800	20	1	7369	SMITH	7902	800	20
2	7499	ALLEN	7698	1600	30	2	7499	ALLEN	7698	1600	30
3	7521	WARD	7698	1250	30	3	7521	WARD	7698	1250	30
4	7566	JONES	7839	2975	20	4	7566	JONES	7839	2975	20
5	7654	MARTIN	7698	1250	30	5	7654	MARTIN	7698	1250	30
6	7698	BLAKE	7839	2850	30	6	7698	BLAKE	7839	2850	30
7	7782	CLARK	7839	2450	10	7	7782	CLARK	7839	2450	10
8	7788	SCOTT	7566	3000	20	8	7788	SCOTT	7566	3000	20
9	7839	KING		5000	10	9	7839	KING		5000	10
10	7844	TURNER	7698	1500	30	10	7844	TURNER	7698	1500	30
11	7876	ADAMS	7788	1100	20	11	7876	ADAMS	7788	1100	20
12	7900	JAMES	7698	950	30	12	7900	JAMES	7698	950	30
13	7902	FORD	7566	3000	20	13	7902	FORD	7566	3000	20
14	7934	MILLER	7782	1300	10	14	7934	MILLER	7782	1300	10

** 결과

SQL1 *				
1	SELECT E1.ENAME AS 직원이름, E1.SAL AS 급여,			
2	E2.ENAME AS 상위관리자이름, E2.SAL AS 상위관리자급여			
3	FROM EMP E1, EMP E2			
4	WHERE E1.MGR = E2.EMPNO;			

Result				
	직원이름	급여	상위관리자이름	상위관리자급여
1	FORD	3000	JONES	2975
2	SCOTT	3000	JONES	2975
3	TURNER	1500	BLAKE	2850
4	ALLEN	1600	BLAKE	2850
5	WARD	1250	BLAKE	2850
6	JAMES	950	BLAKE	2850
7	MARTIN	1250	BLAKE	2850
8	MILLER	1300	CLARK	2450
9	ADAMS	1100	SCOTT	3000
10	BLAKE	2850	KING	5000
11	JONES	2975	KING	5000
12	CLARK	2450	KING	5000
13	SMITH	800	FORD	3000

예제) 셀프 조인을 사용한 누적합(일자별 누적합)

	SALE_ID	PRODUCT_NAME	QUANTITY	SALE_DATE	STORE_ID
1	1	노트북	10	2024/12/25 00:00:00	101
2	2	노트북	20	2024/12/26 00:00:00	102
3	3	노트북	30	2024/12/27 00:00:00	103
4	4	노트북	40	2024/12/28 00:00:00	104
5	5	노트북	50	2024/12/29 00:00:00	105
6	6	노트북	60	2024/12/30 00:00:00	106
7	7	노트북	70	2024/12/31 00:00:00	107
8	8	노트북	80	2025/01/01 00:00:00	108

** 조인 결과

SQL1 *

1 ▶

SELECT S1.SALE_DATE, S2.SALE_DATE, S2.QUANTITY

2

FROM SALES S1, SALES S2

3

WHERE S1.SALE_DATE >= S2.SALE_DATE

4

ORDER BY 1;

5

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	SALE_DATE	SALE_DATE	QUANTITY
1	2024/12/25 00:00:00	2024/12/25 00:00:00	10
2	2024/12/26 00:00:00	2024/12/25 00:00:00	10
3	2024/12/26 00:00:00	2024/12/26 00:00:00	20
4	2024/12/27 00:00:00	2024/12/25 00:00:00	10
5	2024/12/27 00:00:00	2024/12/26 00:00:00	20
6	2024/12/27 00:00:00	2024/12/27 00:00:00	30
7	2024/12/28 00:00:00	2024/12/28 00:00:00	40
8	2024/12/28 00:00:00	2024/12/26 00:00:00	20
9	2024/12/28 00:00:00	2024/12/25 00:00:00	10
10	2024/12/28 00:00:00	2024/12/27 00:00:00	30

이하 생략

이하 생략

** 누적합 연산 결과

SQL1 *

1 ▶

SELECT S1.SALE_DATE, SUM(S2.QUANTITY) AS 누적합

2

FROM SALES S1, SALES S2

3

WHERE S1.SALE_DATE >= S2.SALE_DATE

4

GROUP BY S1.SALE_DATE

5

ORDER BY 1;

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	SALE_DATE	누적합
1	2024/12/25 00:00:00	10
2	2024/12/26 00:00:00	30
3	2024/12/27 00:00:00	60
4	2024/12/28 00:00:00	100
5	2024/12/29 00:00:00	150
6	2024/12/30 00:00:00	210
7	2024/12/31 00:00:00	280
8	2025/01/01 00:00:00	360

☞ 누적 합은 각 날짜 시점까지의 총합을 의미하므로, 각 날짜마다 해당 날짜 이전 또는 같은 날짜에 해당하는 QUANTITY 값을 합산하여 계산할 수 있음

<< 단원 정리 문제 >>

1. 다음 SQL 문을 실행한 결과의 총 행 수는?

<PRODUCT>			<SALES>			
제품번호	제품명	가격	ID	제품번호	거래일자	수량
P001	노트북	100000	1	P001	2024-12-05	10
P002	스마트폰	50000	2	P002	2024-11-10	5
P003	태블릿	30000	3	P003	2024-12-15	7
P004	이어폰	15000	4	P001	2024-10-20	3
P005	키보드	20000	5	P005	2024-12-30	3
P006	마우스	10000	6	P006	2024-12-10	3


```

SELECT *
  FROM PRODUCT P, SALES S
 WHERE P.제품번호 = S.제품번호
       AND S.거래일자 BETWEEN '2024-12-01' AND '2024-12-31'
       AND P.가격 >= 30000;

```

- ① 1
 ② 2
 ③ 3
 ④ 4

2. 다음 빈칸에 들어갈 조건으로 가장 적절한 것은?

<EMP>		
EMPNO	ENAME	MGR
7369	SMITH	7499
7499	ALLEN	7521
7521	WARD	7566
7566	JONES	7654
7654	MARTIN	NULL


```

SELECT E2.ENAME AS 사원명,
       E1.ENAME AS 상위관리자명
  FROM EMP E1, EMP E2
 WHERE _____;

```


<결과>

사원명	상위관리자명
SMITH	ALLEN
ALLEN	WARD
WARD	JONES
JONES	MARTIN

- ① E1.EMPNO = E2.EMPNO
 ② E1.EMPNO = E2.MGR
 ③ E1.MGR = E2.EMPNO
 ④ E1.MGR = E2.MGR

1. ②

12 월에 거래된 품목은 P001, P003, P005, P006 이며, 이 중 가격이 30,000 원 이상인 제품은 P001 과 P003 뿐이다.

2. ②

SELECT 절에서 E2.ENAME 이 직원명이 되려면, E2.MGR 값을 EMPNO 로 갖는 자가 상위 관리자가 된다.

2-8. 표준 조인

● 표준 조인(Standard JOIN)

- ANSI SQL 표준에서 정의한 조인 문법(INNER JOIN, CROSS JOIN, NATURAL JOIN, OUTER JOIN 등)
- 조인 조건을 WHERE 절이 아닌 FROM 절의 JOIN ... ON 구문에 명시함
- 조인 유형과 조건이 명확히 구분되어 가독성과 유지보수성이 뛰어나
- Oracle, SQL Server 등 대부분의 DBMS 에서 공통으로 사용 가능함

● INNER JOIN

- 내부 조인이라고 하며, 조인 조건을 만족하는 행만 결합하여 결과로 반환하는 조인 방식 (Oracle 의 기본 조인 방식에 해당함)
- ANSI 표준 문법에서는 FROM 절에서 테이블 사이에 **INNER JOIN 또는 JOIN** 을 명시함
- 조인 조건은 반드시 **ON 절 또는 USING 절을 통해 지정해야 함**

● ON 절

- ANSI 표준 조인에서는 **ON 절에 조인 조건을 명시**하고, **WHERE 절에는 일반 조회 조건을 명시**함
- 조인할 두 컬럼의 이름이 서로 달라도 사용 가능함
- ON 절에서 조건을 묶는 **괄호를 생략할 수 있음**
- 양쪽 테이블에 동일한 이름의 컬럼이 존재할 경우, 컬럼을 명확히 구분하기 위해 테이블명 또는 테이블 별칭을 반드시 지정해야 함

** 문법

SQL1 *	
1	SELECT 테이블1.컬럼명, 테이블2.컬럼명
2	FROM 테이블1 [INNER] JOIN 테이블2
3	ON (테이블1.컬럼 = 테이블2.컬럼)
4	

예제) EMP 테이블과 DEPT 테이블을 사용하여 각 직원의 이름과 부서명을 함께 출력(INNER JOIN)

<Oracle 표준>

SQL1 *	
1	SELECT EMP.ENAME, DEPT.DNAME
2	FROM EMP, DEPT
3	WHERE EMP.DEPTNO = DEPT.DEPTNO
4	AND JOB = 'CLERK';

	ENAME	DNAME
1	MILLER	ACCOUNTING
2	SMITH	RESEARCH
3	ADAMS	RESEARCH
4	JAMES	SALES

<ANSI 표준>

SQL1 *	
1	SELECT EMP.ENAME, DEPT.DNAME
2	FROM EMP INNER JOIN DEPT
3	ON (EMP.DEPTNO = DEPT.DEPTNO)
4	WHERE JOB = 'CLERK';

	ENAME	DNAME
1	MILLER	ACCOUNTING
2	SMITH	RESEARCH
3	ADAMS	RESEARCH
4	JAMES	SALES

☞ Oracle 표준과 ANSI 표준은 FROM 절과 WHERE 절의 사용 방식이 다름(ANSI 표준에서는 조인 조건을 ON 절에 명시하며, JOB = 'CLERK'와 같은 일반 조회 조건은 WHERE 절에 작성함)

예제) 세 테이블의 조인 문법 비교

** Oracle 표준 작성법

SQL1 *				
1	▶	SELECT	E.EMPLOYEE_ID, D.DEPARTMENT_NAME,	
2			E.SALARY, L.CITY	
3		FROM	EMPLOYEES E,	
4			DEPARTMENTS D,	
5			LOCATIONS L	
6		WHERE	E.DEPARTMENT_ID = D.DEPARTMENT_ID	
7		AND	D.LOCATION_ID = L.LOCATION_ID	
8		AND	E.SALARY >= 14000;	

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	EMPLOYEE_ID	DEPARTMENT_NAME	SALARY	CITY
1	100	Executive	24000	Seattle
2	101	Executive	17000	Seattle
3	102	Executive	17000	Seattle
4	145	Sales	14000	Oxford

** ANSI 표준 작성법

SQL1 *				
1	▶	SELECT	E.EMPLOYEE_ID, D.DEPARTMENT_NAME,	
2			E.SALARY, L.CITY	
3		FROM	EMPLOYEES E JOIN DEPARTMENTS D	
4		ON	E.DEPARTMENT_ID = D.DEPARTMENT_ID	
5			JOIN LOCATIONS L	
6		ON	D.LOCATION_ID = L.LOCATION_ID	
7		WHERE	E.SALARY >= 14000;	

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	EMPLOYEE_ID	DEPARTMENT_NAME	SALARY	CITY
1	100	Executive	24000	Seattle
2	101	Executive	17000	Seattle
3	102	Executive	17000	Seattle
4	145	Sales	14000	Oxford

☞ 여러 테이블을 조인할 경우, JOIN ... ON 절을 연속적으로 명시하여 각 테이블 간의 조인 조건을 작성해야 함

● USING 조건절

- 조인할 두 테이블의 컬럼명이 동일한 경우에 사용하는 조건절
- USING 절에 명시한 컬럼에는 테이블명이나 별칭(Alias)과 같은 접두사(컬럼 구분자)를 사용할 수 없음
- 컬럼명은 반드시 괄호로 묶어 작성해야 함(생략 불가)

**문법

SQL1 *				
1	▶	SELECT	테이블1. 컬럼명, 테이블2. 컬럼명	
2		FROM	테이블1 INNER JOIN 테이블2	
3		USING	(동일컬럼명);	

예제) USING 절을 이용한 사원 이름과 부서 이름 조회

SQL1 *

```

1 SELECT EMP.ENAME, DEPT.DNAME
2 FROM EMP JOIN DEPT
3 USING (DEPTNO);

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	ENAME	DNAME
1	CLARK	ACCOUNTING
2	KING	ACCOUNTING
3	MILLER	ACCOUNTING
4	JONES	RESEARCH
5	FORD	RESEARCH
6	ADAMS	RESEARCH
7	SMITH	RESEARCH
8	SCOTT	RESEARCH

이하 생략

● NATURAL JOIN

- 두 테이블에 공통으로 존재하는 동일한 이름의 모든 컬럼을 기준으로 EQUI JOIN 을 수행하는 조인
- ON 절이나 USING 절을 사용할 수 없으며, WHERE 절에는 일반 조회 조건만 전달 가능함
- 조인에 사용된 컬럼에는 테이블명이나 별칭(Alias)을 붙일 수 없음

** 문법

SQL1 *

```

1 SELECT 테이블1.컬럼명, 테이블2.컬럼명
2 FROM 테이블1 NATURAL JOIN 테이블2;

```

** 조인 예

```

SELECT *
FROM T1 NATURAL JOIN T2;

```

T1		T2		NATURAL JOIN			
NO	NAME	NO	NAME	T1.NO	T2.NO	T1.NAME	T2.NAME
1	A	1	A	1	1	A	A
2	B	2	C				
NULL	C	NULL	C				

** 잘못된 사용 예

SQL1 *

```

1 SELECT T1.NO, NAME
2 FROM T1 NATURAL JOIN T2;

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

ORA-25155: NATURAL 조인에 사용된 열은 식별자를 가질 수 없음

조인에 사용된 컬럼에는 테이블명이나 별칭과 같은 컬럼 구분자를 사용할 수 없음

예제) NATURAL 조인을 사용하여 사원 이름, 부서번호, 부서명 출력

SQL1 *	
1	SELECT EMP.ENAME, DEPTNO, DEPT.DNAME
2	FROM EMP NATURAL JOIN DEPT
3	WHERE JOB = 'CLERK';

Result		
Grid Result	Server Output	Text Output
Explain Plan	Statistics	
ENAME	DEPTNO	DNAME
1 MILLER	10	ACCOUNTING
2 SMITH	20	RESEARCH
3 ADAMS	20	RESEARCH
4 JAMES	30	SALES

☞ 양쪽 테이블에 동일한 이름을 가진 DEPTNO 컬럼의 값이 같은 행끼리 결합됨

☞ 조인에 사용된 컬럼에는 테이블명이나 별칭과 같은 컬럼 구분자를 사용할 수 없음

예제) ORDERS 테이블의 각 주문 내역에 대해, SHIPMENTS 테이블에 저장된 발송 날짜를 함께 출력
(NATURAL JOIN 문법을 사용)

<ORDERS>

ORDER_ID	CUSTOMER_ID	ORDER_DATE
1	100	2025/01/01 00:00:00
2	101	2025/01/02 00:00:00
3	102	2025/01/03 00:00:00
4	100	2025/01/04 00:00:00
5	103	2025/01/05 00:00:00

<SHIPMENTS>

SHIPMENT_ID	ORDER_ID	CUSTOMER_ID	SHIPMENT_DATE
1	1	100	2025/01/02 00:00:00
2	2	101	2025/01/03 00:00:00
3	3	102	2025/01/04 00:00:00
4	4	100	2025/01/05 00:00:00
5	5	103	2025/01/06 00:00:00

< 정답 >

SQL1 *	
1	SELECT ORDER_ID, CUSTOMER_ID,
2	O.ORDER_DATE AS 주문날짜, S.SHIPMENT_DATE AS 발송날짜
3	FROM ORDERS O NATURAL JOIN SHIPMENTS S;

Result				
Grid Result	Server Output	Text Output	Explain Plan	Statistics
ORDER_ID	CUSTOMER_ID	주문날짜	발송날짜	
1	1	100 2025/01/01 00:00:00	2025/01/02 00:00:00	
2	2	101 2025/01/02 00:00:00	2025/01/03 00:00:00	
3	3	102 2025/01/03 00:00:00	2025/01/04 00:00:00	
4	4	100 2025/01/04 00:00:00	2025/01/05 00:00:00	
5	5	103 2025/01/05 00:00:00	2025/01/06 00:00:00	

☞ 양쪽 테이블에 동일한 이름으로 존재하는 ORDER_ID 와 CUSTOMER_ID 를 기준으로 조인하여,
해당 주문에 대한 발송 날짜를 함께 출력할 수 있음

기출) 조인에 대한 설명 중 가장 적절하지 않은 것은?

- ① USING 절을 사용할 경우, 조인에 사용되는 컬럼은 반드시 괄호로 묶어야 하며 테이블 별칭을 사용할 수 없다.
- ② NATURAL JOIN 은 두 테이블에 동일한 이름의 컬럼을 기준으로 자동 조인되며, ON 절을 사용할 수 없다.
- ③ ANSI 표준 조인에서는 조인 조건을 ON 절에 작성하고, 일반 조건은 WHERE 절에 작성한다.
- ④ ON 절에서는 조인 조건을 작성할 때 반드시 괄호를 사용해야 한다.

정답 ④

해설: ON 절에서 조인 조건을 작성할 때 괄호는 필수가 아니며 생략 가능하다. 나머지는 모두 맞는 설명이다.

● CROSS JOIN

- 테이블 간 조인 조건이 없는 경우 사용되며, 두 테이블의 모든 가능한 행 조합(Cartesian Product)을 생성하는 조인 방식임
- 결과 행의 개수는 양쪽 테이블의 행 수를 곱한 값만큼 반환됨

** 문법

SQL1 *

```
1 SELECT 테이블1.컬럼명, 테이블2.컬럼명
2 FROM 테이블1 CROSS JOIN 테이블2;
```

** 조인 예

```
SELECT *
FROM T1 CROSS JOIN T2;
```

T1	
NO	NAME
1	A
2	B
NULL	C

T2	
NO	NAME
1	A
2	B

CROSS JOIN			
T1.NO	T2.NO	T1.NAME	T2.NAME
1	1	A	A
1	2	A	B
2	1	B	A
2	2	B	B
NULL	1	C	A
NULL	2	C	B

기출) 다음 SQL 중 의미가 다른 하나는?

- ① SELECT * FROM T1, T2 WHERE T1.NO = T2.NO;
- ② SELECT * FROM T1 INNER JOIN T2 ON (T1.NO = T2.NO);
- ③ SELECT * FROM T1 JOIN T2 USING (NO);
- ④ SELECT * FROM T1 NATURAL JOIN T2;

정답 ④

해설: NATURAL JOIN 은 두 테이블에 공통으로 존재하는 모든 동일 이름의 컬럼을 기준으로 자동 조인을 수행하므로, NO 외에 동일한 이름의 컬럼이 추가로 존재할 경우 조인 조건이 달라질 수 있어 의미가 다르다.

예제) EMP 테이블과 DEPT 테이블의 CROSS 조인 결과

SQL1 *	
1	▶ SELECT EMP.ENAME, DEPT.DNAME
2	FROM EMP CROSS JOIN DEPT;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

	ENAME	DNAME
1	SMITH	ACCOUNTING
2	ALLEN	ACCOUNTING
3	WARD	ACCOUNTING
4	JONES	ACCOUNTING
5	MARTIN	ACCOUNTING
6	BLAKE	ACCOUNTING
7	CLARK	ACCOUNTING

이하 생략

출력 내용이 많아 일부만 표시했으며, EMP 14 건과 DEPT 4 건의 모든 조합으로 총 56 건(14 × 4)이 출력됨

● OUTER JOIN

- INNER JOIN 과 대비되는 조인 방식으로, 조인 조건을 만족하지 않는 행도 결과에 포함하여 반환함
- INNER JOIN 에서는 조인 컬럼 값이 NULL 이거나 매칭되지 않으면 행이 제외되지만, OUTER JOIN 을 사용하면 해당 행도 출력할 수 있음
- 기준 테이블의 위치에 따라 LEFT / RIGHT / FULL OUTER JOIN 으로 구분됨
- **OUTER 키워드는 생략 가능함**(LEFT OUTER JOIN → LEFT JOIN)

● OUTER JOIN 종류

1. LEFT OUTER JOIN

- FROM 절에 나열된 왼쪽 테이블을 기준으로 결과를 생성함
- 왼쪽 테이블(기준 테이블)의 행은 모두 출력되며, 매칭되는 오른쪽 테이블의 행이 없을 경우 오른쪽 컬럼은 NULL 로 출력됨

2. RIGHT OUTER JOIN

- RIGHT JOIN 의 기준은 오른쪽 테이블임
- 오른쪽 테이블의 행은 모두 출력되며, 매칭되는 왼쪽 테이블의 행이 없을 경우 왼쪽 컬럼은 NULL 로 출력됨
- FROM 절의 테이블 순서를 변경하면 LEFT OUTER JOIN 으로 동일하게 표현 가능함

3. FULL OUTER JOIN

- 양쪽 테이블을 모두 기준으로 결과를 생성하는 조인 방식임
- 매칭되는 행은 한 번만 출력하고, 매칭되지 않는 행은 각각 NULL 을 포함한 상태로 모두 출력함
- **LEFT OUTER JOIN 결과와 RIGHT OUTER JOIN 결과를 UNION(중복 제거)한 것과 동일한 결과를 반환함**
- FULL OUTER JOIN 은 Oracle 구문((+))에서는 불가능하며, ANSI 표준 JOIN 문법에서만 지원됨

예제) STUDENT 테이블과 PROFESSOR 테이블을 OUTER JOIN 하여, 1 학년과 4 학년 학생의 이름, 학년, 지도 교수 이름을 출력

STUDENT TABLE				PROFESSOR TABLE			
STUDNO	NAME	GRADE	PROFNO	PROFNO	NAME	POSITION	DEPTNO
5001	강성근	4	1001	1001	강경연	정교수	101
5002	고대영	4	2001	1002	경윤영	조교수	101
5003	권정현	4	3002	1003	김다훈	전임강사	101
5004	김건일	4	4001	2001	노경우	전임강사	102
5005	김재현	4	4003	2002	이용준	조교수	102
8001	이유나	1		2003	장요한	정교수	102
8002	이민정	1		3001	전홍범	정교수	103
8003	전영훈	1		3002	정성용	조교수	103
8004	허민기	1		3003	백승윤	전임강사	103
8005	최훈	1		4001	한병두	정교수	201
				4002	현기숙	조교수	201
				4003	오하영	조교수	202
				4004	김만중	전임강사	202

일치하는 대상 없음

Oracle 표준 작성법)

```

SQL1 *
1 ▶ SELECT *
2   FROM STUDENT S, PROFESSOR P
3   WHERE S.PROFNO = P.PROFNO(+)
4   AND S.GRADE IN (1, 4);

```

- ☞ Oracle 구문에서는 WHERE 절의 조인 조건에서 (+) 기호를 사용하여 OUTER JOIN 을 작성함
- ☞ WHERE 절에서 기준이 되는 테이블(STUDENT)과 반대편 테이블의 조인 컬럼 뒤에 (+)를 붙임

ANSI 표준 작성법)

```

SQL1 *
1 ▶ SELECT S.STUDNO, S.NAME AS 학생명, S.GRADE, S.PROFNO,
2   P.PROFNO, P.NAME AS 교수명
3   FROM STUDENT S LEFT OUTER JOIN PROFESSOR P
4   ON S.PROFNO = P.PROFNO
5   WHERE S.GRADE IN (1, 4);

```

Result						
	STUDNO	학생명	GRADE	PROFNO	PROFNO	교수명
1	5001	강성근	4	1001	1001	강경연
2	5002	고대영	4	2001	2001	노경우
3	5003	권정현	4	3002	3002	정성용
4	5004	김건일	4	4001	4001	한병두
5	5005	김재현	4	4003	4003	오하영
6	8005	최훈	1			
7	8004	허민기	1			
8	8003	전영훈	1			
9	8002	이민정	1			
10	8001	이유나	1			

INNER JOIN 시 출력되지 X
LEFT OUTER JOIN 시 출력

- ☞ ANSI 표준에서는 FROM 절에서 테이블과 테이블 사이에 OUTER JOIN 을 명시함
- ☞ WHERE 절을 함께 사용하는 경우, JOIN ... ON 절 뒤에 위치해야 하며 작성 순서가 중요함

예제) FULL OUTER JOIN 결과

두 테이블을 ID 컬럼을 기준으로 FULL OUTER JOIN 한 결과는 총 _____행이다.

SQL1 *

```
1 ▶ SELECT * FROM FULL_TAB1;
```

Result

Grid Result Server Output Text Output

	ID	NAME
1	1	홍길동
2	2	김길동
3	3	박길동

SQL1 *

```
1 ▶ SELECT * FROM FULL_TAB2;
```

Result

Grid Result Server Output Text Output

	ID	SCORE
1	2	90
2	3	80
3	4	70

ANSI 표준 작성법)

SQL1 *

```
1 ▶ SELECT *
2   FROM FULL_TAB1 F1 FULL OUTER JOIN FULL_TAB2 F2
3   ON F1.ID = F2.ID
4   ORDER BY 1;
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	ID	NAME	ID	SCORE
1	1	홍길동		
2	2	김길동	2	90
3	3	박길동	3	80
4			4	70

LEFT OUTER JOIN 결과와 RIGHT OUTER JOIN 결과를 동시에 출력하며, 중복된 데이터는 한 번만 표시됨

Oracle 표준 작성법)

SQL1 *

```
1 ▶ SELECT *
2   FROM FULL_TAB1 F1, FULL_TAB2 F2
3   WHERE F1.ID = F2.ID(+)
4   UNION
5   SELECT *
6   FROM FULL_TAB1 F1, FULL_TAB2 F2
7   WHERE F1.ID(+) = F2.ID;
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	ID	NAME	ID	SCORE
1	2	김길동	2	90
2	3	박길동	3	80
3	1	홍길동		
4			4	70

양쪽 모두 존재

LEFT OUTER JOIN에만 존재

RIGHT OUTER JOIN에만 존재

Oracle의 구문에서는 FULL OUTER JOIN 문법을 지원하지 않으며, (+) 기호를 양쪽 컬럼에 동시에 사용할 경우 에러가 발생함

LEFT OUTER JOIN 결과와 RIGHT OUTER JOIN 결과를 UNION 하여 FULL OUTER JOIN 과 동일한 결과를 표현할 수 있음

기출) 다음 SQL 결과로 알맞은 것은?

고객

고객번호	이름	가입일
1	홍길동	2025/12/10
2	김길동	2025/10/25
3	박길동	2025/11/28
4	김길동	2025/09/20
5	최길동	2025/12/25

주문

주문번호	고객번호	상품번호
1000	2	A100
1001	2	B100
1002	3	C100
1003	4	A100
1004	3	B100

```
SELECT COUNT(*)
FROM 고객 LEFT OUTER JOIN 주문 ON (고객.고객번호 = 주문.고객번호);
```

- ① 4
- ② 5
- ③ 6
- ④ 7

정답 ④

해설: LEFT OUTER JOIN 은 왼쪽 테이블인 고객의 모든 행을 유지한다.

주문 정보가 없는 고객도 조인 결과에 포함되며, 이 경우 주문 컬럼은 NULL 로 채워진다.

고객번호별 조인 결과를 보면

고객번호 1 은 주문이 없어 1 행, 고객번호 2 는 주문이 2 건이므로 2 행,

고객번호 3 은 주문이 2 건이므로 2 행, 고객번호 4 는 주문이 1 건이므로 1 행,

고객번호 5 는 주문이 없어 1 행이 생성된다.

따라서 전체 행 수는 $1 + 2 + 2 + 1 + 1 = 7$ 이며, COUNT(*) 결과는 7 이다.

<< 단원 정리 문제 >>

1. 다음 SQL 실행 결과로 가장 적절한 것은?

<TAB1>		<TAB2>	
NO	NAME	NO	SALES
1	A	1	100
2	B	3	200
3	C	4	300
4	D	5	400
NULL	E		


```

SELECT COUNT(TAB1.NO) AS R1,
       COUNT(TAB2.NO) AS R2
FROM TAB1 LEFT OUTER JOIN TAB2
      ON TAB1.NO = TAB2.NO;

```

- ① 4, 2
 ② 4, 3
 ③ 5, 2
 ④ 5, 3

2. 다음 두 SQL 실행 결과를 순서대로 나열한 것은?

<TAB1>		<TAB2>	
NO	NAME	NO	SALES
1	A	1	100
2	B	3	200
3	C	4	300
NULL	E	NULL	100


```

SELECT COUNT(NO)
FROM TAB1 NATURAL JOIN TAB2;

SELECT COUNT(*)
FROM TAB1 CROSS JOIN TAB2;

```

- ① 2, 9
 ② 2, 16
 ③ 3, 9
 ④ 3, 19

1. ②

TAB1 을 기준으로 모든 데이터는 생략 없이 출력되지만, NULL 값은 COUNT 되지 않기 때문에 R1 의 결과는 4 이며, TAB2 에서는 NO 가 1, 3, 4 인 행만 값을 가지고, 나머지 행은 NULL 로 반환되므로 R2 의 결과는 3 이다.

2. ②

NATURAL JOIN 의 결과는 NO 값이 양쪽 테이블에서 동일한 1 과 3 인 행만 출력되므로, R1 의 결과는 2 건이 된다. CROSS JOIN 의 경우, NULL 과 상관없이 모든 경우의 수가 출력되므로 총 16 건이 리턴된다.

단원 종합 문제

1. 다음 중 테이블에 대한 설명으로 가장 적절한 것은?

- ① 테이블명은 숫자로 시작할 수 있다.
- ② 테이블명에는 모든 특수기호(!, _, -, ? 등)를 사용할 수 없다.
- ③ Oracle에서는 테이블 생성 시 소유자를 별도로 명시하지 않으면, 해당 테이블은 생성한 사용자(User)의 소유가 된다.
- ④ Oracle에서는 본인 소유의 테이블에 대해 SELECT, INSERT, UPDATE, DELETE 권한을 다른 사용자에게 임의로 부여할 수 없다.

2. 다음 중 SQL 분류로 적절하지 않은 것은?

- ① DDL - TRUNCATE
- ② DML - MERGE
- ③ DCL - GRANT
- ④ DCL - ROLLBACK

3. SELECT문에 대한 설명 중 가장 적절하지 않은 것은? (단, DBMS는 오라클)

- ① SELECT 절에서는 *와 컬럼을 동시에 사용할 수 없다.
- ② DISTINCT는 단 한번만 사용 가능하다.
- ③ 컬럼 별칭은 항상 AS를 사용하여 정의 가능하다.
- ④ 테이블 별칭은 AS를 사용하지 않는다.

4. 다음 날짜 함수 결과로 적절하지 않은 것은?

단, 오늘 날짜는 2025년 11월 17이다.

- ① TO_DATE('12','MM'): '2025/12/01'
- ② TO_DATE('2025/12','YYYY/MM'): '2025/12/01'
- ③ TO_DATE('2025','YYYY'): '2025/01/01'
- ④ TO_DATE('12','DD'): '2025/11/12'

5. 다음 중 함수 수행 결과가 다른 것은?

COL1	COL2
10	10
20	10
NULL	20

- ① NVL(COL1, 20)
- ② COALESCE(COL1, COL2)
- ③ ISNULL(COL1, 20)
- ④ DECODE(COL1, NULL, 20)

6. 다음 함수의 결과로 적절하지 않은 것은?

- ① SIGN(-100): -1
- ② MOD(7, 3): 1
- ③ CEIL(-2.7): -3
- ④ FLOOR(2.5): 2

7. 다음 함수의 결과로 적절하지 않은 것은?

- ① SUBSTR('SQL SERVER TOOL', 4, 5): ' S'
- ② RPAD(' ', 4, '*'): NULL
- ③ RTRIM('SQL SERVER TOOL', 'L'): 'SQL SERVER TOO'
- ④ LENGTH(REPLACE('SQL SERVER TOOL', 'TOOL')): 11

8. 다음 SQL 중 실행이 불가능 한 구간을 고르시오.

- ① SELECT COL1 C1, SUM(COL3) AS SUM
- ② FROM TABLE1 T1
- ③ WHERE SUM(COL1) >= 100
- ④ GROUP BY COL1, COL2;

9. 아래 SQL의 실행 결과로 알맞은 것은?

<TAB1>		
COL1	COL2	COL3
10	1	NULL
10	2	10
NULL	1	20
0	3	30
20	3	30


```

SELECT SUM(COL1 + COL2)
  FROM TAB1
 WHERE COL3 != 30;

```

- ① NULL
- ② 12
- ③ 13
- ④ 24

10. 다음 문장 수행 결과로 적절한 것은?

<TAB1>

COL1	COL2
A	NULL
A	10
B	20
C	30
C	NULL
C	20
NULL	10
NULL	30

```
SELECT COL1, SUM(COL2) AS CSUM
FROM TAB1
GROUP BY COL1
HAVING COUNT(COL2) = 2;
```

①

COL1	CSUM
A	10

②

COL1	CSUM
C	50

③

COL1	CSUM
C	50
NULL	40

④

COL1	CSUM
A	10
C	50
NULL	30

11. 아래 SQL 수행 결과로 가장 적절한 것은?

<TAB1>

COL1	COL2
A	10
B	20
NULL	30
D	20

<TAB2>

COL1	COL2
A	20
B	20
C	10
NULL	30

```
SELECT COUNT(*)
FROM TAB1 CROSS JOIN TAB2;
```

① 1

② 2

③ 9

④ 16

12. 아래 SQL 수행 결과로 가장 적절한 것은?

<TAB1>

TEL
234-4567
1229-9384
567-8839
2569-4856

```
SELECT SUBSTR(TEL, 1, INSTR(TEL,'-')-1) AS NO
FROM TAB1
ORDER BY NO;
```

①

NO
234
567
1229
2569

②

NO
234-
567-
1229-
2569-

③

NO
1229
234
2569
567

④

NO
1229-
234-
2569-
567-

13. 다음 문장 중 실행이 가능한 것은?

①

```
SELECT T.COL1, T.COL2, SUM(T.COL3) TOTAL
FROM TAB1 T
WHERE T.COL1 != 10
GROUP BY T.COL2;
```

②

```
SELECT COL1, COL2, SUM(COL3) TOTAL
FROM TAB1
GROUP BY COL1, COL2
HAVING SUM(COL3) > 100;
```

③

```
SELECT COL1, SUM(COL2) TOTAL
FROM TAB1
GROUP BY COL1
ORDER BY COL2;
```

④

```
SELECT COL1, COL2, SUM(COL2) TOTAL
FROM TAB1
WHERE TOTAL > 100
GROUP BY COL1, COL2, COL3
ORDER BY TOTAL;
```

14. 아래 SQL 수행 결과로 가장 적절한 것은?

<상품>

코드	후기
100	50
101	0
102	10
103	45
104	60
105	120
106	55

```

SELECT CASE WHEN 후기 < 20 THEN '< 20'
            WHEN 후기 < 100 THEN '< 100'
            ELSE '>= 100'
        END AS 후기수,
        COUNT(후기) AS 상품수
FROM 상품
GROUP BY CASE WHEN 후기 < 20 THEN '< 20'
            WHEN 후기 < 100 THEN '< 100'
            ELSE '>= 100'
        END,
        CASE WHEN 후기 < 20 THEN 1
            WHEN 후기 < 100 THEN 2
            ELSE 3
        END
ORDER BY CASE WHEN 후기 < 20 THEN 1
            WHEN 후기 < 100 THEN 2
            ELSE 3
        END;

```

①

후기수	상품수
< 20	2
< 100	4
>= 100	1

②

후기수	상품수
< 100	4
< 20	2
>= 100	1

③

후기수	상품수
>= 100	1
< 100	4
< 20	2

④

후기수	상품수
>= 100	1
< 20	2
< 100	4

15. 아래 SQL 수행 결과로 가장 적절한 것은?

<TAB1>		<TAB2>	
COL1	COL2	COL1	COL2
A	10	A	20
B	20	B	20
NULL	30	C	10
D	20	NULL	30


```

SELECT SUM(TAB2.COL2)
  FROM TAB1 LEFT OUTER JOIN TAB2
    USING (COL1);

```

- ① NULL
- ② 40
- ③ 50
- ④ 70

16. 조인에 대한 설명으로 가장 적절한 것은?

- ① NATURAL JOIN 시 ON절을 사용할 수 없다.
- ② FULL OUTER JOIN은 LEFT OUTER JOIN과 RIGHT OUTER JOIN 결과의 합집합(UNION ALL) 결과와 같다.
- ③ USING절에는 괄호를 생략할 수 있다.
- ④ ORACLE에서는 FROM절의 테이블 순서에 따라 LEFT OUTER JOIN 결과가 달라진다.

17. 다음 중 NULL 로만 구성된 컬럼에 대한 연산 결과가 다른 하나는?

- ① COUNT(컬럼)
- ② SUM(컬럼)
- ③ AVG(컬럼)
- ④ MAX(컬럼)

18. 다음 테이블에 대해 COL1에 대해 LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN한 결과를 모두 합한 것은?

<TAB1>		<TAB2>	
COL1	COL2	COL1	COL2
A	10	A	20
B	20	B	20
NULL	30	C	10
NULL	30	NULL	30


```

SELECT COUNT(*)
  FROM TAB1 _____ TAB2 ON TAB1.COL1 = TAB2.COL1;

```

- ① 8
- ② 10
- ③ 12
- ④ 14

19. 아래 결과를 얻기 위한 SQL로 가장 적절한 것은?

<고객>

고객번호	이름	포인트
100	홍길동	80
101	박길동	110
102	최길동	150
103	김길동	500
104	안길동	900

<상품>

상품번호	상품명	최소포인트	최대포인트
1000	양말	101	200
1001	세탁세제	201	300
1002	식기세트	301	400
1003	전기담요	401	500
1004	세탁기	501	1000

<결과>

이름	상품명
홍길동	NULL
박길동	양말
최길동	양말
김길동	전기담요
안길동	세탁기

①

```
SELECT 이름, 상품명
FROM 고객, 상품
WHERE 포인트 >= 최소포인트;
```

②

```
SELECT 이름, 상품명
FROM 고객, 상품
WHERE 포인트 BETWEEN 최소포인트 AND 최대포인트;
```

③

```
SELECT 이름, 상품명
FROM 고객, 상품
WHERE 포인트 >= 최소포인트(+);
```

④

```
SELECT 이름, 상품명
FROM 고객, 상품
WHERE 포인트 BETWEEN 최소포인트(+) AND 최대포인트(+);
```

20. 다음 세 SQL 결과의 합으로 가장 적절한 것은?

<TAB1>

COL1	COL2
A	10
B	20
NULL	30
NULL	30

<TAB2>

COL1	COL2
A	10
A	NULL
C	10
NULL	30

- 1) SELECT COUNT(TAB2.COL2)
FROM TAB1 NATURAL JOIN TAB2;
- 2) SELECT COUNT(TAB2.COL2)
FROM TAB1 CROSS JOIN TAB2;
- 3) SELECT COUNT(TAB2.COL2)
FROM TAB1 JOIN TAB2 ON TAB1.COL1 = TAB2.COL1;

- ① 10
- ② 14
- ③ 18
- ④ 20

2-9. 서브쿼리

● 서브쿼리

- 하나의 SQL 문 안에 포함되어 있는 또 다른 SQL 문을 말함
- 반드시 괄호로 묶어서 사용함
- SELECT 문 안에 SELECT 문이 포함되는 형태이거나, INSERT, UPDATE, DELETE 문 안에 포함된 SELECT 문을 의미함

● 서브쿼리 사용 가능한 곳

- SELECT 절
- FROM 절
- WHERE 절
- HAVING 절
- ORDER BY 절
- INSERT, UPDATE, DELETE 와 같은 DML 문 내부

※ GROUP BY 절에서는 서브쿼리 사용 불가

● 서브 쿼리 종류

1. 동작하는 방식에 따라

1) UN-CORRELATED(비연관) 서브쿼리

- 서브쿼리에 메인쿼리(외부쿼리) 테이블의 컬럼을 가지고 있지 않은 형태의 서브쿼리
- 서브쿼리가 메인쿼리의 값을 참조하지 않고 독립적으로 실행함
- 서브쿼리는 한 번만 실행됨

2) CORRELATED(연관) 서브쿼리

- 서브쿼리에 메인쿼리(외부쿼리) 테이블의 컬럼을 가지고 있는 형태의 서브쿼리
- 서브쿼리가 메인쿼리의 컬럼을 참조하고 있기 때문에 **서브쿼리의 실행이 메인쿼리와 독립적이지 않음**
- **메인 쿼리의 각 행에 대해 서브쿼리가 실행됨**

예제) 연관 서브쿼리와 비연관 서브쿼리 비교

SQL1 *	
1	SELECT ENAME, SAL
2	FROM EMP E1
3	WHERE E1.SAL > (SELECT AVG(E2.SAL)
4	FROM EMP E2);

비연관 서브쿼리

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

ENAME	SAL
1 JONES	2975
2 BLAKE	2850
3 CLARK	2450
4 SCOTT	3000
5 KING	5000
6 FORD	3000

☞ SAL 이라는 컬럼이 동일하게 사용된 것처럼 보이지만, 메인쿼리는 E1 테이블의 SAL 을 사용하고 서브쿼리는 E2 테이블의 SAL 을 사용하기 때문에 서로 독립적으로 수행됨

☞ 서브쿼리가 먼저 실행되어 SAL 의 평균값을 구한 뒤, 해당 값을 메인쿼리에 전달하면서 데이터를 선택함

SQL1 *	
1	SELECT ENAME, SAL
2	FROM EMP E1
3	WHERE E1.SAL > (SELECT AVG(E2.SAL)
4	FROM EMP E2
5	WHERE E2.DEPTNO = E1.DEPTNO);

연관 서브쿼리

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

ENAME	SAL
1 ALLEN	1600
2 JONES	2975
3 BLAKE	2850
4 SCOTT	3000
5 KING	5000
6 FORD	3000

☞ 서브쿼리에서 메인쿼리의 컬럼인 E1.DEPTNO 를 참조하고 있기 때문에 서로 독립적으로 수행되지 않고 메인쿼리의 각 행마다 서브쿼리가 실행됨

기출) 다음 중 연관 서브쿼리와 비연관 서브쿼리에 대한 설명으로 옳은 것은?

- ① 연관 서브쿼리는 메인쿼리와 독립적으로 한 번만 실행된다.
- ② 비연관 서브쿼리는 메인쿼리의 각 행마다 반복 실행된다.
- ③ 연관 서브쿼리는 메인쿼리의 컬럼을 참조하므로 독립적으로 수행될 수 없다.
- ④ 비연관 서브쿼리는 반드시 EXISTS 연산자와 함께 사용해야 한다.

정답 ③

해설: 연관 서브쿼리는 서브쿼리에서 메인쿼리의 컬럼을 참조하므로 메인쿼리와 독립적으로 실행될 수 없으며, 메인쿼리의 각 행마다 반복 실행된다.

2. 위치에 따라

1) 스칼라 서브쿼리

- SELECT 절에서 사용되는 서브쿼리를 말함
- 반드시 **단일 행, 단일 컬럼**을 반환해야 함
- 서브쿼리의 실행 결과는 하나의 값으로 반환되며, **컬럼처럼 사용됨**
- 주로 조회 결과에 계산값이나 추가 정보를 함께 출력할 때 사용됨

** 문법

```

SQL1 *
1 SELECT * | 컬럼명 | 표현식,
2   (SELECT * | 컬럼명 | 표현식
3     FROM 테이블명 또는 뷰명
4     WHERE 조건)
5 FROM 테이블명 또는 뷰명;

```

2) 인라인뷰

- FROM 절에서 사용되는 서브쿼리를 말함
- 서브쿼리의 실행 결과를 하나의 가상 테이블처럼 사용함
- 복잡한 SQL 문을 단계적으로 작성할 때 주로 사용함

** 문법

```

SQL1 *
1 SELECT * | 컬럼명 | 표현식
2   FROM (SELECT * | 컬럼명 | 표현식
3         FROM 테이블명 또는 뷰명)
4   WHERE 조건;

```

3) WHERE 절 서브쿼리

- 가장 일반적인 서브쿼리
- 비교 상수 자리에 **값을 전달하기 위한 목적으로 주로 사용함(상수항의 대체)**
- 리턴 데이터의 형태에 따라 단일행 서브쿼리, 다중행 서브쿼리, 다중컬럼 서브쿼리, 상호연관 서브쿼리로 구분됨

** 문법

```

SQL1 *
1 SELECT * | 컬럼명 | 표현식
2   FROM 테이블명 또는 뷰명
3   WHERE 조건대상 연산자 (SELECT * | 컬럼명 | 표현식
4     FROM 테이블명 또는 뷰명
5     WHERE 조건);

```

● WHERE 절 서브쿼리 종류

1. 단일행 서브쿼리

- 서브쿼리 결과가 1 개의 행이 리턴되는 형태

** 단일행 서브쿼리 연산자 종류

연산자	의미
=	같다
<>	같지 않다
>	크다
>=	크거나 같다
<	작다
<=	작거나 같다

예제) EMP 테이블에서 전체 직원의 평균 급여보다 높은 급여를 받는 직원의 정보 출력

STEP1) 비교대상(전체 직원 급여 평균) 확인

SQL1 *	
1	SELECT AVG(SAL)
2	FROM EMP;

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
	AVG(SAL)
1	2073.214285714285714285714285714286

STEP2) 메인쿼리에 비교 상수 자리에 서브쿼리 결과 전달

SQL1 *	
1	SELECT EMPNO, ENAME, SAL
2	FROM EMP
3	WHERE SAL > (SELECT AVG(SAL)
4	FROM EMP);

Result	
Grid Result Server Output Text Output Explain Plan Statistics	
	EMPNO ENAME SAL
1	7566 JONES 2975
2	7698 BLAKE 2850
3	7782 CLARK 2450
4	7788 SCOTT 3000
5	7839 KING 5000
6	7902 FORD 3000

2. 다중행 서브쿼리

- 서브쿼리 결과가 여러 행이 리턴되는 형태
- '=', '>', '<'와 같은 비교 연산자 사용불가(여러 값이랑 비교할 수 없는 연산자들)
- 서브쿼리 결과를 하나로 요약하거나 다중행 서브쿼리 연산자를 사용

** 다중행 서브쿼리 연산자

연산자	의미
IN	같은 값을 찾음
>ANY	최소값을 반환함
<ANY	최대값을 반환함
<ALL	최소값을 반환함
>ALL	최대값을 반환함
EXISTS	서브쿼리가 하나 이상의 결과 반환시 TRUE
NOT EXISTS	서브쿼리가 결과를 반환하지 않으면 TRUE

예제) IN 연산자를 사용하여 서울에 위치한 부서 소속의 직원 이름 출력하기

<EMP3>

	NAME	SAL	DEPTNO
1	DAVID	2500	10
2	JANE	3000	20
3	ALICE	2800	30
4	BOB	2200	10
5	OLIVIA	3500	20
6	SMITH	3500	40

<DEPT3>

	DEPTNO	DNAME	LOC
1	10	HR	SEOUL
2	20	IT	SUWON
3	30	SALES	BUSAN
4	40	MARKETING	SEOUL

잘못된 연산자 사용)

SQL1 *	
1	SELECT NAME
2	FROM EMP3
3	WHERE DEPTNO = (SELECT DEPTNO
4	FROM DEPT3
5	WHERE LOC = 'SEOUL');

Result	4
ORA-01427: 단일 행 하위 질의에 2개 이상의 행이 리턴되었습니다. Col : 1	

☞ 서브쿼리는 SEOUL 에 위치한 부서의 부서번호를 출력하고 있으므로 10, 40 두 개의 행이 리턴됨
 하지만 서브쿼리 앞에 연산자가 '='이므로 두 개의 행과 비교할 연산자로 적합하지 않아 에러가 발생함

해결) = 연산자 대신 IN 연산자로 변경

SQL1 *

```

1 SELECT NAME
2 FROM EMP3
3 WHERE DEPTNO IN (SELECT DEPTNO
4 FROM DEPT3
5 WHERE LOC = 'SEOUL');

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	NAME
1	BOB
2	DAVID
3	SMITH

☞ '=' 대신 IN 연산자로 대체하면 부서번호가 10 번 또는 40 번인 서울에 위치한 부서에 소속된 직원의 이름을 EMP3 테이블에서 출력할 수 있음

● ALL 과 ANY 연산자

- 다중행 서브쿼리에서 여러 개의 리턴값과 비교할 때 사용하는 연산자
- >, >=, <, <= 연산자와 함께 사용될 경우 서브쿼리 결과를 최대값 또는 최소값 중 하나의 값으로 요약하여 비교함

구분	상황	해석
ANY	= ANY(서브쿼리)	서브쿼리 결과 중 하나라도 조건을 만족하면 참
	>= ANY(서브쿼리)	서브쿼리 결과 중 최소값보다 크면 참
	<= ANY(서브쿼리)	서브쿼리 결과 중 최대값보다 작으면 참
ALL	= ALL(서브쿼리)	서브쿼리 결과의 모든 값이 조건을 만족해야 참
	>= ALL(서브쿼리)	서브쿼리 결과 중 최대값보다 커야 참
	<= ALL(서브쿼리)	서브쿼리 결과 중 최소값보다 작아야 참

예제) 다중행 서브쿼리 연산자 오류

10 번 부서원들의 급여보다 많은 급여를 받는 직원의 사번, 이름, 급여를 출력하기 위한 시도

SQL1 *

```

1 SELECT EMPNO, ENAME, SAL
2 FROM EMP
3 WHERE SAL > (SELECT SAL
4 FROM EMP
5 WHERE DEPTNO = 10);

```

Result

ORA-01427: 단일 행 하위 질의에 2개 이상의 행이 리턴되었습니다. Col: 1

☞ 서브쿼리 결과로 3 개의 행이 리턴되므로, 3 개의 SAL 값과 대소 비교를 하게 되어 에러가 발생함

해결 1) 서브쿼리 결과가 하나의 행으로 출력되도록 변경

10 번 부서원들의 급여 중 최솟값(1300)보다 높은 급여를 받는 직원 출력

SQL1 *

```

1 SELECT EMPNO, ENAME, SAL
2 FROM EMP
3 WHERE SAL > (SELECT MIN(SAL)
4 FROM EMP
5 WHERE DEPTNO = 10);

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	EMPNO	ENAME	SAL
1	7499	ALLEN	1600
2	7566	JONES	2975
3	7698	BLAKE	2850
4	7782	CLARK	2450
5	7788	SCOTT	3000
6	7839	KING	5000
7	7844	TURNER	1500
8	7902	FORD	3000

SQL1 *

```

1 SELECT MIN(SAL)
2 FROM EMP
3 WHERE DEPTNO = 10;
4

```

Result

Grid Result Server Output Text Output

	MIN(SAL)
1	1300

출력 요건에 맞게 적절한 집계 함수를 사용하여 서브쿼리 결과를 하나로 요약한 후, 메인쿼리와 비교함

해결 2) 다중행 서브쿼리 연산자로 변경

SQL1 *

```

1 SELECT EMPNO, ENAME, SAL
2 FROM EMP
3 WHERE SAL > ANY(SELECT SAL
4 FROM EMP
5 WHERE DEPTNO = 10);

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	EMPNO	ENAME	SAL
1	7839	KING	5000
2	7902	FORD	3000
3	7788	SCOTT	3000
4	7566	JONES	2975
5	7698	BLAKE	2850
6	7782	CLARK	2450
7	7499	ALLEN	1600
8	7844	TURNER	1500

SQL1 *

```

1 SELECT SAL
2 FROM EMP
3 WHERE DEPTNO = 10;
4

```

Result

Grid Result Server Output Text Output

	SAL
1	2450
2	5000
3	1300

ANY를 사용하여 10 번 부서원들의 급여 중 최솟값보다 급여가 큰 직원들을 찾는 조건으로 해석됨

이때 Oracle 은 내부적으로 정렬을 수행한 후 비교하므로, 출력 결과가 SAL 이 큰 순서대로 출력되고 있음

예제) 2025 년 1 월에 판매되지 않은 책 이름 출력하기

<BOOKS>

BOOK_ID	BOOK_NAME	PRICE
1	파이썬 기초	15000
2	SQLD	20000
3	정보처리기사	25000
4	ADsP	18000
5	빅데이터분석기사	22000

<BOOK_SALES>

SALES_ID	BOOK_ID	QUANTITY	SALE_DATE
1	1	3	2024/11/29 00:00:00
2	2	5	2024/12/01 00:00:00
3	3	2	2025/01/06 00:00:00
4	4	4	2025/01/23 00:00:00
5	5	1	2025/01/23 00:00:00

SQL1 *

```

1 SELECT BOOK_NAME
2 FROM BOOKS
3 WHERE BOOK_ID NOT IN (SELECT BOOK_ID
4                        FROM BOOK_SALES
5                        WHERE SALE_DATE BETWEEN TO_DATE('2025/01/01', 'YYYY/MM/DD')
6                        AND TO_DATE('2025/01/31', 'YYYY/MM/DD'));

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

BOOK_NAME
1 파이썬 기초
2 SQLD

서브쿼리에서 1 월에 판매된 BOOK_ID 를 출력하고, 해당 BOOK_ID 에 포함되지 않는 책을 선택하여 1 월에 판매되지 않은 책 이름을 출력할 수 있음

기출) 다음 SQL 의 실행 결과로 알맞은 것은?

TAB

ID	POINT	CLASS
1	10	A
2	20	A
3	30	A
4	15	B
5	40	B

```

SELECT SUM(POINT)
FROM TAB
WHERE POINT >= ALL(SELECT POINT FROM TAB WHERE CLASS = 'A')

```

- ① 30
- ② 40
- ③ 70
- ④ 100

정답 ③

해설: >= ALL 은 서브쿼리 결과의 최댓값 이상을 의미하므로 POINT >= 30 으로 해석됨
따라서 30 과 40 만 합산되어 SUM(POINT) = 70 이다.

● EXISTS / NOT EXISTS 연산자

- 서브쿼리 조건의 참, 거짓 결과에 따라 메인쿼리의 출력 결과가 달라짐
- 서브쿼리 **SELECT 절의 출력 형태는 중요하지 않음**
- 연관 서브쿼리를 사용하여 **연관 조건의 결과에 따라 메인쿼리 출력을 제한할 수 있음**

구분	설명
EXISTS	서브쿼리 결과가 하나라도 존재하면 메인쿼리 출력
NOT EXISTS	서브쿼리 결과가 존재하지 않으면 메인쿼리 데이터 출력

예제) EXISTS 사용

SQL1 *	
1	SELECT ENAME, SAL
2	FROM EMP
3	WHERE DEPTNO = 10
4	AND EXISTS (SELECT 1
5	FROM DEPT);

Result	
Grid Result	Server Output
Text Output	Explain
ENAME	SAL
1 CLARK	2450
2 KING	5000
3 MILLER	1300

☞ 서브쿼리 결과가 참이므로 메인쿼리의 모든 행이 출력됨

SQL1 *	
1	SELECT ENAME, SAL
2	FROM EMP
3	WHERE DEPTNO = 10
4	AND EXISTS (SELECT 1
5	FROM DEPT
6	WHERE 1=2);

Result	
Grid Result	Server Output
Text Output	Explain
ENAME	SAL

☞ 서브쿼리 결과가 거짓이므로 아무것도 출력되지 않음

예제) NOT EXISTS 사용

SQL1 *	
1	SELECT ENAME, SAL
2	FROM EMP
3	WHERE DEPTNO = 10
4	AND NOT EXISTS (SELECT 1
5	FROM DEPT);

Result	
Grid Result	Server Output
Text Output	Explain
ENAME	SAL

☞ 서브쿼리 결과가 참이므로 NOT EXISTS 에 의해 메인쿼리의 모든 행이 출력되지 않음

SQL1 *	
1	SELECT ENAME, SAL
2	FROM EMP
3	WHERE DEPTNO = 10
4	AND NOT EXISTS (SELECT 1
5	FROM DEPT
6	WHERE 1=2);

Result	
Grid Result	Server Output
Text Output	Explain
ENAME	SAL
1 CLARK	2450
2 KING	5000
3 MILLER	1300

☞ 서브쿼리 결과가 거짓이므로 반대로 10 번 부서원의 행이 출력됨

3. 다중컬럼 서브쿼리

- 서브쿼리 결과가 2 개 이상의 컬럼으로 리턴되는 형태
- 메인쿼리에서는 **여러 컬럼을 동시에 비교하기 위해 사용함**(비교 대상 컬럼의 개수는 반드시 동일해야 함)
- **단일 컬럼 비교 연산자는 사용할 수 없음**(>, >=, <=, <, =, != 등)

예제) EMP 테이블에서 부서별 최대 급여자 확인

STEP1) 부서별 최대 급여 확인

SQL1 *	
1	SELECT DEPTNO, MAX(SAL)
2	FROM EMP
3	GROUP BY DEPTNO;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	
DEPTNO	MAX(SAL)
1	30 2850
2	20 3000
3	10 5000

부서별로 급여를 가장 많이 받는 직원을 찾기 위해 먼저 부서별 최대급여를 확인함

STEP2) 메인쿼리에 비교 대상으로 서브쿼리 결과 전달

```
SQL1 *
1 SELECT EMPNO, ENAME, SAL, DEPTNO
2 FROM EMP
3 WHERE (DEPTNO, SAL) IN (SELECT DEPTNO, MAX(SAL)
4 FROM EMP
5 GROUP BY DEPTNO);
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	EMPNO	ENAME	SAL	DEPTNO
1	7698	BLAKE	2850	30
2	7902	FORD	3000	20
3	7788	SCOTT	3000	20
4	7839	KING	5000	10

부서별 최대 급여가 여러 값이 나오므로 비교 시에는 다중행 연산자인 IN 을 사용(= 사용 시 에러 발생)

예제) 각 부서별로 가장 최근에 입사한 직원의 이름, 부서번호, 입사일 출력하기

SQL1 *

1

2

3

4

5

SELECT *

FROM EMP

WHERE (DEPTNO, HIREDATE) IN (SELECT DEPTNO, MAX(HIREDATE)

FROM EMP

GROUP BY DEPTNO);

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7900	JAMES	CLERK	7698	1981/12/03 00:00:00	950		30
2	7876	ADAMS	CLERK	7788	1987/05/23 00:00:00	1100		20
3	7934	MILLER	CLERK	7782	1982/01/23 00:00:00	1300		10

서브쿼리를 사용해 먼저 부서별 최근 입사날짜를 구한 뒤, 이와 일치하는 대상을 메인쿼리에서 찾아냄

4. 연관 서브쿼리

- 메인쿼리와 서브쿼리가 서로 연결되어 비교를 수행하는 형태의 서브쿼리
- 서브쿼리에서 메인쿼리의 컬럼을 참조하여 조건을 비교함
- 이로 인해 서브쿼리는 메인쿼리와 독립적으로 실행되지 않음(메인쿼리의 각 행마다 서브쿼리가 반복 실행됨)

예제) 소속 부서의 평균 급여보다 높은 급여를 받는 사원 정보 출력

에러 발생) 다중컬럼 서브쿼리는 두 컬럼에 대해 동시에 대소 비교 불가

SQL1 *

```

1 SELECT EMPNO, ENAME, SAL, DEPTNO
2 FROM EMP
3 WHERE (DEPTNO, SAL) > (SELECT DEPTNO, AVG(SAL)
4 FROM EMP
5 GROUP BY DEPTNO);

```

Result

ORA-01796: 연산자의 지정이 부적합합니다 Ln 5, Col 43 0.01 sec.

위 SQL 은 (DEPTNO, SAL)과 (DEPTNO, AVG(SAL))을 동시에 대소 비교하려는 형태로,
다중컬럼 서브쿼리에서는 두 컬럼에 대해 대소 비교를 수행할 수 없기 때문에 에러가 발생함

해결방법)

1. 메인쿼리에서는 사원의 급여(SAL)를 기준으로 비교하고, 서브쿼리에서는 해당 사원이 속한 부서의 평균급여만 구해야 함
2. 따라서 메인쿼리의 부서번호를 서브쿼리에서 참조하는 **연관 서브쿼리 형태로 변경해야 함**

결과)

SQL1 *

```

1 SELECT EMPNO, ENAME, SAL, DEPTNO
2 FROM EMP E1
3 WHERE SAL > (SELECT AVG(SAL)
4 FROM EMP E2
5 WHERE E1.DEPTNO = E2.DEPTNO
6 GROUP BY DEPTNO);

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	EMPNO	ENAME	SAL	DEPTNO
1	7499	ALLEN	1600	30
2	7566	JONES	2975	20
3	7698	BLAKE	2850	30
4	7788	SCOTT	3000	20
5	7839	KING	5000	10
6	7902	FORD	3000	20

- 메인쿼리와 결과적으로 비교해야 할 컬럼은 SAL 이므로 메인쿼리에는 SAL 만 두고 비교를 수행해야 함
- 대신 부서별 평균 급여를 기준으로 비교해야 하므로, GROUP BY 결과에 포함된 DEPTNO 정보가 서로 일치하는 경우에만 SAL 에 대한 비교가 이루어져야 함. 이를 위해 DEPTNO 에 대한 조건은 서브쿼리로 전달함
- 메인쿼리에는 서브쿼리의 테이블 정보가 없으므로 (순서상 메인쿼리부터 해석되기 때문에)
E.DEPTNO = D.DEPTNO 와 같은 조건은 사용할 수 없음

** 상호연관 서브쿼리 해석 순서

1. 메인쿼리 테이블을 먼저 읽음
2. 메인쿼리 WHERE 절에서 조건을 확인함
이때 비교 대상이 되는 컬럼은 SAL 임
3. SAL 비교에 필요한 값을 구하기 위해 서브쿼리를 실행함
4. 서브쿼리 실행 시, 메인쿼리의 현재 행에 해당하는
E1.DEPTNO 값을 서브쿼리에서 요구함
5. 전달받은 E1.DEPTNO 값을 기준으로
서브쿼리의 DEPTNO 컬럼과 비교하여 조건절을 완성함
6. 해당 조건을 만족하는 행들에 대해 그룹 연산을 수행하고 AVG(SAL) 값을 계산함
7. 계산된 평균 급여 값을 메인쿼리에 다시 전달하여
현재 행의 SAL 과 비교한 뒤, 조건을 만족하는 행만 추출함

※ 이 과정이 메인쿼리의 각 행마다 반복 수행됨

※ **연관 조건이 서브쿼리 WHERE 절에 포함되므로 GROUP BY 는 생략 가능함**

예제) EXISTS / NOT EXISTS 연관 서브쿼리

<EXISTS_TAB1>				<EXISTS_TAB2>		
	NO	NAME	PRICE		NO	SALE
1	1	AMERICANO	1000	1	2	10
2	2	LATTE	2000	2	4	15
3	3	MOCHA	3000			
4	4	CAPPUCCINO	3500			

SQL1 *

```

1 SELECT *
2 FROM EXISTS_TAB1 E1
3 WHERE EXISTS (SELECT
4               FROM EXISTS_TAB2 E2
5               WHERE E1.NO = E2.NO);

```

SELECT 절의 표현은 자유롭게 전달가능
(* 또는 1, 여러 컬럼도 가능)

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	NO	NAME	PRICE
1	2	LATTE	2000
2	4	CAPPUCCINO	3500

- ☞ 연관 서브쿼리는 메인쿼리의 각 행이 실행될 때마다 서브쿼리가 함께 실행되며,
E1.NO = E2.NO 조건의 참/거짓 여부에 따라 메인쿼리의 출력 여부가 결정됨
- ☞ 따라서 제품번호(NO)가 EXISTS_TAB2 에 존재하는 경우에만 해당 제품이 출력됨

예제) EXISTS / NOT EXISTS 연산자를 사용하여 2025년 1월에 판매되지 않은 책 이름 출력하기

<BOOKS>

BOOK_ID	BOOK_NAME	PRICE
1	파이썬 기초	15000
2	SQLD	20000
3	정보처리기사	25000
4	ADsP	18000
5	빅데이터분석기사	22000

<BOOK_SALES>

SALES_ID	BOOK_ID	QUANTITY	SALE_DATE
1	1	3	2024/11/29 00:00:00
2	2	5	2024/12/01 00:00:00
3	3	2	2025/01/06 00:00:00
4	4	4	2025/01/23 00:00:00
5	5	1	2025/01/23 00:00:00

SQL1 *

```

1 SELECT BOOK_NAME
2 FROM BOOKS B1
3 WHERE NOT EXISTS (SELECT BOOK_ID
4                     FROM BOOK_SALES B2
5                     WHERE B1.BOOK_ID = B2.BOOK_ID
6                       AND SALE_DATE BETWEEN TO_DATE('2025/01/01', 'YYYY/MM/DD')
7                       AND TO_DATE('2025/01/31', 'YYYY/MM/DD'));

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	BOOK_NAME
1	파이썬 기초
2	SQLD

NOT IN 연산자 대신에 NOT EXISTS 를 사용하여 동일한 결과를 얻을 수 있음

기출) 다음 SQL 의 실행 결과로 알맞은 것은?

TAB1

COL1	COL2
10	A
20	B
30	C
40	A
50	NULL

TAB2

COL1	COL2
10	A
20	B
30	D
40	NULL

```

SELECT SUM(COL1)
  FROM TAB1 T1
 WHERE EXISTS (SELECT 'X'
                FROM TAB2 T2
                WHERE T1.COL2 = T2.COL2);

```

- ① 50
- ② 60
- ③ 70
- ④ 120

정답 ③

해설: TAB2 에 존재하는 COL2 값(A, B)과 동일한 TAB1 행만 선택되며, C 는 제외됨

NULL 은 '=' 비교가 성립하지 않으므로(T1.COL2 = T2.COL2 가 참이 될 수 없음) TAB1 의 50 도 제외되어 10 + 20 + 40 = 70 이다.

● 인라인뷰(Inline View)

- 쿼리 안에서 뷰 형태로 조회할 데이터를 정의하기 위해 사용함
- 테이블명이 존재하지 않기 때문에 다른 테이블과 조인 시에는 반드시 테이블 별칭을 명시해야 함 (단독으로 사용하는 경우에는 불필요함)
- WHERE 절 서브쿼리와 달리, **서브쿼리 결과를 메인쿼리의 어느 절에서도 사용할 수 있음**
- 인라인뷰의 결과를 메인쿼리 테이블과 조인하기 위한 목적으로 주로 사용함

예제) EMP 테이블에서 부서별로 가장 높은 급여를 받는 사원 정보를 조회

SQL1 *				
1	▶	SELECT	E.EMPNO, E.ENAME, E.SAL, I.MAX_SAL	
2		FROM	EMP E, (SELECT DEPTNO, MAX(SAL) AS MAX_SAL	
3			FROM EMP	
4			GROUP BY DEPTNO) I	
5		WHERE	E.DEPTNO = I.DEPTNO	
6		AND	E.SAL = I.MAX_SAL;	

Result				
	EMPNO	ENAME	SAL	MAX_SAL
1	7698	BLAKE	2850	2850
2	7788	SCOTT	3000	3000
3	7839	KING	5000	5000
4	7902	FORD	3000	3000

- ☞ 인라인뷰에서는 EMP 테이블을 부서번호(DEPTNO) 기준으로 그룹화하여 각 부서별 최대 급여(MAX(SAL))를 먼저 구하고, 이 결과를 가상의 테이블로 만든 뒤, 메인쿼리에서 EMP 테이블과 부서번호를 기준으로 조인함
- ☞ 마지막으로 사원의 급여(SAL)가 해당 부서의 최대 급여(MAX_SAL)와 일치하는 경우만 선택하여 각 부서에서 급여가 가장 높은 사원 정보만 출력함

예제) EMP 테이블에서 부서별 평균 급여보다 높은 급여를 받는 사원을 조회

SQL1 *				
1	▶	SELECT E.EMPNO, E.ENAME, E.SAL, I.AVG_SAL		
2		FROM EMP E, (SELECT DEPTNO, AVG(SAL) AS AVG_SAL		
3		FROM EMP		
4		GROUP BY DEPTNO) I		
5		WHERE E.DEPTNO = I.DEPTNO		
6		AND E.SAL > I.AVG_SAL;		

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	EMPNO	ENAME	SAL	AVG_SAL
1	7499	ALLEN	1600	1566.666...
2	7566	JONES	2975	2175
3	7698	BLAKE	2850	1566.666...
4	7788	SCOTT	3000	2175
5	7839	KING	5000	2916.666...
6	7902	FORD	3000	2175

인라인뷰에서 부서별 평균 급여를 먼저 구한 뒤, 해당 평균 급여보다 높은 급여를 받는 사원만 메인쿼리에서 선택하여 출력함

● 스칼라 서브쿼리

- 하나의 컬럼처럼 표현하기 위해 SELECT 절에서 사용하는 서브쿼리를 말함
- 각 행마다 스칼라 서브쿼리의 결과는 하나여야 하며, 단일 행 서브쿼리 형태여야 함
(한 개의 행, 한 개의 컬럼)
- 스칼라 서브쿼리를 사용한 조인 시에는 OUTER JOIN 이 기본으로 적용되며, 일치하는 대상이 없을 경우 NULL 로 출력됨

예제) EMP 테이블에서 10 번 부서 직원의 이름, 부서번호, 급여와 함께 전체 직원의 평균 급여도 함께 출력
에러발생)

SQL1 *				
1	▶	SELECT ENAME, DEPTNO, SAL, AVG(SAL)		
2		FROM EMP		
3		WHERE DEPTNO = 10;		
4				
5				

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
ORA-00937: 단일 그룹의 그룹 함수가 아닙니다				

AVG(SAL)은 여러 행을 하나의 값으로 요약하는 집계 함수이므로,
ENAME, DEPTNO, SAL 처럼 행마다 값이 달라질 수 있는 컬럼과 함께 SELECT 절에 올 수 없음

해결)

SQL1 *				
1	▶	SELECT ENAME, DEPTNO, SAL,		
2		(SELECT ROUND(AVG(SAL))		
3		FROM EMP) AS AVG_SAL		
4		FROM EMP		
5		WHERE DEPTNO = 10;		

Result				
		Grid Result	Server Output	Text Output
		Explain Plan	Statistics	
ENAME	DEPTNO	SAL	AVG_SAL	
1 CLARK	10	2450	2073	
2 KING	10	5000	2073	
3 MILLER	10	1300	2073	

ENAME	JOB	SAL	DEPTNO
SMITH	CLERK	800	20
ALLEN	SALESMAN	1600	30
WARD	SALESMAN	1250	30
JONES	MANAGER	2975	20
MARTIN	SALESMAN	1250	30
BLAKE	MANAGER	2850	30
CLARK	MANAGER	2450	10
SCOTT	ANALYST	3000	20
KING	PRESIDENT	5000	10
TURNER	SALESMAN	1500	30
ADAMS	CLERK	1100	20
JAMES	CLERK	950	30
FORD	ANALYST	3000	20
MILLER	CLERK	1300	10

- ☞ 집계 함수는 일반 컬럼과 함께 직접 사용할 수 없으므로
 평균 급여를 스칼라 서브쿼리 형태로 분리하여 SELECT 절에서 조회함

예제) JOB 이 CLERK 인 직원의 이름, 부서번호, 급여를 출력하고, 각 직원이 소속된 부서의 평균 급여도 함께 출력함

SQL1 *				
1	▶	SELECT ENAME, DEPTNO, SAL,		
2		(SELECT ROUND(AVG(SAL))		
3		FROM EMP E2		
4		WHERE E1.DEPTNO = E2.DEPTNO) AS AVG_SAL		
5		FROM EMP E1		
6		WHERE JOB = 'CLERK'		
7		ORDER BY DEPTNO;		

Result				
		Grid Result	Server Output	Text Output
		Explain Plan	Statistics	
ENAME	DEPTNO	SAL	AVG_SAL	
1 MILLER	10	1300	2917	
2 SMITH	20	800	2175	
3 ADAMS	20	1100	2175	
4 JAMES	30	950	1567	

- ☞ 메인쿼리에서 출력되는 사원(E1)이 속한 부서번호(DEPTNO)에 따라 서브쿼리에서 구해야 하는 평균 급여가 부서별로 달라져야 함
- ☞ 따라서 서브쿼리에서 E1.DEPTNO = E2.DEPTNO 조건으로 메인쿼리의 부서번호를 참조해야 하고, 이로 인해 메인쿼리의 각 행마다 서브쿼리가 실행되는 연관 서브쿼리 형태가 필요함

예제) 스칼라 서브쿼리를 사용하여 EMP 테이블의 각 직원과 상위관리자 이름 출력

SQL1 *	
1	SELECT E1.ENAME AS 사원명,
2	(SELECT E2.ENAME
3	FROM EMP E2
4	WHERE E1.MGR = E2.EMPNO) AS 상위관리자명
5	FROM EMP E1;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

	사원명	상위관리자명
1	SMITH	FORD
2	ALLEN	BLAKE
3	WARD	BLAKE
4	JONES	KING
5	MARTIN	BLAKE
6	BLAKE	KING
7	CLARK	KING
8	SCOTT	JONES
9	KING	

이하 생략

☞ KING의 경우 MGR 컬럼 값이 NULL 이므로 MGR = EMPNO 조건에 해당하는 E2.ENAME 값이 존재하지 않음

☞ 하지만 스칼라 서브쿼리는 메인쿼리에서 출력되는 각 행에 대해 항상 하나의 값을 반환해야 하므로, 해당 값이 존재하지 않는 경우에도 행이 생략되지 않고 결과는 NULL 로 출력됨(아우터 조인 역할)

● 서브쿼리 주의 사항

- 특별한 경우(TOP-N 분석 등)를 제외하고, **서브쿼리에서는 ORDER BY 절 사용 불가**
- 단일행 서브쿼리와 다중행 서브쿼리에서는 사용 가능한 연산자가 다르므로 연산자 선택에 주의해야 함

예제) 서브쿼리에 ORDER BY 전달 시 에러 발생

SQL1 *	
1	SELECT *
2	FROM EMP
3	WHERE SAL IN (SELECT SAL
4	FROM EMP
5	WHERE DEPTNO = 10
6	ORDER BY SAL);

Result	
ORA-00907: 누락된 우괄호	Col : 1 Ln 6, Col 31 0.00 sec.

☞ 서브쿼리에 ORDER BY 절을 사용하여 에러가 발생함

<< 단원 정리 문제 >>

1. 다음 중 서브쿼리에 대한 설명으로 가장 적절하지 않은 것은?

- ① 서브쿼리에는 ORDER BY 를 사용할 수 없다.
- ② 스칼라 서브쿼리가 연관 서브쿼리 형태일 경우 조건이 일치하지 않는 행은 출력되지 않는다.
- ③ 비연관 서브쿼리는 메인쿼리와 독립적으로 한 번 수행된다.
- ④ 연관 서브쿼리는 메인쿼리가 실행될 때마다 수행된다.

2. 다음 SQL 수행 결과로 적절한 것은?

<TAB1>		<TAB2>	
COL1	COL2	COL1	COL2
1	A	2	A
2	A	3	A
3	A	4	B
4	B	6	B
5	B		


```

SELECT SUM(COL1)
  FROM TAB1
 WHERE COL1 >= ANY(SELECT COL1
                    FROM TAB2);

```

- ① NULL
- ② 0
- ③ 14
- ④ 15

3. 다음 SQL 수행 결과로 적절한 것은?

<TAB1>		<TAB2>	
COL1	COL2	COL1	COL2
10	A	2	A
20	A	3	B
30	C	4	D
40	B	6	NULL
50	NULL		


```

SELECT SUM(COL1)
  FROM TAB1
 WHERE COL2 NOT IN (SELECT COL2
                    FROM TAB2);

```

- ① 0
- ② 30
- ③ 80
- ④ NULL

4. 다음 SQL 수행 결과로 적절한 것은?

<TAB1>

COL1	COL2
10	15
10	5
20	20
30	25
30	NULL
40	50
40	NULL
40	40

<TAB2>

COL1	COL2
10	10
20	20
20	30
30	10
30	10
30	40
40	NULL
40	50

```
SELECT SUM(COL2)
FROM TAB1
WHERE COL2 >= (SELECT AVG(COL2)
                FROM TAB2
                WHERE TAB1.COL1 = TAB2.COL1));
```

- ① 30
- ② 40
- ③ 90
- ④ NULL

5. 다음 SQL 수행 결과로 적절한 것은?

<사원>

사원번호	부서코드	급여
100	A	3000
101	A	4000
102	B	3500
103	B	NULL
104	C	5000

<부서>

부서코드	기준급여
A	3500
B	3000
C	NULL

```
SELECT SUM((SELECT 기준급여
            FROM 부서 D
            WHERE D.부서코드 = S.부서코드))
FROM 사원 S
WHERE 급여 IS NOT NULL;
```

- ① 6500
- ② 9500
- ③ 10000
- ④ NULL

1. ②

스칼라 서브쿼리는 아우터 조인이 기본 연산이므로, 조건이 맞지 않아도 결과가 생략되지 않고 NULL 로 출력된다.

2. ③

ANY 는 크다와 함께 사용 시 서브쿼리 결과 중 최솟값을 리턴하므로 COL1 >= 2 가 최종 조건이 된다.

3. ④

서브쿼리 결과에 NULL 이 포함되어 있으므로 NOT IN 비교 결과는 모두 UNKNOWN 이 된다.
조건을 만족하는 행이 없어 SUM 결과는 NULL 이 리턴된다.

4. ③

서브쿼리는 TAB1 의 각 행에 대해 COL1 값이 같은 TAB2 행들만 대상으로 하여 COL2 의 평균을 계산한다.
이 평균값과 TAB1.COL2 를 비교하여, TAB1.COL2 가 평균 이상인 행만 선택한다.

이때 COL2 값이 NULL 인 경우에는 비교 결과가 UNKNOWN 이 되므로 조건을 만족하지 않아 제외된다.

조건을 만족하는 TAB1 의 행은 다음과 같다.

COL1 = 10, COL2 = 15 ($15 \geq 10$)

COL1 = 30, COL2 = 25 ($25 \geq 20$)

COL1 = 40, COL2 = 50 ($50 \geq 50$)

조건을 만족한 행들의 COL2 값만 합산하므로 최종 결과는

$15 + 25 + 50 = 90$ 이다.

5. ③

SELECT 절에 위치한 서브쿼리는 스칼라 서브쿼리로, 사원 테이블의 각 행마다 하나의 값만 반환해야 한다.
해당 스칼라 서브쿼리는 사원의 부서코드와 동일한 부서의 기준급여를 반환한다.

WHERE 절에서 급여가 NULL 인 사원은 제외되므로, 사원번호 103 은 집계 대상에서 제외된다.

사원별로 스칼라 서브쿼리 결과는 다음과 같다.

사원 100 (A) → 3500

사원 101 (A) → 3500

사원 102 (B) → 3000

사원 104 (C) → NULL

SUM 함수는 NULL 을 제외하고 합산하므로,

$3500 + 3500 + 3000 = 10000$ 이 된다.

2-10. 집합 연산자

● 집합 연산자

- 두 개 이상의 SELECT 결과를 하나의 결과 집합으로 결합할 때 사용하는 연산자
- 각 SELECT 문의 결과는 **컬럼 개수와 데이터 타입이 동일해야 함**
- 종 결과 집합의 **컬럼명과 데이터 타입은 첫 번째 SELECT 문의 기준으로 결정됨**

● 집합 연산자 종류

1. UNION / UNION ALL

- 두 집합의 합집합을 출력하는 연산자
- UNION: 두 SELECT 결과를 합쳐 **중복 행을 제거**한 결과를 반환(내부적으로 DISTINCT + SORT 수행)
- UNION ALL: 두 SELECT 결과를 그대로 **모두 반환**(중복 제거 및 정렬이 없으므로 성능이 가장 좋음)

예제) A 지점과 B 지점에서 판매하는 모든 메뉴 출력

<MENU_A>

ID	NAME	PRICE
1	아메리카노	2500
2	카페라떼	3000
3	바닐라라떼	3500
4	아이스 아메리카노	2700
5	콜드브루	4000

<MENU_B>

ID	NAME	PRICE
1	아메리카노	2500
2	카페라떼	3000
3	바닐라라떼	3500
6	녹차라떼	3500
7	허니브레드	5000

SQL1 *

```

1 SELECT NAME, PRICE
2   FROM MENU_A
3   UNION ALL
4 SELECT NAME, PRICE
5   FROM MENU_B;

```

Result

	NAME	PRICE
1	아메리카노	2500
2	카페라떼	3000
3	바닐라라떼	3500
4	아이스 아메리카노	2700
5	콜드브루	4000
6	아메리카노	2500
7	카페라떼	3000
8	바닐라라떼	3500
9	녹차라떼	3500
10	허니브레드	5000

☞ MENU_A 와 MENU_B 에 공통으로 존재하는 메뉴(빨간 박스 영역)가 두 번 출력됨

SQL1 *

```

1 SELECT NAME, PRICE
2   FROM MENU_A
3   UNION
4 SELECT NAME, PRICE
5   FROM MENU_B;

```

Result

	NAME	PRICE
1	녹차라떼	3500
2	바닐라라떼	3500
3	아메리카노	2500
4	아이스 아메리카노	2700
5	카페라떼	3000
6	콜드브루	4000
7	허니브레드	5000

☞ MENU_A 와 MENU_B 에 공통으로 존재하는 메뉴는 중복 제거되어 한 번만 출력됨

2. INTERSECT

- 두 SELECT 결과에 공통으로 존재하는 행만 반환하는 연산자
- 중복 제거 후 반환

예제) A 지점과 B 지점에서 상품명과 가격이 동일하게 등록된 메뉴를 출력

SQL1 *	
1	SELECT NAME, PRICE
2	FROM MENU_A
3	INTERSECT
4	SELECT NAME, PRICE
5	FROM MENU_B;

Result	
Grid Result	Server Output
Text Output	
NAME	PRICE
1 바닐라라떼	3500
2 아메리카노	2500
3 카페라떼	3000

☞ 상품명과 가격이 모두 동일한 상품만 출력됨(두 메뉴의 교집합)

3. MINUS

- 두 집합의 차집합을 반환하는 연산자
- 차집합이란 첫 번째 집합에는 존재하지만 두 번째 집합에는 존재하지 않는 데이터를 의미
- (A - B)와 (B - A)는 결과가 다르므로, 집합의 순서에 유의해야 함

예제) A 지점에는 있지만 B 지점에는 없는 메뉴 이름 출력

SQL1 *	
1	SELECT NAME
2	FROM MENU_A
3	MINUS
4	SELECT NAME
5	FROM MENU_B;

Result	
Grid Result	Server Output
Text Output	
NAME	
1 아이스 아메리카노	
2 콜드브루	

☞ MENU_A 에는 있지만 MENU_B 에는 없는 상품명만 출력함

기출) 다음 SQL 수행 결과로 출력되는 행의 수는?

TAB1

COL1
ABC
AAA
A%A
A_BA
AA

TAB2

COL1
A_BA
AAA
A_A
ABC

```
SELECT COL1
  FROM TAB1
 WHERE COL1 LIKE 'A%_A'
    UNION
SELECT COL1
  FROM TAB2
 WHERE COL1 LIKE 'A\_A' ESCAPE '\';
```

- ① 2
- ② 3
- ③ 4
- ④ 5

정답 ③

해설: 첫 번째 SELECT 는 LIKE 'A%_A' 조건이므로 A로 시작하고 마지막이 A이며, 마지막 A 바로 앞에 문자 1 개가 존재해야 한다. TAB1 에서 AAA, A%A, A_BA 가 조건을 만족하므로 3 행이다.

두 번째 SELECT 는 LIKE 'A_A' ESCAPE '\' 조건이므로 \는 언더스코어(_)를 문자 그대로 의미해 A_로 시작하고 마지막이 A 인 값을 찾는다. TAB2 에서 A_BA, A_A 가 만족하므로 2 행이다.

UNION 은 중복 행을 제거하므로 공통값 A_BA 1 건이 제거되어

최종 결과는 AAA, A%A, A_BA, A_A 총 4 행이다.

● 집합 연산자 주의 사항

- 두 집합의 SELECT 절의 컬럼 수가 동일해야 함
- 두 집합의 SELECT 절의 컬럼 나열 순서에 주의해야 함
- 두 집합의 SELECT 절에서 같은 위치에 있는 컬럼끼리 **데이터 타입이 일치해야 함**
- 집합 연산자 사용 시 각 SELECT 절에는 ORDER BY 절을 사용할 수 없으며,
ORDER BY 절은 마지막 SELECT 문 뒤에서만 사용 가능함

예제) 컬럼의 데이터타입이 다른 경우의 예러

SQL1 *			
1 ▶ DESC EMP;			
Result			
	Column	Nullable	Type
1	EMPNO	NOT NULL	NUMBER(4)
2	ENAME		VARCHAR2(10)
3	JOB		VARCHAR2(9)
4	MGR		NUMBER(4)
5	HIREDATE		DATE
6	SAL		NUMBER(7,2)
7	COMM		NUMBER(7,2)
8	DEPTNO		NUMBER(2)

SQL1 *			
1 ▶ DESC EMP_T1;			
Result			
	Column	Nullable	Type
1	EMPNO		VARCHAR2(40)
2	ENAME		VARCHAR2(10)
3	SAL		NUMBER(7,2)
4	DEPTNO		NUMBER(2)

에러 발생)

SQL1 *			
1 ▶ SELECT EMPNO, ENAME, SAL, DEPTNO			
2 FROM EMP			
3 WHERE DEPTNO = 10			
4 UNION			
5 SELECT EMPNO, ENAME, SAL, DEPTNO			
6 FROM EMP_T1;			
Result			
ORA-01790: 대응하는 식과 같은 데이터 유형이어야 합니다 Col : 1 Ln 6, Col 15			

EMPNO 컬럼의 데이터 타입이 서로 다르기 때문에 UNION 연산 시 에러 발생함

해결)

SQL1 *			
1 ▶ SELECT EMPNO, ENAME, SAL, DEPTNO			
2 FROM EMP			
3 WHERE DEPTNO = 10			
4 UNION			
5 SELECT TO_NUMBER(EMPNO),			
6 ENAME, SAL, DEPTNO			
7 FROM EMP_T1;			
Result			
	EMPNO	ENAME	SAL
1	7369	SMITH	800
2	7499	ALLEN	1600
3	7521	WARD	1250
4	7566	JONES	2975
5	7654	MARTIN	1250
6	7698	BLAKE	2850
7	7782	CLARK	2450
8	7788	SCOTT	3000

TO_NUMBER 함수를 사용하여 아래 집합의 EMPNO 컬럼을 숫자형 데이터 타입으로 변환하여 처리함

예제) 집합연산자와 ORDER BY 의 사용

에러 발생)

```

SQL1 *
1 SELECT ENAME, SAL
2   FROM EMP
3   WHERE DEPTNO = 10
4   ORDER BY SAL
5 UNION ALL
6 SELECT ENAME, SAL
7   FROM EMP
8   WHERE DEPTNO = 20
9   ORDER BY SAL;

```

Result

ORA-00933: SQL 명령어가 올바르게 종료되지 않았습니다 Col : 1 Ln 9, Col 15

☞ 집합 연산자 사용 시 개별 쿼리에는 ORDER BY 절을 전달할 수 없음

해결)

```

SQL1 *
1 (SELECT ENAME, SAL
2   FROM EMP
3   WHERE DEPTNO = 10
4   UNION ALL
5   SELECT ENAME, SAL
6   FROM EMP
7   WHERE DEPTNO = 20)
8   ORDER BY SAL;

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	ENAME	SAL
1	SMITH	800
2	ADAMS	1100
3	MILLER	1300
4	CLARK	2450
5	JONES	2975
6	FORD	3000
7	SCOTT	3000
8	KING	5000

☞ 집합 연산자 수행 후 전체 결과에 ORDER BY 절 전달 가능(괄호 생략 가능)

기출) 다음 중 집합 연산자 사용 시 주의사항으로 옳지 않은 것은?

- ① 집합 연산자로 결합되는 두 SELECT 문에서, 첫 번째 집합의 컬럼 길이는 두 번째 집합의 컬럼 길이보다 반드시 커야 한다.
- ② 집합 연산자로 결합되는 각 SELECT 문의 컬럼 수는 반드시 동일해야 한다.
- ③ 집합 연산자로 결합되는 각 SELECT 문에서 같은 위치의 컬럼끼리는 데이터 타입이 일치해야 한다.
- ④ 집합 연산자 사용 시 ORDER BY 절은 전체 결과에 대해 마지막에 한 번만 사용할 수 있다.

정답 ①

해설: 집합 연산자는 컬럼 길이 크기 비교 조건은 없고, 각 SELECT 문의 컬럼 수 동일 및 같은 위치 컬럼의 데이터 타입 일치가 핵심 조건이다. 따라서 ①은 옳지 않다.

<< 단원 정리 문제 >>

1. 다음 중 두 테이블에서 판매하는 제품의 가격이 같은 메뉴를 출력하는 문장으로 가장 적절한 것은?

<PRODUCT1>			<PRODUCT2>		
NO	NAME	PRICE	NO	NAME	PRICE
1	빼빼로	2500	1	빼빼로	2500
2	초코칩	3000	2	초코송이	1400
3	초코송이	1400	3	감자칩	2200
4	새우깡	1800	4	새우깡	1600
6	버터링	3200	5	버터링	3200

①

```
SELECT *
  FROM PRODUCT1
UNION
SELECT *
  FROM PRODUCT2;
```

②

```
SELECT NAME, PRICE
  FROM PRODUCT1
UNION
SELECT NAME, PRICE
  FROM PRODUCT2;
```

③

```
SELECT *
  FROM PRODUCT1
INTERSECT
SELECT *
  FROM PRODUCT2;
```

④

```
SELECT NAME, PRICE
  FROM PRODUCT1
INTERSECT
SELECT NAME, PRICE
  FROM PRODUCT2;
```

2. 다음 중 집합 연산자에 대한 설명으로 가장 적절한 것은?

- ① 두 번째 집합에서의 컬럼 별칭은 무시된다.
- ② 각 집합을 정렬하여 합집합을 출력할 수 있다.
- ③ 각 집합에 GROUP BY 절을 사용할 수 없다.
- ④ 각 집합에서 출력하는 컬럼 수가 일치하지 않아도 정상적으로 출력된다.

1. ④

두 집합에서의 교집합을 찾는 문제이다. 상품 번호는 다를 수 있으므로 이름과 가격 정보만 비교해야 한다.

2. ①

집합 연산자 결과는 첫 번째 집합의 데이터 타입과 컬럼 이름으로 출력된다

2-11. 그룹 함수

● 그룹 함수(Grouping Extensions)

- 여러 수준의 집계를 한 번의 SQL 로 처리하기 위한 기능을 갖는 함수
예) EMP 테이블에서 부서별 급여 총합과 전체 급여 총합을 동시에 출력

※ 이 책에서는 SUM, AVG, COUNT 등과 같은 함수를 집계 함수로 통일하여 표현하기로 함

예제) 일반 GROUP BY 기능

SQL1 *	
1 ▶	SELECT DEPTNO, SUM(SAL)
2	FROM EMP
3	GROUP BY DEPTNO;

Result	
Grid Result	Server Output
Text Output	Explain Plan
Statistics	

	DEPTNO	SUM(SAL)
1	30	9400
2	20	10875
3	10	8750

☞ DEPTNO 별 급여 총합만 출력되므로 전체 총합과 함께 출력될 수 없음

● 그룹 함수 종류

GROUPING SETS 는 여러 그룹핑 쿼리를 UNION ALL 한 것과 같은 결과를 만들 수 있어 조금 더 유연하게 소계, 합계를 집계할 수 있음

1. GROUPING SETS

- 사용자가 원하는 그룹핑 조합만 명시적으로 지정함
- 컬럼 나열 순서는 중요하지 않음
- 기본적으로 전체 집계 결과는 출력되지 않음
- NULL 또는 ()를 사용하여 전체 집계 결과를 출력할 수 있음

** 예시

- GROUPING SETS ((A, B), (A), ()) → (A, B) 기준 집계, (A) 기준 집계, 전체 집계 결과를 함께 출력함

** 시험 출제 공식

- GROUPING SETS(A): (A)
- GROUPING SETS(A,B): (A) + (B)
- GROUPING SETS((A,B), (B,C)): (A,B) + (B,C)
- A, GROUPING SETS(B): (A,B)
- A, B, GROUPING SETS(C): (A,B,C)
- A, GROUPING SETS(B, C): (A,B) + (A,C)

예제) EMP 테이블에서 부서별 급여 총합과 직무별 급여 총합을 함께 출력함

SQL1 *			
1	SELECT DEPTNO, JOB, SUM(SAL)		
2	FROM EMP		
3	GROUP BY GROUPING SETS(DEPTNO, JOB);		

Result			
	DEPTNO	JOB	SUM(SAL)
1		CLERK	4150
2		SALESMAN	5600
3		PRESIDENT	5000
4		MANAGER	8275
5		ANALYST	6000
6	30		9400
7	20		10875
8	10		8750

JOB별 급여 총합

DEPTNO별 급여 총합

GROUPING SETS 에 나열한 대상별로 각각 GROUP BY 를 수행한 결과를 출력해 줌

UNION ALL 로 대체 가능)

SQL1 *			
1	SELECT DEPTNO, NULL AS JOB, SUM(SAL)		
2	FROM EMP		
3	GROUP BY DEPTNO		
4	UNION ALL		
5	SELECT NULL, JOB, SUM(SAL)		
6	FROM EMP		
7	GROUP BY JOB;		

Result			
	DEPTNO	JOB	SUM(SAL)
1	30		9400
2	20		10875
3	10		8750
4		CLERK	4150
5		SALESMAN	5600
6		PRESIDENT	5000
7		MANAGER	8275
8		ANALYST	6000

부서별 급여 집계 결과와 직무별 급여 집계 결과를 각각 GROUP BY 로 계산한 뒤 UNION ALL 로 결합하여, GROUPING SETS 를 사용한 것과 동일한 결과를 출력할 수 있음

예제) 부서별 급여 총합과, 직무별 급여 총합, 그리고 전체 급여 총합을 출력

SQL1 *			
1	▶	SELECT DEPTNO, JOB, SUM(SAL)	
2		FROM EMP	
3		GROUP BY GROUPING SETS(DEPTNO, JOB, ());	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	DEPTNO	JOB	SUM(SAL)
1		CLERK	4150
2		SALESMAN	5600
3		PRESIDENT	5000
4		MANAGER	8275
5		ANALYST	6000
6	10		8750
7	20		10875
8	30		9400
9			29025

GROUPING SETS (DEPTNO, JOB, ())를 사용하여 부서별 집계, 직무별 집계, 전체 집계 결과를 한 번에 출력함 → () 대신 NULL 사용 가능

2. ROLLUP

- GROUP BY 컬럼을 계층적으로 확장하여 집계함
- 소계 → 총계 형태의 결과를 출력함
- 컬럼 순서에 따라 결과가 달라짐

** 예시

- ROLLUP(A, B) → (A, B) 기준 집계 → (A) 기준 소계 → 전체 기준 총계 순서로 결과를 출력함

** 시험 출제 공식

- ROLLUP(A): (A) + (전체)
- ROLLUP(A,B): (A) + (A,B) + (전체)
- A, ROLLUP(B): (A) + (A,B)
- A, B, ROLLUP(C): (A,B) + (A,B,C)
- A, ROLLUP(B,C): (A) + (A,B) + (A,B,C)

예제) EMP 테이블에서 부서와 직무별 급여 총합, 부서별 급여 총합, 그리고 전체 급여 총합을 함께 출력함

SQL1 *

```

1 SELECT DEPTNO, JOB, SUM(SAL)
2 FROM EMP
3 GROUP BY ROLLUP(DEPTNO, JOB);

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	DEPTNO	JOB	SUM(SAL)	
1	10	CLERK	1300	
2	10	MANAGER	2450	DEPTNO, JOB 별 급여 총합
3	10	PRESIDENT	5000	
4	10		8750	DEPTNO 별 급여 총합
5	20	CLERK	1900	
6	20	ANALYST	6000	
7	20	MANAGER	2975	
8	20		10875	
9	30	CLERK	950	
10	30	MANAGER	2850	
11	30	SALESMAN	5600	
12	30		9400	
13			29025	전체 급여 총합

☞ ROLLUP(DEPTNO, JOB)을 사용하여 부서·직무별 급여 총합 → 부서별 급여 총합 → 전체 급여 총합을 단계적으로 출력함

UNION ALL 로 대체)

SQL1 *

```

1 SELECT DEPTNO, JOB, SUM(SAL)
2 FROM EMP
3 GROUP BY DEPTNO, JOB
4 UNION ALL
5 SELECT DEPTNO, NULL, SUM(SAL)
6 FROM EMP
7 GROUP BY DEPTNO
8 UNION ALL
9 SELECT NULL, NULL, SUM(SAL)
10 FROM EMP;

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	DEPTNO	JOB	SUM(SAL)
1	20	CLERK	1900
2	30	SALESMAN	5600
3	20	MANAGER	2975
4	30	CLERK	950
5	10	PRESIDENT	5000
6	30	MANAGER	2850
7	10	CLERK	1300
8	10	MANAGER	2450
9	20	ANALYST	6000
10	30		9400
11	20		10875
12	10		8750
13			29025

☞ 부서·직무별 집계, 부서별 집계, 전체 집계를 각각 GROUP BY 로 계산한 뒤 UNION ALL 로 결합하여, ROLLUP(DEPTNO, JOB)과 동일한 집계 결과를 구현함

3. CUBE

- GROUP BY 컬럼의 모든 조합에 대한 집계를 수행함
- ROLLUP 보다 결과 행 수가 많음
- 컬럼 순서와 무관함

** 예시

- CUBE (A, B) → (A, B) 기준 집계, (A) 기준 소계, (B) 기준 소계, 전체 기준 총계를 출력함

** 시험 출제 공식

- CUBE(A): (A) + (전체)
- CUBE(A,B): (A) + (B) + (A,B) + (전체)
- A, CUBE(B): (A) + (A,B)
- A, B, CUBE(C): (A,B) + (A,B,C)
- A, CUBE(B,C): (A) + (A,B) + (A,C) + (A,B,C)

예제) 부서별, 직무별, 부서와 직무별, 그리고 전체 급여 총합을 함께 출력

SQL1 *

1

2

3

SELECT DEPTNO, JOB, SUM(SAL)
FROM EMP
GROUP BY CUBE(DEPTNO, JOB);

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	DEPTNO	JOB	SUM(SAL)	
1			29025	전체 급여 총합
2		CLERK	4150	직무별 급여 총합
3		ANALYST	6000	
4		MANAGER	8275	
5		SALESMAN	5600	
6		PRESIDENT	5000	부서별 급여 총합
7	10		8750	
8	10	CLERK	1300	
9	10	MANAGER	2450	
10	10	PRESIDENT	5000	부서와 직무별 급여 총합
11	20		10875	직무별 급여 총합
12	20	CLERK	1900	
13	20	ANALYST	6000	
14	20	MANAGER	2975	
15	30		9400	직무별 급여 총합
16	30	CLERK	950	
17	30	MANAGER	2850	
18	30	SALESMAN	5600	

CUBE(DEPTNO, JOB)을 사용하여 부서·직무별 급여 총합, 부서별 급여 총합, 직무별 급여 총합, 그리고 전체 급여 총합을 모두 출력함

UNION ALL 로 대체)

SQL1 *			
1	▶	SELECT NULL AS DEPTNO, NULL AS JOB, SUM(SAL)	
2		FROM EMP	
3		UNION ALL	
4		SELECT NULL, JOB, SUM(SAL)	
5		FROM EMP	
6		GROUP BY JOB	
7		UNION ALL	
8		SELECT DEPTNO, NULL, SUM(SAL)	
9		FROM EMP	
10		GROUP BY DEPTNO	
11		UNION ALL	
12		SELECT DEPTNO, JOB, SUM(SAL)	
13		FROM EMP	
14		GROUP BY DEPTNO, JOB;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	DEPTNO	JOB	SUM(SAL)
1			29025
2		CLERK	4150
3		SALESMAN	5600
4		PRESIDENT	5000
5		MANAGER	8275
6		ANALYST	6000
7	30		9400
8	20		10875
9	10		8750

이하 생략

☞ 전체 집계, 직무별 집계, 부서별 집계, 부서·직무별 집계를 각각 GROUP BY 로 계산한 뒤 UNION ALL 로 결합하여, CUBE(DEPTNO, JOB)과 동일한 집계 결과를 구현함

GROUPING SETS 로 대체)

SQL1 *			
1	▶	SELECT DEPTNO, JOB, SUM(SAL)	
2		FROM EMP	
3		GROUP BY GROUPING SETS(DEPTNO, JOB, (DEPTNO, JOB), ());	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	DEPTNO	JOB	SUM(SAL)
1	10	CLERK	1300
2	20	CLERK	1900
3	30	CLERK	950
4	20	ANALYST	6000
5	10	MANAGER	2450
6	20	MANAGER	2975
7	30	MANAGER	2850
8	30	SALESMAN	5600
9	10	PRESIDENT	5000

이하 생략

기출) 다음 중 출력 결과가 다른 하나는? (단, A, B 는 NULL 을 허용하지 않는 컬럼이다.)

- ① GROUP BY A, ROLLUP(B)
- ② GROUP BY GROUPING SETS ((A, B), (A))
- ③ GROUP BY ROLLUP(A, B)
- ④ GROUP BY A, CUBE(B)

정답 ③

해설: ①과 ②는 (A, B) 기준 집계와 (A) 기준 소계 결과만 출력하며, 전체 집계는 포함하지 않는다.

④는 ①의 결과와 같이 (A, B) → (A)를 출력한다.

③의 ROLLUP(A, B)는 (A, B) → (A) → 전체 총계를 출력하므로, 나머지와 결과가 다르다.

● 소계·총계 구분 함수

- 집계 결과에 포함된 NULL 값의 의미를 구분하기 위해 사용되는 함수임
- 동시에 출력되는 여러 집계 수준의 결과를 구분하는 역할을 함

1. GROUPING

- 특정 컬럼이 집계에 의해 생성된 값인지 여부를 판단하는 함수임
- 단 하나의 인수만 전달 가능
- 반환 값
 - 0: 실제 데이터에 의해 출력된 값임
 - 1: 집계(소계·총계)로 인해 생성된 값임

예제) GROUP_TAB 테이블에서 성별과 학과별 평균 점수, 성별별 평균 점수, 학과별 평균 점수, 그리고 전체 평균 점수를 함께 출력함

SQL1 *					
1	▶	SELECT	GENDER, DNAME, AVG(SCORE),		
2			GROUPING(GENDER) AS GROUP_GENDER,		
3			GROUPING(DNAME) AS GROUP_DNAME		
4		FROM	GROUP_TAB		
5		GROUP BY	CUBE(GENDER, DNAME);		
6					

Result					
	GENDER	DNAME	AVG(SCORE)	GROUP_GENDER	GROUP_DNAME
1			89.5	1	1
2		경영학	87	1	0
3		컴퓨터공학	92	1	0
4	남자		85	0	1
5	남자	경영학	85	0	0
6	남자	컴퓨터공학	85	0	0
7	여자		94	0	1
8	여자	경영학	89	0	0
9	여자	컴퓨터공학	99	0	0

GROUPING 함수를 사용하여 각 컬럼이 집계로 인해 생성된 값인지 여부를 0, 1로 구분하여 확인할 수 있음

2. GROUPING_ID

- 여러 컬럼에 대해 집계 레벨을 하나의 값으로 식별하기 위한 함수임
- GROUPING 함수의 결과를 이진법(비트값)으로 결합한 형태임
- 집계 결과를 레벨별로 정렬하거나 필터링할 때 유용함

**** GROUPING_ID(NAME, BRANCHID) 출력 결과**

GROUPING(NAME)	GROUPING(BRANCHID)	GROUPING_ID(NAME, BRANCHID)
0	0	$2 \times 0 + 1 \times 0 = 0$
0	1	$2 \times 0 + 1 \times 1 = 1$
1	0	$2 \times 1 + 1 \times 0 = 2$
1	1	$2 \times 1 + 1 \times 1 = 3$

SQL1 *

```

1 SELECT NAME, BRANCHID, SUM(ACHIEVEMENT) AS TOTAL,
2     GROUPING(NAME) AS GROUP_NAME,
3     GROUPING(BRANCHID) AS GROUP_BRANCHID,
4     GROUPING_ID(NAME, BRANCHID) AS GROUP_NUMBER
5 FROM PERFORMANCE
6 GROUP BY CUBE(NAME, BRANCHID);

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	NAME	BRANCHID	TOTAL	GROUP_NAME	GROUP_BRANCHID	GROUP_NUMBER
1			18687	1	1	3
2		100	3646	1	0	2
3		101	7509	1	0	2
4		102	3618	1	0	2
5		103	3914	1	0	2
6	EVA		1750	0	1	1
7	EVA	101	1750	0	0	0
8	SAM		1686	0	1	1
9	SAM	102	1686	0	0	0
10	TOM		1827	0	1	1
11	TOM	101	1827	0	0	0
12	ANNA		2035	0	1	1
13	ANNA	101	2035	0	0	0
14	JANE		1897	0	1	1
15	JANE	101	1897	0	0	0
16	JOHN		1773	0	1	1
17	JOHN	100	1773	0	0	0
18	LUKE		1932	0	1	1
19	LUKE	102	1932	0	0	0
20	MIKE		1873	0	1	1
21	MIKE	100	1873	0	0	0
22	SARA		1781	0	1	1
23	SARA	103	1781	0	0	0
24	CHRIS		2133	0	1	1
25	CHRIS	103	2133	0	0	0

GROUPING과 GROUPING_ID를 함께 사용하여 각 행이 상세 데이터인지, 소계인지, 총계인지 집계 레벨을 구분하여 확인할 수 있음

기출) 다음 중 출력 결과가 그림과 동일한 SQL 로 알맞은 것은?

	GENDER	DNAME	AVG_SCORE
1			89.5
2	소계	경영학	87
3	소계	컴퓨터공학	92
4	남자	소계	85
5	남자	경영학	85
6	남자	컴퓨터공학	85
7	여자	소계	94
8	여자	경영학	89
9	여자	컴퓨터공학	99

①

```
SELECT NVL(GENDER, '소계') AS GENDER,
       NVL(DNAME, '소계') AS DNAME,
       AVG(SCORE) AS AVG_SCORE
FROM GROUP_TAB
GROUP BY CUBE(GENDER, DNAME);
```

②

```
SELECT CASE WHEN GROUPING(GENDER)=1 AND GROUPING(DNAME)=0 THEN '소계' END AS GENDER,
       CASE WHEN GROUPING(DNAME)=1 AND GROUPING(GENDER)=0 THEN '소계' END AS DNAME,
       AVG(SCORE) AS AVG_SCORE
FROM GROUP_TAB
GROUP BY CUBE(GENDER, DNAME);
```

③

```
SELECT CASE WHEN GROUPING(GENDER)=1 THEN '소계' ELSE GENDER END AS GENDER,
       CASE WHEN GROUPING(DNAME)=1 THEN '소계' ELSE DNAME END AS DNAME,
       AVG(SCORE) AS AVG_SCORE
FROM GROUP_TAB
GROUP BY CUBE(GENDER, DNAME);
```

④

```
SELECT CASE WHEN GROUPING_ID(GENDER, DNAME)=2 THEN '소계' ELSE GENDER END AS GENDER,
       CASE WHEN GROUPING_ID(GENDER, DNAME)=1 THEN '소계' ELSE DNAME END AS DNAME,
       AVG(SCORE) AS AVG_SCORE
FROM GROUP_TAB
GROUP BY CUBE(GENDER, DNAME);
```

정답 ④

해설: ①, ③은 전체 집계계의 NULL 까지 모두 '소계'로 치환하므로 요구사항에 부적절하다.

②는 CASE 문에 ELSE 절을 생략했으므로 GENDER 와 DNAME 컬럼이 NULL 이면 '소계'로 치환되고 NULL 이 아니면 NULL 로 치환되므로 적절하지 않다.

④는 GROUPING_ID 로 집계 레벨을 구분하여, 2(학과별 소계 행: GENDER 가 집계로 NULL)에서만 GENDER 를 치환하고, 1(성별 소계 행: DNAME 이 집계로 NULL)에서만 DNAME 을 치환하므로 전체 총계(3)는 NULL 이 유지된다.

<< 단원 정리 문제 >>

1. 다음 중 출력 결과가 다른 하나를 고르시오

①

```
SELECT COL1, COL2, SUM(COL3), SUM(COL4)
  FROM TAB1
 GROUP BY COL1, ROLLUP(COL2);
```

②

```
SELECT COL1, COL2, SUM(COL3), SUM(COL4)
  FROM TAB1
 GROUP BY COL1, CUBE(COL2);
```

③

```
SELECT COL1, COL2, SUM(COL3), SUM(COL4)
  FROM TAB1
 GROUP BY GROUPING SETS(COL1, (COL1, COL2));
```

④

```
SELECT COL1, COL2, SUM(COL3), SUM(COL4)
  FROM TAB1
 GROUP BY GROUPING SETS(COL1, ROLLUP(COL2));
```

2. 다음 구문과 동일한 결과를 출력하는 것은?

GROUP BY GROUPING SETS (A, ROLLUP(B))

① GROUP BY GROUPING SETS ((A), (B), ())

② GROUP BY CUBE(A, B)

③ GROUP BY ROLLUP(A, B)

④ GROUP BY GROUPING SETS (A, B)

3. 다음 집합을 출력한 결과와 같은 SQL 문장을 고르시오

GRADE	GENDER	COUNT(*)
1	남자	3
1	여자	2
1		5
2	남자	2
2	여자	3
2		5
3	남자	2
3	여자	3
3		5
4	남자	5
4		5
		20

①

```
SELECT GRADE, GENDER, COUNT(*)
  FROM STUDENT
 GROUP BY ROLLUP(GRADE, GENDER);
```

②

```
SELECT GRADE, GENDER, COUNT(*)
  FROM STUDENT
 GROUP BY CUBE(GRADE, GENDER);
```

③

```
SELECT GRADE, GENDER, COUNT(*)
  FROM STUDENT
 GROUP BY GROUPING SETS(GRADE, GENDER);
```

④

```
SELECT GRADE, GENDER, COUNT(*)
  FROM STUDENT
 GROUP BY GROUPING SETS(GRADE, (GRADE, GENDER));
```

1. ④

4 번은 (COL1) + (COL2) + (전체) 집계 연산 결과가 출력되며, 나머지는 (COL1) + (COL1, COL2) GROUP BY 연산 결과가 리턴된다.

2. ①

GROUPING SETS(A, ROLLUP(B))는 ROLLUP(B)가 (B), (전체)로 확장되므로 최종적으로 (A), (B), (전체) 집계를 출력한다. 따라서 GROUPING SETS((A),(B),())와 동일하다.

3. ①

(GRADE), (GRADE, GENDER), (전체) 집계 연산 결과가 출력되었으므로 1 번 문장이 적합하다.

2-12. 윈도우 함수

● 윈도우 함수

- 윈도우 함수는 행 단위 결과를 유지한 상태에서 집계, 순위, 분석 기능을 수행하는 함수임
- 일반 그룹 함수와 달리 GROUP BY 절을 사용하지 않으며, **결과 행 수가 감소하지 않음**
- 대표적인 함수: LAG, LEAD, SUM, AVG, MIN, MAX, COUNT, RANK 등
- SELECT, ORDER BY 절에서만 사용 가능(WHERE, GROUP BY, HAVING 절 사용 불가)

** 문법

SQL1 *	
1	SELECT 윈도우함수([대상]) OVER([PARTITION BY 컬럼]
2	[ORDER BY 컬럼 ASC DESC]
3	[ROWS RANGE BETWEEN A AND B])

1. PARTITION BY 절

- 결과 집합을 그룹 단위로 분할함
- 각 파티션 내에서 윈도우 함수가 독립적으로 수행됨

2. ORDER BY 절

- 파티션 내에서 행의 순서를 정의함
- 누적 집계, 순위 계산 등에 사용됨

3. ROWS|RANGE BETWEEN A AND B

- 윈도우 함수가 적용될 행 범위를 지정함
- 연산 범위를 지정하려면 ORDER BY 절을 이전에 반드시 명시해야 함
- 생략 시 기본 범위: **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**
(BETWEEN AND CURRENT ROW 생략 가능)

문법	의미
ROWS	• 물리적인 행의 개수 기준으로 범위를 지정함 • ORDER BY 결과에서 현재 행을 기준으로 앞·뒤 몇 행을 대상으로 집계함
RANGE	• ORDER BY 절에 지정된 컬럼의 값 범위 기준으로 범위를 지정함 • 현재 행의 값과 같은 값 또는 특정 값 범위에 해당하는 모든 행을 포함함
A(시작점)	• CURRENT ROW: 현재부터 • UNBOUNDED PRECEDING: 처음부터(DEFAULT) • N PRECEDING: N 이전부터
B(끝점)	• CURRENT ROW: 현재까지(DEFAULT) • UNBOUNDED FOLLOWING: 마지막까지 • N FOLLOWING: N 이후까지

** 주의 사항

- 나열 순서 중요: PARTITION BY → ORDER BY → ROWS|BETWEEN 순

● 윈도우 함수 종류

구분	설명
누적 계산 함수	- SUM, COUNT, AVG, MIN, MAX - ORDER BY 절을 사용하여 누적 연산 수행
순위 관련 함수	- RANK, DENSE_RANK, ROW_NUMBER, NTILE - ORDER BY 절 필수
이전/이후 행 참조 함수	LAG, LEAD, FIRST_VALUE, LAST_VALUE
비율 관련 함수	RATIO_TO_REPORT, CUME_DIST, PERCENT_RANK

● 누적 계산 함수

- SUM, COUNT, AVG, MIN, MAX 연산 결과를 다양한 형태로 출력함
- OVER 절 안에 ORDER BY 절을 사용하면 누적 연산 결과가 리턴됨
- PARTITION BY 절을 함께 사용하면 그룹별 누적 연산 결과를 출력할 수 있음

** 문법

SQL1 *
1 SELECT SUM(대상) OVER([PARTITION BY 컬럼]
2 [ORDER BY 컬럼 ASC DESC]
3 [ROWS RANGE BETWEEN A AND B])

※ COUNT, AVG, MIN, MAX 함수 사용법도 위와 같음

1. SUM OVER

- 전체 총합, 그룹별 총합, 누적합 출력 가능

예제) EMP 테이블에서 각 직원 정보와 함께 급여 총합을 출력함

에러 발생)

SQL1 *
1 SELECT EMPNO, ENAME, SAL, DEPTNO, SUM(SAL)
2 FROM EMP;

EMPNO, ENAME, SAL, DEPTNO 는 각 행의 값을 그대로 출력하는 컬럼인 반면, SUM(SAL)은 여러 행을 하나의 값으로 요약하여 출력하는 집계 함수이므로 GROUP BY 절 없이 함께 출력할 수 없어 에러가 발생함

해결 1) 스칼라 서브쿼리 사용

SQL1 *					
1	SELECT EMPNO, ENAME, SAL, DEPTNO,				
2	(SELECT SUM(SAL) FROM EMP) AS TOTAL				
3	FROM EMP;				

Result					
	EMPNO	ENAME	SAL	DEPTNO	TOTAL
1	7369	SMITH	800	20	29025
2	7499	ALLEN	1600	30	29025
3	7521	WARD	1250	30	29025
4	7566	JONES	2975	20	29025
5	7654	MARTIN	1250	30	29025
6	7698	BLAKE	2850	30	29025
7	7782	CLARK	2450	10	29025
8	7788	SCOTT	3000	20	29025
9	7839	KING	5000	10	29025
10	7844	TURNER	1500	30	29025
11	7876	ADAMS	1100	20	29025
12	7900	JAMES	950	30	29025
13	7902	FORD	3000	20	29025
14	7934	MILLER	1300	10	29025

☞ 서브쿼리를 사용하여 급여 총합을 하나의 값으로 계산한 뒤, 해당 값을 각 행에 동일하게 함께 출력함
(집계 결과를 스칼라 서브쿼리로 처리하여 행 단위 출력과 함께 사용 가능함)

해결 2) 윈도우 함수 사용

SQL1 *					
1	SELECT EMPNO, ENAME, SAL, DEPTNO,				
2	SUM(SAL) OVER() AS TOTAL				
3	FROM EMP;				

Result					
	EMPNO	ENAME	SAL	DEPTNO	TOTAL
1	7369	SMITH	800	20	29025
2	7499	ALLEN	1600	30	29025
3	7521	WARD	1250	30	29025
4	7566	JONES	2975	20	29025
5	7654	MARTIN	1250	30	29025
6	7698	BLAKE	2850	30	29025
7	7782	CLARK	2450	10	29025
8	7788	SCOTT	3000	20	29025
9	7839	KING	5000	10	29025
10	7844	TURNER	1500	30	29025
11	7876	ADAMS	1100	20	29025
12	7900	JAMES	950	30	29025
13	7902	FORD	3000	20	29025
14	7934	MILLER	1300	10	29025

☞ 윈도우 함수(SUM(SAL) OVER())를 사용하여 집계 결과를 각 행에 함께 출력할 수 있으며, 서브쿼리 없이도 전체 급여 총합을 행 단위 결과와 함께 조회할 수 있음

예제) EMP 테이블에서 각 직원의 급여 정보와 함께 부서별 급여 총합을 출력함

SQL1 *				
1	SELECT ENAME, SAL, DEPTNO,			
2	SUM(SAL) OVER(PARTITION BY DEPTNO) AS DEPTNO_SUM_SAL			
3	FROM EMP;			

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	ENAME	SAL	DEPTNO	DEPTNO_SUM_SAL
1	CLARK	2450	10	8750
2	KING	5000	10	8750
3	MILLER	1300	10	8750
4	JONES	2975	20	10875
5	FORD	3000	20	10875
6	ADAMS	1100	20	10875
7	SMITH	800	20	10875
8	SCOTT	3000	20	10875
9	WARD	1250	30	9400
10	TURNER	1500	30	9400
11	ALLEN	1600	30	9400
12	JAMES	950	30	9400
13	BLAKE	2850	30	9400
14	MARTIN	1250	30	9400

☞ PARTITION BY DEPTNO 를 사용하여 부서별로 그룹을 나눈 뒤, 각 부서의 급여 총합을 해당 부서에 속한 모든 행에 함께 출력함

☞ GROUP BY 를 사용하지 않으므로 결과 행 수는 감소하지 않음

예제) EMP 테이블에서 급여(SAL)를 기준으로 정렬한 후 누적 급여 합계를 출력함(기본 연산 범위)

SQL1 *			
1	SELECT ENAME, SAL,		
2	SUM(SAL) OVER(ORDER BY SAL) AS CUMSUM_SAL		
3	FROM EMP;		

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	ENAME	SAL	CUMSUM_SAL
1	SMITH	800	800
2	JAMES	950	1750
3	ADAMS	1100	2850
4	WARD	1250	5350
5	MARTIN	1250	5350
6	MILLER	1300	6650
7	TURNER	1500	8150
8	ALLEN	1600	9750
9	CLARK	2450	12200
10	BLAKE	2850	15050
11	JONES	2975	18025
12	SCOTT	3000	24025
13	FORD	3000	24025
14	KING	5000	29025

☞ ORDER BY SAL 만 지정한 경우 기본 윈도우 프레임이 RANGE 방식으로 적용됨

☞ 동일한 SAL 값을 가진 행은 같은 값 범위로 인식되어 동시에 누적합에 포함되므로, 급여가 같은 직원(WARD, MARTIN / SCOTT, FORD)은 누적 급여 합계가 동일하게 출력됨

예제) EMP 테이블에서 급여(SAL)를 기준으로 정렬한 후 누적 급여 합계를 출력함(ROWS 연산 범위)

SQL1 *				
1	SELECT ENAME, SAL, DEPTNO,			
2	SUM(SAL) OVER(ORDER BY SAL			
3	ROWS UNBOUNDED PRECEDING) AS CUMSUM_SAL			
4	FROM EMP;			

Result				
	ENAME	SAL	DEPTNO	CUMSUM_SAL
1	SMITH	800	20	800
2	JAMES	950	30	1750
3	ADAMS	1100	20	2850
4	WARD	1250	30	4100
5	MARTIN	1250	30	5350
6	MILLER	1300	10	6650
7	TURNER	1500	30	8150
8	ALLEN	1600	30	9750
9	CLARK	2450	10	12200
10	BLAKE	2850	30	15050
11	JONES	2975	20	18025
12	SCOTT	3000	20	21025
13	FORD	3000	20	24025
14	KING	5000	10	29025

- ROWS UNBOUNDED PRECEDING 을 사용하여 물리적인 행 기준으로 누적합을 계산함
- 동일한 SAL 값을 가진 행이라도 정렬된 순서대로 한 행씩 누적되므로,
급여가 같은 직원(WARD → MARTIN, SCOTT → FORD)의 누적 급여 합계가 서로 다르게 출력됨

예제) RANGE 기본 연산 범위

SQL1 *				
1	SELECT NO, SAL,			
2	SUM(SAL) OVER(ORDER BY SAL) AS RESULT1,			
3	SUM(SAL) OVER(ORDER BY SAL			
4	RANGE BETWEEN UNBOUNDED PRECEDING			
5	AND CURRENT ROW) AS RESULT2			
6	FROM WINDOW_TAB1;			

Result				
	NO	SAL	RESULT1	RESULT2
1	1	10	10	10
2	2	11	21	21
3	3	12	45	45
4	4	12	45	45
5	5	25	70	70
6	6	30	100	100
7	7	35	135	135

<WINDOW_TAB1>

	NO	SAL
1	1	10
2	2	11
3	3	12
4	4	12
5	5	25
6	6	30
7	7	35

- ORDER BY 절만 지정한 경우 기본 윈도우 프레임은
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW 로 적용됨
- 따라서 SAL 값이 12로 동일한 경우, $10 + 11 + 12 + 12 = 45$ 로 동일한 누적합 결과가 출력됨

예제) RANGE의 다양한 연산 범위

SQL1 *			
1	▶	SELECT NO, SAL,	
2	□	SUM(SAL) OVER(ORDER BY SAL	
3		RANGE BETWEEN 10 PRECEDING	
4		AND 10 FOLLOWING) AS RESULT	
5		FROM WINDOW_TAB1;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	NO	SAL	RESULT
1	1	10	45
2	2	11	45
3	3	12	45
4	4	12	45
5	5	25	90
6	6	30	90
7	7	35	90

- ☞ RANGE BETWEEN 10 PRECEDING AND 10 FOLLOWING 은 행 기준이 아닌 값 기준으로 범위를 지정함
- ☞ 현재 SAL 값을 기준으로 ± 10 범위에 해당하는 모든 행을 포함하여 집계하므로,
SAL 값이 10~12 인 구간에서는 동일하게 45, SAL 값이 25~35 인 구간에서는 90 으로 동일한 결과가 출력됨

예제) ROWS 연산 범위

SQL1 *			
1	▶	SELECT NO, SAL,	
2	□	SUM(SAL) OVER(ORDER BY SAL	
3		ROWS BETWEEN UNBOUNDED PRECEDING	
4		AND CURRENT ROW) AS RESULT1	
5		FROM WINDOW_TAB1;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	NO	SAL	RESULT1
1	1	10	10
2	2	11	21
3	3	12	33
4	4	12	45
5	5	25	70
6	6	30	100
7	7	35	135

- ☞ ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW 는 물리적인 행 기준으로 범위를 계산함
- ☞ 동일한 SAL 값을 가진 행이라도 정렬된 순서에 따라 한 행씩 누적되므로,
첫 번째 SAL=12 행에서는 $10 + 11 + 12 = 33$,
다음 SAL=12 행에서는 $10 + 11 + 12 + 12 = 45$ 로 누적 결과가 서로 다르게 출력됨

예제) ROWS 연산 범위 응용

SQL1 *				
1	▶	SELECT	NO, SAL,	
2			SUM(SAL) OVER(ORDER BY SAL	
3			ROWS CURRENT ROW) AS RESULT1,	
4			SUM(SAL) OVER(ORDER BY SAL	
5			ROWS 1 PRECEDING) AS RESULT2	
6		FROM	WINDOW_TAB1;	

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	NO	SAL	RESULT1	RESULT2
1	1	10	10	10
2	2	11	11	21
3	3	12	12	23
4	4	12	12	24
5	5	25	25	37
6	6	30	30	55
7	7	35	35	65

- ROWS CURRENT ROW, ROWS 1 PRECEDING 과 같이 BETWEEN 절을 생략한 경우, 시작 지점만 지정한 것으로 해석되며 종료 범위는 자동으로 CURRENT ROW 가 적용됨
- RESULT1: 현재 행만을 범위로 지정하므로, 각 행의 SAL 값이 그대로 출력됨
- RESULT2: 현재 행과 바로 이전 1 행을 범위로 지정하여 합계를 계산함. 따라서 두 번째 행부터는 이전 행의 SAL 값이 함께 누적되어 출력됨

2. AVG, MIN, MAX, COUNT 도 SUM 함수와 사용 방법 동일

기출) 다음 SQL 결과를 순서대로 나열한 것은?

TAB

COL1
10
20
40
40
60

```
SELECT SUM(COL1) OVER(ORDER BY COL1
                      RANGE BETWEEN 10 PRECEDING AND 20 FOLLOWING)
FROM TAB;
```

- ① 30, 110, 140, 140, 60
- ② 30, 70, 140, 140, 60
- ③ 10, 30, 110, 140, 200
- ④ 30, 110, 120, 140, 60

정답 ①

해설: 현재 COL1 값을 기준으로 (현재 값 - 10) ~ (현재 값 + 20) 범위에 해당하는 모든 행을 합산한다.

● 순위 관련 함수

- 값의 순서대로 순위를 리턴하는 함수
- ORDER BY 절에 나열된 순서대로 순위를 계산함(함수에 따라 **동순위 처리 방식이 서로 다름**)

1. RANK

- 값이 같은 경우 동일한 순위를 반환함
- 동순위가 발생하는 경우, 다음 순위는 **연속적으로 증가하지 않음**
- **동순위의 개수만큼 순위가 건너뛴**
- 예) 값이 100, 90, 90, 80 인 경우 → 큰 순서대로의 순위는 1, 2, 2, 4 로 부여됨

** 문법

SQL1 *	
1	SELECT RANK() OVER([PARTITION BY 컬럼]
2	ORDER BY 컬럼 ASC DESC)

2. DENSE_RANK

- 값이 같은 경우 동일한 순위를 반환함
- 동순위가 발생하더라도, **다음 순위는 연속적으로 부여됨**
- 동순위의 개수와 관계없이 **순위 번호가 건너뛰지 않음**
- 예) 값이 100, 90, 90, 80 인 경우 → 큰 순서대로의 순위는 1, 2, 2, 3 으로 부여됨

3. ROW_NUMBER

- 값이 같더라도 동일한 순위를 반환하지 않음
- ORDER BY 절 기준으로 연속된 행 번호를 부여함
- 동일한 값이 존재하더라도 **순위 중복은 발생하지 않음**
- 예) 값이 100, 90, 90, 80 인 경우 → 큰 순서대로의 행 번호는 1, 2, 3, 4 로 부여됨

※ RANK, DENSE_RANK, ROW_NUMBER 문법 동일

예제) EMP 테이블에서 부서번호가 10 인 직원들의 급여를 기준으로 급여 순위를 출력함

SQL1 *	
1	SELECT EMPNO, ENAME, DEPTNO, SAL,
2	RANK() OVER(ORDER BY SAL DESC) AS 급여순위
3	FROM EMP
4	WHERE DEPTNO = 10;

EMPNO	ENAME	DEPTNO	SAL	급여순위
1	7839 KING	10	5000	1
2	7782 CLARK	10	2450	2
3	7934 MILLER	10	1300	3

☞ RANK() 함수는 ORDER BY SAL DESC 기준으로 급여가 높은 순서대로 순위를 부여함

예제) EMP 테이블에서 부서별로 급여를 기준으로 급여 순위를 출력함(급여가 큰 순서대로)

```
SQL1 *
1 ▶ SELECT ENAME, DEPTNO, SAL,
2       RANK() OVER(PARTITION BY DEPTNO ORDER BY SAL DESC) AS RANK1
3       FROM EMP;
```

Result 4

Grid Result Server Output Text Output Explain Plan Statistics

	ENAME	DEPTNO	SAL	RANK1
1	KING	10	5000	1
2	CLARK	10	2450	2
3	MILLER	10	1300	3
4	SCOTT	20	3000	1
5	FORD	20	3000	1
6	JONES	20	2975	3
7	ADAMS	20	1100	4
8	SMITH	20	800	5
9	BLAKE	30	2850	1
10	ALLEN	30	1600	2
11	TURNER	30	1500	3
12	MARTIN	30	1250	4
13	WARD	30	1250	4
14	JAMES	30	950	6

☞ PARTITION BY DEPTNO 를 사용하여 부서별로 독립적인 순위를 계산함

☞ 각 부서 내에서 ORDER BY SAL DESC 기준으로 급여가 높은 순서대로 순위를 부여함

☞ 동일한 급여 값이 존재하는 경우 동일한 순위를 부여하며, 그 다음 순위는 동순위 수만큼 건너뛰어 계산됨

예제) RANK, DENSE_RANK, ROW_NUMBER 비교

```
SQL1 *
1 ▶ SELECT SAL,
2       RANK() OVER(ORDER BY SAL) AS RANK1,
3       DENSE_RANK() OVER(ORDER BY SAL) AS RANK2,
4       ROW_NUMBER() OVER(ORDER BY SAL) AS RANK3
5       FROM EMP
6       WHERE DEPTNO = 30;
```

Result 4

Grid Result Server Output Text Output Explain Plan Statistics

	SAL	RANK1	RANK2	RANK3	
1	950	1	1	1	
2	1250	2	2	2	
3	1250	2	2	3	동순위 처리 차이
4	1500	4	3	4	후순위 처리 차이
5	1600	5	4	5	
6	2850	6	5	6	

☞ 동일한 SAL 값을 가진 경우 RANK와 DENSE_RANK는 동일한 순위를 부여하지만,

RANK는 동순위만큼 다음 순위를 건너뛰고, DENSE_RANK는 연속적으로 순위를 부여함

☞ ROW_NUMBER는 값이 같더라도 순위 중복 없이 행 순서대로 번호를 부여함

4. NTILE

- 특정 컬럼값의 정렬 순서에 따라 행을 지정한 개수의 그룹으로 분할하는 함수
- 각 행이 **어느 그룹에 속하는지에 대한 그룹 번호를 반환함**
- PARTITION BY 를 사용하여 그룹별로 다시 지정한 개수만큼 분할할 수 있음
- 전체 행 수가 그룹 수로 균등하게 나누어지지 않는 경우, 앞쪽 그룹의 행 수가 더 많도록 분리됨
- 예) 전체 14 행을 3 개 그룹으로 분리하면 → 앞 그룹부터 5, 5, 4 로 나뉨

** 문법

```
SQL1 *
1 SELECT NTILE(N) OVER([PARTITION BY 컬럼]
2 ORDER BY 컬럼 ASC|DESC)
```

예제) EMP 테이블에서 급여를 기준으로 정렬한 후, 전체 데이터를 2 개의 그룹으로 분할하여 출력함

SQL1 *				
1	SELECT ENAME, SAL, DEPTNO,			
2	NTILE(2) OVER(ORDER BY SAL) AS GROUP_NUMBER			
3	FROM EMP;			

Result				
	ENAME	SAL	DEPTNO	GROUP_NUMBER
1	SMITH	800	20	1
2	JAMES	950	30	1
3	ADAMS	1100	20	1
4	WARD	1250	30	1
5	MARTIN	1250	30	1
6	MILLER	1300	10	1
7	TURNER	1500	30	1
8	ALLEN	1600	30	2
9	CLARK	2450	10	2
10	BLAKE	2850	30	2
11	JONES	2975	20	2
12	SCOTT	3000	20	2
13	FORD	3000	20	2
14	KING	5000	10	2

- ☞ NTILE(2)는 ORDER BY SAL 기준으로 정렬된 결과를 두 개의 그룹으로 분리함
- ☞ 전체 행 수가 균등하게 나누어지지 않는 경우, 앞쪽 그룹에 더 많은 행이 배정됨
- ☞ 각 행에는 자신이 속한 그룹 번호가 함께 출력됨

기출) 다음 출력 결과에 알맞은 빈칸에 들어갈 내용을 순서대로 나열한 것은?

TAB

NAME	SCORE
SMITH	100
ALLEN	90
WARD	90
SCOTT	80
BLAKE	70

```
SELECT NAME, SCORE,
        _____ OVER (ORDER BY SCORE DESC) AS RANK1,
        _____ OVER (ORDER BY SCORE DESC) AS RANK2,
        _____ OVER (ORDER BY SCORE DESC) AS RANK3
FROM TAB;
```

출력 결과

NAME	SCORE	RANK1	RANK2	RANK3
SMITH	100	1	1	1
ALLEN	90	2	2	2
WARD	90	2	2	3
SCOTT	80	3	4	4
BLAKE	70	4	5	5

- ① RANK, DENSE_RANK, ROW_NUMBER
- ② DENSE_RANK, RANK, ROW_NUMBER
- ③ RANK, ROW_NUMBER, DENSE_RANK
- ④ ROW_NUMBER, DENSE_RANK, RANK

정답 ②

해설: RANK1 은 동순위가 발생해도 다음 순위가 연속적으로 부여되므로 DENSE_RANK 의 결과이다.

RANK2 는 동순위 다음 순위가 건너뛰어 부여되므로 RANK 의 결과이다.

RANK3 는 값이 같더라도 서로 다른 번호를 부여하므로 ROW_NUMBER 의 결과이다.

● 이전/이후 행 참조 함수

1. LAG

- 각 행마다 정렬 기준에 따라 이전 행의 값을 참조할 수 있음
- **ORDER BY 절**을 기준으로 이전 행이 결정되므로 **필수로 사용해야 함**
- 이전 행이 존재하지 않는 경우 NULL 이 반환됨
- 기본적으로 바로 이전 1 행의 값을 참조함(필요 시 값을 지정하여 여러 행 이전 값 참조 가능)

** 문법

SQL1 *
1 SELECT LAG(컬럼 [,N] [,VALUE]) OVER([PARTITION BY ...] ORDER BY ...)

** 옵션

구분	설명
컬럼	• 참조할 대상 컬럼
N	• N 번째 이전 행을 참조함 (DEFAULT 1)
VALUE	• 이전 행이 존재하지 않을 경우 출력되는 값 (DEFAULT NULL)

2. LEAD

- 각 행마다 정렬 기준에 따라 이후 행의 값을 참조할 수 있음

** LAG 와 문법 동일

예제) EMP 테이블에서 부서번호가 20 인 직원들을 입사일 순으로 정렬한 후, 각 직원의 직전 사원의 급여를 함께 출력함

SQL1 *
1 SELECT ENAME, HIREDATE, SAL,
2 LAG(SAL) OVER(ORDER BY HIREDATE) AS 직전상사급여
3 FROM EMP
4 WHERE DEPTNO = 20;

Result
Grid Result Server Output Text Output Explain Plan Statistics
ENAME HIREDATE SAL 직전상사급여
1 SMITH 1980/12/17 00:00:00 800
2 JONES 1981/04/02 00:00:00 2975 800
3 FORD 1981/12/03 00:00:00 3000 2975
4 SCOTT 1987/04/19 00:00:00 3000 3000
5 ADAMS 1987/05/23 00:00:00 1100 3000

☞ LAG(SAL)은 ORDER BY HIREDATE 기준으로 이전 행의 급여 값을 참조함

☞ 가장 먼저 입사한 직원은 이전 행이 존재하지 않으므로 NULL 이 출력됨

☞ 이후 행부터는 직전에 입사한 사원의 급여가 차례대로 출력됨

두 번째 이전 급여 출력)

SQL1 *				
1	▶	SELECT	ENAME, HIREDATE, SAL,	
2			LAG(SAL, 2) OVER(ORDER BY HIREDATE) AS 직전상사급여	
3		FROM	EMP	
4		WHERE	DEPTNO = 20;	

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	ENAME	HIREDATE	SAL	직전상사급여
1	SMITH	1980/12/17 00:00:00	800	
2	JONES	1981/04/02 00:00:00	2975	
3	FORD	1981/12/03 00:00:00	3000	800
4	SCOTT	1987/04/19 00:00:00	3000	2975
5	ADAMS	1987/05/23 00:00:00	1100	3000

☞ HIREDATE 기준으로 두 번째 이전 행의 급여 값을 참조함(이전 행이 존재하지 않는 경우 NULL 이 출력됨)

이전 값이 없을 경우의 출력 값 지정)

SQL1 *				
1	▶	SELECT	ENAME, HIREDATE, SAL,	
2			LAG(SAL, 2, 0) OVER(ORDER BY HIREDATE) AS 직전상사급여	
3		FROM	EMP	
4		WHERE	DEPTNO = 20;	

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	ENAME	HIREDATE	SAL	직전상사급여
1	SMITH	1980/12/17 00:00:00	800	0
2	JONES	1981/04/02 00:00:00	2975	0
3	FORD	1981/12/03 00:00:00	3000	800
4	SCOTT	1987/04/19 00:00:00	3000	2975
5	ADAMS	1987/05/23 00:00:00	1100	3000

☞ 두 단계 이전 행이 존재하지 않는 경우, 기본값으로 지정한 0 이 출력됨

예제) POINT 테이블에서 등급(GRADE)과 점수(POINT) 기준으로 정렬한 후, 각 행의 다음 점수와 현재 점수 간의 차이를 출력함

SQL1 *				
1	▶	SELECT	ID, POINT, GRADE,	
2			LEAD(POINT) OVER(ORDER BY GRADE DESC, POINT) AS LEAD_VALUE,	
3			LEAD(POINT, 1, POINT) OVER(ORDER BY GRADE DESC, POINT) - POINT AS POINT_GAP	
4		FROM	POINT;	

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	ID	POINT	GRADE	LEAD_VALUE POINT_GAP
1	1	100	C	140 40
2	2	140	C	200 60
3	3	200	B	260 60
4	4	260	B	310 50
5	5	310	A	350 40
6	6	350	A	

☞ LEAD 함수를 사용하여 정렬 기준에 따른 다음 행의 점수를 참조하고, 현재 점수와 차이를 계산함

3. FIRST_VALUE

- ORDER BY 절로 지정한 정렬 순서에 따라 첫 번째 값을 반환함
- ORDER BY 절을 통해 **최솟값 또는 최댓값을 선택적으로 반환할 수 있음**
- PARTITION BY 절을 사용하여 그룹별 첫 번째 값을 반환할 수 있음(PARTITION BY 절은 생략 가능)
- **ORDER BY 절 생략 가능**(생략 시 기본 출력 순서대로의 첫 번째 값 반환)

**문법

```
SQL1 *
1 SELECT FIRST_VALUE(대상) OVER([PARTITION BY 컬럼]
2                                [ORDER BY 컬럼]
3                                [RANGE|ROWS BETWEEN A AND B])
```

4. LAST_VALUE

- ORDER BY 절로 지정한 정렬 순서에 따라 마지막 값을 반환함
- ** FIRST_VALUE 와 문법 동일

예제) EMP 테이블에서 부서번호가 10 인 직원들의 급여 중 최솟값을 각 행과 함께 출력함(FIRST_VALUE 사용)

```
SQL1 *
1 SELECT ENAME, SAL,
2        FIRST_VALUE(SAL) OVER(ORDER BY SAL) AS MIN_SAL
3 FROM EMP
4 WHERE DEPTNO = 10;
```

Result

	ENAME	SAL	MIN_SAL
1	MILLER	1300	1300
2	CLARK	2450	1300
3	KING	5000	1300

☞ FIRST_VALUE(SAL)을 ORDER BY SAL 기준으로 사용하여 부서 내 최저 급여 값을 모든 행에 함께 출력함

예제) EMP 테이블에서 부서번호가 10 인 직원들의 급여 중 최댓값을 각 행과 함께 출력함 (FIRST_VALUE 사용)

```
SQL1 *
1 SELECT ENAME, SAL,
2        FIRST_VALUE(SAL) OVER(ORDER BY SAL DESC) AS MAX_SAL
3 FROM EMP
4 WHERE DEPTNO = 10;
```

Result

	ENAME	SAL	MAX_SAL
1	KING	5000	5000
2	CLARK	2450	5000
3	MILLER	1300	5000

☞ FIRST_VALUE(SAL)을 ORDER BY SAL DESC 기준으로 사용하여 부서 내 최고 급여 값을 모든 행에 함께 출력함

예제) EMP 테이블에서 부서번호가 10 인 직원들의 급여 중 최댓값을 각 행과 함께 출력함(LAST_VALUE 사용)
잘못된 최대 급여 출력)

SQL1 *			
1	▶	SELECT ENAME, SAL,	
2		LAST_VALUE(SAL) OVER(ORDER BY SAL) AS MAX_SAL	
3		FROM EMP	
4		WHERE DEPTNO = 10;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	ENAME	SAL	MAX_SAL
1	MILLER	1300	1300
2	CLARK	2450	2450
3	KING	5000	5000

기본 윈도우 프레임이 RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW 이므로,
각 행에서 자기 자신까지의 범위 중 마지막 값이 반환되어 결과적으로 SAL 과 동일하게 출력됨

올바른 최대 급여 출력)

SQL1 *			
1	▶	SELECT ENAME, SAL,	
2		LAST_VALUE(SAL) OVER(ORDER BY SAL	
3		ROWS BETWEEN UNBOUNDED PRECEDING	
4		AND UNBOUNDED FOLLOWING) AS MAX_SAL	
5		FROM EMP	
6		WHERE DEPTNO = 10;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	ENAME	SAL	MAX_SAL
1	MILLER	1300	5000
2	CLARK	2450	5000
3	KING	5000	5000

ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING 을 지정하여 연산 범위를
전체 행으로 확장하고, 급여의 최댓값을 각 행마다 동일하게 출력함

예제) EMP 테이블에서 부서번호가 10 인 직원들의 급여 중 최솟값을 각 행과 함께 출력함(LAST_VALUE 사용)

SQL1 *			
1	▶	SELECT ENAME, SAL,	
2		LAST_VALUE(SAL) OVER(ORDER BY SAL DESC	
3		ROWS BETWEEN UNBOUNDED PRECEDING	
4		AND UNBOUNDED FOLLOWING) AS MIN_SAL	
5		FROM EMP	
6		WHERE DEPTNO = 10;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	ENAME	SAL	MIN_SAL
1	KING	5000	1300
2	CLARK	2450	1300
3	MILLER	1300	1300

연산 범위를 전체 행으로 확장한 뒤 급여 역순으로 정렬하여, 부서 내 전체 급여 중 최솟값을 반환함

기출) 다음 중 MIN 값과 동일한 값을 출력할 수 있는 함수로 옳은 것은?
(단, ORDER BY SCORE ASC 기준이며, 필요한 경우 윈도우 프레임 설정을 적절히 지정한다.)

- ① LAST_VALUE(SCORE) OVER (ORDER BY SCORE ASC)
- ② FIRST_VALUE(SCORE) OVER (ORDER BY SCORE ASC)
- ③ RANK() OVER (ORDER BY SCORE ASC)
- ④ NTILE(1) OVER (ORDER BY SCORE ASC)

정답 ②

해설: FIRST_VALUE(SCORE) OVER (ORDER BY SCORE ASC)는 오름차순 정렬 기준으로 가장 먼저 위치한 값, 즉 최소값(MIN)을 반환한다. LAST_VALUE는 기본 윈도우 프레임 특성상 현재 행 기준의 마지막 값을 반환하므로 MIN과 동일한 결과를 보장하지 않는다.

● 비율 관련 함수

1. RATIO_TO_REPORT

- 각 값이 전체 합에서 차지하는 비율을 계산하는 함수임
- 결과 값은 0과 1 사이의 실수 형태로 반환됨
- OVER 절을 사용하나, ORDER BY 절은 사용할 수 없음
- 정렬 순서와 무관하게 전체 대비 비율만 계산함

** 문법

SQL1 *

```
1 RATIO_TO_REPORT(대상) OVER([PARTITION BY 컬럼])
```

예제) PERFORMANCE 테이블에서 특정 날짜와 지점(101번)에 대해 각 사원의 실적이 전체 실적 합에서 차지하는 비율을 출력함

SQL1 *

```
1 SELECT EMPNO, NAME, ACHIEVEMENT,
2        ROUND(RATIO_TO_REPORT(ACHIEVEMENT) OVER(), 2) AS 실적비율
3 FROM PERFORMANCE
4 WHERE RDATE = TO_DATE('2024/01/31', 'YYYY/MM/DD')
5 AND BRANCHID = 101;
```

Result

Grid Result	Server Output	Text Output	Explain Plan	Statistics
EMPNO	NAME	ACHIEVEMENT	실적비율	
1	2 JANE	140	0.26	
2	4 TOM	122	0.23	
3	6 ANNA	150	0.28	
4	9 EVA	125	0.23	

☞ RATIO_TO_REPORT(ACHIEVEMENT)를 사용하여 각 행의 실적 ÷ 전체 실적 합으로 비율을 계산함

☞ 예를 들어 JANE의 경우, 전체 실적 합(140 + 122 + 150 + 125 = 537) 중
JANE의 실적 140이 차지하는 비율은 $140 \div 537 \approx 0.26$ 으로 계산됨

2. CUME_DIST

- 현재 값이 전체 데이터 중 어디까지의 위치인지를 비율로 알려주는 함수임
- 정렬 기준으로 현재 값이 **작은 쪽부터 몇 번째 위치까지 포함되는지**를 전체 개수로 나눈 비율을 반환함
- 결과는 0보다 크고 1 이하의 값으로 출력됨
- **가장 작은 값이라도 0이 아닌 값이 출력되며, 가장 큰 값은 항상 1로 출력됨**
- 계산을 위해 **ORDER BY 절이 반드시 필요함**

** 문법

```
SQL1 *
1 CUME_DIST() OVER([PARTITION BY 컬럼]
2 ORDER BY 컬럼 ASC|DESC)
```

** 계산 방법

$CUME_DIST = (\text{현재 값보다 작거나 같은 값의 개수}) \div (\text{전체 행 수})$

예) [10, 20, 30, 40, 50] 데이터가 존재할 경우

- $CUME_DIST(10) = 1 / 5 = 0.2$
- $CUME_DIST(30) = 3 / 5 = 0.6$
- $CUME_DIST(50) = 5 / 5 = 1$

예제) CUME_DIST 함수 결과

```
SQL1 *
1 SELECT SAL,
2 COUNT(SAL) OVER() AS TCOUNT,
3 ROUND(CUME_DIST() OVER(ORDER BY SAL), 4) AS RESULT
4 FROM EMP
5 WHERE DEPTNO = 30;
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	SAL	TCOUNT	RESULT
1	950	6	0.1667 ^{1/6}
2	1250	6	0.5 ^{3/6}
3	1250	6	0.5
4	1500	6	0.6667
5	1600	6	0.8333
6	2850	6	1

- ☞ CUME_DIST()에서 범위를 별도로 지정하지 않은 경우, ORDER BY 기준으로 값의 크기(RANGE)를 기준으로 해석되어 계산됨
- ☞ 따라서 SAL 값이 같은 경우(1250), 같은 값 범위로 함께 처리되므로, 두 행 모두 1250 이하 값이 3 개라는 기준으로 $3/6 = 0.5$ 가 동일하게 출력됨

3. PERCENT_RANK

- 정렬 기준으로 봤을 때, 현재 값의 순위가 전체에서 어느 정도 위치에 있는지를 비율로 나타내는 함수임
- 계산 결과는 0 과 1 사이의 값으로 출력됨
- 가장 작은 값은 항상 0, 가장 큰 값은 항상 1 로 출력됨
- RANK 함수 순위를 기준으로 계산하므로 ORDER BY 절이 반드시 필요함

** 문법

SQL1 *
1 PERCENT_RANK() OVER([PARTITION BY 컬럼]
2 ORDER BY 컬럼 ASC DESC)

** 계산 방법

$$\text{PERCENT_RANK} = (\text{RANK} - 1) \div (\text{전체 행 수} - 1)$$

예) [10, 20, 30, 50] 데이터가 존재할 경우(순위는 RANK 함수로 계산함)

- PERCENT_RANK(10) = (1 - 1) / (5 - 1) = 0
- PERCENT_RANK(30) = (3 - 1) / (5 - 1) = 0.5
- PERCENT_RANK(50) = (5 - 1) / (5 - 1) = 1

예제) PERCENT_RANK 결과

SQL1 *
1 SELECT SAL,
2 COUNT(SAL) OVER() AS TCOUNT,
3 ROUND(PERCENT_RANK() OVER(ORDER BY SAL), 4) AS RESULT
4 FROM EMP
5 WHERE DEPTNO = 30;

Result
Grid Result Server Output Text Output Explain Plan Statistics
SAL TCOUNT RESULT
1 950 6 0
2 1250 6 0.2
3 1250 6 0.2
4 1500 6 0.6
5 1600 6 0.8
6 2850 6 1

- ☞ 총 6 개의 행이므로, PERCENT_RANK 는 각 순위마다 1/(6 - 1) 단위로 비율을 계산함
- ☞ 값이 같은 경우 동순위를 가지므로, PERCENT_RANK 결과도 동일하게 출력됨
- ☞ SAL 이 1250 인 경우 순위가 2 위이므로 1/5,
SAL 이 1500 인 경우 순위가 4 위이므로 3/5 가 각각 반환됨

<< 단원 정리 문제 >>

1. 다음 SQL 실행 결과로 적절한 것은? (단, DBMS 는 Oracle)

<TAB1>

COL1	COL2
A	10
A	20
A	20
A	30
B	10
B	10
B	10
B	NULL

```
SELECT RANK() OVER(PARTITION BY COL1 ORDER BY COL2) AS R1
FROM TAB1;
```

①

R1
1
2
2
3
1
1
1
2

②

R1
1
2
2
3
2
2
2
1

③

R1
1
2
2
4
1
1
1
4

④

R1
1
2
2
4
4
4
4
1

2. 다음 SQL 실행 결과로 적절한 것은?

<TAB1>

COL1	COL2
A	5
A	20
A	20
A	40

```
SELECT SUM(COL2) OVER(PARTITION BY COL1 ORDER BY COL2) AS R1,
       SUM(COL2) OVER(PARTITION BY COL1 ORDER BY COL2
                       ROWS UNBOUNDED PRECEDING) AS R2
FROM TAB1;
```

①

R1	R2
5	5
25	25
45	45
85	85

②

R1	R2
5	5
45	45
45	45
85	85

③

R1	R2
5	5
45	25
45	45
85	85

④

R1	R2
5	85
45	85
45	85
85	85

1. ③

Oracle 의 경우 오름차순 정렬 시 NULL 은 가장 마지막에 배치된다. 또한 RANK 함수는 동순위를 인정하며, 그 다음 순위는 연속적이지 않기 때문에 정답은 3 이 된다.

2. ③

R1 은 RANGE 로 누적합을 계산하므로 값이 같은 2, 3 번째 행은 하나로 묶여 동시에 계산된다. 따라서 R1 은 5, 45, 45, 85 순서로 출력된다.

R2 는 행 단위로 각 행마다 처음부터 현재 행까지의 누적합을 계산하기 때문에 순서대로 5, 25, 45, 85 가 출력된다.

2-13. TOP N QUERY

● TOP N QUERY

- 전체 결과 집합에서 상위 N 개의 행을 추출할 때 사용하는 쿼리임
- 정렬 기준(ORDER BY)을 기준으로 상위 데이터를 제한적으로 조회할 수 있음
- 페이징 처리를 효과적으로 수행하기 위해 사용됨
- 예) 최근 결제 내역 상위 10 건을 출력한 후, 그 다음 결제 내역을 10 건씩 순차적으로 출력

● TOP-N 추출 방법

구분	종류
Oracle	• ROWNUM • RANK 함수 • FETCH (12c 이후)
SQL Server	• TOP N • RANK 함수 • OFFSET-FETCH

1. ROWNUM

- 출력되는 순서대로 부여되는 가상의 행 번호를 의미함
- 실제 테이블에 저장된 값이 아닌 조회 과정에서 임시로 부여되는 번호이므로,
절대적인 행 번호로 사용할 수 없음
- 주로 행의 개수를 제한하기 위한 용도로 사용됨
- 예) WHERE ROWNUM <= 10 → 10 개 행만 출력

** 주의

- ROWNUM 은 1 부터 순차적으로 부여되므로, ROWNUM = 1 을 제외한 다른 값에 대해 =(EQUAL) 연산자를 사용할 수 없음
- ROWNUM > N 조건은 의미가 없으므로 사용 불가

예제) ROWNUM 출력

SQL1 *					
1	▶	SELECT	ROWNUM,	EMPNO, ENAME, SAL, JOB	
2		FROM	EMP		
3		WHERE	DEPTNO = 20;		

Result					
	ROWNUM	EMPNO	ENAME	SAL	JOB
1	1	7369	SMITH	800	CLERK
2	2	7566	JONES	2975	MANAGER
3	3	7788	SCOTT	3000	ANALYST
4	4	7876	ADAMS	1100	CLERK
5	5	7902	FORD	3000	ANALYST

SQL1 *					
1	▶	SELECT	ROWNUM,	EMPNO, ENAME, JOB	
2		FROM	EMP		
3		WHERE	SAL >= 2500;		

Result					
	ROWNUM	EMPNO	ENAME	JOB	
1	1	7566	JONES	MANAGER	
2	2	7698	BLAKE	MANAGER	
3	3	7788	SCOTT	ANALYST	
4	4	7839	KING	PRESIDENT	
5	5	7902	FORD	ANALYST	

☞ ROWNUM 은 WHERE 절 조건에 의해 추출된 결과 집합에 대해 다시 순서대로 부여되므로, 조건이 달라지면 두 그림처럼 ROWNUM 값도 달라짐

예제) 잘못된 ROWNUM 의 사용

CASE1) > 조건 사용 시

SQL1 *	
1	SELECT *
2	FROM EMP
3	WHERE ROWNUM > 1;

Result							
Grid Result	Server Output	Text Output	Explain Plan	Statistics			
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO

☞ ROWNUM 은 행이 출력되면서 1 부터 순차적으로 부여되기 때문에, 아직 1 이 부여되지 않은 상태에서는 ROWNUM > 1 조건을 만족할 수 없어 결과가 출력되지 않는다.

CASE2) EQUAL(=) 조건 사용 시

SQL1 *	
1	SELECT *
2	FROM EMP
3	WHERE ROWNUM = 4;

Result							
Grid Result	Server Output	Text Output	Explain Plan	Statistics			
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO

☞ ROWNUM 은 1 부터 순차적으로 부여되기 때문에 중간 번호를 직접 지정하는 조건(ROWNUM = 4)은 성립할 수 없어 결과가 출력되지 않는다.

예제) 올바른 ROWNUM 의 사용

SQL1 *	
1	SELECT EMPNO, ENAME, DEPTNO, SAL
2	FROM EMP
3	WHERE ROWNUM <= 5;

Result							
Grid Result	Server Output	Text Output	Explain Plan	Statistics			
EMPNO	ENAME	DEPTNO	SAL				
1	7369	SMITH	20	800			
2	7499	ALLEN	30	1600			
3	7521	WARD	30	1250			
4	7566	JONES	20	2975			
5	7654	MARTIN	30	1250			

☞ 첫 번째 행이 반드시 정의되어야 두 번째 행(ROWNUM=2)이 정의될 수 있기 때문에, 첫 번째 행을 포함하는 ROWNUM 에 대한 선택은 가능함

예제) EMP 테이블에서 급여가 높은 순서대로 상위 5 명의 직원 정보를 출력
(잘못된 TOP 5 출력)

SQL1 *

```

1 SELECT EMPNO, ENAME, DEPTNO, SAL
2 FROM EMP
3 WHERE ROWNUM <= 5
4 ORDER BY SAL DESC;

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	EMPNO	ENAME	DEPTNO	SAL
1	7566	JONES	20	2975
2	7499	ALLEN	30	1600
3	7654	MARTIN	30	1250
4	7521	WARD	30	1250
5	7369	SMITH	20	800

SQL1 *

```

1 SELECT SAL
2 FROM EMP
3 ORDER BY SAL DESC;
4

```

Result

Grid Result Server Output Text O

	SAL
1	5000
2	3000
3	3000
4	2975
5	2850
6	2450
7	1600
8	1500
9	1300
10	1250
11	1250
12	1100
13	950
14	800

☞ 실제로 급여 기준 상위 5 명이 출력되지 않음(전체 최대 급여는 5000 임)

☞ 추출 원리: WHERE 절에서 먼저 5 개 행을 추출한 뒤, 해당 결과 집합에 대해 정렬을 수행함

☞ 해결: 실행 순서를 변경하여 정렬을 먼저 수행한 후(ORDER BY → ROWNUM), 상위 N 개를 추출해야 함

올바른 TOP 5 출력)

SQL1 *

```

1 SELECT *
2 FROM (SELECT EMPNO, ENAME, SAL, JOB
3 FROM EMP
4 ORDER BY SAL DESC)
5 WHERE ROWNUM <= 5;

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	EMPNO	ENAME	SAL	JOB
1	7839	KING	5000	PRESIDENT
2	7788	SCOTT	3000	ANALYST
3	7902	FORD	3000	ANALYST
4	7566	JONES	2975	MANAGER
5	7698	BLAKE	2850	MANAGER

☞ 서브쿼리에서 ORDER BY SAL DESC 를 먼저 수행하여 급여 기준으로 정렬한 뒤, 그 결과 집합에 대해 바깥 쿼리에서 ROWNUM <= 5 를 적용하면 급여 기준 상위 5 명이 의도대로 정확히 출력됨

예제) EMP 테이블에서 급여가 높은 순서대로 4 번째부터 6 번째에 해당하는 직원 정보를 출력
잘못된 출력 1)

SQL1 *				
1	▶	SELECT *		
2	□	FROM (SELECT EMPNO, ENAME, SAL, JOB		
3	□	FROM EMP		
4	□	ORDER BY SAL DESC)		
5		WHERE ROWNUM BETWEEN 4 AND 6;		

Result				
Grid Result	Server Output	Text Output	Explain Plan	Statistics
EMPNO	ENAME	SAL	JOB	

☞ ROWNUM 은 시작 값(1)을 건너뛴 상태에서 그 이후 행 번호를 바로 추출할 수 없음

잘못된 출력 2)

SQL1 *

```
1 SELECT *
2 FROM (SELECT EMPNO, ENAME, SAL, JOB, ROWNUM AS RN
3        FROM EMP
4        ORDER BY SAL DESC)
5 WHERE RN BETWEEN 4 AND 6
6 ORDER BY RN;
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	EMPNO	ENAME	SAL	JOB	RN
1	7566	JONES	2975	MANAGER	4
2	7654	MARTIN	1250	SALESMAN	5
3	7698	BLAKE	2850	MANAGER	6

SQL1 *				
1	▶	SELECT SAL		
2		FROM EMP		
3		ORDER BY SAL DESC;		
4				

Result				
Grid Result	Server Output	Text O		
SAL				
1	5000			
2	3000			
3	3000			
4	2975			
5	2850			
6	2450			
7	1600			
8	1500			
9	1300			
10	1250			
11	1250			
12	1100			
13	950			
14	800			

☞ 실제 4,5,6 순위가 출력되지 않음

☞ 원인: 서브쿼리에서 ROWNUM AS RN 을 부여하는 시점은 ORDER BY SAL DESC 정렬 결과가 확정되기 전이므로, RN 은 급여 내림차순 순서를 보장하지 못함

☞ 해결: 정렬이 먼저 수행된 결과에 대해 ROWNUM 을 부여하도록 추가 서브쿼리를 사용해야 함

올바른 출력)

SQL1 *

1 ▶

2

3

4

5

6

7

```
SELECT *
FROM (SELECT EMPNO, ENAME, SAL, JOB, ROWNUM AS RN
      FROM (SELECT EMPNO, ENAME, SAL, JOB
            FROM EMP
            ORDER BY SAL DESC))
WHERE RN BETWEEN 4 AND 6
ORDER BY RN;
```

Result

Grid ResultServer OutputText OutputExplain PlanStatistics

	EMPNO	ENAME	SAL	JOB	RN
1	7566	JONES	2975	MANAGER	4
2	7698	BLAKE	2850	MANAGER	5
3	7782	CLARK	2450	MANAGER	6

☞ 정렬을 가장 안쪽 서브쿼리에서 먼저 수행한 뒤 ROWNUM 을 부여했기 때문에,

급여 내림차순 기준으로 정확한 순위가 매겨지고, 그 결과 4~6 번째 행을 올바르게 추출할 수 있음

2. 순위 계열 윈도우 함수

- 윈도우 함수의 순위 관련 함수를 이용하여 상위 N 개의 데이터를 추출할 수 있음
- 순위 부여 방식에 따라 RANK, DENSE_RANK, ROW_NUMBER 중 적절한 함수를 선택하여 사용함
- 동순위 처리 여부 및 다음 순위 부여 방식이 함수마다 다르므로 목적에 맞게 구분해서 활용해야 함

예제) EMP 테이블에서 급여가 높은 순서대로 4 번째부터 6 번째에 해당하는 직원 정보를 출력

SQL1 *					
1	SELECT *				
2	FROM (SELECT EMPNO, ENAME, SAL, JOB,				
3	RANK() OVER(ORDER BY SAL DESC) AS RN				
4	FROM EMP)				
5	WHERE RN BETWEEN 4 AND 6				
6	ORDER BY RN;				

EMPNO	ENAME	SAL	JOB	RN
1	7566 JONES	2975	MANAGER	4
2	7698 BLAKE	2850	MANAGER	5
3	7782 CLARK	2450	MANAGER	6

SQL1 *					
1	SELECT *				
2	FROM (SELECT EMPNO, ENAME, JOB, SAL,				
3	ROW_NUMBER() OVER(ORDER BY SAL DESC) AS RN				
4	FROM EMP)				
5	WHERE RN BETWEEN 4 AND 6				
6	ORDER BY RN;				

EMPNO	ENAME	JOB	SAL	RN
1	7566 JONES	MANAGER	2975	4
2	7698 BLAKE	MANAGER	2850	5
3	7782 CLARK	MANAGER	2450	6

☞ RANK, DENSE_RANK, ROW_NUMBER 와 같은 윈도우 함수를 사용하여 급여 순위를 구한 뒤, 이를 통해 4~6 위에 해당하는 급여 순위를 추출할 수 있음

3. FETCH 절

- 결과 집합에서 출력할 행의 수를 제한하기 위한 절
- Oracle 12c 이상부터 사용 가능하며, 이전 버전에서는 주로 ROWNUM 을 사용함
- SQL Server 에서도 지원됨
- ORDER BY 절 뒤에 작성해야 하며, 정렬된 결과를 기준으로 행을 제한함

** 문법

SQL1 *	
1	SELECT
2	FROM
3	WHERE
4	GROUP BY
5	HAVING
6	ORDER BY
7	OFFSET N { ROW ROWS }
8	FETCH { FIRST NEXT } N { ROW ROWS } { ONLY WITH TIES };

** 옵션

구분	설명
OFFSET N	• 건너뛴 행의 수를 지정 • 예) 성적이 높은 순으로 정렬 후 상위 1 건을 제외하고 나머지 3 건 출력 • Oracle 의 경우 건너뛴 행이 없으면 OFFSET 절을 생략할 수 있음 • SQL Server 에서는 OFFSET 절이 필수
ROW ROWS	• 단수/복수 표현 차이만 있으며 의미상 동일(결과에 영향 없음)
FETCH	• 출력할 행의 수를 지정하는 구문
FIRST NEXT	• 구분하지 않고 사용
ONLY WITH TIES	• ONLY: 정해진 개수만 출력(중복값 있더라도 무시) • WITH TIES: 중복값도 함께 출력

예제) EMP 테이블에서 급여가 높은 순서대로 상위 5 명의 직원 정보를 출력

SQL1 *				
1	▶	SELECT	EMPNO, ENAME, JOB, SAL	
2		FROM	EMP	
3		ORDER BY	SAL DESC	
4		FETCH FIRST 5 ROWS ONLY;		

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	EMPNO	ENAME	JOB	SAL
1	7839	KING	PRESIDENT	5000
2	7788	SCOTT	ANALYST	3000
3	7902	FORD	ANALYST	3000
4	7566	JONES	MANAGER	2975
5	7698	BLAKE	MANAGER	2850

SQL1 *	
1	▶ SELECT SAL
2	FROM EMP
3	ORDER BY SAL DESC;
4	

Result	
Grid Result Server Output Text O	
	SAL
1	5000
2	3000
3	3000
4	2975
5	2850
6	2450
7	1600
8	1500
9	1300
10	1250
11	1250
12	1100
13	950
14	800

☞ FETCH 를 사용하여 급여를 내림차순으로 정렬한 결과 중에서 상위 5 명의 직원만 정상적으로 출력할 수 있음

예제) EMP 테이블에서 급여가 높은 순서대로 4 번째부터 6 번째에 해당하는 직원 정보를 출력

SQL1 *				
1	▶	SELECT	EMPNO, ENAME, JOB, SAL	
2		FROM	EMP	
3		ORDER BY	SAL DESC	
4		OFFSET 3 ROW		
5		FETCH FIRST 3 ROW ONLY;		
6				

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	EMPNO	ENAME	JOB	SAL
1	7566	JONES	MANAGER	2975
2	7698	BLAKE	MANAGER	2850
3	7782	CLARK	MANAGER	2450

☞ 급여를 내림차순으로 정렬한 후 OFFSET 을 사용하여 상위 3 명을 건너뛰고, FETCH 를 통해 그 다음 3 명의 직원만 출력함

기출) 다음 SQL 출력 결과를 순서대로 나열한 것은? (단, DBMS 는 Oracle)

TAB

COL1
NULL
80
80
80
70
NULL
90

SELECT * FROM TAB ORDER BY COL1 DESC OFFSET 2 ROW FETCH FIRST 3 ROW ONLY;

- ① 90 → 80 → 80
- ② 80 → 80 → 70
- ③ 80 → 70 → NULL
- ④ NULL → 90 → 80

정답 ①

해설: 오라클에서는 ORDER BY COL1 DESC 수행 시 NULL 이 기본적으로 앞쪽에 배치된다(NULLS FIRST). 따라서 정렬 결과는 NULL, NULL, 90, 80, 80, 80, 70 순서이며, OFFSET 2 로 NULL 2 개를 제외한 뒤 FETCH FIRST 3 으로 90, 80, 80 을 출력한다.

4. TOP N 쿼리

- SQL Server 에서 상위 N 개 행을 추출하기 위한 문법임
- TOP 절은 SELECT 바로 뒤에 위치함
- 정렬 기준이 없으면 임의의 N 행이 반환될 수 있으므로 일반적으로 ORDER BY 와 함께 사용함
- 서브쿼리 없이 단일 쿼리로 정렬된 결과에서 상위 N 개 행을 바로 추출할 수 있음
- **WITH TIES** 옵션을 사용하면, 상위 N 개에 포함된 값과 **동일한 순위의 행도 함께 출력됨**

** 문법

SQL1 *
1 SELECT TOP N [WITH TIES] 컬럼1, 컬럼2, ...
2 FROM 테이블명
3 ORDER BY 정렬컬럼1 [ASC DESC], 정렬컬럼2 [ASC DESC], ...;

예제) SQL Server 에서 EMP 테이블의 급여가 높은 순서대로 상위 2 명의 직원 정보 출력

SQL1 *	
1	SELECT TOP 2 ENAME, SAL
2	FROM EMP
3	ORDER BY SAL DESC;

Result	
ENAME	SAL
1 KING	5000
2 SCOTT	3000

SQL1 *	
1	SELECT TOP 2 WITH TIES ENAME, SAL
2	FROM EMP
3	ORDER BY SAL DESC;

Result	
ENAME	SAL
1 KING	5000
2 FORD	3000
3 SCOTT	3000

☞ SAL을 내림차순으로 정렬하면 5000, 3000, 3000 순서가 되며, TOP 2를 적용할 경우 5000과 첫 번째 3000만 출력됨

☞ WITH TIES를 사용하면 TOP N에 포함된 값과 동일한 급여(3000)를 가진 행도 함께 출력됨

기출) 다음 SQL 출력 결과를 순서대로 나열한 것은? (단, DBMS 는 SQL Server)

TAB

COL1
NULL
80
90
80
70
NULL
90

SELECT TOP 3 WITH TIES COL1 FROM TAB ORDER BY COL1 DESC;

- ① 90 → 90 → 80
- ② 90 → 90 → 80 → 80
- ③ 90 → 80 → 80
- ④ 80 → 80 → 70

정답 ②

해설: SQL Server 에서 ORDER BY COL1 DESC 를 수행하면 NULL 값은 마지막에 배치된다.

따라서 정렬 결과는 다음과 같다. 90 → 90 → 80 → 80 → 70 → NULL → NULL

TOP 3 에 의해 상위 3 개의 값은 90, 90, 80 이다.

WITH TIES 옵션을 사용하였으므로, 3 번째 행의 값(80)과 동일한 값을 가진 행도 함께 출력된다.

따라서 나머지 80 도 포함되어 최종 출력 결과는 90 → 90 → 80 → 80 이 된다.

<< 단원 정리 문제 >>

1. 다음 SQL 수행 결과로 가장 적절한 것은? (단, DBMS는 SQL Server)

<TAB1>

NAME	SCORE
김길동	1000
박길동	2000
최길동	1800
엄길동	2500
홍길동	3000
마길동	2000

```
SELECT TOP 3 WITH TIES NAME, SCORE
FROM TAB1
ORDER BY SCORE DESC, NAME;
```

①

NAME	SCORE
홍길동	3000
엄길동	2500
박길동	2000

②

NAME	SCORE
홍길동	3000
엄길동	2500
마길동	2000

③

NAME	SCORE
홍길동	3000
엄길동	2500
박길동	2000
마길동	2000

④

NAME	SCORE
홍길동	3000
엄길동	2500
마길동	2000
박길동	2000

2. 다음 SQL 수행 결과로 가장 적절한 것은? (단, DBMS는 Oracle)

<TAB1>

ID	POINT
1	500
2	600
3	600
4	0
5	900
6	1000

```
SELECT ID, POINT
FROM TAB1
WHERE ROWNUM <= 3
ORDER BY POINT DESC;
```

①

ID	SCORE
1	500
2	600
3	600

②

ID	SCORE
6	1000
5	900
2	600

③

ID	SCORE
2	600
3	600
1	500

④

ID	SCORE
3	600
2	600
1	500

3. 다음 중 SQL 실행 결과가 다른 하나를 고르시오
(단, 테이블 데이터는 입력된 순서대로 아래와 같다)

<TAB1>	
NAME	SCORE
김길동	1000
박길동	2000
최길동	1800
엄길동	1800

①

```
SELECT NAME, SCORE
  FROM (SELECT NAME, SCORE, ROW_NUMBER() OVER(ORDER BY SCORE DESC) AS RN
        FROM TAB1)
 WHERE RN <= 3;
```

②

```
SELECT NAME, SCORE
  FROM (SELECT NAME, SCORE, RANK() OVER(ORDER BY SCORE DESC, NAME DESC) AS RN
        FROM TAB1)
 WHERE RN <= 3;
```

③

```
SELECT NAME, SCORE
  FROM TAB1
 WHERE ROWNUM <= 3
 ORDER BY SCORE DESC;
```

④

```
SELECT TOP 3 NAME, SCORE
  FROM TAB1
 ORDER BY SCORE DESC;
```

1. ②

ORDER BY 절의 2차 정렬 컬럼으로 인해 SAL 값이 같더라도 NAME 오름차순 순서대로 순위가 결정된다.

2. ③

오라클에서 ROWNUM은 ORDER BY보다 먼저 부여된다. 따라서 이 SQL은 POINT 기준으로 정렬한 후 상위 3건을 가져오는 것이 아니라, 테이블에서 조회된 순서대로 먼저 3행을 선택한 뒤 정렬한다.

그 결과 (1, 500), (2, 600), (3, 600) 행이 먼저 선택된다.

이후 ORDER BY POINT DESC를 통해 정렬이 수행되며, POINT 값이 동일한 600인 경우에는 추가적인 정렬 기준이 없으므로 입력된 순서대로 ID가 2 → 3 순으로 유지된다.

따라서 최종 출력 결과의 정렬 순서는 (2, 600) → (3, 600) → (1, 500) 이다.

3. ③

데이터의 입력(조회) 순서대로 먼저 ROWNUM이 부여된 후 ORDER BY가 수행되므로, SCORE가 높은 순서대로 상위 3명이 출력되지 않는다.

2-14. 계층형 질의

● 계층형 질의

- 하나의 테이블 내에서 각 행이 서로 관계를 가질 때, 부모-자식 간의 연결고리를 통해 행들 간의 계층(depth)을 표현하는 기법
- 예: DEPT2 테이블에서 부서 간 상·하 관계 표현
- CONNECT BY 절에서 **PRIOR**의 위치에 따라 연결되는 데이터(탐색 방향)가 달라짐
- **LEVEL** 값을 통해 각 행이 계층 구조 내에서 어느 위치(상·하 관계, 깊이)에 있는지 확인할 수 있음

** 문법

SQL1 *	
1	SELECT 컬럼명
2	FROM 테이블명
3	START WITH 시작조건
4	CONNECT BY [NOCYCLE] PRIOR 연결 조건; -- 시작점을 지정하는 조건 전달 -- 시작점 기준으로 연결 데이터를 찾아가는 조건

** 옵션

구분	설명
START WITH	• 계층형 질의에서 데이터를 출력할 시작 행(루트)을 지정하는 조건
CONNECT BY	• 부모 행과 자식 행을 연결하여 계층 구조를 생성하는 조건 • PRIOR의 위치에 따라 데이터 탐색 방향이 달라짐
NOCYCLE	• 계층 구조에서 순환이 발생할 경우 무한 루프가 될 수 있으므로, 이를 방지하기 위해 사용하는 옵션

● 계층형 질의절에서 사용하는 조건의 형태

구분	설명
CONNECT BY 절	• 계층 구조를 설정하는 조건 • START WITH 절에서 시작한 데이터로부터 부모-자식 관계를 탐색하는 규칙을 정의 • 이 조건이 성립하지 않으면, 해당 행을 기준으로 더 이상 하위(자식) 계층을 연결하지 않음
WHERE 절	• 계층 구조가 생성된 이후, 전체 결과를 필터링하는 데 사용 • START WITH 절과 CONNECT BY 절에 의해 연결된 모든 결과가 먼저 생성된 후, WHERE 절 조건에 맞는 데이터만 선택하여 출력 • START WITH 절에서 지정된 데이터라도 WHERE 절 조건에 맞지 않는 경우 최종 결과에서 제외됨

예제) DEPT2 테이블을 이용한 부서의 상·하 구조 표현

SQL1 *					
1	SELECT D.*, LEVEL				
2	FROM DEPT2 D				
3	START WITH PDEPT IS NULL				
4	CONNECT BY PRIOR DCODE = PDEPT;				

Result					
	DCODE	DNAME	PDEPT	AREA	LEVEL
1	0001	사장실		포항본사	1
2	1000	경영지원부	0001	서울지사	2
3	1001	재무관리팀	1000	서울지사	3
4	1002	총무팀	1000	서울지사	3
5	1003	기술부	0001	포항본사	2
6	1004	H/W지원	1003	대전지사	3
7	1005	S/W지원	1003	경기지사	3
8	1006	영업부	0001	포항본사	2
9	1007	영업기획팀	1006	포항본사	3
10	1008	영업1팀	1007	부산지사	4
11	1009	영업2팀	1007	경기지사	4
12	1010	영업3팀	1007	서울지사	4
13	1011	영업4팀	1007	울산지사	4

START WITH 를 사용하여 상위부서코드(PDEPT)가 NULL 인 사장실부터 시작하여 하위 레벨을 탐색한다.
 이때 사장실의 부서번호(DCODE)를 기준으로, 해당 값을 상위부서코드로 갖는 하위 부서를 순차적으로 찾아 내려가게 된다. 따라서 하위 레벨을 탐색하기 위해 먼저 참조되는 DCODE 앞에 PRIOR 가 위치해야 한다.

잘못된 PRIOR 위치 전달)

SQL1 *					
1	SELECT D.*, LEVEL				
2	FROM DEPT2 D				
3	START WITH PDEPT IS NULL				
4	CONNECT BY DCODE = PRIOR PDEPT;				

Result					
	DCODE	DNAME	PDEPT	AREA	LEVEL
1	0001	사장실		포항본사	1

CONNECT BY 절에서 DCODE = PRIOR PDEPT 로 PRIOR 의 위치가 잘못 지정되어,
 상위 행의 PDEPT 값(NULL)을 DCODE 값으로 갖는 하위 데이터를 탐색하게 된다.
 이로 인해 CONNECT BY 조건이 성립하지 않아 START WITH 절에서 지정된 사장실만 출력된다.

기출) 다음은 메뉴 테이블 MENU의 데이터이다. 다음 SQL을 수행한 결과로 가장 적절한 것은?

MENU

MENU_ID	PARENT_ID	MENU_NAME
10	NULL	전체메뉴
11	10	음료
12	10	디저트
111	11	커피
112	11	티
121	12	케이크
122	12	쿠키

```
SELECT MENU_ID, MENU_NAME, LEVEL
FROM MENU
START WITH PARENT_ID IS NULL CONNECT BY PRIOR MENU_ID = PARENT_ID;
```

①

MENU_ID	PARENT_ID	LEVEL
10	전체메뉴	1
11	음료	2
111	커피	3
112	티	3
12	디저트	2
121	케이크	3
122	쿠키	3

②

MENU_ID	PARENT_ID	LEVEL
11	음료	1
111	커피	2
112	티	2
12	디저트	1
121	케이크	2
122	쿠키	2

③

MENU_ID	PARENT_ID	LEVEL
10	전체메뉴	1
11	음료	2
12	디저트	2

④

MENU_ID	PARENT_ID	LEVEL
10	전체메뉴	1

정답 ①

해설: START WITH 절에 의해 전체메뉴(10)가 LEVEL 1이 된다. 이후 CONNECT BY 절 조건(PRIOR MENU_ID = PARENT_ID)에 따라 상위 행의 MENU_ID를 먼저 읽고, 그 값을 PARENT_ID로 갖는 행이 다음 LEVEL로 연결된다. 따라서 10을 PARENT_ID로 갖는 음료(11)와 디저트(12)가 LEVEL 2가 되고, 같은 방식으로 음료의 하위 메뉴인 커피(111)·티(112), 디저트의 하위 메뉴인 케이크(121)·쿠키(122)가 LEVEL 3으로 출력되므로 ①이 정답이다.

예제) CONNECT BY 절을 이용한 조건 전달

SQL1 *				
1	▶	SELECT *		
2		FROM DEPT2		
3		START WITH PDEPT IS NULL		
4		CONNECT BY PRIOR DCODE = PDEPT		
5		AND AREA = '서울지사';		

Result				
	DCODE	DNAME	PDEPT	AREA
1	0001	사장실		포항본사
2	1000	경영지원부	0001	서울지사
3	1001	재무관리팀	1000	서울지사
4	1002	총무팀	1000	서울지사

DCODE	DNAME	PDEPT	AREA	
0001	사장실		포항본사	시작점
1000	경영지원부	0001	서울지사	2-LEVEL 자식
1001	재무관리팀	1000	서울지사	3-LEVEL 자식
1002	총무팀	1000	서울지사	
1003	기술부	0001	포항본사	
1004	H/W지원	1003	대전지사	
1005	S/W지원	1003	경기지사	
1006	영업부	0001	포항본사	
1007	영업기획팀	1006	포항본사	
1008	영업1팀	1007	부산지사	
1009	영업2팀	1007	경기지사	
1010	영업3팀	1007	서울지사	
1011	영업4팀	1007	울산지사	

- CONNECT BY 절에 AND AREA = '서울지사' 조건이 포함되어 있어,
 상위 부서로부터 하위 부서를 탐색하는 과정에서 AREA 가 '서울지사'인 부서만 계층 연결 대상이 된다.
 이로 인해 사장실을 시작으로 하되, 하위 부서 중 서울지사에 속한 부서들만 계층 구조로 연결되어 출력된다.

예제) WHERE 절을 이용한 조건 전달

SQL1 *				
1	▶	SELECT D.*, LEVEL		
2		FROM DEPT2 D		
3		WHERE AREA = '서울지사'		
4		START WITH PDEPT IS NULL		
5		CONNECT BY PRIOR DCODE = PDEPT;		

Result					
	DCODE	DNAME	PDEPT	AREA	LEVEL
1	1000	경영지원부	0001	서울지사	2
2	1001	재무관리팀	1000	서울지사	3
3	1002	총무팀	1000	서울지사	3
4	1010	영업3팀	1007	서울지사	4

DCODE	DNAME	PDEPT	AREA	LEVEL
0001	사장실		포항본사	1
1000	경영지원부	0001	서울지사	2
1001	재무관리팀	1000	서울지사	3
1002	총무팀	1000	서울지사	3
1003	기술부	0001	포항본사	2
1004	H/W지원	1003	대전지사	3
1005	S/W지원	1003	경기지사	3
1006	영업부	0001	포항본사	2
1007	영업기획팀	1006	포항본사	3
1008	영업1팀	1007	부산지사	4
1009	영업2팀	1007	경기지사	4
1010	영업3팀	1007	서울지사	4
1011	영업4팀	1007	울산지사	4

- WHERE 절에 AREA = '서울지사' 조건이 적용되어,
 계층 구조가 생성된 이후 전체 결과 중에서 서울지사에 속한 부서만 선택하여 출력된다.
 이로 인해 START WITH 절에서 지정된 사장실은 계층 구조에는 포함되지만, WHERE 절 조건에 맞지 않아 결과에서는 출력되지 않는다.

기출) 다음은 사내 문서 분류 테이블의 데이터이다. 다음 SQL 을 수행한 결과로 가장 적절한 것은?

MENU

CAT_ID	PARENT_ID	CAT_NAME	TEAM
10	NULL	전자문서	공통
20	10	인사	경영지원
30	10	재무	경영지원
40	20	채용	경영지원
50	20	평가	경영지원
60	30	예산	경영지원
70	30	세무	경영지원

```
SELECT CAT_ID, CAT_NAME, LEVEL
  FROM DOC_CAT
 WHERE TEAM = '경영지원'
START WITH PARENT_ID IS NULL CONNECT BY PRIOR CAT_ID = PARENT_ID;
```

①

CAT_ID	CAT_NAME	LEVEL
10	전자문서	1
20	인사	2
30	재무	2
40	채용	3
50	평가	3
60	예산	3
70	세무	3

②

CAT_ID	CAT_NAME	LEVEL
20	인사	2
30	재무	2
40	채용	3
50	평가	3
60	예산	3
70	세무	3

③

CAT_ID	CAT_NAME	LEVEL
10	전자문서	1
20	인사	2
30	재무	2

④

CAT_ID	CAT_NAME	LEVEL
--------	----------	-------

정답 ②

해설: START WITH 와 CONNECT BY 에 의해 전자문서(10)를 루트로 계층 구조가 먼저 생성된 후, WHERE 절 조건(TEAM='경영지원')을 만족하는 행만 최종적으로 출력된다. 따라서 TEAM 이 '공통'인 전자문서(10)는 결과에서 제외되고, 그 하위 분류부터 출력된다.

● 계층형 질의 가상 컬럼 및 함수

구분	종류	설명
가상 컬럼	LEVEL	- 각 DEPTH 를 표현하는 숫자 - 시작점을 1 로 하여 하위 DEPTH 로 내려갈수록 1 씩 증가함
	CONNECT_BY_ISLEAF	LEAF NODE(최하위 노드) 여부를 나타냄 - 최하위 노드일 경우 1, 그렇지 않으면 0 을 반환함
	CONNECT_BY_ISCYCLE	- 계층형 질의 결과에서 순환이 발생했는지 여부를 나타냄 - 해당 행에서 순환이 발생한 경우 1, 그렇지 않으면 0 을 반환
가상 함수	CONNECT_BY_ROOT 컬럼명	루트 노드(시작점)에 해당하는 컬럼 값을 반환
	SYS_CONNECT_BY_PATH (컬럼, 구분자)	루트 노드부터 현재 행까지의 경로를 구분자를 기준으로 이어서 출력
	ORDER SIBLINGS BY 컬럼	같은 LEVEL 내에서 정렬할 컬럼을 전달

예제) CONNECT_BY_ISLEAF 를 이용한 최하위 노드 확인

SQL1 *					
1	▶	SELECT	DEPTNO, DNAME, PDEPT,		
2			LEVEL,		
3			CONNECT_BY_ISLEAF		
4		FROM	DEPARTMENT		
5		START WITH	PDEPT IS NULL		
6		CONNECT BY	PRIOR DEPTNO = PDEPT;		
Result					
Grid Result Server Output Text Output Explain Plan Statistics					
	DEPTNO	DNAME	PDEPT	LEVEL	CONNECT_BY_ISLEAF
1	10	공과대학		1	0
2	100	컴퓨터공학부	10	2	0
3	101	컴퓨터공학과	100	3	1
4	102	빅데이터융합과	100	3	1
5	103	소프트웨어공학과	100	3	1
6	200	자연과학부	10	2	0
7	201	수학과	200	3	1
8	202	통계학과	200	3	1
9	203	화학공학과	200	3	1
10	20	인문대학		1	0
11	300	경영학부	20	2	0
12	301	문헌정보학과	300	3	1
13	302	경영학과	300	3	1
14	400	인문사회학부	20	2	1

CONNECT_BY_ISLEAF 는 해당 행이 계층 구조에서 더 이상 하위 부서를 가지지 않는 최하위 노드(LEAF NODE) 인지를 판단하는 가상 컬럼으로, 하위 노드가 존재하지 않으면 1, 하위 노드가 존재하면 0 을 반환함

예제) CONNECT_BY_ROOT 와 SYS_CONNECT_BY_PATH 를 이용한 계층 경로 표현

SQL1 *					
1	SELECT DEPTNO, DNAME, PDEPT,				
2	CONNECT_BY_ROOT DNAME AS 최상위노드,				
3	SYS_CONNECT_BY_PATH(DNAME, '-') AS 연결과정				
4	FROM DEPARTMENT				
5	START WITH PDEPT IS NULL				
6	CONNECT BY PRIOR DEPTNO = PDEPT;				

Result					
	DEPTNO	DNAME	PDEPT	최상위노드	연결과정
1	10	공과대학		공과대학	- 공과대학
2	100	컴퓨터공학부	10	공과대학	- 공과대학 - 컴퓨터공학부
3	101	컴퓨터공학과	100	공과대학	- 공과대학 - 컴퓨터공학부 - 컴퓨터공학과
4	102	빅데이터융합과	100	공과대학	- 공과대학 - 컴퓨터공학부 - 빅데이터융합과
5	103	소프트웨어공학과	100	공과대학	- 공과대학 - 컴퓨터공학부 - 소프트웨어공학과
6	200	자연과학부	10	공과대학	- 공과대학 - 자연과학부
7	201	수학과	200	공과대학	- 공과대학 - 자연과학부 - 수학과
8	202	통계학과	200	공과대학	- 공과대학 - 자연과학부 - 통계학과
9	203	화학공학과	200	공과대학	- 공과대학 - 자연과학부 - 화학공학과
10	20	인문대학		인문대학	- 인문대학
11	300	경영학부	20	인문대학	- 인문대학 - 경영학부
12	301	문헌정보학과	300	인문대학	- 인문대학 - 경영학부 - 문헌정보학과
13	302	경영학과	300	인문대학	- 인문대학 - 경영학부 - 경영학과
14	400	인문사회학부	20	인문대학	- 인문대학 - 인문사회학부

☞ CONNECT_BY_ROOT 를 사용하여 각 행이 속한 최상위 노드의 부서명을 함께 출력함

☞ SYS_CONNECT_BY_PATH 함수를 사용하여 루트 노드부터 현재 행까지의 연결 과정을 하나의 경로로 표현

예제) ORDER SIBLINGS BY 절을 이용한 형제 노드 정렬

SQL1 *				
1	SELECT DEPTNO, DNAME, PDEPT, LEVEL			
2	FROM DEPARTMENT			
3	START WITH PDEPT IS NULL			
4	CONNECT BY PRIOR DEPTNO = PDEPT			
5	ORDER SIBLINGS BY DNAME;			

Result				
	DEPTNO	DNAME	PDEPT	LEVEL
1	10	공과대학		1
2	200	자연과학부	10	2
3	201	수학과	200	3
4	202	통계학과	200	3
5	203	화학공학과	200	3
6	100	컴퓨터공학부	10	2
7	102	빅데이터융합과	100	3
8	103	소프트웨어공학과	100	3
9	101	컴퓨터공학과	100	3
10	20	인문대학		1
11	300	경영학부	20	2
12	302	경영학과	300	3
13	301	문헌정보학과	300	3
14	400	인문사회학부	20	2

☞ ORDER SIBLINGS BY 절을 사용하여 같은 상위 부서(형제 관계)에 속한 부서들을 DNAME 기준으로 정렬하여 출력함

☞ 계층 구조는 그대로 유지하면서, 동일한 LEVEL 내에서만 정렬이 수행됨

예제) 계층형 질의에서 순환(CYCLE) 발생 오류 및 해결 방법

< EMPLOYEES2 DATA >

	EMPLOYEE_ID	DEPARTMENT_ID	MANAGER_ID	NAME	HIRE_DATE
1	1000	10	2000	홍은혜	2024/06/24 00:10:21
2	2000	10	1000	박승민	2024/06/24 00:10:22

순환 구조로 인한 오류 발생)

SQL1 *	
1	SELECT EMPLOYEE_ID, NAME, LEVEL
2	FROM EMPLOYEES2
3	START WITH EMPLOYEE_ID = 1000
4	CONNECT BY PRIOR EMPLOYEE_ID = MANAGER_ID;

Result		
Grid Result	Server Output	Text Output
Explain Plan	Statistics	
EMPLOYEE_ID	NAME	LEVEL

ORA-01436: CONNECT BY의 루프가 발생되었습니다 Col : 1 Ln 4, Col 43 0.01 sec.

- 해당 데이터에서는 사원 1000의 상위 관리자(MANAGER_ID)가 2000이고, 사원 2000의 상위 관리자가 다시 1000으로 설정되어 있다. 이로 인해 1000 → 2000 → 1000으로 이어지는 순환 구조가 형성되며, CONNECT BY 절에서 동일한 경로를 반복 탐색하게 되어 순환(CYCLE)이 발생한다.

NOCYCLE 옵션을 사용한 해결)

SQL1 *

```
1 SELECT EMPLOYEE_ID, NAME, LEVEL,
2     CONNECT_BY_ISCYCLE AS IS_CYCLE
3     FROM EMPLOYEES2
4     START WITH EMPLOYEE_ID = 1000
5     CONNECT BY NOCYCLE PRIOR EMPLOYEE_ID = MANAGER_ID;
```

Result

 Grid Result

 Server Output

 Text Output

 Explain Plan

 Statistics

	EMPLOYEE_ID	NAME	LEVEL	IS_CYCLE
1	1000	홍은혜	1	0
2	2000	박승민	2	1

- CONNECT BY 절에 NOCYCLE 옵션을 사용하여 순환 구조가 존재하더라도 오류 없이 계층형 질의를 수행함
- CONNECT_BY_ISCYCLE 가상 컬럼을 통해 각 행이 순환 경로에 포함되는지 여부를 확인할 수 있음

<< 단위 정리 문제 >>

1. 다음 중 계층형 질의에 대한 설명으로 가장 적절하지 않은 것은?

- ① 계층형 질의는 하나의 테이블 내에서 부모-자식 관계를 정의하여 데이터를 계층적으로 조회하는 기법이다.
- ② START WITH 절은 계층형 질의를 시작할 최상위 레벨의 데이터를 지정한다.
- ③ CONNECT BY 절은 부모와 자식 간의 관계를 정의하며, PRIOR 키워드를 사용해 부모와 자식 간의 연결을 만든다.
- ④ ORDER BY 절은 계층형 질의에서 레벨을 기준으로 데이터를 정렬하는 데 사용된다.

2. 다음 SQL 실행 결과로 가장 적절한 것은?

<지점>

ID	DNAME	PID
1	본부	
2	서울지사	1
3	부산지사	1
4	강남지점	2
5	강북지점	2
6	동부지점	3
7	서부지점	3

```
SELECT *
  FROM 지점
 START WITH PID IS NULL
CONNECT BY PID = PRIOR ID
 ORDER SIBLINGS BY DNAME DESC;
```

①

ID	DNAME	PID
1	본부	

②

ID	DNAME	PID
1	본부	
2	서울지사	1
4	강남지점	2
5	강북지점	2
3	부산지사	1
6	동부지점	3
7	서부지점	3

③

ID	DNAME	PID
1	본부	
3	부산지사	1
6	동부지점	3
7	서부지점	3
2	서울지사	1
4	강남지점	2
5	강북지점	2

④

ID	DNAME	PID
1	본부	
2	서울지사	1
5	강북지점	2
4	강남지점	2
3	부산지사	1
7	서부지점	3
6	동부지점	3

1. ④

ORDER SIBLINGS BY 를 사용하여 레벨이 같은 경우 정렬을 수행한다.

2. ④

ORDER SIBLINGS BY 를 사용하여 같은 레벨인 서울지사와 부산지사가 이름 내림차순으로 출력되며, 서울지사는 강북지점 > 강남지점 순으로, 부산지사는 서부지점 > 동부지점 순으로 출력된다.

2-15. PIVOT 과 UNPIVOT (데이터의 구조를 변경하는 기능)

● 데이터의 구조

1. LONG DATA(Tidy data)

- 하나의 속성이 하나의 컬럼으로 정의되고, 값들은 여러 행으로 쌓이는 구조
- 관계형 데이터베이스(RDBMS)의 기본적인 테이블 설계 방식
- 데이터가 정규화된 형태로 저장됨
- 다른 테이블과의 **조인 연산이 가능한 구조**
- 데이터 분석, 집계, 가공에 유리함

** LONG DATA 형태

	EMPNO	ENAME	JOB	MGR	HIREDATE
1	7369	SMITH	CLERK	7902	1980/12/17 00:00:00
2	7499	ALLEN	SALESMAN	7698	1981/02/20 00:00:00
3	7521	WARD	SALESMAN	7698	1981/02/22 00:00:00
4	7566	JONES	MANAGER	7839	1981/04/02 00:00:00
5	7654	MARTIN	SALESMAN	7698	1981/09/28 00:00:00
6	7698	BLAKE	MANAGER	7839	1981/05/01 00:00:00
7	7782	CLARK	MANAGER	7839	1981/06/09 00:00:00
8	7788	SCOTT	ANALYST	7566	1987/04/19 00:00:00
9	7839	KING	PRESIDENT		1981/11/17 00:00:00

☞ 하나의 속성은 하나의 컬럼으로 정의되고, 각 직원의 정보가 행(Row) 단위로 누적되어 저장된 구조

☞ EMPNO, ENAME, JOB, MGR, HIREDATE 등 속성별로 컬럼이 명확히 분리되어 있음

2. WIDE DATA(Cross table)

- 행과 컬럼 자체가 의미 있는 정보 전달을 목적으로 구성된 구조
- 하나의 속성값이 여러 컬럼으로 분리되어 표현됨
- **교차표(Cross Table) 형태**로 데이터가 요약됨
- RDBMS 에서 WIDE 형식으로 테이블을 설계할 경우, 값이 추가될 때마다 컬럼을 추가해야 하므로 비효율적
- 구조가 고정적이어서 다른 테이블과의 조인 연산이 사실상 불가능
- 주로 데이터 요약, 보고서, 출력용 결과로 사용됨

** WIDE DATA 형태

	JOB	10	20	30
1	CLERK	1	2	1
2	SALESMAN	0	0	4
3	PRESIDENT	1	0	0
4	MANAGER	1	1	1
5	ANALYST	0	2	0

☞ JOB 이 행에 위치하고, 부서 번호(10, 20, 30)가 컬럼으로 분리되어 있는 교차표(Cross Table) 구조

☞ 각 컬럼의 값은 JOB 별·부서별 인원 수(집계 결과) 를 의미함

● 데이터 구조 변경

1. PIVOT: LONG DATA → WIDE DATA 로 변환

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	1980/12/17 00:00:00	800		20
2	7499	ALLEN	SALESMAN	7698	1981/02/20 00:00:00	1600	300	30
3	7521	WARD	SALESMAN	7698	1981/02/22 00:00:00	1250	500	30
4	7566	JONES	MANAGER	7839	1981/04/02 00:00:00	2975		20
5	7654	MARTIN	SALESMAN	7698	1981/09/28 00:00:00	1250	1400	30
6	7698	BLAKE	MANAGER	7839	1981/05/01 00:00:00	2850		30
7	7782	CLARK	MANAGER	7839	1981/06/09 00:00:00	2450		10
8	7788	SCOTT	ANALYST	7566	1987/04/19 00:00:00	3000		20
9	7839	KING	PRESIDENT		1981/11/17 00:00:00	5000		10
10	7844	TURNER	SALESMAN	7698	1981/09/08 00:00:00	1500	0	30
11	7876	ADAMS	CLERK	7788	1987/05/23 00:00:00	1100		20
12	7900	JAMES	CLERK	7698	1981/12/03 00:00:00	950		30
13	7902	FORD	ANALYST	7566	1981/12/03 00:00:00	3000		20
14	7934	MILLER	CLERK	7782	1982/01/23 00:00:00	1300		10



	JOB	10	20	30
1	CLERK	1	2	1
2	SALESMAN	0	0	4
3	PRESIDENT	1	0	0
4	MANAGER	1	1	1
5	ANALYST	0	2	0

2. UNPIVOT: WIDE DATA → LONG DATA 로 변환

	MONTH	월	화	수	목	금	토	일
1	01	10	20	30	40	45	35	60
2	02	39	45	75	34	65	45	33
3	03	13	24	67	43	45	34	56
4	04	34	50	34	22	24	43	44



	MONTH	요일	구매건수
1	01	월	10
2	01	화	20
3	01	수	30
4	01	목	40
5	01	금	45
6	01	토	35
7	01	일	60
8	02	월	39
9	02	화	45
10	02	수	75
11	02	목	34
12	02	금	65
13	02	토	45
14	02	일	33
15	03	월	13
16	03	화	24
17	03	수	67
18	03	목	43
19	03	금	45
20	03	토	34
21	03	일	56

● PIVOT

- LONG DATA 를 WIDE DATA 형태의 교차표로 변환하는 기능
- 교차표의 3 가지 요소인 STACK 컬럼, UNSTACK 컬럼, VALUE 컬럼의 구분이 핵심

		UNSTACK 컬럼						
	MONTH	월	화	수	목	금	토	일
1	01	10	20	30	40	45	35	60
2	02	39	45	75	34	65	45	33
3	03	13	24	67	43	45	34	56
4	04	34	50	34	22	24	43	44

STACK 컬럼 VALUE

** 문법

SQL1 *	
1	SELECT *
2	FROM 테이블명 또는 서브쿼리
3	PIVOT (VALUE FOR UNSTACK컬럼명 IN (값1, 값2, ...));
4	

** 옵션

구분	설명
FROM 절	• STACK 컬럼, UNSTACK 컬럼, VALUE 컬럼만 정의 • 필요 없는 컬럼은 서브쿼리에서 미리 제거하여 대상 컬럼을 제한할 수 있음 • FROM 절에 선언된 컬럼을 기준으로 PIVOT 연산이 수행됨
PIVOT 절	• 교차표로 변환될 컬럼과 값의 구조를 정의 • 어떤 컬럼을 기준으로 컬럼을 펼칠지(UNSTACK), 어떤 값을 집계할지(VALUE)를 지정하는 부분
VALUE	• 교차표에서 요약되어 표시될 값 • 반드시 집계 함수 형태(COUNT, SUM, AVG 등) 로 작성해야 함
UNSTACK 컬럼명	• 컬럼으로 펼쳐질 기준이 되는 컬럼 • 이 컬럼의 값들이 교차표의 컬럼명으로 변환됨
값 1, 값 2	• UNSTACK 컬럼의 값 중, 최종적으로 컬럼명으로 생성할 값들을 명시적으로 지정

** 구조

- 반드시 FROM 절에 STACK 컬럼, UNSTACK 컬럼, VALUE 컬럼을 모두 명시
- FROM 절에 선언된 컬럼 중 UNSTACK, VALUE 컬럼을 제외한 컬럼은 모두 STACK 컬럼으로 처리됨

예제) EMP 테이블에서 아래와 같이 EMP 테이블에서 JOB 별·DEPTNO 별 직원수를 출력

<EMP>

	JOB	10	20	30
1	CLERK	1	2	1
2	SALESMAN	0	0	4
3	PRESIDENT	1	0	0
4	MANAGER	1	1	1
5	ANALYST	0	2	0

올바른 출력)

SQL1 *				
1	SELECT *			
2	FROM (SELECT EMPNO, JOB, DEPTNO FROM EMP)			
3	PIVOT (COUNT(EMPNO) FOR DEPTNO IN (10, 20, 30));			

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	JOB	10	20	30
1	CLERK	1	2	1
2	SALESMAN	0	0	4
3	PRESIDENT	1	0	0
4	MANAGER	1	1	1
5	ANALYST	0	2	0

JOB 컬럼은 STACK 컬럼으로 행(Row) 방향에 배치되고, DEPTNO 컬럼은 UNSTACK 컬럼으로 그 중 값 10, 20, 30 이 각각 컬럼명으로 생성되며, COUNT(EMPNO)는 VALUE 값으로 각 JOB 과 DEPTNO 조합에 해당하는 직원 수(도수)를 의미함

잘못된 출력 1) 이 때 FROM 절 서브쿼리 안에 JOB 이 없으면 아래와 같이 그냥 부서별로의 도수가 출력됨

SQL1 *				
1	SELECT *			
2	FROM (SELECT EMPNO, DEPTNO FROM EMP)			
3	PIVOT (COUNT(EMPNO) FOR DEPTNO IN (10, 20, 30));			

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	10	20	30	
1	3	5	6	

이 때 FROM 절의 서브쿼리에서 JOB 컬럼을 제외하면 STACK 컬럼이 존재하지 않으므로 결과는 하나의 행으로 집계되며, PIVOT 절의 UNSTACK 컬럼인 DEPTNO 값(10, 20, 30)만 컬럼으로 생성되어 부서별 직원 수(도수)만 출력됨

잘못된 출력 2)

SQL1 *										
1	SELECT *									
2	FROM EMP									
3	PIVOT (COUNT(EMPNO) FOR DEPTNO IN (10, 20, 30));									

Result										
	ENAME	JOB	MGR	HIREDATE	SAL	COMM	10	20	30	
1	JAMES	CLERK	7698	1981/12/03 00:00:00	950		0	0	1	
2	MILLER	CLERK	7782	1982/01/23 00:00:00	1300		1	0	0	
3	SCOTT	ANALYST	7566	1987/04/19 00:00:00	3000		0	1	0	
4	ADAMS	CLERK	7788	1987/05/23 00:00:00	1100		0	1	0	
5	SMITH	CLERK	7902	1980/12/17 00:00:00	800		0	1	0	
6	CLARK	MANAGER	7839	1981/06/09 00:00:00	2450		1	0	0	

이하 생략

FROM 절에서 서브쿼리로 컬럼을 제한하지 않고 FROM EMP 처럼 전체 컬럼을 그대로 사용하면, PIVOT 절에서 사용한 UNSTACK 컬럼(DEPTNO)과 VALUE 컬럼(집계 대상)을 제외한 나머지 컬럼들이 모두 STACK 컬럼으로 처리된다. 따라서 ENAME, JOB, MGR, HIREDATE, SAL, COMM 등 여러 컬럼 조합별로 행이 분리되어 결과가 불필요하게 세분화되므로, 원하는 교차표를 만들기 위해서는 서브쿼리에서 필요한 컬럼만 선택하는 것이 바람직함

예제) 다음 테이블을 이용하여 아래와 같이 성별·연도별 구매량 총합을 교차표 형태로 작성

SQL1 *				
1	SELECT *			
2	FROM UNSTACK_TEST;			

	년도	성별	지역	구매량
1	2008	남자	서울	1
2	2008	남자	경기	2
3	2008	여자	서울	3
4	2008	여자	경기	4
5	2009	남자	서울	5
6	2009	남자	경기	6
7	2009	여자	서울	7
8	2009	여자	경기	8



	성별	2008	2009
1	남자	3	11
2	여자	7	15

정답)

SQL1 *				
1	SELECT *			
2	FROM (SELECT 년도, 성별, 구매량 FROM UNSTACK_TEST)			
3	PIVOT (SUM(구매량) FOR 년도 IN (2008, 2009));			

	성별	2008	2009
1	남자	3	11
2	여자	7	15

FROM 절에는 연도 컬럼과 구매량 컬럼, 그리고 STACK 컬럼인 성별 컬럼만 정의되어야 함

기출) 다음 SQL 수행 결과로 가장 적절한 것은?

실적 테이블

사번	이름	지점	지역	실적
1001	김철수	A	서울	10
1002	김정민	A	경기	20
1003	이영희	A	서울	15
1004	홍민정	A	경기	25
1005	박민수	B	서울	30
1006	박나라	B	경기	40

SQL 수행 결과

지점	서울	경기
A	25	45
B	30	40

①

```
SELECT * FROM (SELECT 지점, 지역, 실적 FROM 실적)
PIVOT (SUM(실적) FOR 지역 IN ('서울' AS 서울, '경기' AS 경기)) ORDER BY 지점;
```

②

```
SELECT * FROM (SELECT 지점, 실적 FROM 실적)
PIVOT (SUM(실적) FOR 지역 IN ('서울' AS 서울, '경기' AS 경기)) ORDER BY 지점;
```

③

```
SELECT * FROM (SELECT * FROM 실적)
PIVOT (SUM(실적) FOR 지역 IN ('서울' AS 서울, '경기' AS 경기))
ORDER BY 지점;
```

④

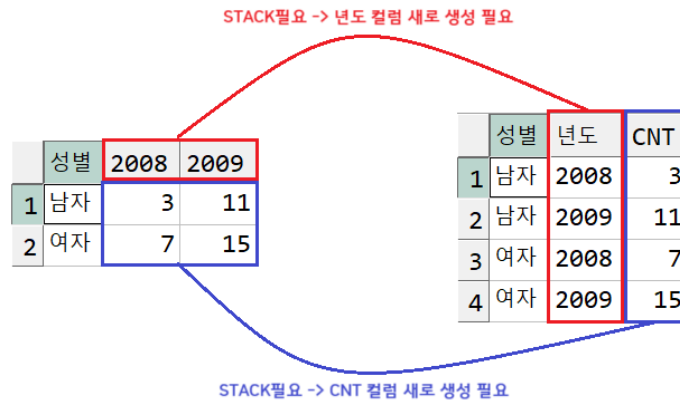
```
SELECT * FROM (SELECT 지점, 지역, 실적 FROM 실적)
PIVOT (SUM(지역) FOR 지역 IN (서울, 경기)) ORDER BY 지점;
```

정답 ①

해설: 지점은 STACK 컬럼(행), 지역은 UNSTACK 컬럼(컬럼), 실적은 VALUE 컬럼(집계 대상)이다. 따라서 FROM 절(서브쿼리)에 지점·지역·실적이 모두 포함되어야 하며, PIVOT 절에서 SUM(실적)로 지역별 실적 총합을 집계하고 FOR 지역 IN ('서울','경기')로 컬럼을 생성해야 한다.

● UNPIVOT

- WIDE DATA(교차표)를 LONG DATA 형태로 변환하는 연산
- 여러 컬럼으로 분리되어 있던 값을 행(Row) 단위로 풀어내는 기능
- 분석, 조인, 추가 집계를 위해 원천 데이터 구조로 되돌릴 때 사용



** 문법

SQL1 *	
1	SELECT *
2	FROM 테이블명 또는 서브쿼리
3	UNPIVOT (VALUE 컬럼명 FOR STACK컬럼명 IN (값1, 값2, ...));

** 옵션

구분	설명
FROM 절	• UNSTACK 처리할 대상 컬럼이 포함된 테이블(또는 서브쿼리)를 지정
UNPIVOT 절	• WIDE DATA 를 LONG DATA 형태로 변환하기 위한 구조를 정의 • 여러 컬럼으로 나뉘어 있는 값을 행(Row) 단위로 풀어내는 역할
VALUE 컬럼명	• UNPIVOT 결과에서 각 컬럼에 저장되어 있던 실제 값이 저장될 컬럼명을 정의 • 사용자가 임의로 정의
STACK 컬럼명	• 기존에 컬럼이었던 항목들의 이름을 값으로 저장하기 위해 생성되는 컬럼 • 사용자가 임의로 정의
값 1, 값 2	• 지정한 컬럼들만 UNPIVOT 대상이 되며, 해당 컬럼명이 STACK 컬럼의 값으로 출력됨

** 구조

- 교차표 형태로 분리된 여러 컬럼의 값을 하나의 컬럼으로 통합함
- 교차표에서 요약된 값들을 하나의 컬럼으로 통합함
- 총 두 개의 새로운 컬럼으로 생성되며, **컬럼명은 사용자가 정의해야 함**

예제) 다음 테이블을 사용하여 아래와 같은 LOGN DATA 형태로 작성

SQL1 *			
1	▶	SELECT *	
2		FROM STACK_TEST;	
3			

Result			
		Grid Result	Server Output
		Text Output	
성별	2008	2009	
1 남자	3	11	
2 여자	7	15	



	성별	년도	CNT
1	남자	2008	3
2	남자	2009	11
3	여자	2008	7
4	여자	2009	15

정답)

SQL1 *			
1	▶	SELECT *	
2		FROM STACK_TEST	
3		UNPIVOT (CNT FOR 년도 IN ("2008", "2009"));	

Result			
		Grid Result	Server Output
		Text Output	Explain Plan
		Statistics	
성별	년도	CNT	
1 남자	2008	3	
2 남자	2009	11	
3 여자	2008	7	
4 여자	2009	15	

- 연도 컬럼(2008, 2009)은 STACK 컬럼(년도)으로 변환되어 기존 컬럼명이 값으로 저장되고, 각 연도의 구매량은 VALUE 컬럼(CNT)에 저장됨
- 2008, 2009 는 숫자 상수가 아니라 컬럼명이므로, Oracle 에서는 이를 식별자로 인식시키기 위해 쌍따옴표로 감싸야 함

예제) 아래 테이블을 STACK 처리하여 LONG DATA 형태로 변환

SQL1 *			
1	▶	SELECT *	
2		FROM TT5;	

Result			
		Grid Result	Server Output
		Text Output	Explain Plan
		Statistics	
MONTH	월	화	수
1 01	10	20	30
2 02	39	45	75
3 03	13	24	67
4 04	34	50	34

정답)

SQL1 *			
1	▶	SELECT *	
2		FROM TT5	
3		UNPIVOT (구매건수 FOR 요일 IN (월, 화, 수, 목, 금, 토, 일));	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	MONTH	요일	구매건수
1	01	월	10
2	01	화	20
3	01	수	30
4	01	목	40
5	01	금	45
6	01	토	35
7	01	일	60
8	02	월	39
9	02	화	45
10	02	수	75
11	02	목	34

이하 생략

- ☞ 요일(월~일)별로 컬럼에 분리되어 있던 구매건수 데이터를 UNPIVOT 을 통해 행(Row) 단위로 쌓아 LONG DATA 형태로 변환한 결과
- ☞ 월~일 컬럼은 STACK 컬럼(요일)으로 변환되어 값 형태로 저장되고, 10, 20, ..., 44 와 같은 구매건수 데이터는 VALUE 컬럼(구매건수)에 저장됨

기출) 다음 SQL 수행 결과로 가장 적절한 것은?

실적_요약 테이블

지점	서울	경기
A	25	45
B	30	40

SQL 수행 결과

지점	지역	실적
A	서울	25
A	경기	45
B	서울	30
B	경기	40

①

```
SELECT 지점, 지역, 실적 FROM 실적_요약
UNPIVOT (실적 FOR 지역 IN (서울, 경기)) ORDER BY 지점, 지역 DESC;
```

②

```
SELECT 지점, 지역, 실적 FROM 실적_요약
UNPIVOT (실적 FOR 지역 IN (서울 AS '서울', 경기 AS '경기')) ORDER BY 지점, 지역;
```

③

```
SELECT 지점, 지역, 실적 FROM (SELECT 지점, 지역, 실적 FROM 실적_요약)
PIVOT (SUM(실적) FOR 지역 IN ('서울' AS 서울, '경기' AS 경기));
```

④

```
SELECT 지점, 지역, SUM(실적) AS 실적 FROM 실적_요약 GROUP BY 지점, 지역;
```

정답 ①

해설: 주어진 데이터는 서울/경기가 컬럼으로 분리된 WIDE 구조이므로, 이를 행으로 “쌓아” LONG 형태로 만들기 위해서는 UNPIVOT 을 사용해야 한다.

① 서울/경기 컬럼의 값을 실적 컬럼으로 통합하고, 원래 컬럼명(서울/경기)을 지역 컬럼 값으로 생성하여 목표 형태와 일치한다.

② Oracle 에서 IN (서울 AS '서울')처럼 문자열 별칭을 주는 방식은 일반적으로 허용되지 않는다.

<< 단원 정리 문제 >>

1. 다음 중 PIVOT 과 UNPIVOT 에 대한 설명 중 가장 적절하지 않은 것은?

- ① PIVOT 연산은 데이터를 행에서 열로 변환하는 데 사용되며, UNPIVOT 연산은 열에서 행으로 변환하는 데 사용된다.
- ② PIVOT 은 그룹화된 데이터의 집계 결과로 변환하여 출력하는 데 유용하다.
- ③ UNPIVOT 은 데이터를 변환할 때 행의 수가 줄어들게 하며, 여러 행을 하나의 행으로 합치는 데 사용된다.
- ④ PIVOT 과 UNPIVOT 모두 특정 조건에 맞는 데이터를 동적으로 변경할 수 있으며, 데이터의 형태를 효율적으로 조작할 수 있다.

2. 데이터를 아래와 같이 변환하기 위한 가장 적절한 문장은?

<SALES>

ID	REGION	YEAR	SALES_AMOUNT
1	서울	2023	1000
2	서울	2024	1200
3	부산	2023	900
4	부산	2024	1100
5	대구	2023	800
6	대구	2024	950



REGION	2023	2024
부산	900	1100
대구	800	950
서울	1000	1200

①

```
SELECT *
FROM (SELECT *
      FROM SALES)
PIVOT (SUM(SALES_AMOUNT) FOR YEAR IN (2023,2024));
```

②

```
SELECT *
FROM (SELECT REGION, YEAR, SALES_AMOUNT
      FROM SALES)
PIVOT (SUM(SALES_AMOUNT) FOR YEAR IN (2023,2024));
```

③

```
SELECT *
FROM (SELECT *
      FROM SALES)
UNPIVOT (SUM(SALES_AMOUNT) FOR YEAR IN (2023,2024));
```

④

```
SELECT *
FROM (SELECT REGION, YEAR, SALES_AMOUNT
      FROM SALES)
UNPIVOT (SUM(SALES_AMOUNT) FOR YEAR IN (2023,2024));
```

1. ③

여러 개의 컬럼을 하나로 합쳐서 행으로 나열하기 때문에 행의 수가 늘어난다.

2. ②

하나의 컬럼이 여러 컬럼으로 분해되는 과정은 PIVOT 으로 처리할 수 있다.

PIVOT 연산 시 FROM 절에 명시된 컬럼에서 PIVOT 절에 사용한 컬럼을 제외하고는 원래 형태를 유지하므로, ID 값이 FROM 절에 남아있으면 안 된다.

2-16. 정규 표현식

● 정규 표현식

- 문자열이 가지는 공통된 규칙(패턴)을 일반화하여 표현하는 방법
- 특정 형식의 문자열을 검색, 추출, 치환, 검증하는 데 사용됨
- SQL 은 정규 표현식을 처리하기 위한 문자 함수를 제공함
- 예) 숫자를 포함하는, 숫자로 시작하는 4 자리, 두 번째 자리가 A 인 5 글자

예제) 일반화 규칙 찾아내기



** 정규 표현식에서 사용하는 메타문자

\d	Digit, 0, 1, 2, ..., 9	[ab]	a 또는 b의 한 글자	\n	그룹번호
\D	숫자가 아닌 것	[^ab]	a 와 b 제외한 모든 문자	[:alnum:]	문자와 숫자
\s	공백	[0-9]	숫자	[:alpha:]	문자
\S	공백이 아닌 것	[A-Z]	영어 대문자	[:blank:]	공백
\w	단어	[a-z]	영어 소문자	[:cntrl:]	제어문자
\W	단어가 아닌 것	[A-z]	모든 영문자	[:digit:]	digits
\t	Tab	i+	i가 1회 이상 반복	[:graph:]	graphical characters [:alnum:], [:punct:]
\n	New line(엔터문자)	i*	i가 0회 이상 반복	[:lower:]	소문자
^	시작되는글자	i?	0회에서 1회 반복	[:print:]	숫자, 문자, 특수문자, 공백 모두
\$	마지막글자	i{n}	i가 n회 반복	[:punct:]	특수문자
\	Escape character (뒤에 기호 의미 제거)	i{n1,n2}	i가 n1에서 n2회 반복	[:space:]	공백문자
	또는	i{n,}	i가 n회 이상 반복	[:upper:]	대문자
.	엔터를 제외한 모든 한 글자	()	그룹지정	[:xdigit:]	16진수

예제) 전화번호의 일반화

글자	1차 일반화 결과	2차 일반화 결과
tel)02-999-4456	tel\[0-9-]+\	tel\)?[0-9-]+\
tel02-999-4456	tel[0-9-]+\	

※ 1차 일반화 결과

: tel) 뒤에 오는 전화번호 부분은 숫자와 '-' 문자로만 구성되어 있음

→ 따라서 전화번호 부분은 숫자(0~9)와 하이픈(-)만 허용하는 정규식 패턴으로 표현할 수 있음

→ 숫자 0~9는 [0-9], 하이픈은 -로 표현하며, 이들이 1회 이상 반복되므로 +를 사용함

→ 결과적으로 전화번호 부분은 [0-9-]+\로 일반화할 수 있음

→ ')' 문자는 정규 표현식에서 메타문자이므로, 문자 그대로 인식시키기 위해 escape character를 사용하여 '\)'로 표현함

※ 2차 일반화 결과

: 전화번호 형식에 따라 'tel)'처럼 ')'가 존재하는 경우와, 'tel'처럼 괄호가 없는 경우가 모두 발생할 수 있음

→ 즉, ')' 문자는 있을 수도 있고 없을 수도 있는 선택적인 문자임

→ '?'는 앞의 문자가 0회 또는 1회 등장함을 의미하므로, '\)?' 형태로 작성하면

'\)'가 있는 경우와 없는 경우를 모두 처리할 수 있음

● REGEXP_REPLACE

- 정규 표현식을 사용하여 문자열의 특정 패턴을 다른 문자열로 치환하는 함수

- 찾을 문자열에 해당하는 정규식 패턴을 지정하여, 이를 다른 문자열로 변경하거나 제거할 수 있음

** 문법

SQL1 *
1 REGEXP_REPLACE(대상, 찾을문자열, [바꿀문자열], [검색위치], [발견횟수], [옵션])

** 구성

구분	설명
대상	• 정규 표현식을 적용할 원본 문자열
찾을문자열	• 정규 표현식 패턴으로, 변경 또는 제거할 문자열을 지정
바꿀문자열	• 찾을문자열을 대체할 문자열 • 생략하거나 빈 문자열('')을 지정하면 찾을문자열을 삭제함
검색위치	• 검색을 시작할 문자 위치 • 생략 시 처음부터 검색 (기본값 1)
발견횟수	• 치환할 패턴의 발생 횟수 지정 • 생략 시 모든 패턴을 치환 (기본값 0)
옵션	• 정규식 동작 방식 지정 • 예: i(대소문자 구분 없음), c(대소문자 구분, 기본값) 등

예제) REGEXP_REPLACE 를 이용한 숫자 제거

SQL1 *			
1	SELECT ID,		
2	REGEXP_REPLACE(ID, '\d', '') AS RESULT1,		
3	REGEXP_REPLACE(ID, '[:digit:]', '') AS RESULT2		
4	FROM ID_TEST;		
5			

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	ID	RESULT1	RESULT2
1	abc1004	abc	abc
2	hong_1000	hong_	hong_
3	sqld!!	sqld!!	sqld!!
4	sql.oracle	sql.oracle	sql.oracle
5	hdata-lab	hdata-lab	hdata-lab

- ☞ 숫자를 의미하는 정규식 패턴인 '\d' 또는 '[:digit:]'를 사용하여, REGEXP_REPLACE 함수의 바꿀문자열에 빈 문자열('')을 전달하면 문자열에 포함된 모든 숫자를 삭제할 수 있다.
- ☞ 이로 인해 abc1004, hong_1000 와 같이 숫자가 포함된 ID 값은 숫자 부분이 제거되어 문자만 남게 되며, 숫자가 포함되지 않은 문자열은 원본 그대로 출력된다.

예제) REGEXP_REPLACE 를 이용한 특수기호 제거

SQL1 *				
1	SELECT ID,			
2	REGEXP_REPLACE(ID, '\W', '') AS RESULT1,			
3	REGEXP_REPLACE(ID, '\W _', '') AS RESULT2,			
4	REGEXP_REPLACE(ID, '[:punct:]', '') AS RESULT3			
5	FROM ID_TEST;			
6				

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	ID	RESULT1	RESULT2	RESULT3
1	abc1004	abc1004	abc1004	abc1004
2	hong_1000	hong_1000	hong1000	hong1000
3	sqld!!	sqld	sqld	sqld
4	sql.oracle	sqloracle	sqloracle	sqloracle
5	hdata-lab	hdata lab	hdata lab	hdata lab

- ☞ '\W'는 문자, 숫자, 언더바(_)를 제외한 문자를 의미하므로, 이를 사용하여 특수문자를 제거할 경우 언더바는 제거되지 않는다. 따라서 RESULT2 와 같이 언더바까지 함께 제거하려면 OR 연산자인 '|'를 사용하여 '\W|_' 형태로 언더바를 명시적으로 지정하거나, 모든 특수문자를 의미하는 '[:punct:]'를 사용해야 한다.

예제) ID 에서 문자 바로 뒤에 오는 숫자를 문자와 함께 삭제(대소문자는 구분하지 않음)

```
SQL1 *
1 SELECT ID,
2     REGEXP_REPLACE(ID, '[a-z][0-9]') AS 성공1,
3     REGEXP_REPLACE(ID, '[a-zA-Z][0-9]', '') AS 성공2,
4     REGEXP_REPLACE(ID, '[A-Z][0-9]', '') AS 성공3,
5     REGEXP_REPLACE(ID, '[A-z0-9]', '') AS 실패,
6     REGEXP_REPLACE(ID, '[a-z][0-9]', '', 1, 0, 'i') AS 성공4
7 FROM ID_TEST;
8
```

Result					
	ID	성공1	성공2	성공3	실패 성공4
1	abc1004	ab004	ab004	ab004	ab004
2	hong_1000	hong_1000	hong_1000	hong000	hong_1000
3	sql!!	sql!!	sql!!	sql!!	!! sql!!
4	sql.oracle	sql.oracle	sql.oracle	sql.oracle	. sql.oracle
5	hdata-lab	hdata-lab	hdata-lab	hdata-lab	- hdata-lab

- 영문 소문자 뒤에 숫자가 오는 패턴을 의미하는 '[a-z][0-9]' 또는 영문 대소문자 뒤에 숫자가 오는 패턴을 의미하는 '[a-zA-Z][0-9]', '[A-Z][0-9]'를 사용하여 REGEXP_REPLACE 함수의 바꿀문자열에 빈 문자열을 전달하면 해당 패턴에 일치하는 문자 조합이 삭제되어 abc1004 는 ab004 로 변환된다.
- '[A-z0-9]'는 영문 대소문자 또는 숫자값을 모두 삭제하므로 위와 다른 결과가 출력된다.
- 7 번째 인수에 'i'를 전달하면 대소문자를 구분하지 않고 정규 표현식을 적용하므로, 영문 대소문자 여부와 관계없이 동일한 패턴으로 인식되어 처리된다.

예제) PRODUCT 테이블의 상품명에서 괄호포함, 괄호안에 들어가는 모든 글자를 삭제하여 출력

```
SQL1 *
1 SELECT 상품명,
2     REGEXP_REPLACE(상품명, '\(.+\)') AS 상품명2
3 FROM PRODUCT;
4
5
6
```

Result	
상품명	상품명2
1 머거본 꿀땅콩(135g)	머거본 꿀땅콩
2 무가염버터(450g)	무가염버터
3 매일유업 빠로가는 칼슘치즈(270g)	매일유업 빠로가는 칼슘치즈
4 양반 들기름이 그윽한 김	양반 들기름이 그윽한 김

- 정규식 패턴 '\(.+\)'는 여는 괄호 '('와 닫는 괄호 ')' 사이에 위치한 모든 문자열을 하나의 패턴으로 인식하며, 이를 빈 문자열로 치환함으로써 상품명에서 괄호 및 괄호 내부 문자열이 모두 삭제된다.

예제) REGEXP_REPLACE 를 사용한 문자열 삭제

SQL1 *			
1	▶	SELECT NAME,	
2		REGEXP_REPLACE(NAME, '[:a]pha:]', '') AS RESULT1,	
3		REGEXP_REPLACE(NAME, '[:a]pha:]', '', 1, 1) AS RESULT2	
4		FROM STUDENT;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	NAME	RESULT1	RESULT2
1	강성근		성근
2	고대영		대영
3	권정현		정현
4	김건일		건일
5	김재현		재현
6	나세희		세희
7	박주현		주현

이하 생략

RESULT1: 전체 글자를 삭제한다.

RESULT2: 이름에서 처음부터 찾아 첫 번째로 발견되는 글자만 삭제한다.

예제) REGEXP_REPLACE 를 사용한 글자 치환

SQL1 *			
1	▶	SELECT NAME,	
2		REGEXP_REPLACE(NAME, '[:a]pha:]', 'X') AS RESULT1,	
3		REGEXP_REPLACE(NAME, '[:a]pha:]', 'X', 1, 2) AS RESULT2	
4		FROM STUDENT;	

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
	NAME	RESULT1	RESULT2
1	강성근	XXX	강X근
2	고대영	XXX	고X영
3	권정현	XXX	권X현
4	김건일	XXX	김X일
5	김재현	XXX	김X현
6	나세희	XXX	나X희
7	박주현	XXX	박X현

이하 생략

RESULT: 전체 글자를 'X'로 치환한다.

RESULT2: 전체 글자 중 처음부터 찾아 두 번째로 발견되는 글자만 'X'로 치환한다.

기출) 정규식 메타문자에 대한 설명으로 옳지 않은 것은?

- ① ?: 문자열이 0 회에서 1 회 반복
- ② *: 문자열이 1 회 이상 반복
- ③ +: 문자열이 1 회 이상 반복
- ④ {4,}: 문자열이 최소 4 회 이상 반복

정답 ②

해설: '*'는 문자열이 최소 0 회 이상의 반복을 의미하는 메타문자이다.

● REGEXP_SUBSTR

- 정규 표현식을 사용하여 문자열에서 특정 패턴에 일치하는 부분을 추출하는 함수
- 단순 위치 기반 추출이 아니라 정규식 패턴에 일치하는 문자열만 선택적으로 반환함
- 옵션(검색 위치, 발생 횟수, 옵션 등)은 REGEXP_REPLACE 와 동일하게 사용됨

** 문법

SQL1 *

1 REGEXP_SUBSTR(대상, 패턴, [검색위치], [발견횟수], [옵션], [추출그룹])

** 구성

구분	설명
대상	• 정규 표현식을 적용할 원본 문자열
패턴	• 추출할 문자열을 정의하는 정규식 패턴
검색위치	• 검색을 시작할 문자 위치 • 생략 시 처음부터 검색 (기본값 1)
발견횟수	• 치환할 패턴의 발생 횟수 지정 • 생략 시 첫 번째 패턴 반환 (기본값 1)
옵션	• 정규식 동작 방식 지정 • 예: i(대소문자 구분 없음), c(대소문자 구분, 기본값) 등
추출그룹	• 정규식 내 괄호로 묶은 그룹 중 반환할 그룹 번호 지정

예제) REGEXP_SUBSTR 을 사용한 문자열 추출

SQL1 *	
1	SELECT EMAIL,
2	REGEXP_SUBSTR(EMAIL, '[A-z]+') AS RESULT
3	FROM PROFESSOR
4	WHERE DEPTNO = 101;
5	

Result	
EMAIL	RESULT
1 captain@abc.net	captain
2 sweety@abc.net	sweety
3 pman@power.com	pman

☞ 정규식 패턴 [a-z]+는 영문 소문자가 1 회 이상 연속으로 등장하는 문자열을 의미하며, REGEXP_SUBSTR 는 해당 패턴에 만족하는 문자열 중에서 처음 발견되는 부분만을 추출한다.

예제) REGEXP_SUBSTR 을 사용한 지역번호 추출

SQL1 *	
1	SELECT TEL,
2	REGEXP_SUBSTR(TEL,
3	'(\d+)\)(\d+)-(\d+)',
4	1,
5	1,
6	NULL,
7	1) AS 지역번호
8	FROM STUDENT;

TEL	지역번호
1 02)399-3947	02
2 031)304-3047	031
3 032)394-3048	032
4 02)3048-3039	02
5 051)304-3048	051
6 041)204-0383	041
7 02)4933-0382	02

이하 생략

☞ 전화번호는 숫자 여러 개 + ')' + 숫자 여러 개 + '-' + 숫자 여러 개로 구성되어 있으므로, 이를 차례대로 \d+, \), \d+, -, \d+로 표현할 수 있다.
 각 전화번호의 숫자 부분을 괄호로 그룹화한 뒤, 그중 첫 번째 그룹을 추출한다.

예제) 이메일 아이디 추출(서브패턴 활용)

```

SQL1 *
1 SELECT EMAIL,
2     REGEXP_SUBSTR(EMAIL, '(.+)\@.+', 1, 1, NULL, 1) AS EMAIL_ID1,
3     REGEXP_SUBSTR(EMAIL, '(\w|\W+)\@[a-z.]+', 1, 1, NULL, 1) AS EMAIL_ID2
4 FROM PROFESSOR;

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	EMAIL	EMAIL_ID1	EMAIL_ID2
1	loveu@abc.net	loveu	loveu
2	a1234@abc.net	a1234	a1234
3	pman@power.com	pman	pman
4	power1234@hamail.net	power1234	power1234
5	kong-12@naver.com	kong-12	kong-12
6	bdragon@naver.com	bdragon	bdragon
7	love-1004@hanmir.com	love-1004	love-1004

이하 생략

- ☞ 첫 번째 정규식 '(.+)\@.+ '는 '@' 기호를 기준으로 앞쪽에 위치한 모든 문자를 하나의 그룹으로 묶어 추출하는 방식으로, 이메일 ID 가 어떤 문자로 구성되어 있는 상관없이 '@' 앞의 문자열을 반환한다.
- ☞ 두 번째 정규식 '(\w|\W+)\@[a-z.]+'는 이메일 ID 부분을 문자(\w) 또는 비문자(\W)의 조합으로 보다 명확하게 정의하고, @ 뒤쪽은 영문 소문자와 점(.)으로 구성된 형식으로 제한하여 표현한 것이다.

기출) 다음 SQL 로 추출되지 않는 것은? (단, 보기의 문자열은 COL1 의 값이다)

```
SELECT REGEXP_SUBSTR(COL1, '\w+\w+(\.\w+)+') FROM DUAL;
```

- ① user_01@datahub.com
- ② Abc123@hdatalab.co.kr
- ③ testUser@oracle.sql.dev
- ④ admin@server

정답 ④

해설: 정규식 '\w+\w+(\.\w+)+'는 아이디 부분과 도메인 부분이 모두 '\w'(영문자, 숫자, 언더스코어)로 구성되고, '@' 뒤에 '.'이 포함된 도메인 확장자(.\w+)+가 최소 1 회 이상 반복되어야 한다.

①, ②, ③은 모두 아이디@도메인.확장자 구조를 만족하므로 추출된다.

④ 'admin@server' 는 '@' 뒤에 '.'을 포함한 도메인 확장자가 없으므로 정규식 패턴에 만족하지 않아 추출되지 않는다.

● REGEXP_INSTR

- 주어진 문자열에서 특정패턴의 시작 위치를 반환
- 찾는 패턴이 없는 경우 0 리턴

** 문법

SQL1 *
1 REGEXP_INSTR (대상, 패턴, [검색위치], [발견횟수], [출력옵션], [옵션], [추출그룹])

** 구성

구분	설명
대상	• 정규 표현식을 적용할 원본 문자열
패턴	• 위치를 찾을 정규식 패턴
검색위치	• 검색을 시작할 문자 위치 • 생략 시 문자열의 처음부터 검색 (기본값 1)
발견횟수	• 몇 번째로 발견된 패턴의 위치를 반환할지 지정 • 생략 시 첫 번째 패턴 반환 (기본값 1)
출력옵션	• 반환할 위치 지정 - 0: 패턴의 시작 위치 반환 (기본값) - 1: 패턴의 끝 위치 다음 위치 반환
옵션	• 정규식 동작 방식 지정 • 예: i(대소문자 구분 없음) , c(대소문자 구분, 기본값)
추출그룹	• 정규식 내 괄호로 묶은 그룹 중 반환할 그룹 번호 지정

예제) REGEXP_INSTR 을 사용하여 처음 발견되는 숫자의 위치 출력

SQL1 *
1 SELECT ID,
2 REGEXP_INSTR (ID, '\d+')
3 FROM ID_TEST;
4

Result		
Grid Result Server Output Text Output Explain Plan Statistics		
ID	REGEXP_INSTR(ID, '\D+')	
1 abc1004		4
2 hong_1000		6
3 sqld!!		0
4 sql.oracle		0
5 hdata-lab		0

☞ 정규식 패턴 '\d+'를 사용하여 연속된 숫자 문자열을 검색하고, 해당 패턴에 처음으로 일치하는 문자열의 시작 위치를 반환한다.

예제) REGEXP_INSTR 에서 발견횟수를 지정하여 두 번째 숫자 위치 찾기

SQL1 *		
1	SELECT ID,	
2	REGEXP_INSTR(ID, '\d', 1, 2)	
3	FROM ID_TEST;	
4		

Result		
Grid Result Server Output Text Output Explain Plan Statistics		
ID	REGEXP_INSTR(ID, '\D', 1, 2)	
1 abc1004		5
2 hong_1000		7
3 sqld!!		0
4 sql.oracle		0
5 hdata-lab		0

정규식 패턴 '\d'를 사용하여 숫자 한 자리를 검색하고, 검색위치는 문자열의 처음부터(1), 발견횟수를 2로 지정하여 두 번째로 등장하는 숫자의 시작 위치를 반환한다.

예제) REGEXP_INSTR 에서 출력옵션 1 을 사용한 패턴 종료 위치 반환

SQL1 *		
1	SELECT ID,	
2	REGEXP_INSTR(ID, '\d+', 1, 1, 1)	
3	FROM ID_TEST;	
4		

Result		
Grid Result Server Output Text Output Explain Plan Statistics		
ID	REGEXP_INSTR(ID, '\D+', 1, 1, 1)	
1 abc1004		8
2 hong_1000		10
3 sqld!!		0
4 sql.oracle		0
5 hdata-lab		0

정규식 패턴 '\d+'를 사용하여 연속된 숫자 문자열을 검색한 뒤, 출력옵션을 1로 지정하면 패턴이 끝난 다음 문자의 위치를 반환한다. 따라서 'abc1004'의 경우 숫자 '1004'가 4번째 위치에서 시작하여 7번째 위치에서 끝나므로, 그 다음 위치인 8이 반환되며, 숫자가 존재하지 않는 문자열에 대해서는 0이 반환된다.

예제) REGEXP_INSTR 에서 발견횟수를 이용한 특정 문자 위치 찾기

SQL1 *		
1	SELECT REGEXP_INSTR('500 ORACLE PARKWAY, REDWOOD SHORES, CA',	
2	'[^]+', 1, 2) AS "REGEXP_INSTR"	
3	FROM DUAL;	

Result		
Grid Result Server Output Text Output Explain Plan Statistics		
REGEXP_INSTR		
1		5

정규식 패턴 '[^]+'는 공백이 아닌 문자가 1회 이상 연속으로 등장하는 문자열을 의미하며, 이를 사용하여 문자열에서 공백으로 구분된 단어 단위를 검색한다. 따라서 두 번째로 발견되는 단어인 'ORACLE'의 첫 위치인 5를 출력한다.

예제) 서브그룹에 대한 위치 출력(2024 기출)

SQL1 *

```

1 SELECT REGEXP_INSTR('2025/01/01 12:50:21',
2 '((\d+)/(\d+)/(\d+)) ((\d+):(\d+):(\d+))',
3 1, 1, 0, NULL, 5) AS RESULT
4 FROM DUAL;

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

RESULT	
1	12

아래와 같은 순서로 서브그룹의 번호가 정해지기 때문에 5 번째 서브그룹은 12:50:21 전체를 나타낸다.
 이때 출력옵션(5 번째 인수)이 0 이므로 12:50:21 문자열의 시작인 1 의 위치를 리턴하게 된다.
 (만약 출력옵션이 1 인 경우 12:50:21 문자열의 끝인 1 의 다음 위치가 리턴됨)

① ((\d+)/(\d+)/(\d+)) ⑤ ((\d+):(\d+):(\d+))

② ③ ④ ⑥ ⑦ ⑧

기출) 다음 SQL 의 수행 결과로 알맞은 것은?

SELECT REGEXP_INSTR('12345678', '(123)(4(56)(78))', 1, 1, 0, 'i', 2) FROM DUAL;

- ① 1
- ② 4
- ③ 5
- ④ 7

정답 ②

해설: REGEXP_INSTR 함수에서 마지막 인자 2 는 두 번째 서브패턴의 시작 위치를 반환하라는 의미이다.
 정규식 '(123)(4(56)(78))'에서 첫 번째 서브패턴 (123)의 시작 위치는 1,
 두 번째 서브패턴 (4(56)(78))의 시작 위치는 문자 4 가 시작되는 위치인 4 이다.
 따라서 두 번째 서브패턴의 시작 위치인 4 가 반환된다.

● REGEXP_LIKE

- 주어진 문자열에서 특정패턴을 갖는 경우 반환(WHERE 절 사용만 가능)
- 옵션 REGEXP_REPLACE 와 동일

** 문법

SQL1 *	
1	REGEXP_LIKE(원본, 찾을문자열, [옵션])

예제) 영문자 뒤에 숫자가 연속으로 포함된 문자열 검색

SQL1 *	
1	SELECT *
2	FROM ID_TEST
3	WHERE REGEXP_LIKE(ID, '\w\d');
4	

Result	
	Grid Result Server Output Text Output Explain Plan Statistics
ID	
1	abc1004
2	hong_1000

☞ \w: 영문자, 숫자, 언더스코어 중 한 문자

☞ \d: 숫자 한 자리

→ 즉, 문자 + 숫자 패턴이 포함되기만 하면 매칭된다.

예제) 괄호 안에 g 단위 중량이 표시된 상품명 검색

SQL1 *	
1	SELECT 상품명
2	FROM PRODUCT
3	WHERE REGEXP_LIKE(상품명, '(\w*g)');
4	

Result	
	Grid Result Server Output Text Output Explain Plan Statistics
상품명	
1	머거본 꿀망콩(135g)
2	무가염버터(450g)
3	매일유업 빠로가는 칼슘치즈(270g)

☞ '(\w*g)' 패턴을 사용하여 상품명에 괄호로 감싼 g 단위 중량 정보가 포함된 경우만 추출한다.

☞ (135g), (450g), (270g)처럼 숫자와 문자 조합 뒤에 g로 끝나는 문자열이 괄호 안에 있는 상품이 검색된다.

기출) 다음 SQL 조건을 만족하여 출력되는 요일이 아닌 것은? (단, 보기의 문자열은 COL 컬럼의 값이다)

```
SELECT COL
  FROM DAY_T
 WHERE REGEXP_LIKE(COL, '^[^mc][a-z]*u[a-z]*day$', 'i');
```

- ① TUESDAY
- ② THURSDAY
- ③ SUNDAY
- ④ FRIDAY

정답 ④

해설: `^[^mc]`는 문자열이 M 또는 C(대소문자 무시 i 적용)로 시작하면 제외한다는 뜻인데, 보기 ①~④는 모두 M/C로 시작하지 않으므로 이 조건은 전부 통과한다. 결국 핵심은 뒤쪽의 u 포함 여부와 day로 끝나는지 여부이다. ①, ②, ③은 u를 포함하고 day로 끝나므로 출력된다. ④ FRIDAY는 u가 없어서 정규식에 매칭되지 않으므로 출력되지 않는다.

● REGEXP_COUNT

- 주어진 문자열에서 특정 패턴이 등장한 횟수를 반환하는 함수
- 패턴, 시작 위치, 옵션 등 옵션 사용법은 REGEXP_REPLACE와 동일
- 시작 위치를 생략하면 문자열의 처음부터 스캔하여 패턴의 등장 횟수를 계산함

** 문법

```
SQL1 *
1 REGEXP_COUNT(원본, 찾을문자열, [시작위치], [옵션])
```

예제) ID 값에서의 숫자의 수

SQL1 *			
1	SELECT	ID,	
2		REGEXP_COUNT(ID, '\d') AS RESULT1,	
3		REGEXP_COUNT(ID, '\d+') AS RESULT2	
4	FROM	ID_TEST;	
5			

Result			
Grid Result Server Output Text Output Explain Plan Statistics			
ID	RESULT1	RESULT2	
1 abc1004	4	1	
2 hong_1000	4	1	
3 sqld!!	0	0	
4 sql.oracle	0	0	
5 hdata-lab	0	0	

☞ \d는 한 자리수의 숫자를 의미하며 \d+는 연속적인 숫자를 의미. 따라서 COUNT 시 연속적인 숫자를 하나로 취급함

<< 단원 정리 문제 >>

1. 다음 실행 결과로 가장 적절한 것은?

```
SELECT REGEXP_SUBSTR('http://www.hdatalab.com/main',
                     'http://([[:alpha:]]+\.?){3,4}/?') AS RESULT
FROM DUAL;
```

- ① http://www.
- ② http://www.hdatalab.
- ③ http://www.hdatalab.com/
- ④ http://www.hdatalab.com/main

2. 다음 실행 결과로 가장 적절한 것은?

```
SELECT REGEXP_REPLACE('http://www.hdatalab.com/main',
                      '[:alnum:]+', '', 5, 2) AS RESULT
FROM DUAL;
```

- ① http/www.hdatalab.com/main
- ② http://.hdatalab.com/main
- ③ http://..com/main
- ④ http://www..com/main

3. 다음 결과로 올바른 것은?

```
SELECT REGEXP_SUBSTR('aabbcc', '(a|b)+c') AS C1,
       REGEXP_SUBSTR('aabbcc', 'ab+c') AS C2
FROM DUAL;
```

- ① C1 = aabbcc, C2 = abb
- ② C1 = aabbcc, C2 = abbc
- ③ C1 = aabbcc, C2 = abbc
- ④ C1 = abbc, C2 = abbc

4. 다음 결과로 올바른 것은?

```
SELECT REGEXP_SUBSTR('aaaaaaaaab', 'a{2,4}') FROM DUAL;
```

- ① aa
- ② aaaa
- ③ aaaaaa
- ④ aaaaaaaaa

5. 다음 결과로 올바른 것은?

```
SELECT REGEXP_INSTR('123123123', '312') FROM DUAL;
```

- ① 1
- ② 2
- ③ 3
- ④ 4

6. 다음 빈칸에 들어가면 NULL 이 출력되는 문자열은?

```
SELECT REGEXP_SUBSTR(_____, '^\\w*$')FROM DUAL;
```

- ① 'abc_123'
- ② 'ABC'
- ③ 'a b'
- ④ ''

1. ③

문자가 1 회 이상 반복되며 그 뒤에 마침표(.)가 있거나 없어도 되는 문자열을 3 개 또는 4 개까지 추출하되, 마지막에는 /가 있거나 없어도 된다.

(www.), (hdata lab.), (com/) 까지만 추출되며, com 뒤에 있는 /는 `[[alpha:]]+\.?`에 포함되지 않는다.

2. ④

문자 또는 숫자가 1 회 이상 반복되는 문자열을 5 의 위치에서부터 찾아서 두 번째로 발견되는 경우 삭제한다.

3. ②

REGEXP_SUBSTR 는 조건을 만족하는 문자열 중 가장 왼쪽에서 처음 발견되는 매칭 결과를 반환한다.

C1 의 `'(a|b)+c'`는 a 또는 b 가 1 회 이상 반복된 뒤 c 가 오는 패턴이다. 문자열 aabbcc 에서 가능한 매칭은 aabbcc(첫 번째 c 까지)이며, 더 길게 aabbcc 로는 매칭될 수 없다(패턴의 마지막이 c 1 개로 끝나기 때문). 따라서 C1 은 aabbcc 이다.

C2 의 `ab+c` 는 a 다음에 b 가 1 회 이상 반복되고 c 가 오는 패턴이다.

aabbcc 에서 abbc 가 처음으로 매칭되므로 C2 는 abbc 이다.

4. ②

정규식 `a{2,4}`는 문자 a 가 최소 2 번, 최대 4 번 반복되는 경우를 의미한다.

REGEXP_SUBSTR 함수는 조건을 만족하는 문자열 중 가장 왼쪽에서 처음 발견되는 패턴을 매칭하여 반환한다.

문자열 'aaaaaaaab'의 시작 부분에서 a 가 연속으로 나오며, 허용 가능한 최대 반복 횟수는 4 이므로 처음 위치에서 'aaaa'가 매칭된다.

5. ③

REGEXP_INSTR 함수는 주어진 문자열에서 정규식 패턴이 처음으로 매칭되는 시작 위치를 반환한다.

문자열 '123123123'에서 패턴 '312'가 처음 나타나는 부분은 3 번째 문자부터이다.

6. ③

정규식 `^\w*$`는 문자열의 시작부터 끝까지(`^`, `$`) `\w`(영문자, 숫자, 언더스코어)만 0 개 이상(`*`)으로 구성된 경우에만 매칭된다.

①, ②는 영문자/숫자/언더스코어로만 이루어져 매칭되며, ④는 빈 문자열도 * 조건(0 개 이상)을 만족하므로 매칭된다. ③ 'a b'는 공백이 포함되어 `\w`에 해당하지 않으므로 패턴에 매칭되지 않아 REGEXP_SUBSTR 결과가 NULL 이 된다.

단원 종합 문제

1. 아래 SQL 수행 결과로 알맞은 것은?

<CAR>

CAR_ID	MODEL	RENT_FEE	CAR_TYPE
1	소나타	60000	SEDAN
2	K5	65000	SEDAN
3	셀토스	80000	SUV
4	K9	220000	LUXURY
5	EV6	95000	ELECTRIC

<CAR_RENTAL_HISTORY>

RENTAL_ID	CAR_ID	START_DATE	END_DATE
1000	1	2024/01/02 10:30:00	2024/01/05 14:45:00
1001	2	2024/07/12 08:00:00	2024/07/18 11:35:00
1002	4	2024/07/25 09:15:00	2024/07/27 10:45:00
1003	5	2024/02/15 08:45:00	2024/02/20 11:20:00
1004	1	2024/07/05 14:00:00	2024/07/10 16:50:00

```
SELECT COUNT(RENTAL_ID) AS RESULT
FROM CAR_RENTAL_HISTORY
WHERE CAR_ID IN (SELECT CAR_ID
                  FROM CAR
                  WHERE CAR_TYPE = 'SEDAN')
AND TO_CHAR(START_DATE, 'YYYYMM') = '202407';
```

- ① 0
- ② 1
- ③ 2
- ④ 3

2. 서브쿼리에 대한 설명으로 가장 적절하지 않은 것은?

- ① 연관 서브쿼리는 서브쿼리에 메인 테이블의 컬럼이 존재하는 경우이다.
- ② 다중 컬럼 서브쿼리는 모든 비교연산에 대한 처리가 가능하다.
- ③ 단일 행 서브쿼리는 대소비교 연산자의 전달이 가능하다.
- ④ > ALL 연산자는 서브쿼리 내 최댓값보다 큰 조건을 완성한다.

3. 아래 SQL 수행 결과로 알맞은 것은?

<TAB1>

COL1
AB
ABC
BC
BCD
CAB

```
SELECT COUNT(COL1) AS C1
  FROM TAB1
 WHERE COL1 LIKE (SELECT '%'||MIN(COL1)||'%' FROM TAB1);
```

- ① 0
- ② 1
- ③ 2
- ④ 3

4. 다음 중 항상 오류가 발생하는 것은?

①

```
SELECT *
  FROM TAB1
 WHERE (COL1, COL2) = (SELECT COL1, MAX(COL2)
                        FROM TAB2
                        GROUP BY COL1
                        HAVING MAX(COL2) > 100);
```

②

```
SELECT COL1, (SELECT COL3
               FROM TAB2
               WHERE TAB1.COL2 = TAB2.COL2)
  FROM TAB1;
```

③

```
SELECT TAB1.COL1, TAB1.COL2, I.AVGSAL
  FROM TAB1, (SELECT COL1, AVG(COL2) AS AVGSAL
               FROM TAB1
               GROUP BY COL1) I
 WHERE TAB1.COL1 = I.COL1
        AND TAB1.COL2 > I.AVGSAL;
```

④

```
SELECT T1.COL1, T2.AVGSAL
  FROM TAB1 T1
 WHERE COL1 < (SELECT AVG(COL1) AS AVGSAL
               FROM TAB1 T2
               WHERE COL2 = T1.COL2);
```


5. 아래 SQL 수행 결과로 알맞은 것은?

<EMP>

EMPNO	ENAME	COMM	MGR
1000	SMITH	200	1001
1001	ALLEN	100	1002
1002	FORD	200	1003
1003	WARD	300	1004
1004	KING	200	NULL

```
UPDATE EMP E1
  SET COMM = (SELECT (E1.COMM + E2.COMM)/2
              FROM EMP E2
              WHERE E1.MGR = E2.EMPNO
              AND E1.COMM > E2.COMM);
```

```
SELECT SUM(COMM)
  FROM EMP;
```

- ① 300
- ② 400
- ③ 800
- ④ NULL

6. 다음 SQL의 수행 결과로 가장 적절한 것은?

<TAB1>

COL1	COL2
A	10
B	20
C	30
E	50
NULL	40

<TAB2>

COL1	COL2
A	10
C	20
D	30
NULL	40

```
SELECT SUM(COL2)
  FROM TAB1
 WHERE NOT EXISTS (SELECT COL1
                  FROM TAB2
                  WHERE TAB1.COL1 = TAB2.COL1);
```

- ① 40
- ② 70
- ③ 80
- ④ 110

7. 아래의 SQL 의 출력 결과 중 알맞은 것은?

<TAB1>

COL1	COL2
A	25
B	20
C	40
D	10

<TAB2>

COL1	COL2
A	30
A	20
B	20
B	10
C	30
C	40
D	10
D	20

```
SELECT SUM(COL2)
  FROM TAB1
 WHERE COL2 < ANY(SELECT COL2
                   FROM TAB2
                   WHERE TAB1.COL1 = TAB2.COL1);
```

- ① 25
- ② 35
- ③ 55
- ④ 95

8. 다음 SQL 실행 결과로 가장 알맞은 것은?

<실적>

지점	연도	월	실적금액
A	2024	01	2000
A	2024	02	1000
A	2024	03	2000
A	2024	04	3000
B	2024	01	2000
B	2024	02	4000
B	2024	03	2000

```
SELECT SUM(실적금액)
  FROM (SELECT 실적.*,
              LAG(실적금액) OVER(PARTITION BY 지점 ORDER BY 월) AS VAL
        FROM 실적)
 WHERE 실적금액 < VAL;
```

- ① 3000
- ② 5000
- ③ 7000
- ④ 9000

9. 아래 결과를 얻기 위한 빈칸에 들어갈 문장으로 가장 적절한 것은?

<EMP>

ENAME	JOB	DNAME	SAL
CLARK	MANAGER	ACCOUNTING	2450
KING	PRESIDENT	ACCOUNTING	5000
MILLER	CLERK	ACCOUNTING	1300
JONES	MANAGER	RESEARCH	2975
FORD	ANALYST	RESEARCH	3000
ADAMS	CLERK	RESEARCH	1100
SMITH	CLERK	RESEARCH	800
SCOTT	ANALYST	RESEARCH	3000
WARD	SALESMAN	SALES	1250
TURNER	SALESMAN	SALES	1500
ALLEN	SALESMAN	SALES	1600
JAMES	CLERK	SALES	950
BLAKE	MANAGER	SALES	2850
MARTIN	SALESMAN	SALES	1250

<결과>

DNAME	JOB	SUMSAL
ACCOUNTING	CLERK	1300
ACCOUNTING	MANAGER	2450
ACCOUNTING	PRESIDENT	5000
ACCOUNTING		8750
RESEARCH	ANALYST	6000
RESEARCH	CLERK	1900
RESEARCH	MANAGER	2975
RESEARCH		10875
SALES	CLERK	950
SALES	MANAGER	2850
SALES	SALESMAN	5600
SALES		9400
	ANALYST	6000
	CLERK	4150
	MANAGER	8275
	PRESIDENT	5000
	SALESMAN	5600
		29025

```
SELECT DNAME, JOB, SUM(SAL) AS SUMSAL
  FROM EMP
 GROUP BY _____
 ORDER BY 1, 2;
```

- ① CUBE(DNAME, JOB)
- ② CUBE(DNAME, JOB, NULL)
- ③ ROLLUP(DNAME, JOB)
- ④ GROUPING SETS(DNAME, JOB, NULL)

10. 다음 중 수행 불가능한 SQL은?

<TAB1>	
COL1	NUMBER(10)
COL2	NUMBER(6)
<TAB2>	
COL1	NUMBER(10)
COL2	NUMBER(6)

①

```
SELECT COL2, COL1
  FROM TAB1
 UNION ALL
SELECT COL1, COL2
  FROM TAB2;
```

②

```
SELECT COL1, COL2
  FROM TAB1
 UNION
SELECT COL1, COL2
  FROM TAB2
 ORDER BY COL1;
```

③

```
(SELECT COL1, COL2
  FROM TAB1
 UNION
SELECT COL1, COL2
  FROM TAB2)
 ORDER BY COL1;
```

④

```
SELECT COL1, COL2
  FROM TAB1
 ORDER BY COL1
 UNION ALL
SELECT COL1, COL2
  FROM TAB2
 ORDER BY COL1;
```

11. 다음 SQL 실행 결과로 가장 알맞은 것은?

<직원>

사번	이름	부서번호	급여
1000	SMITH	01	5000
1001	ALLEN	01	5000
1002	SCOTT	01	3000
1003	CLARK	01	2000
1004	BLAKE	02	4000
1005	JAMES	02	2000
1006	ADAMS	02	2000

```
SELECT RANK() OVER(PARTITION BY 부서번호 ORDER BY 급여 DESC) AS 순위1,
       DENSE_RANK() OVER(PARTITION BY 부서번호 ORDER BY 급여 DESC) AS 순위2,
       ROW_NUMBER() OVER(PARTITION BY 부서번호 ORDER BY 급여 DESC) AS 순위3
FROM 직원;
```

①

순위1	순위2	순위3
1	1	1
1	1	2
2	3	3
3	4	4
1	1	1
2	2	2
2	2	3

②

순위1	순위2	순위3
1	1	1
2	1	2
3	2	3
4	3	4
1	1	1
2	2	2
3	2	3

③

순위1	순위2	순위3
1	1	1
1	1	1
3	2	2
4	3	3
1	1	1
2	2	2
2	2	3

④

순위1	순위2	순위3
1	1	1
1	1	2
3	2	3
4	3	4
1	1	1
2	2	2
2	2	3

12. 다음 SQL 실행 결과로 가장 알맞은 것은?

<포인트>

고객번호	포인트	날짜
1000	500	2024/01/01 14:34:00
1000	300	2024/01/03 19:30:00
1000	300	2024/01/05 09:50:10
1000	100	2024/01/07 11:10:00
1001	400	2024/01/09 14:00:00
1001	200	2024/01/10 15:30:00
1001	100	2024/01/11 16:34:00

```
SELECT p.*, SUM(포인트) OVER(PARTITION BY 고객번호
                                ORDER BY 날짜
                                ROWS BETWEEN 1 PRECEDING AND CURRENT ROW ) as 누적포인트
FROM 포인트 p
```

①

고객번호	포인트	날짜	누적포인트
1000	500	2024/01/01 14:34:00	500
1000	300	2024/01/03 19:30:00	300
1000	300	2024/01/05 09:50:10	300
1000	100	2024/01/07 11:10:00	100
1001	400	2024/01/09 14:00:00	400
1001	200	2024/01/10 15:30:00	200
1001	100	2024/01/11 16:34:00	100

②

고객번호	포인트	날짜	누적포인트
1000	500	2024/01/01 14:34:00	500
1000	300	2024/01/03 19:30:00	800
1000	300	2024/01/05 09:50:10	600
1000	100	2024/01/07 11:10:00	400
1001	400	2024/01/09 14:00:00	400
1001	200	2024/01/10 15:30:00	600
1001	100	2024/01/11 16:34:00	300

③

고객번호	포인트	날짜	누적포인트
1000	500	2024/01/01 14:34:00	500
1000	300	2024/01/03 19:30:00	800
1000	300	2024/01/05 09:50:10	1100
1000	100	2024/01/07 11:10:00	1200
1001	400	2024/01/09 14:00:00	400
1001	200	2024/01/10 15:30:00	600
1001	100	2024/01/11 16:34:00	700

④

고객번호	포인트	날짜	누적포인트
1000	500	2024/01/01 14:34:00	500
1000	300	2024/01/03 19:30:00	1100
1000	300	2024/01/05 09:50:10	1100
1000	100	2024/01/07 11:10:00	1200
1001	400	2024/01/09 14:00:00	400
1001	200	2024/01/10 15:30:00	600
1001	100	2024/01/11 16:34:00	700

13. 다음 결과를 얻기 위한 SQL 결과로 가장 적절한 것은?

<EXAM>	
이름	성적
홍길동	98
김길동	80
이길동	80
최길동	72
박길동	60
<결과>	
이름	성적
홍길동	98
김길동	80

①

```
SELECT TOP(2) 이름, 성적
FROM EXAM
ORDER BY 성적 DESC;
```

②

```
SELECT TOP(3) 이름, 성적
FROM EXAM
ORDER BY 성적 DESC;
```

③

```
SELECT TOP(2) WITH TIES 이름, 성적
FROM EXAM
ORDER BY 성적 DESC;
```

④

```
SELECT TOP(3) WITH TIES 이름, 성적
FROM EXAM
ORDER BY 성적 DESC;
```

14. 다음 수행 결과로 가장 적절한 것은?

<DEPARTMENT>

DEPTNO	DNAME	PART	BUILD
101	컴퓨터공학과	100	정보관
102	멀티미디어공학과	100	멀티미디어관
103	소프트웨어공학과	100	소프트웨어관
201	전자공학과	200	전자제어관
202	기계공학과	200	기계실험관
203	화학공학과	200	화학실습관
301	문헌정보학과	300	인문관
100	컴퓨터정보학부	10	
200	메카트로닉스학부	10	
300	인문사회학부	20	
10	공과대학		
20	인문대학		

```
SELECT LEVEL, DNAME
FROM DEPARTMENT
START WITH DEPTNO = 10
CONNECT BY PRIOR DEPTNO = PART
ORDER BY 1, 2;
```

①

LEVEL	DNAME
1	공과대학
2	컴퓨터정보학부
3	컴퓨터공학과
3	멀티미디어공학과
3	소프트웨어공학과
2	메카트로닉스학부
3	전자공학과
3	기계공학과
3	화학공학과
1	인문대학
2	인문사회학부
3	문헌정보학과

②

LEVEL	DNAME
1	공과대학
2	메카트로닉스학부
2	컴퓨터정보학부
3	기계공학과
3	멀티미디어공학과
3	소프트웨어공학과
3	전자공학과
3	컴퓨터공학과
3	화학공학과

③

LEVEL	DNAME
1	공과대학
1	인문대학

④

LEVEL	DNAME
1	공과대학

15. 아래 결과를 얻기 위한 빈칸에 들어갈 문장으로 가장 적절한 것은?

<학생>

학번	이름	클래스	점수
1	홍길동	A	90
2	박길동	A	80
3	송길동	A	70
4	김길동	B	85
5	구길동	B	80
6	이길동	B	75

<결과>

학번	이름	클래스	점수	학번	이름	클래스	점수
1	홍길동	A	90				
2	박길동	A	80	1	홍길동	A	90
3	송길동	A	70	1	홍길동	A	90
3	송길동	A	70	2	박길동	A	80
4	김길동	B	85				
5	구길동	B	80	4	김길동	B	85
6	이길동	B	75	4	김길동	B	85
6	이길동	B	75	5	구길동	B	80

```
SELECT S1.*, S2.*
  FROM 학생 S1, 학생 S2
 WHERE _____
 ORDER BY S1.학번, S2.학번;
```

- ① S1.점수 > S2.점수
- ② S1.점수 < S2.점수
- ③ S1.점수 > S2.점수(+) AND S1.클래스 = S2.클래스(+)
- ④ S1.점수 < S2.점수(+) AND S1.클래스 = S2.클래스(+)

16. 다음 SQL 실행 결과로 가장 적절한 것은?

```
SELECT REGEXP_SUBSTR('ORA-00933: SQL command not properly ended', '[A-z]+', 1, 3) FROM DUAL;
```

- ① null
- ② ORA
- ③ SQL
- ④ command

17. 다음 출력 결과를 얻기 위한 SQL로 가장 적절한 것은?

<학생>

학생번호	이름	학년	성별	성적
1000	홍길동	4	남	90
1001	박길동	4	여	80
1002	최길동	3	남	90
1003	유길동	3	남	70
1004	신길동	3	여	90
1005	김길동	2	남	100
1006	간길동	2	남	60
1007	문길동	2	여	80
1008	송길동	1	여	60
1009	구길동	1	남	90

<결과>

	남	여
1	90	60
2	80	80
3	80	90
4	90	80

①

```
SELECT *
FROM (SELECT 학년, 성별, 성적 FROM 학생)
PIVOT (AVG(성적) FOR 학년 IN (1,2,3,4));
```

②

```
SELECT *
FROM (SELECT 학년, 성별, 성적 FROM 학생)
UNPIVOT (AVG(성적) FOR 학년 IN (1,2,3,4));
```

③

```
SELECT *
FROM (SELECT 학년, 성별, 성적 FROM 학생)
PIVOT (AVG(성적) FOR 성별 IN ('남','여'));
```

④

```
SELECT *
FROM (SELECT 학년, 성별, 성적 FROM 학생)
UNPIVOT (AVG(성적) FOR 성별 IN ('남','여'));
```

18. 다음 SQL 실행 결과로 가장 적절한 것은?

<TAB1>

A12345A
12345BA
AA
B00A

```
SELECT COUNT(ID)
```

```
FROM TAB1
```

```
WHERE REGEXP_LIKE(ID, '^A|B.+A$');
```

① 0

② 1

③ 2

④ 3

19. 다음 SQL로 추출되지 않는 것은? (단, 보기의 문자열은 COL1의 값이다)

```
SELECT REGEXP_SUBSTR(COL1, '^\d{3}\d{3}') FROM DUAL;
```

① (031)123-4567

② (064)555-1212

③ (051)987-0000

④ (02)345-6789

20. 빈칸에 들어갔을 때 출력되지 않는 것은?

```
SELECT COL1 FROM (SELECT _____ AS COL1 FROM DUAL)
```

```
WHERE REGEXP_LIKE(COL1, '^[A-Z]{2}\d{2}\1$');
```

① CD34CD

② ZZ99ZZ

③ AA00AB

④ AB12AB

2-17. DML

● DML(Data Manipulation Language)

- 테이블에 저장된 데이터를 삽입(INSERT), 수정(UPDATE), 삭제(DELETE), 병합(MERGE) 하는 명령어
- 트랜잭션을 발생시키므로 **작업 후 반드시 COMMIT 또는 ROLLBACK 이 필요함**

● INSERT

- 테이블에 행(row)을 삽입할 때 사용하는 명령어
- 각 컬럼의 데이터 타입과 크기에 맞는 값만 삽입 가능
- Oracle 은 서브쿼리 없이 사용 시 **한 번에 한 행만 삽입 가능(SQL Server. 여러 행 동시 삽입 가능)**
- Oracle 은 INSERT ALL 을 사용하여 서로 다른 테이블에 각각 입력 가능(서브쿼리 필수)

** 문법

SQL1 *	
1	INSERT INTO 테이블명 VALUES(값1, 값2, ...); -- 전체 컬럼 값을 입력
2	INSERT INTO 테이블명(컬럼1, 컬럼2, ...) VALUES(값1, 값2, ...); -- 선택한 컬럼만 데이터 입력

** 주의

- INTO 절에 컬럼명을 명시하면 일부 컬럼만 데이터 입력 가능하며, **명시하지 않은 컬럼에는 NULL 이 입력됨**
 ☞ 단, **NOT NULL 제약조건이 있는 컬럼은 오류 발생**
- 전체 컬럼에 대한 데이터를 입력하는 경우, 테이블명 뒤의 컬럼명 목록은 생략 가능

예제) INSERT 문을 이용한 단일 행 데이터 삽입

SQL1 *

1

2

DESC MERGE_OLD;

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	Column	Nullable	Type	Comment
1	NO		NUMBER(10)	
2	NAME		VARCHAR2(10)	
3	PRICE		NUMBER(10)	

SQL1 *

1

INSERT INTO MERGE_OLD VALUES(1, 'AMERICANO', 1000);

2

INSERT INTO MERGE_OLD VALUES(2, 'LATTE', 2000);

3

INSERT INTO MERGE_OLD VALUES(3, 'MILK', 3000);

4

Result

Grid Result

Server Output

Text Output

Explain Plan

Statistics

	Column	Nullable	Type	Comment
1	NO		NUMBER(10)	
2	NAME		VARCHAR2(10)	
3	PRICE		NUMBER(10)	

- ☞ 데이터는 각 컬럼의 데이터 타입과 크기(size)에 맞게 입력해야 함
- ☞ 문자형 컬럼에는 숫자 형태의 값도 입력 가능하나, 암묵적 형 변환이 발생하므로 권장하지 않음
- ☞ 숫자형 컬럼에는 '001'과 같이 숫자처럼 보이는 문자값도 입력 가능하나 권장하지 않음

결과)

SQL1 *		
1	SELECT *	
2	FROM MERGE_OLD;	
3		

Result		
Grid Result	Server Output	Text Output
Explain Plan	Statistics	

	NO	NAME	PRICE
1	1	AMERICANO	1000
2	2	LATTE	2000
3	3	MILK	3000

☞ 각 INSERT 문장이 정상적으로 수행되어 총 3 개의 행이 테이블에 삽입됨

예제) INSERT ALL 을 이용한 다중 행 입력

SQL1 *		
1	CREATE TABLE INSERT_TAB1(	
2	NO NUMBER PRIMARY KEY,	
3	NAME VARCHAR2(10)	
4	);	
5		
6	INSERT ALL	
7	INTO INSERT_TAB1 VALUES(1, 'A')	
8	INTO INSERT_TAB1 VALUES(2, 'B')	
9	SELECT * FROM DUAL;	
10	COMMIT;	
11		
12	SELECT *	
13	FROM INSERT_TAB1;	
14		

Result		
Grid Result	Server Output	Text Output
Explain Plan	Statistics	

	NO	NAME
1	1	A
2	2	B

☞ INSERT ALL 을 사용하면 하나의 INSERT 문으로 동일 테이블에 여러 행을 동시에 입력할 수 있으며, 실제 데이터 조회를 위해 SELECT * FROM DUAL 을 함께 사용함

☞ 서로 다른 테이블에도 데이터를 함께 입력할 수 있음

예제) 서브쿼리를 이용한 다중 행 INSERT

SQL1 *		
1	CREATE TABLE INSERT_TAB2(	
2	NO NUMBER,	
3	NAME VARCHAR2(10)	
4	);	
5		
6	INSERT INTO INSERT_TAB2	
7	SELECT *	
8	FROM INSERT_TAB1;	
9		
10	COMMIT;	
11		
12	SELECT *	
13	FROM INSERT_TAB2;	
14		

Result		
Grid Result	Server Output	Text Output
Explain Plan	Statistics	

	NO	NAME
1	1	A
2	2	B

☞ 서브쿼리를 사용하면 조회 결과가 여러 행일 경우, 해당 행 수만큼 데이터를 한 번에 입력할 수 있음

예제) INSERT 문 컬럼 리스트 생략 시 입력 규칙

```

SQL1 *
1  INSERT INTO INSERT_TAB1 VALUES(3);           -- 에러
2  INSERT INTO INSERT_TAB1(NAME) VALUES('C');    -- 에러
3  INSERT INTO INSERT_TAB1(NO) VALUES(3);        -- 정상
4
5  COMMIT;
6
7  SELECT *
8     FROM INSERT_TAB1;
9

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	NO	NAME
1	1	A
2	2	B
3	3	

- ☞ 컬럼 리스트를 생략하면 테이블에 정의된 모든 컬럼 개수와 순서대로 값을 입력해야 하므로, VALUES(3)처럼 값 개수가 부족하면 오류가 발생함
- ☞ INSERT 문에서는 일부 컬럼만 선택하여 입력할 수 있으나, 이 경우 언급되지 않은 컬럼에는 NULL 이 입력되며, 해당 컬럼들이 모두 NULL 을 허용하는 경우에만 정상 처리됨

● UPDATE

- 테이블에 저장된 데이터를 수정할 때 사용하는 명령어
- 컬럼 단위로 수행되며, **하나 이상의 컬럼을 동시에 수정할 수 있음**

** 문법

1) 단일 컬럼 수정

```

SQL1 *
1  UPDATE 테이블명
2     SET 수정컬럼 = 값
3     [WHERE 조건]           -- 수정 대상
4

```

- ☞ WHERE 절로 수정 대상을 선택 가능

2) 다중 컬럼 수정

문법 1)

```

SQL1 *
1  UPDATE 테이블명
2     SET 수정컬럼1 = 값1, 수정컬럼2 = 값2, ...
3     [WHERE 조건]
4

```

문법 2)

```

SQL1 *
1  UPDATE 테이블명
2     SET (수정컬럼명1, 수정컬럼명2, ...) = (SELECT 수정값1, 수정값2 ...)
3     WHERE 조건;

```

예제) 조건에 따른 단일 컬럼 PRICE 값 변경

SQL1 *

```

1 ▶ UPDATE MERGE_OLD
2   SET PRICE = 1500
3   WHERE NAME = 'AMERICANO';

```

Result

1 rows Updated.

WHERE 절 조건(NAME = 'AMERICANO')에 만족하는 행의 PRICE 값이 1500 으로 수정된다.

예제) 특정 행의 다중 컬럼(NAME, PRICE) 수정

SQL1 *

```

1 ▶ UPDATE MERGE_OLD
2   SET NAME = 'HOT_MILK', PRICE = 2500
3   WHERE NO = 3;

```

Result

1 rows Updated.

WHERE 절 조건(NO = 3)에 만족하는 행의 NAME 과 PRICE 값이 동시에 수정된다.

예제) 서브쿼리를 이용한 다중 컬럼(SAL, COMM) 수정

SQL1 *

```

1 ▶ UPDATE UPDATE_TEST
2   SET (SAL, COMM) = (SELECT MAX(SAL), MAX(COMM)
3                       FROM UPDATE_TEST)
4   WHERE ENAME = 'SCOTT';
5

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

1 rows Updated.

수정 전)

SQL1 *

```

1 ▶ SELECT *
2   FROM UPDATE_TEST;
3

```

Result

Grid Result Server Output Text Output

	EMPNO	ENAME	SAL	COMM	DEPTNO
1	7839	KING	5000	100	10
2	7566	JONES	2975	300	20
3	7902	FORD	3000		20
4	7788	SCOTT	3000	300	20
5	7698	BLAKE	2850	600	30

수정 후)

SQL1 *

```

1 ▶ SELECT *
2   FROM UPDATE_TEST;
3

```

Result

Grid Result Server Output Text Output

	EMPNO	ENAME	SAL	COMM	DEPTNO
1	7839	KING	5000	100	10
2	7566	JONES	2975	300	20
3	7902	FORD	3000		20
4	7788	SCOTT	5000	600	20
5	7698	BLAKE	2850	500	30

SMITH의 SAL 과 COMM 값이 각각 5000, 600 으로 수정됨

예제) 연관 서브쿼리를 이용한 부서별 평균 급여로 SAL 수정

SQL1 *					
1	UPDATE UPDATE_TEST U1				
2	SET SAL = (SELECT AVG(SAL)				
3	FROM UPDATE_TEST U2				
4	WHERE U1.DEPTNO = U2.DEPTNO);				
5					
6	SELECT *				
7	FROM UPDATE_TEST;				
8					

Result					
	EMPNO	ENAME	SAL	COMM	DEPTNO
1	7839	KING	5000	100	10
2	7566	JONES	3658.33	300	20
3	7902	FORD	3658.33		20
4	7788	SCOTT	3658.33	600	20
5	7698	BLAKE	2850	500	30

각 행마다 소속 부서의 평균 급여가 계산되어 해당 사원의 SAL 값이 부서 평균으로 갱신됨

● DELETE

- 테이블에 저장된 데이터를 행 단위로 삭제할 때 사용하는 명령어
- WHERE 절을 사용하여 삭제 대상을 선택할 수 있으며, **생략할 경우 테이블의 모든 행이 삭제됨**

** 문법

SQL1 *					
1	DELETE [FROM] 테이블명				
2	[WHERE] 조건				
3					

-- 삭제할 행 선택

예제) 특정 조건에 따른 행 삭제

SQL1 *					
1	DELETE MERGE_OLD				
2	WHERE NO = 3;				

Result					
1 rows Deleted.					

WHERE 절 조건(NO = 3)에 만족하는 1 개의 행이 삭제됨

예제) 서브쿼리를 이용한 평균 COMM 미만 데이터 삭제

SQL1 *					
1	DELETE FROM UPDATE_TEST				
2	WHERE COMM < (SELECT AVG(COMM)				
3	FROM UPDATE_TEST);				
4					
5	SELECT *				
6	FROM UPDATE_TEST;				
7					

Result					
	EMPNO	ENAME	SAL	COMM	DEPTNO
1	7902	FORD	3658.33		20
2	7788	SCOTT	3658.33	600	20
3	7698	BLAKE	2850	500	30

전체 데이터의 평균 COMM(375) 값보다 작은 COMM 을 가진 행이 삭제됨

4) MERGE

- 기준이 되는 참조 테이블과의 연결 조건(ON 절)을 기준으로, 대상 테이블의 데이터를 동기화하는 작업
(참조 테이블의 데이터 삽입, 참조 테이블 값 기준 수정 등)
- 조건에 따라 INSERT, UPDATE, DELETE 작업을 동시에 수행 가능

** 문법

```

SQL1 *
1  MERGE INTO 테이블명 T1          -- 수정할 테이블명
2  USING 참조테이블명 T2          -- 참조할 테이블명
3  ON (조건)                      -- 괄호 필수
4  WHEN MATCHED THEN
5  UPDATE                        -- 테이블명 생략
6  SET T1.컬럼 = 값              --
7  [DELETE (조건)]              -- 괄호 생략 가능
8  [WHEN NOT MATCHED THEN
9  INSERT                        -- INTO 테이블명 생략
10 VALUES(T2.컬럼1, T2.컬럼2, ...)]
11 ;

```

** 주의

1) WHEN MATCHED THEN 절

- 조건이 만족하는 경우 수행됨
- UPDATE 만 수행하거나, UPDATE 와 DELETE 를 함께 수행할 수 있음
- DELETE 는 UPDATE 후 수행되며, 수정된 행을 대상으로만 삭제됨

2) WHEN NOT MATCHED THEN 절

- 조건이 만족하지 않는 경우 수행할 명령어를 지정
- 생략 가능
- INSERT 만 수행 가능

예제) OLD 테이블을 NEW 테이블과 동일하게 MERGE 문 작성

STEP1) MERGE 할 TABLE 확인

SQL1 *	
1	SELECT *
2	FROM MERGE_NEW;

Result			
Grid Result			
	NO	NAME	PRICE
1	1	AMERICANO	1000
2	2	LATTE	2500
3	3	MOCHA	3000

SQL1 *	
1	SELECT *
2	FROM MERGE_OLD;

Result			
Grid Result			
	NO	NAME	PRICE
1	1	AMERICANO	1500
2	2	LATTE	2000

STEP2) MERGE 문 작성

```

SQL1 *
1  MERGE INTO MERGE_OLD M1
2  USING MERGE_NEW M2
3  ON (M1.NO = M2.NO)
4  WHEN MATCHED THEN
5  UPDATE
6  SET M1.PRICE = M2.PRICE
7  WHEN NOT MATCHED THEN
8  INSERT VALUES(M2.NO, M2.NAME, M2.PRICE);

```

Result

3 rows upserted.

- ☞ 수정할 테이블명을 MERGE INTO 절에 명시하고, 참조 테이블을 USING 절에 명시
- ☞ 두 테이블의 데이터를 비교할 연결 조건을 ON 절에 명시하며, 괄호 필수
- ☞ UPDATE 문에서는 테이블명을 명시하지 않음
- ☞ SET 절에서 왼쪽은 수정 테이블 컬럼, 오른쪽은 참조 테이블 컬럼
- ☞ INSERT 문에서는 INTO 절 없이 VALUES 절만 사용하여 참조 테이블 컬럼 값을 전달
- ☞ 연결 조건에 따라 일치하는 행은 UPDATE, 일치하지 않는 행은 INSERT 수행

STEP3) MERGE 결과 확인

```

SQL1 *
1  SELECT *
2  FROM MERGE_OLD;

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	NO	NAME	PRICE
1	1	AMERICANO	1000
2	2	LATTE	2500
3	3	MOCHA	3000

- ☞ 처음 테이블에는 없던 MOCHA 행이 삽입되고, 기존 LATTE의 가격은 2000에서 2500으로 수정됨

<< 단원 정리 문제 >>

1. 다음 중 DML 에 대한 설명으로 가장 적절하지 않은 것은?

- ① DML 은 데이터베이스의 데이터를 삽입, 수정, 삭제하는 데 사용되는 SQL 명령어 집합이다.
- ② TRUNCATE 명령어는 DML 의 대표적인 명령어로 데이터를 삭제하는 데 사용된다.
- ③ INSERT 명령어는 새로운 값을 입력하는 데 사용된다.
- ④ DELETE 명령어는 테이블에서 데이터를 삭제하는 데 사용된다.

2. 다음 SQL 결과로 가장 적절한 것은?

<SALES>

ID	REGION	YEAR	SALES_AMOUNT
1	서울	2023	1000
2	서울	2024	1200
3	부산	2023	900
4	부산	2024	1100
5	대구	2023	900
6	대구	2024	800

```
UPDATE SALES S1
  SET SALES_AMOUNT = (SELECT MAX(SALES_AMOUNT) FROM SALES S2
                      WHERE S1.REGION = S2.REGION);

DELETE FROM SALES WHERE YEAR = 2023;
COMMIT;

SELECT SUM(SALES_AMOUNT) FROM SALES;
```

- ① 3100
- ② 3200
- ③ 3600
- ④ 5900

3. 다음은 MERGE 문에 대한 설명이다. 옳은 것을 고르시오 (단, DBMS 는 Oracle)

- ① WHEN MATCHED THEN 절은 생략 가능하며, WHEN NOT MATCHED THEN 절은 반드시 작성해야 한다.
- ② WHEN MATCHED THEN 절에서는 DELETE 만 단독으로 수행할 수 있다.
- ③ WHEN NOT MATCHED THEN 절에서는 INSERT, UPDATE 중 하나를 선택하여 수행할 수 있다.
- ④ WHEN MATCHED THEN 절에서 UPDATE 와 DELETE 를 함께 사용할 경우, UPDATE 후 DELETE 가 수행된다.

4. 다음 중 Oracle 에서 DML 문에 대한 설명으로 옳지 않은 것은? (단, DBMS 는 Oracle)

- ① INSERT 문에서 서브쿼리를 사용하지 않으면 한 번에 한 행만 삽입할 수 있다.
- ② UPDATE 문은 컬럼 단위로 수행되며, 하나의 UPDATE 문으로 여러 컬럼을 동시에 수정할 수 있다.
- ③ DELETE 문에서 WHERE 절을 생략하면 테이블의 모든 행이 삭제된다.
- ④ INSERT 문에서 컬럼명을 생략하더라도 일부 컬럼에만 값을 입력할 수 있다.

1. ②

TRUNCATE 는 데이터를 삭제하는 명령어이지만 AUTO COMMIT 특성으로 인해 DDL 에 속한다.

2. ②

SALES_AMOUNT 를 각 지역의 최댓값으로 수정 후, 2023 연도 행을 삭제하면 서울은 1200, 부산은 1100, 대구는 900 이 남는다.

3. ④

WHEN MATCHED THEN 절은 생략할 수 없으며, UPDATE 만 수행하거나 UPDATE 와 DELETE 를 함께 수행할 수 있다. DELETE 는 단독으로 사용할 수 없고 반드시 UPDATE 와 함께 사용되며, 이 경우 UPDATE 가 먼저 수행된 후 수정된 행을 기준으로 DELETE 조건이 평가된다. WHEN NOT MATCHED THEN 절은 생략 가능하며 INSERT 만 수행할 수 있다.

4. ④

INSERT 문에서 컬럼명을 생략하는 경우 테이블의 전체 컬럼 수와 순서에 맞게 값을 모두 입력해야 한다. 일부 컬럼에만 값을 입력하려면 반드시 테이블명 뒤에 컬럼명을 명시해야 한다. 따라서 ④번 설명은 옳지 않다.

2-18. TCL

● TCL(Transaction Control Language)

- **트랜잭션 제어어**로 COMMIT, ROLLBACK 이 포함됨
- DML 에 의해 조작된 결과를 작업 단위(트랜잭션)별로 제어하는 명령어
- DML 수행 후 트랜잭션을 정상적으로 종료하지 않으면 LOCK 이 발생할 수 있음



LOCK



트랜잭션 수행 중 다른 트랜잭션이 동일한 데이터에 동시에 접근하지 못하도록 제어해 데이터의 일관성과 무결성을 보장하는 메커니즘

● 트랜잭션

1. 개념

- 트랜잭션은 데이터베이스에서 하나의 연속적인 업무를 처리하기 위한 분할할 수 없는 논리적 연산 단위
- 하나의 트랜잭션에는 하나 이상의 SQL 문장이 포함됨
- 트랜잭션 내 모든 작업은 **모두 COMMIT 되거나 전체가 ROLLBACK 처리되어야 함(ALL OR NOTHING)**

2. 특징

구분	설명
원자성 (Atomicity)	• 트랜잭션에 정의된 연산들은 모두 성공적으로 실행되거나, 전혀 실행되지 않은 상태로 남아 있어야 함
일관성 (Consistency)	• 트랜잭션 실행 전 데이터베이스 내용에 문제가 없다면, 실행 이후에도 데이터베이스 내용에 문제가 발생해서는 안됨
고립성 (Isolation)	• 트랜잭션 실행 도중 다른 트랜잭션의 영향을 받아 잘못된 결과가 발생해서는 안됨
지속성 (Durability)	• 트랜잭션이 성공적으로 수행되면, 갱신된 데이터베이스 내용은 영구적으로 저장

● COMMIT/ROLLBACK

구분	설명
COMMIT	• 입력, 수정, 삭제한 데이터에 이상이 없을 경우 트랜잭션을 확정하여 데이터베이스에 영구 저장하는 명령어 • 한 번 COMMIT 을 수행하면 COMMIT 이전에 수행된 모든 DML 은 저장되며 ROLLBACK 으로 되돌릴 수 없음 • 일반적으로 DDL 문은 DBMS 에 관계없이 자동 COMMIT 되며, 트랜잭션 제어 대상이 아님
ROLLBACK	• 입력, 수정, 삭제한 데이터 중 아직 COMMIT 되지 않은 변경 사항을 취소하는 명령어 • 변경 내용은 데이터베이스에 저장되지 않으며, 최종 COMMIT 이전 상태 또는 특정 SAVEPOINT 지점으로 원복됨 • SAVEPOINT 를 설정하면 최종 COMMIT 시점이 아닌, 지정한 SAVEPOINT 이후 작업만 선택적으로 취소 가능

● SAVEPOINT

- 트랜잭션 내에서 ROLLBACK 을 부분적으로 수행하기 위해 기준이 되는 지점을 지정하는 데 사용
- 사용자가 원하는 위치에 원하는 이름으로 설정 가능
- **ROLLBACK TO SAVEPOINT** 명을 사용하면 **해당 SAVEPOINT 지점 이후 작업만 취소 가능**
- 단, COMMIT 이후에는 어떤 SAVEPOINT 로도 원복 불가

** 문법

```
SQL1 *
1  SAVEPOINT SAVEPOINT_NAME;
```

예제) COMMIT 과 ROLLBACK 후 최종 데이터 상태

```
SQL1 *
1  CREATE TABLE ROLLBACK_TEST (
2      EMPNO    NUMBER(4),
3      ENAME    VARCHAR2(20),
4      DEPTNO   NUMBER(2),
5      SAL      NUMBER(7,2)
6  );
7
8  INSERT INTO ROLLBACK_TEST (EMPNO, ENAME, DEPTNO) VALUES (9999, 'HONG', 10);
9  INSERT INTO ROLLBACK_TEST (EMPNO, ENAME, DEPTNO) VALUES (9998, 'PARK', 20);
10
11 COMMIT;
12
13 UPDATE ROLLBACK_TEST
14     SET SAL = 3000
15     WHERE EMPNO = 9999;
16
17 ROLLBACK;
18
19 UPDATE ROLLBACK_TEST
20     SET SAL = 3000
21     WHERE EMPNO = 9998;
22
23 COMMIT;
24
25 UPDATE ROLLBACK_TEST
26     SET SAL = 4000
27     WHERE EMPNO = 9999;
28
29 SAVEPOINT A;
30
31 DELETE ROLLBACK_TEST WHERE EMPNO = 9999;
32
33 UPDATE ROLLBACK_TEST
34     SET SAL = 9000
35     WHERE EMPNO = 9998;
36
37 ROLLBACK TO A;
38 COMMIT;
39
```

최종 결과)

SQL1 *				
1	▶	SELECT	*	
2		FROM	ROLLBACK_TEST;	
3				

Result				
Grid Result Server Output Text Output Explain Plan Statistics				
	EMPNO	ENAME	DEPTNO	SAL
1	9999	HONG	10	4000
2	9998	PARK	20	3000

- ☞ 9999(HONG)과 9998(PARK)을 INSERT 후 COMMIT 했으므로 두 행은 영구 저장된다.
- ☞ 이후 9999 의 SAL 을 3000 으로 UPDATE 했지만 곧바로 ROLLBACK 했으므로 해당 UPDATE 는 취소되어 SAL 은 변경 전(예: NULL) 상태로 돌아간다.
- ☞ 9998 의 SAL 을 3000 으로 UPDATE 후 COMMIT 했으므로 9998 의 SAL=3000 은 저장된다.
- ☞ 그 다음 9999 의 SAL 을 4000 으로 UPDATE 한 뒤 SAVEPOINT A 를 찍었으므로, A 시점까지의 변경(9999 SAL=4000)은 유지되는 기준점이 된다.
- ☞ 이후 9999 를 DELETE 하고 9998 의 SAL 을 9000 으로 UPDATE 했지만, ROLLBACK TO A 를 수행했으므로 SAVEPOINT A 이후 작업(DELETE 9999, UPDATE 9998=9000)은 모두 취소된다.
- ☞ 마지막 COMMIT 으로 SAVEPOINT A 시점의 상태가 확정되므로 최종적으로
 - 9999 는 삭제되지 않고 남으며 SAL=4000 으로 저장
 - 9998 은 SAL=3000(이전 COMMIT 값)이 유지된다.

<< 단원 정리 문제 >>

1. 다음 중 TCL 에 대한 설명으로 가장 적절하지 않은 것은?

- ① TCL 은 데이터베이스에서 트랜잭션의 시작, 종료 및 관리에 사용되는 명령어 집합이다.
- ② COMMIT 명령어는 트랜잭션에서 수행된 작업을 영구적으로 데이터베이스에 반영한다.
- ③ ROLLBACK 명령어를 사용하여 작업을 취소하고 이전 상태로 되돌릴수 있지만, COMMIT 이전 시점으로는 돌아갈 수 없다
- ④ SAVEPOINT 명령어는 트랜잭션의 특정 시점에 대한 부분적인 커밋을 수행한다.

2. 다음 중 트랜잭션에 대한 설명으로 가장 적절하지 않은 것은?

- ① 트랜잭션은 데이터베이스의 일관성과 무결성을 유지하기 위해 실행되는 일련의 연산이다.
- ② 트랜잭션은 COMMIT 명령어를 통해 확정되며, COMMIT 을 수행하지 않으면 발생한 모든 변경 사항은 데이터베이스에 반영되지 않는다.
- ③ 트랜잭션 고립성이란 트랜잭션 간의 격리가 없음을 의미한다.
- ④ 트랜잭션 원자성이란 트랜잭션 내의 모든 작업이 완전하게 수행되거나 전혀 수행되지 않는 특성을 의미한다.

3. 다음은 TAB 테이블에서 트랜잭션 제어를 섞어 수행한 SQL 이다. 최종 COMMIT 이후, SAL 의 총합(SUM)은 얼마인가?

< ROLLBACK_TEST >

EMPNO	ENAME	DEPTNO	SAL
9999	HONG	10	1000
9998	PARK	20	2000

```
UPDATE ROLLBACK_TEST SET SAL = 3000 WHERE EMPNO = 9999;
INSERT INTO ROLLBACK_TEST VALUES (9997, 'KIM', 30, 1500);
COMMIT;
```

```
UPDATE ROLLBACK_TEST SET SAL = 5000 WHERE EMPNO = 9998;
ROLLBACK;
```

```
UPDATE ROLLBACK_TEST SET SAL = 4000 WHERE EMPNO = 9999;
SAVEPOINT A;
```

```
DELETE ROLLBACK_TEST WHERE EMPNO = 9997;
UPDATE ROLLBACK_TEST SET SAL = 9000 WHERE EMPNO = 9998;
ROLLBACK TO A;
```

- ① 5000
- ② 6000
- ③ 7500
- ④ 8000

1. ④

SAVEPOINT 명령어는 트랜잭션의 특정 시점에 대한 ROLLBACK 시점을 만들어 줄뿐, 부분적인 커밋은 수행하지 않는다.

2. ③

고립성은 트랜잭션 간에 서로 영향을 미치지 않도록 보장하는 특성입니다.

3. ③

1~3 에서 9999 의 SAL 은 3000 으로 변경되고, 9997 이 1500 으로 추가된 상태가 COMMIT 되어 확정이다.

4 에서 9998 의 SAL 을 5000 으로 변경했지만 5 에서 ROLLBACK 했으므로 9998 의 SAL 은 2000 으로 유지된다.

6 에서 9999 의 SAL 을 4000 으로 변경하고 SAVEPOINT A 를 설정한다.

8 과 9 는 SAVEPOINT A 이후 작업이므로 10 에서 ROLLBACK TO A 로 모두 취소된다.

최종 COMMIT 이후 남는 데이터는 9999=4000, 9998=2000, 9997=1500 이며,

급여 총합은 $4000 + 2000 + 1500 = 7500$ 이 된다.

2-19. DDL

● DDL(Data Definition Language)

- 테이블, 인덱스 등 데이터베이스 객체의 구조를 정의·변경·삭제하는 데이터 정의어
- 주요 명령어: CREATE(객체 생성), ALTER(객체 변경), DROP(객체 삭제), TRUNCATE(데이터 삭제)
- DDL 문은 AUTO COMMIT 되어 실행 즉시 저장되며 ROLLBACK 으로 원복 불가

● CREATE

- 테이블, 뷰, 인덱스, 시퀀스 등 데이터베이스 객체를 생성하는 DDL 명령어
- 객체의 구조(컬럼, 데이터 타입, 제약조건 등)를 정의할 때 사용

1. 테이블 생성

- 새로운 테이블을 생성하여 컬럼 구조와 데이터 타입을 정의하는 방식

** 문법

```
SQL1 *
1 CREATE TABLE [소유자.]테이블명(
2   컬럼1 데이터타입 [DEFAULT 기본값] [제약조건],
3   컬럼2 데이터타입 [DEFAULT 기본값] [제약조건],
4   ...
5 );
```

** 특징

- 테이블 생성 시 **테이블명, 컬럼명, 컬럼순서, 데이터타입 정의 필수**
- 테이블 생성 시 컬럼 크기는 데이터타입에 따라 지정하며, **각 컬럼의 제약조건 및 기본값은 생략 가능**
- **테이블 생성 시 소유자 명시 가능**(생략 시 명령어 수행 계정 소유)
- 숫자 컬럼의 경우 컬럼 사이즈 생략 가능, 날짜 컬럼은 사이즈 명시하지 않음

2. 테이블 복제

- 서브쿼리를 사용하여 기존 테이블의 구조와 데이터를 기준으로 새로운 테이블을 생성하는 방식
- CTAS(Create Table As Select)라고 부름

** 문법

```
SQL1 *
1 CREATE TABLE 새테이블명
2 AS
3 SELECT * FROM 테이블명;
```

** 특징

- **복제 테이블의 컬럼명과 컬럼의 데이터 타입이 복제됨**
- SELECT 문에서 컬럼 별칭을 사용하거나 CREATE 문에서 컬럼명을 변경하여 **기존 컬럼명과 다르게 테이블 생성 가능**
- **NULL 속성도 그대로 복제됨**
- 테이블에 정의된 **제약조건과 INDEX 는 복제되지 않으며**, 컬럼 구조와 데이터만 복제됨

● 데이터 타입

구분	종류	설명
문자	CHAR(n)	- 고정 길이 문자형 - 지정한 길이보다 짧으면 공백으로 채움
	VARCHAR2(n)	- 가변 길이 문자형 - 실제 입력한 길이만큼만 저장
	NCHAR(n) NVARCHAR2(n)	- 유니코드 문자 저장용 - 다국어 지원 목적
숫자	NUMBER(p, s)	- 정수와 실수 모두 저장 가능 - p는 전체 자릿수, s는 소수점 이하 자릿수 - 정수부 자릿수는 최대 (p - s)자리까지만 허용 - 소수점은 s 자리까지만 허용하고 그 이후 자리는 자동 반올림 처리됨
	NUMBER(n)	- 정수만 저장할 때 사용(소수점 이하는 반올림되어 저장됨) - 자리수 생략 가능(최대 38 자리까지 허용)
날짜	DATE	- 날짜와 시간(시·분·초)까지 저장 - 컬럼 사이즈 지정 불가
	TIMESTAMP	- 소수점 이하 초 단위까지 표현 가능 - 컬럼 사이즈 지정 불가

예제) 새 테이블 생성

```

SQL1 *
1 CREATE TABLE CREATE_TAB1(
2     NO1 NUMBER(5),
3     NO2 NUMBER(5,2),
4     NAME1 CHAR(5),
5     NAME2 VARCHAR2(5)
6 );
7

```

- 👁 CREATE TABLE 문을 사용하여 새로운 테이블을 생성하며, 컬럼명과 컬럼의 데이터 타입은 반드시 정의함
- 👁 테이블 생성 시 컬럼 순서는 작성한 순서대로 결정됨
- 👁 컬럼의 제약조건, 기본값(DEFAULT)은 선택 사항으로 생략 가능
- 👁 테이블 생성 시 소유자(스키마)는 생략 가능하며, 생략 시 명령어를 수행한 계정이 소유자가 됨

예제) CTAS 를 이용한 복제 테이블 생성

```

SQL1 *
1 ▶ CREATE TABLE CREATE_TAB2
2   AS
3   SELECT *
4   FROM EMP;

```

결과)

SQL1 *	SQL1 *
1 ▶ DESC EMP;	1 ▶ DESC CREATE_TAB2;
2	2
Result	Result
Grid Result Server Output Text Output Explain Pla	Grid Result Server Output Text Output Explain Pla
Column	Column
Nullable	Nullable
Type	Type
Comment	Comment
1 EMPNO	1 EMPNO
2 ENAME NOT NULL	2 ENAME NOT NULL
3 JOB	3 JOB
4 MGR	4 MGR
5 HIREDATE	5 HIREDATE
6 SAL	6 SAL
7 COMM	7 COMM
8 DEPTNO	8 DEPTNO

- ☞ 서브쿼리를 사용하여 기존 테이블(EMP)의 컬럼 구조와 데이터를 그대로 복제하여 새로운 테이블을 생성함
- ☞ SELECT *를 사용했으므로 EMP 테이블의 모든 컬럼이 동일한 이름과 데이터 타입으로 생성됨
- ☞ 테이블의 데이터만 복제되며, 제약조건과 INDEX 등은 복제되지 않는다.

예제) CTAS 를 이용한 컬럼명 변경 테이블 생성

```

SQL1 *
1 ▶ CREATE TABLE CREATE_TAB3(NO, NAME)
2   AS
3   SELECT EMPNO, ENAME
4   FROM EMP;
5

```

- ☞ CREATE TABLE AS SELECT 문에서 테이블 생성 시 컬럼 목록을 명시하면, SELECT 절의 컬럼과 매핑되어 새로운 컬럼명으로 생성 가능함

예제) CTAS 에서 WHERE 조건으로 데이터 없이 테이블 구조만 생성

```

SQL1 *
1 ▶ CREATE TABLE CREATE_TAB4(NO, NAME)
2   AS
3   SELECT EMPNO, ENAME
4   FROM EMP
5   WHERE 1=2;
6

```

- ☞ CREATE TABLE AS SELECT 문에서 WHERE 조건을 거짓(1=2)으로 설정하여 조회 결과가 없도록 처리함
- ☞ 기존 테이블의 컬럼 구조만 복제하고 데이터는 복제하지 않을 때 사용

● ALTER

- 테이블의 구조를 변경하는 DDL 명령어
- 컬럼명 변경, 컬럼 데이터타입 변경, 컬럼 사이즈 변경, DEFAULT 값 변경, 컬럼 추가·삭제, 제약조건 추가·삭제 가능
- 컬럼 순서 변경 불가(컬럼 순서를 바꾸려면 테이블 재생성으로 해결)

1. 컬럼 추가

- 새로 추가된 컬럼의 위치는 항상 맨 마지막이며, 중간 위치에 추가 불가
- 컬럼 추가 시 데이터타입은 필수, DEFAULT 값과 제약조건은 선택 사항
- 여러 컬럼을 동시에 추가 가능하며, 이 경우 반드시 괄호 사용

** 문법

```
SQL1 *
1 ALTER TABLE 테이블명 ADD 컬럼명 데이터타입 [DEFAULT] [제약조건];
```

예제) ALTER TABLE 을 이용한 여러 컬럼 동시 추가

```
SQL1 *
1 ALTER TABLE CREATE_TAB3 ADD (COL1 NUMBER, COL2 CHAR(5));
2
```

여러 컬럼을 동시에 추가할 경우 반드시 괄호를 사용해야 함

2. 컬럼 이름 변경

- 컬럼에 데이터가 존재하더라도 컬럼 이름 변경은 가능
- 여러 컬럼의 이름을 동시에 변경할 수 없으며, 한 번에 하나의 컬럼만 변경 가능

** 문법

```
SQL1 *
1 ALTER TABLE TABLE_NAME RENAME COLUMN 기존컬럼명 TO 새컬럼명;
```

예제) 컬럼 이름 변경

```
SQL1 *
1 ALTER TABLE CREATE_TAB4 RENAME COLUMN NO TO NO#;
2
```

실행 전)

```
SQL1 *
1  ▶ DESC CREATE_TAB4;
2
```

Result

 Grid Result  Server Output  Text Output  Explain

	Column	Nullable	Type	Comment
1	NO		NUMBER(4)	
2	NAME		VARCHAR2(10)	

실행 후)

```
SQL1 *
1 ► DESC CREATE_TAB4;
2
```

Result

 Grid Result  Server Output  Text Output  Explain

	Column	Nullable	Type	Comment
1	NO#		NUMBER(4)	
2	NAME		VARCHAR2(10)	

3. 컬럼(속성) 변경

- 컬럼의 사이즈, 데이터 타입, DEFAULT 값 변경 가능
- 여러 컬럼을 동시에 변경 가능

** 문법

SQL1 *
1 ALTER TABLE 테이블명 MODIFY(컬럼명 DEFAULT 값);

👁️ 괄호 생략 가능

1) 컬럼 사이즈 변경

- **컬럼 사이즈 증가는 항상 가능**
- **컬럼 사이즈 축소는** 데이터 존재 여부에 따라 제한되며, 데이터가 존재하는 경우 **현재 저장된 데이터의 최대 사이즈까지만 축소 가능**
- 여러 컬럼 동시 변경 가능(반드시 괄호 사용)

예제) 여러 컬럼 사이즈 수정

SQL1 *
1 ALTER TABLE EMP MODIFY (EMPNO NUMBER(10), ENAME VARCHAR2(6));
2

👁️ EMPNO 컬럼은 NUMBER(5)에서 NUMBER(10)으로 컬럼 사이즈가 증가하므로 정상 수행됨

👁️ ENAME 컬럼은 VARCHAR2(10)에서 VARCHAR2(6)으로 컬럼 사이즈가 축소되었지만, 기존 데이터의 최대 길이가 6 이하이므로 정상 수행됨

2) 데이터 타입 변경

- **컬럼이 비어 있는 경우 데이터 타입 변경 가능**
- CHAR ↔ VARCHAR 계열처럼 서로 호환되는 타입은 데이터가 있어도 변경 가능
- 데이터가 존재하는 경우 **서로 호환되지 않는 데이터 타입 간 변경은 불가능**

예제) 데이터 존재 여부와 데이터 타입 호환성에 따른 컬럼 타입 변경 가능 여부

SQL1 *
1 ALTER TABLE EMP MODIFY (ENAME CHAR(6));
2

👁️ ENAME 컬럼에 데이터가 존재하더라도, VARCHAR2 → CHAR 처럼 서로 호환되는 문자형 데이터 타입 간 변경은 가능함

오류)

SQL1 *
1 ALTER TABLE EMP MODIFY (SAL VARCHAR2(10));
2

Result

ORA-01439: 데이터 유형을 변경할 열은 비어 있어야 합니다

👁️ SAL 컬럼에 데이터가 존재하므로, NUMBER → VARCHAR2 처럼 서로 호환되지 않는 데이터 타입으로는 변경 불가하여 오류가 발생함

3) DEFAULT 값 변경

- DEFAULT 값이란 INSERT 시 해당 컬럼의 값이 생략될 경우 자동으로 부여되는 값
- INSERT 시 DEFAULT 값이 선언된 컬럼의 값을 생략하면 DEFAULT 값이 자동으로 저장됨
- INSERT 시 DEFAULT 값이 선언된 컬럼에 **NULL 을 직접 입력하면 DEFAULT 값이 아닌 NULL 이 저장됨**
- 이미 **데이터가 존재하는 테이블에 DEFAULT 값을 선언해도 기존 데이터는 변경되지 않으며, 이후 입력되는 데이터부터 적용됨**
- DEFAULT 값은 NULL 로 선언하여 해제 가능

예제) DEFAULT 값 변경 후 INSERT 시 NULL 입력과 컬럼 생략에 따른 동작 비교

SQL1 *

```

1 CREATE TABLE ORDER_TEST (
2     ORDER_NAME VARCHAR2(20),
3     STATUS      VARCHAR2(10)
4 );
5
6 INSERT INTO ORDER_TEST VALUES ('주문A', 'READY');
7 INSERT INTO ORDER_TEST VALUES ('주문B', 'DONE');
8
9 COMMIT;
10
11 ALTER TABLE ORDER_TEST MODIFY STATUS DEFAULT 'READY';
12
13 -- ① DEFAULT 컬럼에 NULL 직접 입력
14 INSERT INTO ORDER_TEST VALUES ('주문C', NULL);
15
16 -- ② DEFAULT 컬럼 생략
17 INSERT INTO ORDER_TEST (ORDER_NAME) VALUES ('주문D');
18
19 COMMIT;
20
21 SELECT ORDER_NAME, STATUS
22 FROM ORDER_TEST
23 ORDER BY ORDER_NAME;
24

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	ORDER_NAME	STATUS
1	주문A	READY
2	주문B	DONE
3	주문C	
4	주문D	READY

- STATUS 컬럼에 DEFAULT 값을 설정하였지만, 이미 존재하던 데이터는 변경되지 않는다.
- INSERT 시 DEFAULT 값이 선언된 컬럼에 NULL 을 직접 입력하면 DEFAULT 값은 적용되지 않고 NULL 이 저장됨
- INSERT 시 DEFAULT 값이 선언된 컬럼을 생략한 경우에만 DEFAULT 값이 자동으로 적용됨
- 따라서 최종 결과에서 주문 C 는 NULL, 주문 D 는 DEFAULT 값인 READY 가 저장됨

4. 컬럼 삭제

- 컬럼에 데이터가 존재하더라도 삭제 가능
- 컬럼 삭제는 **자동 저장되므로 ROLLBACK 으로 복구 불가**
- 여러 컬럼을 동시에 삭제할 수 없으며, 한 번에 하나의 컬럼만 삭제 가능

● DROP

- 테이블, 인덱스 등 데이터베이스 객체를 삭제하는 DDL 명령어
- DROP 수행 후에는 객체 자체가 제거되므로 조회 불가
- 일반적으로 테이블을 삭제하면 RECYCLEBIN 이라는 공간에 임시 보관되며, FLASHBACK 명령어를 사용하여 삭제 이전 상태로 복구 가능하다.

** 문법

SQL1 *
1 DROP TABLE 테이블명 [PURGE];

※ PURGE 옵션으로 테이블 삭제 시 영구 삭제

** 특징

- 테이블에 정의된 **인덱스는 함께 삭제됨**
- 테이블에 정의된 **제약조건(PRIMARY KEY, UNIQUE 등)도 함께 삭제됨**
- 자식 테이블이 **외래키(FK)로 부모 테이블을 참조 중인 경우,**
부모 테이블은 외래키를 먼저 삭제하거나 자식 테이블을 삭제하지 않으면 DROP TABLE 수행 불가

● TRUNCATE

- 테이블 구조는 유지하고 데이터만 즉시 삭제하는 DDL 명령어
- **실행 즉시 AUTO COMMIT 되며 ROLLBACK 불가**

** 문법

SQL1 *
1 TRUNCATE TABLE 테이블명;

예제) TRUNCATE 수행 후 테이블 데이터 및 구조 확인

SQL1 *
1 TRUNCATE TABLE CREATE_TAB2;
2
3 SELECT *
4 FROM CREATE_TAB2;
5

Result
Grid Result Server Output Text Output Explain Plan Statistics
EMPNO ENAME JOB MGR HIREDATE SAL COMM DEPTNO

☞ 데이터는 삭제되지만 테이블 구조(컬럼 정의)는 그대로 유지되므로, 이후 SELECT 문 수행 시 컬럼 정보는 조회됨

● DELETE / TRUNCATE / DROP 차이

1. DELETE: 데이터 일부 또는 전체 삭제 가능, ROLLBACK 가능
2. TRUNCATE: 데이터 전체 삭제만 가능(일부 삭제 불가), 즉시 반영(AUTO COMMIT, ROLLBACK 불가)
3. DROP: 테이블 구조와 데이터 동시 삭제, 즉시 반영(AUTO COMMIT, ROLLBACK 불가)

● 제약조건

- 데이터 무결성을 유지하기 위해 컬럼(또는 테이블)에 설정하는 제약 장치
- 테이블 생성 시, 컬럼 추가 시 함께 정의 가능
- 이미 생성된 컬럼에 제약조건만 추가하는 것도 가능

1. PRIMARY KEY(기본키)

- 테이블에서 각 행을 유일하게 식별하는 제약조건
- 중복 불가, NULL 불가
- 테이블당 하나만 정의 가능
- PRIMARY KEY 생성 시 자동으로 INDEX 생성

** 문법

1) 테이블 생성 시 제약조건 생성

```
SQL1 *
1 CREATE TABLE 테이블명 (
2   컬럼1 데이터타입 [DEFAULT 기본값] [[CONSTRAINT 제약조건명] 제약조건종류],
3   컬럼2 데이터타입 [DEFAULT 기본값] [[CONSTRAINT 제약조건명] 제약조건종류],
4   ...
5   [[CONSTRAINT 제약조건명] 제약조건종류]
6 );
```

👁 컬럼 단위 선언은 해당 컬럼에만 적용되며,

테이블 단위 선언은 여러 컬럼을 묶는 제약조건(PRIMARY KEY, FOREIGN KEY 등)에 사용됨

2) 컬럼 추가 시 제약 조건 생성

```
SQL1 *
1 ALTER TABLE 테이블명
2 ADD 컬럼명 데이터타입 [DEFAULT 기본값] [[CONSTRAINT 제약조건명] 제약조건종류];
```

3) 이미 생성된 컬럼에 제약조건만 추가

```
SQL1 *
1 ALTER TABLE 테이블명 ADD [CONSTRAINT 제약조건명] 제약조건종류;
```

4) 제약조건 삭제

```
SQL1 *
1 ALTER TABLE 테이블명 DROP CONSTRAINT 제약조건명;
```

예제) 컬럼 단위로 PRIMARY KEY 와 UNIQUE 제약조건 정의

```
SQL1 *
1 CREATE TABLE TEST_1(
2     NO    NUMBER(10) CONSTRAINT PK_TEST1_NO PRIMARY KEY,
3     NAME  VARCHAR2(10) UNIQUE
4 );
```

- ☞ 제약조건은 컬럼 단위로 각 컬럼 정의 시 함께 설정 가능하
- ☞ 제약조건에 이름을 부여하려면 'CONSTRAINT 제약조건명'을 명시해야 하며, 이를 생략하면 시스템이 제약조건 이름을 자동으로 생성함

예제) 테이블 단위로 제약조건 정의(복합 PRIMARY KEY)

```
SQL1 *
1 CREATE TABLE TEST_2(
2     NO    NUMBER(10),
3     NAME  VARCHAR2(10),
4     PRIMARY KEY(NO, NAME)
5 );
```

- ☞ 제약조건은 컬럼 정의 이후 테이블 단위로 별도 선언 가능
- ☞ 테이블 단위 제약조건은 여러 컬럼을 묶는 제약조건(복합 PRIMARY KEY 등)을 정의할 때 사용함

2. UNIQUE

- 컬럼 값의 중복을 허용하지 않음
- **NULL 값 허용**
- UNIQUE 생성 시 자동으로 INDEX 생성

3. NOT NULL

- 컬럼에 NULL 입력 불가
- 다른 제약조건과 달리 컬럼의 속성(특징)을 나타내는 제약조건
 - 따라서 CTAS(Create Table As Select)로 **테이블 복제 시에도 NOT NULL 속성은 함께 복제됨**
- NOT NULL 을 선언하지 않으면 기본적으로 NULL 허용 컬럼(Nullable)으로 생성됨
- **이미 생성된 컬럼에 NOT NULL 설정 시, 제약조건 추가가 아니라 컬럼 수정(MODIFY) 방식으로 처리함**

예제) NOT NULL 제약조건 설정 문법

```
SQL1 *
1 ALTER TABLE CREATE_TAB3 ADD NOT NULL(NO);    --에러
2 ALTER TABLE CREATE_TAB3 MODIFY NO NOT NULL;   --정상
3
```

- ☞ NOT NULL 제약조건은 일반 제약조건처럼 ADD 로 추가할 수 없음
- ☞ 이미 생성된 컬럼에 NOT NULL 을 설정할 경우 MODIFY 문을 사용해야 함

4. FOREIGN KEY

- 다른 테이블의 PRIMARY KEY 또는 UNIQUE 컬럼을 참조하는 제약조건(참조키에 먼저 PK/UK 설정 필요)
- 부모 테이블에 존재하는 값만 입력 가능(NULL 허용)
- 부모 테이블 삭제 시 자식 테이블에서 참조 중이면 삭제 불가

** 문법

```
SQL1 *
1 CREATE TABLE 테이블명 (
2   컬럼1 데이터타입 [DEFAULT 값] REFERENCES 참조테이블(참조키),
3   ...
4 );
```

예제) 외래키 제약조건에 따른 데이터 입력 및 수정 동작 확인

```
SQL1 *
1 -- 부모 테이블
2 CREATE TABLE CATEGORY (
3   CAT_ID NUMBER(2) PRIMARY KEY,
4   CAT_NAME VARCHAR2(20)
5 );
6
7 -- 자식 테이블
8 CREATE TABLE MENU (
9   MENU_ID NUMBER(3) PRIMARY KEY,
10  MENU_NAME VARCHAR2(20),
11  CAT_ID NUMBER(2),
12  CONSTRAINT FK_MENU_CATEGORY
13     FOREIGN KEY (CAT_ID)
14     REFERENCES CATEGORY(CAT_ID)
15 );
16
17 -- 부모 데이터 입력
18 INSERT INTO CATEGORY VALUES (10, '음료');
19 INSERT INTO CATEGORY VALUES (20, '디저트');
20 INSERT INTO CATEGORY VALUES (30, '굿즈');
21
22 -- 자식 데이터 입력
23 INSERT INTO MENU VALUES (100, '아메리카노', 10);
24 INSERT INTO MENU VALUES (200, '치즈케이크', 20);
25 INSERT INTO MENU VALUES (300, '텀블러', 30);
26
27 COMMIT;
28
29 INSERT INTO MENU VALUES (400, '샌드위치', 99); -- 에러
30
31 UPDATE MENU -- 에러
32   SET CAT_ID = 50
33   WHERE MENU_ID = 100;
34
35 INSERT INTO MENU VALUES (500, '머그컵', NULL); -- 정상
36
```

결과)

```
SQL1 *
1 SELECT *
2 FROM MENU;
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	MENU_ID	MENU_NAME	CAT_ID
1	100	아메리카노	10
2	200	치즈케이크	20
3	300	텀블러	30
4	500	머그컵	

● FOREIGN KEY 옵션(생성 시 정의, 변경 불가 -> 재생성)

1. **ON DELETE CASCADE**: 부모 데이터 삭제 시 자식 데이터 함께 삭제
2. **ON DELETE SET NULL**: 부모 데이터 삭제 시 자식 데이터의 참조값은 NULL 로 수정

예제) 외래키 삭제 옵션에 따른 부모 데이터 삭제 동작 비교(ON DELETE CASCADE)

SQL1 *

```

1  -- 외래키가 있는 상태에서 부모 데이터 삭제 시도 (삭제 불가)
2  DELETE FROM CATEGORY
3     WHERE CAT_ID = 10;
4
5  -- 기존 외래키 제약조건 삭제
6  ALTER TABLE MENU
7     DROP CONSTRAINT FK_MENU_CATEGORY;
8
9  -- ON DELETE CASCADE 옵션으로 외래키 재생성
10 ALTER TABLE MENU
11    ADD CONSTRAINT FK_MENU_CATEGORY
12        FOREIGN KEY (CAT_ID)
13        REFERENCES CATEGORY(CAT_ID)
14        ON DELETE CASCADE;
15
16 -- 동일한 부모 데이터 다시 삭제 테스트 (정상 수행)
17 DELETE FROM CATEGORY
18    WHERE CAT_ID = 10;
19
20 ▶ SELECT *
21    FROM CATEGORY;
22

```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	CAT_ID	CAT_NAME
1	20	디저트
2	30	굿즈

- ☞ 외래키가 설정된 상태에서는 자식 테이블이 부모 데이터를 참조 중일 경우 부모 데이터 삭제가 제한됨
- ☞ 기존 외래키 제약조건을 삭제한 후, ON DELETE CASCADE 옵션을 사용하여 외래키를 재생성하면 부모 데이터 삭제 시 자식 데이터도 함께 자동 삭제됨
- ☞ 동일한 부모 데이터에 대해 삭제를 다시 수행한 결과, CAT_ID = 10 데이터만 삭제되고 나머지 부모 데이터는 정상적으로 유지됨을 확인할 수 있음

5. CHECK

- 컬럼에 입력 가능한 값의 범위를 제한
- 조건식에 TRUE 인 값만 입력 가능(**NULL 은 입력 가능**)

예제) CHECK 제약조건을 이용한 컬럼 값 범위 제한

SQL1 *

```

1 ▶ ALTER TABLE CATEGORY
2    ADD CONSTRAINT CK_CATEGORY_CAT_ID CHECK (CAT_ID > 0);
3

```

- ☞ CHECK 제약조건을 사용하여 CAT_ID 에 0 보다 큰 값만 입력되도록 제한함

● 뷰(VIEW)

1. 개념

- 하나 이상의 테이블 또는 다른 뷰를 기반으로 정의된 가상의 테이블
- 데이터를 저장하지 않으며 **별도의 저장공간을 차지하지 않음**

2. 사용 목적

- 복잡한 SQL 단순화
- 보안성 향상 (컬럼/행 제한)
- 사용자 편의성 제공

3. 종류

구분	설명
단순 뷰	• 하나의 테이블을 조회하는 뷰(VIEW) • 경우에 따라 데이터 수정이 가능함
복합 뷰	• 둘 이상의 테이블을 조인하여 조회하는 뷰(VIEW) • 일반적으로 데이터 수정이 불가능함

4. 특징

- **뷰를 통해 데이터를 수정하면 기본 테이블의 데이터가 변경됨**
- 기본 테이블을 삭제해도 **뷰는 자동으로 삭제되지 않지만, 참조 대상이 없어져 사용할 수 없게 됨**
- 뷰에는 인덱스를 생성할 수 없음
(인덱스는 테이블에만 생성 가능하며, 뷰 조회 시 테이블의 인덱스가 사용됨)
- 뷰를 통해서 테이블 구조를 변경할 수 없음

5. 뷰를 통한 데이터 수정

- 뷰의 한 행이 기본 테이블의 한 행으로 명확하게 대응되지 않는 경우 수정 불가
- 원래 테이블에 없던 **값을 인위적으로 만들어 낸 경우, 해당 컬럼에 대해서는 수정 불가**
(계산식, 함수식, 스칼라 서브쿼리, 인라인 뷰)
- 복합 뷰에서도 키 보존 테이블의 컬럼에 대해서는 뷰를 통한 수정이 가능
- 다음의 경우 뷰를 통해 데이터 수정이 불가능함
 - **GROUP BY 절이 포함된 경우**
 - **DISTINCT 가 포함된 경우**

** 문법

SQL1 *	
1	CREATE [OR REPLACE] VIEW 뷰이름
2	AS
3	조회쿼리;



키 보존(Key-Preserved)



조인 결과에서도 기본 키(PK)가 중복되지 않고 그대로 유지되는 테이블(조인 결과에서 1:1로 매칭되어 각 행이 늘어나지 않는 경우)

<< 단원 정리 문제 >>

1. 다음 중 제약조건에 대한 설명으로 가장 적절하지 않은 것은?

- ① 제약조건은 데이터베이스에서 데이터를 삭제하거나 수정하는 작업을 제어하는 기능이다.
- ② PRIMARY KEY 제약조건을 여러 컬럼으로 구성할 수 있다.
- ③ FOREIGN KEY 제약조건은 다른 테이블과의 관계를 정의하고, 참조 무결성을 보장하는 데 사용된다.
- ④ UNIQUE 제약조건은 각 행에서 동일한 값이 들어올 수 없도록 제약을 두며, NULL 값 역시 허용하지 않는다.

2. 다음 중 수행이 불가능 한 문장은? (단, DBMS 는 Oracle 이며 명령어는 순서대로 실행됨을 가정)

```
CREATE TABLE TABLE1(NO NUMBER PRIMARY KEY, NAME VARCHAR2(10), VDATE DATE);
INSERT INTO TABLE1 VALUES(1, 'ALLEN', SYSDATE);
INSERT INTO TABLE1 VALUES(2, 'SMITH', SYSDATE);
COMMIT;
```

- ① ALTER TABLE TABLE1 ADD COLUMN(ADDR VARCHAR2(10));
- ② ALTER TABLE TABLE1 MODIFY(NAME NOT NULL, VDATE NOT NULL);
- ③ ALTER TABLE TABLE1 DROP COLUMN NAME;
- ④ ALTER TABLE TABLE1 MODIFY VDATE DEFAULT SYSDATE;

3. 다음 중 뷰(View)에 대한 설명으로 옳지 않은 것은?

- ① 뷰는 하나 이상의 테이블 또는 다른 뷰를 기반으로 생성되는 가상의 테이블이다.
- ② 뷰는 데이터를 저장하지 않으므로 별도의 저장공간을 차지하지 않는다.
- ③ 뷰를 통해 데이터를 수정하면 기본 테이블의 데이터도 함께 변경된다.
- ④ 기본 테이블이 삭제되면 이를 참조하는 뷰도 자동으로 함께 삭제된다.

4. 다음 중 외래키(Foreign Key)에 대한 설명으로 옳지 않은 것은?

- ① 외래키 제약조건이 설정된 경우, 자식 테이블에 참조 데이터가 존재하면 부모 테이블의 행은 삭제할 수 없다.
- ② 외래키 컬럼은 NULL 값을 가질 수 있으며, 이 경우 부모 테이블의 데이터 존재 여부와는 무관하다.
- ③ 외래키 제약조건이 설정되어 있더라도, 자식 테이블의 외래키 컬럼에 NULL 값이 저장되어 있으면 부모 테이블의 행은 항상 삭제할 수 있다.
- ④ ON DELETE CASCADE 옵션이 설정된 경우, 부모 테이블의 행을 삭제하면 이를 참조하는 자식 테이블의 행도 함께 삭제된다.

5. 다음 중 DELETE, TRUNCATE, DROP 에 대한 설명으로 옳지 않은 것은?

- ① DELETE 는 WHERE 절을 사용할 수 있으며, 트랜잭션 제어(COMMIT, ROLLBACK)가 가능하다.
- ② TRUNCATE 는 DML 명령어이므로 실행 후 ROLLBACK 을 통해 데이터를 복구할 수 있다.
- ③ DROP 은 테이블 구조와 데이터 모두를 제거하며, 실행 후 해당 테이블을 더 이상 사용할 수 없다.
- ④ TRUNCATE 는 테이블의 모든 데이터를 제거하며, 실행 후 ROLLBACK 이 불가능하다.

1. ④

UNIQUE 제약조건은 동일한 값은 삽입될 수 없지만, NULL 값은 허용된다.

2. ①

컬럼을 추가할 때는 ADD COLUMN 이 아닌 ADD 만 전달한다.

3. ④

뷰는 SELECT 문 정의만 저장하는 객체이므로 기본 테이블이 삭제되더라도 자동으로 삭제되지는 않는다. 다만 참조 대상이 없어져 뷰를 사용할 수 없게 된다.

4. ③

외래키 컬럼에 NULL 값이 저장된 행이 존재하더라도, 동일한 부모 키 값을 참조하는 다른 자식 행이 존재하면 부모 테이블의 행은 삭제할 수 없다. 외래키 컬럼의 NULL 허용 여부만으로 부모 테이블 삭제 가능 여부가 결정되지는 않는다.

5. ②

TRUNCATE 는 DDL 명령어로 분류되며, 테이블의 모든 데이터를 제거하지만 트랜잭션 제어가 불가능하다. 따라서 실행 후 ROLLBACK 을 통해 데이터를 복구할 수 없다. DELETE 만이 DML 명령어로 트랜잭션 제어가 가능하다.

2-20. DCL

● DCL(Data Control Language)

- 데이터 제어어로 객체에 대한 권한을 부여(GRANT)하거나 회수(REVOKE) 하는 기능을 부여하는 언어
- 테이블 소유자는 타 계정에 자신의 테이블에 대한 조회 및 수정 권한을 부여·회수할 수 있음

● 권한

1. 개념

- 일반적으로 본인(접속한 계정) 소유가 아닌 테이블은 조회 및 수정 불가
- 업무적으로 필요할 경우, 테이블 소유자가 타 계정에 권한을 부여하여 접근 가능

2. 종류

구분	설명
오브젝트 권한 (Object Privilege)	- 특정 객체(테이블 등)에 대한 접근 권한 제어 예) 특정 테이블에 대한 SELECT, INSERT, UPDATE, DELETE, MERGE 권한 - 테이블 소유자가 자신의 테이블에 대해 타 계정에 권한 부여 및 회수 가능
시스템 권한 (System Privilege)	- 데이터베이스 시스템 작업에 대한 권한 제어 예) 테이블 생성 권한, 인덱스 삭제 권한 등 - 관리자 권한을 가진 계정만 권한 부여 및 회수 가능

● GRANT

- 객체에 대한 권한을 부여하는 명령어
- 권한 부여는 일반적으로 테이블 소유자 또는 관리자 계정(SYS, SYSTEM)으로 접속하여 수행해야 함
- 여러 사용자에게 동시에 권한 부여 가능
- 여러 권한을 동시에 부여 가능
- 여러 객체에 대한 권한을 하나의 GRANT 문으로 동시에 부여할 수는 없음

** 문법

SQL1 *

```
1 GRANT 권한 ON 테이블명 TO 유저;
2
```


예제) GRANT 문에서 다중 사용자·다중 권한 지정 규칙

```

SQL1 *
1  -- 단일 사용자 권한 부여 (가능)
2  GRANT SELECT ON PROFESSOR TO HR;
3
4  -- 다중 사용자 권한 부여 (가능)
5  GRANT SELECT ON PROFESSOR TO ORDDATA, BI;
6
7  -- 다중 권한 부여 (가능)
8  GRANT SELECT, UPDATE, INSERT ON PROFESSOR TO HR;
9
10 -- 다중 객체 권한 부여 (예러)
11 GRANT SELECT ON PROFESSOR, DEPT2 TO HR;
12

```

☞ GRANT 문에서는 여러 사용자에게 동시에 권한 부여하거나 여러 권한을 동시에 부여하는 것은 가능하지만, 여러 객체에 대한 권한을 하나의 GRANT 문으로 동시에 부여할 수는 없음

● REVOKE

- 부여된 권한을 회수하는 명령어
- 권한 회수는 시스템 관리자 또는 객체 소유자만 가능하며, 객체 소유자는 자신의 객체에 대해서만 권한을 회수할 수 있음
- 여러 사용자로부터 동시에 권한 회수 가능
- 여러 권한을 동시에 회수 가능

** 문법

```

SQL1 *
1  REVOKE 권한 ON 테이블명 FROM 유저;
2

```

예제) REVOKE 를 이용한 다중 권한 및 다중 사용자 권한 회수

```

SQL1 *
1  -- 다중 권한 회수 (가능)
2  REVOKE SELECT, UPDATE, INSERT ON PROFESSOR FROM HR;
3
4  -- 다중 사용자로부터 권한 회수 (가능)
5  REVOKE SELECT ON PROFESSOR FROM ORDDATA, BI;
6

```

☞ REVOKE 문을 사용하면 하나의 SQL 문으로 여러 권한을 동시에 회수하거나, 여러 사용자로부터 동일한 객체에 대한 권한을 동시에 회수할 수 있음

● 롤(ROLE)

1. 개념

- 여러 개의 권한을 하나로 묶어 관리하기 위한 권한 집합
- 사용자에게 권한을 직접 부여하지 않고 **롤을 통해 간접적으로 권한 부여**
- 권한 관리의 편의성과 일관성을 높이기 위한 객체

2. 특징

- **여러 개의 시스템 권한 및 오브젝트 권한을 포함할 수 있음**
- 하나의 롤을 여러 사용자에게 동시에 부여 가능
- 사용자에게 여러 롤을 동시에 부여 가능
- 롤을 통해 부여된 권한은 REVOKE 로 한 번에 회수 가능
- **롤을 통해 부여된 권한은 사용자에게 직접 회수할 수 없으며**, 해당 롤에서 권한을 제거하거나 사용자의 롤을 회수하는 방식으로만 권한 회수가 가능함

** 문법

SQL1 *

```
1 CREATE ROLE 롤이름;
2 GRANT 권한1, 권한2, ... TO 롤이름;
```

예제) 롤(Role)을 이용한 권한 부여

SQL1 *

```
1 -- 롤 생성
2 CREATE ROLE ROLE_SEL;
3
4 -- 롤에 SELECT 권한 부여 (다중 객체)
5 GRANT SELECT ON EMP TO ROLE_SEL;
6 GRANT SELECT ON STUDENT TO ROLE_SEL;
7 GRANT SELECT ON DEPT TO ROLE_SEL;
8 GRANT SELECT ON DEPARTMENT TO ROLE_SEL;
9
10 -- 롤을 사용자에게 부여
11 GRANT ROLE_SEL TO HR;
12
13 -- HR 계정으로 수행
14 SELECT * FROM SCOTT.DEPT;
15
```

Result

Grid Result Server Output Text Output Explain Plan Statistics

	DEPTNO	DNAME	LOC
1	10	ACCOUNTING	NEW YORK
2	20	RESEARCH	DALLAS
3	30	SALES	CHICAGO
4	40	OPERATIONS	BOSTON

☞ 여러 테이블에 대한 SELECT 권한을 하나의 롤로 묶어 사용자에게 부여할 수 있으며, 사용자는 해당 롤을 통해 개별 테이블에 직접 권한을 부여받지 않아도 조회가 가능함

예제) 롤을 통해 부여된 권한의 회수 방법

```

SQL*
1  -- ROLE을 통해 부여된 권한은 사용자에게 직접 REVOKE 할 수 없음 (예러)
2  REVOKE SELECT ON SCOTT.DEPT FROM HR;
3
4  -- 사용자에게 부여된 ROLE 회수 (정상)
5  REVOKE ROLE_SEL FROM HR;
6
7  -- HR 계정으로 수행 : 테이블 조회 불가(권한 부족)
8  SELECT * FROM SCOTT.DEPT;
9

```

Result

ORA-00942: 테이블 또는 뷰가 존재하지 않습니다

- ☞ 롤을 통해 사용자에게 부여된 권한은 사용자에게 직접 REVOKE 할 수 없으며 오류가 발생함
- ☞ 해당 권한을 회수하려면 롤에서 권한을 제거하거나 사용자로부터 롤 자체를 회수해야 하며, 롤이 회수된 이후에는 해당 테이블에 대한 조회가 불가능해짐

● 권한부여 옵션(중간관리자의 권한)

- 권한 부여 옵션을 사용하면 **권한을 직접 관리하지 않고 중간관리자를 통해 권한을 위임·관리할 수 있음**
- 적용 대상과 권한 회수 방식에 따라 WITH GRANT OPTION 과 WITH ADMIN OPTION 으로 구분

구분	설명
WITH GRANT OPTION	• 오브젝트 권한을 다른 사용자에게 부여할 수 있는 옵션 • WITH GRANT OPTION 으로 부여받은 오브젝트 권한을 제 3 자에게 부여할 수 있음 • 중간관리자가 제 3 자에게 부여한 권한은 중간관리자만 회수 가능 • 중간관리자에게 부여된 권한을 회수하면, 해당 중간관리자를 통해 제 3 자에게 부여된 권한도 함께 회수됨
WITH ADMIN OPTION	• 시스템 권한 또는 롤 권한을 중간관리자를 통해 다른 사용자에게 부여하기 위한 옵션 • WITH ADMIN OPTION 으로 부여받은 시스템 및 롤 권한을 제 3 자에게 부여할 수 있음 • 관리자가 중간관리자를 거치지 않고 직접 회수 가능 • 관리자가 중간관리자의 권한을 회수하더라도 제 3 자 권한은 유지됨

<< 단원 정리 문제 >>

1. 다음 중 권한에 대한 설명 중 가장 적절하지 않은 것은?

- ① 관리자 계정을 제외한 일반 계정의 경우 별도의 권한이 부여되지 않은 경우라면 다른 계정 소유의 테이블을 조회할 수 없다.
- ② 일반 계정이라도 CREATE TABLE 권한 없이 자기 자신 소유의 테이블을 만들 수 있다.
- ③ WITH GRANT OPTION 으로 부여받은 권한은 다른 계정에게 권한을 부여할 수 있다.
- ④ 여러 권한을 동시에 부여 및 회수할 수 있다.

2. 다음 중 DCL 에 대한 설명으로 가장 적절하지 않은 것은?

- ① DCL(Data Control Language)은 데이터베이스에서 사용자의 권한을 관리하는 데 사용되는 SQL 명령어 집합이다.
- ② GRANT 명령어는 사용자가 특정 작업을 수행할 수 있도록 권한을 부여하는 데 사용된다.
- ③ REVOKE 명령어는 사용자가 부여받은 권한을 취소하거나 철회하는 데 사용된다.
- ④ 권한을 부여하거나 회수한 이후, 반드시 COMMIT 을 통해 변경 사항을 확정해야만 영구적으로 반영된다.

1. ②

CREATE TABLE 권한이 필요하다.

2. ④

권한을 부여하거나 회수하는 경우 즉시 반영되므로 별도의 COMMIT 을 수행하지 않아도 영구 반영된다.

단원 종합 문제

1. 다음 UPDATE 문장 중 수행이 불가능한 문장은? (단, DBMS는 Oracle)

①

```
UPDATE EMP
```

```
SET SAL = 5000, COMM = 300
```

```
WHERE ENAME = 'SMITH';
```

②

```
UPDATE EMP
```

```
SET (SAL, COMM) = (5000, 300)
```

```
WHERE ENAME = 'SMITH';
```

③

```
UPDATE EMP
```

```
SET (SAL, COMM) = (SELECT MAX(SAL), MAX(COMM)
                    FROM EMP)
```

```
WHERE ENAME = 'SMITH';
```

④

```
UPDATE EMP E1
```

```
SET SAL = (SELECT MAX(SAL)
            FROM EMP)
```

```
WHERE DEPTNO = E1.DEPTNO)
```

```
WHERE ENAME = 'SMITH';
```

2. 외래키에 대한 설명 중 적절하지 않은 것은?

① 외래키를 생성하기 위해서는 반드시 참조키에 기본키 또는 고유키가 먼저 생성되어야 한다.

② 외래키 설정 후에는 참조키에 생성된 PK 또는 UNIQUE 제약조건을 삭제할 수 없다.

③ 외래키 설정 후 부모 테이블 삭제 시 CASCADE 옵션을 사용하면 자식 테이블을 함께 삭제할 수 있다.

④ 외래키를 ON DELETE CASCADE 옵션과 함께 생성하면 부모 데이터 삭제 시 부모키를 참조하고 있는 자식 데이터도 함께 삭제할 수 있다.

3. 사용자와 권한에 대한 설명으로 가장 적절한 것은?

① 자신이 소유한 테이블에 대해서는 조회 권한을 다른 사용자에게 언제나 부여할 수 있다.

② SYSTEM 계정에서 테이블을 생성하는 경우 항상 SYSTEM 소유로 생성된다.

③ 동시에 여러 테이블에 대한 조회 권한을 부여할 수 있다.

④ 권한을 동시에 여러 사용자에게 부여할 수 없다.

4. 다음 SQL 문장 수행 후 TAB1의 최종 총합(COL2 합계)으로 알맞은 것은?

<TAB1>

COL1	COL2
A	10
A	20
B	30
B	40

UPDATE TAB1

SET COL2 = 20

WHERE COL1 = 'B';

COMMIT;

INSERT INTO TAB1 VALUES ('C', 50);

SAVEPOINT AAA;

UPDATE TAB1

SET COL2 = 99

WHERE COL1 = 'C';

DELETE TAB1 WHERE COL1 = 'A';

INSERT INTO TAB1 VALUES ('C', 60);

ROLLBACK TO AAA;

INSERT INTO TAB1 VALUES ('D', 70);

COMMIT;

- ① 150
- ② 170
- ③ 190
- ④ 200

5. 다음 중 DDL(Data Definition Language)에 대한 설명으로 옳지 않은 것은?

- ① DROP TABLE을 수행하면 해당 테이블에 생성되어 있던 인덱스도 함께 삭제된다.
- ② ALTER TABLE을 이용하여 컬럼을 삭제한 경우, 해당 작업은 ROLLBACK을 통해 복구할 수 없다.
- ③ TRUNCATE TABLE은 테이블 구조는 유지한 채 데이터만 삭제하며, 트랜잭션 제어가 불가능하다.
- ④ DDL 문장을 사용하면 하나의 SQL 문으로 여러 테이블을 동시에 삭제할 수 있다.

6. 다음 중 뷰(View)에 대한 설명으로 옳지 않은 것은?

- ① 뷰는 데이터를 저장하지 않는 가상의 테이블로, 별도의 저장공간을 차지하지 않는다.
- ② 뷰를 통해 데이터를 수정하면, 수정 내용은 기본 테이블에 반영된다.
- ③ 기본 테이블이 삭제되면 이를 참조하는 뷰도 자동으로 함께 삭제된다.
- ④ 뷰에는 인덱스를 생성할 수 없으며, 뷰 조회 시에는 기본 테이블의 인덱스가 사용될 수 있다.

7. 다음 문장을 차례대로 수행한 결과로 알맞은 것은?

```
CREATE TABLE TAB1(COL1 NUMBER, COL2 VARCHAR2(10));
INSERT INTO TAB1 VALUES(1, 'A');
INSERT INTO TAB1 VALUES(2, 'B');
COMMIT;

ALTER TABLE TAB1 ADD COL3 NUMBER;
ALTER TABLE TAB1 MODIFY COL3 DEFAULT 100;

INSERT INTO TAB1(COL1, COL2) VALUES(3, 'C');
INSERT INTO TAB1 VALUES(4, 'D', NULL);
COMMIT;

SELECT SUM(COL3)
FROM TAB1;
```

- ① 100
- ② 200
- ③ 300
- ④ 400

8. 다음 중 MERGE 문에 대한 설명으로 옳지 않은 것은?

- ① MERGE 문은 하나의 SQL 문으로 대상 테이블에 대해 INSERT와 UPDATE를 동시에 수행할 수 있다.
- ② ON 절의 조건을 기준으로 일치하는 행이 존재하면 UPDATE가 수행되고, 일치하는 행이 없으면 INSERT가 수행된다.
- ③ MERGE 문에서 INSERT가 수행되는 경우, 대상 테이블의 모든 컬럼에 대해 반드시 값을 전달해야 한다.
- ④ MERGE 문은 DML 명령어이므로 실행 후 COMMIT 또는 ROLLBACK을 통해 트랜잭션 제어가 가능하다.

9. 다음 중 CTAS(Create Table As Select)에 대한 설명으로 옳지 않은 것은?

- ① CTAS를 사용하면 SELECT 결과를 이용해 테이블을 생성할 수 있으며, 구조만 복제할 수도 있다.
- ② CTAS로 생성된 테이블은 원본 테이블의 컬럼 데이터 타입과 길이를 그대로 따른다.
- ③ CTAS 수행 시 컬럼 이름은 SELECT 절에서 별칭을 사용하여 변경할 수 있다.
- ④ CTAS로 생성된 테이블에는 원본 테이블에 정의된 PRIMARY KEY와 제약조건이 자동으로 함께 생성된다.

10. 다음 중 DML(Data Manipulation Language)에 대한 설명으로 옳지 않은 것은?

- ① 오라클에서는 INSERT ALL을 사용하여 하나의 SQL 문으로 서로 다른 테이블에 데이터를 입력할 수 있다.
- ② INSERT 문에서 컬럼을 명시하지 않은 경우, 해당 컬럼에 DEFAULT 값이 정의되어 있더라도 NULL이 입력된다.
- ③ DELETE 문은 WHERE 절을 사용하지 않으면 테이블의 모든 행을 삭제하며, 트랜잭션 제어가 가능하다.
- ④ UPDATE 문에서는 하나의 SQL 문으로 여러 컬럼의 값을 동시에 수정할 수 있다.

기출문제

1회차

1. 다음은 어떤 데이터 모델링에 대한 설명인가?

추상화 수준이 높고 업무 중심적이고 포괄적인 수준의 모델링 진행. 전사적 데이터 모델링

- ① 논리적 데이터 모델링
- ② 물리적 데이터 모델링
- ③ 개괄적 데이터 모델링
- ④ 개념적 데이터 모델링

2. 다음중 엔터티의 특징이 아닌 것은?

- ① 반드시 해당 업무에서 필요하고 관리되어야 하는 정보이어야 한다.
- ② 유일한 식별자에 의해 식별이 가능해야 한다.
- ③ 엔터티는 반드시 속성이 있어야 한다.
- ④ 정규화 이론에 근간하여 정해진 주식별자는 함수적 종속성을 가져야 한다.

3. 속성의 분류 중 속성의 특성에 따른 분류가 아닌 것은?

- ① 기본속성
- ② 파생속성
- ③ 일반속성
- ④ 설계속성

4. 아래 ERD에 대한 설명으로 가장 적절하지 않은 것은?



- ① 한 명의 고객은 여러 개의 서비스를 구매할 수 있다.
- ② 한 명의 고객은 서비스를 구매하지 않을 수 있다.
- ③ 하나의 서비스 구매는 반드시 한 명의 고객에 의해 주문된다.
- ④ 하나의 서비스 구매 이력은 고객 정보가 없을 수 있다.

5. 다음은 식별자의 특징 중 무엇을 설명하고 있는가?

사원번호 없는 회사직원은 있을 수 없음

- ① 유일성
- ② 최소성
- ③ 불변성
- ④ 존재성

6. 다음 엔터티를 제 3차 정규화를 수행했을 때 도출되는 엔터티의 수는(도서대출 포함)?
(단, 하나의 대출번호에 대해 하나의 도서만 대출할 수 있다고 가정)

도서대출

대출번호
대출자번호(FK)
대출자명
대출자직업
대출일자
대출도서번호
대출도서명
출판사명
출판년월
대표저자명
반납일자

- ① 1
- ② 2
- ③ 3
- ④ 4

7. 관계에 대한 설명으로 가장 적절하지 않은 것은?

회원은 반드시 개인회원 또는 법인회원으로 회원가입을 한다.
회원 가입 후 개인회원 또는 법인회원으로 로그인하여 서비스를 이용할 수 있다.

- ① 관계는 존재적 관계와 행위에 의한 관계로 나누어 볼 수 있다.
- ② 부서와 사원 엔터티 간의 '소속' 관계는 존재적 관계이다.
- ③ 고객과 주문내역 엔터티 간의 '주문' 관계는 행위에 의한 관계이다.
- ④ 개인회원 또는 법인회원 둘 중 하나로 주문 가능할 경우 고객과 주문 엔터티는 상호포함적 관계이다.

8. 트랜잭션의 특징 중 보기는 무엇을 설명하고 있는가?

트랜잭션이 실행되기 전의 데이터베이스 내용이 잘못되어 있지 않다면
트랜잭션이 실행된 이후에도 데이터베이스의 내용에 잘못이 있으면 안된다.

- ① 원자성
- ② 일관성
- ③ 고립성
- ④ 지속성

9. NULL에 대한 설명으로 가장 적절하지 않은 것은?

- ① 정해지지 않은 값을 의미한다.
- ② NULL과의 모든 비교(IS NULL 제외)는 알 수 없음을 반환한다.
- ③ NULL로만 구성된 컬럼을 COUNT한 결과는 공집합이다.
- ④ 공백문자 혹은 숫자 0과는 다른 의미를 갖는다.

10. 다음 식별자에 대한 설명으로 가장 적절한 것은?

엔터티 내의 여러 인스턴스 중 하나를 유일하게 구분할 수 있으나, 대표성을 가지지 못하는 식별자

- ① 보조식별자
- ② 인조식별자
- ③ 본질식별자
- ④ 복합식별자

11. SELECT 문에 대한 설명으로 가장 적절하지 않은 것은?

- ① 오라클에서는 GROUP BY절 위에 HAVING절을 명시할 수 있다.
- ② ORDER BY절은 문법 순서도 맨 마지막에 위치하며, 실행 순서 역시 마지막이다.
- ③ FROM 절은 모든 DBMS에서 생략 가능하다.
- ④ SELECT절에 DISTINCT는 항상 SELECT 바로 뒤에 위치한다.

12. SQL의 종류와 해당되는 명령어를 연결한 것 중 틀린 것은?

- ① DML - TRUNCATE
- ② TCL - COMMIT
- ③ DCL - GRANT
- ④ DDL - ALTER

13. SQL 문을 실행했을 때 오류가 발생하는 부분으로 가정 적절한 것은?

- ① SELECT DEPTNO, ROUND(AVG(SAL)) AS ROUND_VALUE
- ② FROM EMP E
- ③ WHERE ROUND_VALUE >= 2000
- ④ GROUP BY DEPTNO;

14. 다음의 함수 실행 결과 중 틀린 것은?

- ① SUBSTR('WWW.HDATA LAB.CO.KR', -5) : 'CO.KR'
- ② LPAD('X',5,'X') : 'XXXXXX'
- ③ INSTR('WWW.HDATA LAB.CO.KR','.', 5, 2) : 16
- ④ LTRIM('AABABAA', 'A') : 'BABAA'

15. 함수의 실행 결과로 적절하지 않은 것은?

- ① CEIL(3.5) : 4
- ② FLOOR(3.5) : 3
- ③ ROUND(12345.678, -2) : 12350
- ④ SIGN(120) : 1

16. 함수의 실행 결과 중 다른 하나는?

COMM
NULL
500

- ① NVL(COMM, 100)
- ② NULLIF(COMM, 100)
- ③ COALESCE(COMM, 100)
- ④ ISNULL(COMM, 100)

17. 다음 쿼리의 실행 결과로 알맞은 것은?

<TAB1>	
SAL	
3000	
3500	
4000	
4200	
6000	
SELECT SUM(DECODE(SIGN(SAL-4000), 1, 1, 0))	
FROM TAB1;	

- ① 0
- ② 1
- ③ 2
- ④ 3

18. 다음 SQL 중 실행 결과가 다른 것은?

<TAB1>	
JUMIN(CHAR(13))	
9012231223456	
9606272349876	

- ① SELECT DECODE(SUBSTR(JUMIN, 7, 1), '1', '남자', '여자')
FROM TAB1;
- ② SELECT CASE WHEN SUBSTR(JUMIN, 7, 1) = 1 THEN '남자' ELSE '여자' END
FROM TAB1;
- ③ SELECT CASE SUBSTR(JUMIN, 7, 1) WHEN 1 THEN '남자' ELSE '여자' END
FROM TAB1;
- ④ SELECT CASE SUBSTR(JUMIN, 7, 1) WHEN '1' THEN '남자' ELSE '여자' END
FROM TAB1;

19. 아래 SQL의 실행 결과로 가장 적절한 것은?(단, DBMS는 ORACLE로 가정)

```
SELECT TO_CHAR(TO_DATE('2024/08/24 10:00', 'YYYY/MM/DD HH24:MI')
               - 30/24/60, 'YYYY.MM.DD HH24:MI:SS')
FROM DUAL;
```

- ① 2024.07.25 10:00:00
- ② 2024.08.24 06:00:00
- ③ 2024.08.24 09:30:00
- ④ 2024.08.24 09:59:30

20. 다음 SQL 중 실행 결과가 다른 것은?

- ① SELECT JOB,
COUNT(CASE WHEN DEPTNO = 10 THEN 1 ELSE 0 END) AS "10번부서원수",
COUNT(CASE WHEN DEPTNO = 20 THEN 1 ELSE 0 END) AS "20번부서원수",
COUNT(CASE WHEN DEPTNO = 30 THEN 1 ELSE 0 END) AS "30번부서원수"
FROM EMP
GROUP BY JOB;
- ② SELECT JOB,
COUNT(CASE WHEN DEPTNO = 10 THEN 1 END) AS "10번부서원수",
COUNT(CASE WHEN DEPTNO = 20 THEN 1 END) AS "20번부서원수",
COUNT(CASE WHEN DEPTNO = 30 THEN 1 END) AS "30번부서원수"
FROM EMP
GROUP BY JOB;
- ③ SELECT JOB,
SUM(CASE WHEN DEPTNO = 10 THEN 1 ELSE 0 END) AS "10번부서원수",
SUM(CASE WHEN DEPTNO = 20 THEN 1 ELSE 0 END) AS "20번부서원수",
SUM(CASE WHEN DEPTNO = 30 THEN 1 ELSE 0 END) AS "30번부서원수"
FROM EMP
GROUP BY JOB;
- ④ SELECT JOB,
NVL(SUM(CASE WHEN DEPTNO = 10 THEN 1 END),0) AS "10번부서원수",
NVL(SUM(CASE WHEN DEPTNO = 20 THEN 1 END),0) AS "20번부서원수",
NVL(SUM(CASE WHEN DEPTNO = 30 THEN 1 END),0) AS "30번부서원수"
FROM EMP
GROUP BY JOB;

21. 다음 SQL 수행 결과로 알맞은 것은?

<TAB1>	
CODE	
A1C1	
A2C2	
B1A0	
B2C1	
C1A1	
D1B2	


```

SELECT COUNT(CODE)
  FROM TAB1
 WHERE CODE LIKE '_1%'
        OR CODE LIKE '%A%';

```

- ① 4
- ② 5
- ③ 7
- ④ 8

22. 다음 SQL 문장 중 실행 결과가 다른 하나는?

- ① SELECT COUNT(*) FROM EMP WHERE DEPTNO = 10 OR DEPTNO = 20 AND JOB = 'CLERK';
- ② SELECT COUNT(*) FROM EMP WHERE (DEPTNO = 10 OR DEPTNO = 20) AND JOB = 'CLERK';
- ③ SELECT COUNT(*) FROM EMP WHERE DEPTNO = 10 OR (DEPTNO = 20 AND JOB = 'CLERK');
- ④ SELECT COUNT(*) FROM EMP WHERE (DEPTNO = 10 OR DEPTNO = 20 AND JOB = 'CLERK');

23. 아래 SQL 수행 결과로 가장 적절한 것은?

<TAB1>		
COL1	COL2	COL3
100	NULL	100
NULL	200	400
0	300	NULL


```

SELECT SUM(COL2) + SUM(COL3) FROM TAB1;
SELECT SUM(COL2) + SUM(COL3) FROM TAB1 WHERE COL1 > 0;
SELECT SUM(COL2) + SUM(COL3) FROM TAB1 WHERE COL1 IS NOT NULL;
SELECT SUM(COL2) + SUM(COL3) FROM TAB1 WHERE COL1 IS NULL;

```

- ① 500, 100, 400, 600
- ② 500, NULL, 400, 600
- ③ 1000, 100, 400, 600
- ④ 1000, NULL, 400, 600

24. 다음 중 GROUP BY절 대한 설명 중 틀린 것은?

- ① GROUP BY절에는 컬럼 별칭을 사용할 수 없다.
- ② GROUP BY절에는 SUM, COUNT 함수를 사용할 수 없다.
- ③ GROUP BY절에 명시되지 않은 컬럼을 그룹함수 없이 SELECT절에 사용할 수 없다.
- ④ GROUP BY절에 나열되는 컬럼 순서에 따라 SELECT절의 그룹함수의 연산 결과가 달라질 수 있다.

25. 아래 SQL 수행 결과로 가장 적절한 것은?

<TAB1>		
COL1	COL2	COL3
100	NULL	100
NULL	200	400
0	300	NULL

SELECT COUNT(*) AS RESULT FROM TAB1 WHERE COL1 = 200 GROUP BY COL1;
 SELECT SUM(COL2) AS RESULT FROM TAB1 WHERE COL1 = 100;
 SELECT COUNT(COL1) AS RESULT FROM TAB1 WHERE COL3 = 400;

①

RESULT	RESULT	RESULT
0	0	0

②

RESULT	RESULT	RESULT
		0

③

RESULT	RESULT	RESULT

④

RESULT	RESULT	RESULT
		0

26. 다음 중 실행이 불가능한 구문은?

- ① SELECT COL1, COL2, COL3 C1
FROM TAB1 T
ORDER BY 1, COL2, COL3;
- ② SELECT COL1, COL2, COL3 C1
FROM TAB1 T
ORDER BY 1, 2, 3;
- ③ SELECT COL1, COL2, COL3 C1
FROM TAB1 T
ORDER BY 1, COL2, C1;
- ④ SELECT COL1, COL2, COL3 C1
FROM TAB1 T
ORDER BY COL1, COL1, T.C1;

27. 다음 SQL 구문의 실행 결과로 가장 적절한 것은?

<TAB1>	
SAL	
1300	
800	
2500	
300	


```

SELECT SAL
  FROM TAB1
 ORDER BY TO_CHAR(SAL);

```

- ① 1300 2500 300 800
- ② 300 800 1300 2500
- ③ 300 1300 800 2500
- ④ 2500 1300 800 300

28. 아래의 SQL 의 결과로 알맞은 것은?

<TAB1>	
COL1	COL2
1	A
2	
3	B
4	C

<TAB2>	
COL1	COL2
1	A
2	
3	B


```

SELECT SUM(A.COL1)
  FROM TAB1 A, TAB2 B
 WHERE A.COL2 <> B.COL2;

```

- ① 8
- ② 10
- ③ 12
- ④ 30

29. 아래 SQL 실행 결과로 알맞은 것은?

<TAB1>

EMPNO	NAME
1000	SCOTT
2000	SMITH
3000	FORD
4000	TIGER

<TAB2>

NO	RULE_NAME
1	%O%
2	F%

```
SELECT COUNT(*)
  FROM TAB1, TAB2
 WHERE NAME LIKE RULE_NAME;
```

- ① 0
- ② 1
- ③ 2
- ④ 3

30. 아래와 같은 테이블 TAB1, TAB2가 있을 때 아래의 SQL의 결과 건수를 알맞게 나열한 것은?

<TAB1>

COL1	COL2	KEY1
A	100	BB
B	200	CC
C	300	DD
NULL	400	EE

<TAB2>

COL1	COL2	KEY2
A	100	AA
B	200	BB
C	300	BB
NULL	400	EE

```
SELECT * FROM TAB1 A INNER JOIN TAB2 B ON (A.KEY1 = B.KEY2);
SELECT * FROM TAB1 A LEFT OUTER JOIN TAB2 B ON (A.KEY1 = B.KEY2);
SELECT * FROM TAB1 A FULL OUTER JOIN TAB2 B ON (A.KEY1 = B.KEY2);
SELECT * FROM TAB1 A CROSS JOIN TAB2 B;
SELECT * FROM TAB1 A NATURAL JOIN TAB2 B;
```

- ① 2 4 6 8 3
 ② 2 5 6 16 4
 ③ 3 5 6 16 3
 ④ 3 6 8 12 4

31. 아래와 같은 테이블 데이터가 있다. 각 SQL에 대한 결과값이 잘못된 것은?

<TAB1>		<TAB2>	
N1	V1	N1	V1
1	A	1	A
2		2	
3	B	3	B
4	C		

- ① SELECT *
 FROM TAB1
 WHERE V1 IN (SELECT V1 FROM TAB2);

N1	V1
1	A
3	B

- ② SELECT *
 FROM TAB1
 WHERE V1 NOT IN (SELECT V1 FROM TAB2);

N1	V1
4	C

- ③ SELECT *
 FROM TAB1 A
 WHERE EXISTS (SELECT 'X'
 FROM TAB2 B
 WHERE A.V1 = B.V1);

N1	V1
1	A
3	B

- ④ SELECT *
 FROM TAB1 A
 WHERE NOT EXISTS (SELECT 'X'
 FROM TAB2 B
 WHERE A.V1 = B.V1);

N1	V1
2	
4	C

32. 아래와 같은 테이블이 있다. 다음 SQL 실행 결과로 올바른 것은?

<EMPLOYEES>

EMPNO	DEPTNO	HIREDATE
1	NULL	1998/12/23
2	10	2001/05/06
3	20	2020/01/16
4	30	2020/02/01
5	40	2020/02/28
6	50	2021/06/07

```
SELECT COUNT(DEPTNO)
  FROM EMPLOYEES
 WHERE DEPTNO <= ALL(SELECT DEPTNO
                     FROM EMPLOYEES
                     WHERE HIREDATE >= TO_DATE('2020/02', 'YYYY/MM'));
```

- ① 2
- ② 3
- ③ 4
- ④ 5

33. 다음 SQL 중 정상 수행이 불가능한 것은?

- ① SELECT *
FROM TAB1 A
WHERE NOT EXISTS (SELECT 'X'
FROM TAB2 B
WHERE A.NO = B.NO);
- ② SELECT A.NO, (SELECT B.NAME
FROM TAB2 B)
FROM TAB1 A
WHERE A.NO = B.NO;
- ③ SELECT NO
FROM TAB1 A JOIN TAB2 B
USING (NO);
- ④ UPDATE TAB1 A
SET A.NAME = (SELECT B.NAME
FROM TAB2 B
WHERE A.NO = B.NO);

34. 아래의 SQL의 출력 결과 중 알맞은 것은?

<TAB1>

COL1	COL2
A	10
B	20
C	30

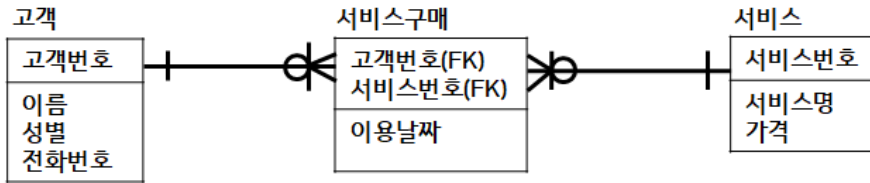
<TAB2>

COL1	COL2
B	20
C	30
D	40

```
SELECT COUNT(*)  
  FROM TAB1  
 WHERE EXISTS (SELECT 1  
                FROM TAB2  
               WHERE TAB2.COL1 = 'A');
```

- ① NULL
- ② 0
- ③ 1
- ④ 3

35. 다음 ERD를 보고, 고객의 성별로 서비스 이용횟수와 이용금액의 총합을 출력하는 SQL로 적절하지 않은 것은?



- ① SELECT 고객.성별, COUNT(서비스.서비스번호) AS CNT, SUM(서비스.가격) AS SUM_PRICE
FROM 고객, 서비스구매, 서비스
WHERE 고객.고객번호= 서비스구매.고객번호
AND 서비스구매.서비스번호 = 서비스.서비스번호
GROUP BY 고객.성별;
- ② SELECT 고객.성별, COUNT(고객.고객번호) AS CNT, SUM(서비스.가격) AS SUM_PRICE
FROM 고객 LEFT OUTER JOIN 서비스구매
ON 고객.고객번호= 서비스구매.고객번호 LEFT OUTER JOIN 서비스
ON 서비스구매.서비스번호 = 서비스.서비스번호
GROUP BY 고객.성별;
- ③ SELECT 고객.성별, COUNT(I.SNO) AS CNT, SUM(I.가격) AS SUM_PRICE
FROM 고객 INNER JOIN (SELECT 서비스구매.고객번호 AS GNO, 서비스.서비스번호 AS SNO, 서비스.가격
FROM 서비스구매 INNER JOIN 서비스
ON 서비스구매.서비스번호 = 서비스.서비스번호) I
ON 고객.고객번호= I.GNO
GROUP BY 고객.성별;
- ④ SELECT 고객.성별,
COUNT((SELECT 서비스번호
FROM 서비스
WHERE 서비스구매.서비스번호 = 서비스.서비스번호)) AS CNT,
SUM((SELECT 가격
FROM 서비스
WHERE 서비스구매.서비스번호 = 서비스.서비스번호)) AS SUM_PRICE
FROM 고객 INNER JOIN 서비스구매
ON 고객.고객번호= 서비스구매.고객번호
GROUP BY 고객.성별;

36. 아래 SQL 중 수행이 정상적이지 않은 것은?

<EMP>	
COLUMN	DATA_TYPE
EMPNO	NUMBER(4)
ENAME	VARCHAR2(10)
SAL	NUMBER(7,2)
DEPTNO	NUMBER(2)

<EMPLOYEES>	
COLUMN	DATA_TYPE
EMPLOYEE_ID	VARCHAR2(7)
NAME	VARCHAR2(10)
SALARY	NUMBER(8,2)
DEPARTMENT_ID	VARCHAR2(5)

- ① SELECT DEPTNO, SUM(SAL) AS SUM_SAL
FROM EMP
GROUP BY DEPTNO
UNION ALL
SELECT DEPARTMENT_ID, SUM(SALARY)
FROM EMPLOYEES
GROUP BY DEPARTMENT_ID;
- ② SELECT ENAME, SAL
FROM EMP
MINUS
SELECT NAME, SALARY
FROM EMPLOYEES;
- ③ SELECT TO_CHAR(EMPNO) AS EMPNO, ENAME, SAL
FROM EMP
UNION
SELECT EMPLOYEE_ID, NAME, SALARY
FROM EMPLOYEES
ORDER BY EMPNO;
- ④ (SELECT TO_CHAR(EMPNO) AS EMPNO, ENAME, SAL
FROM EMP
UNION
SELECT EMPLOYEE_ID AS EMPNO, NAME, SALARY
FROM EMPLOYEES)
ORDER BY EMPNO;

37. 아래 쿼리중 결과값이 다른 하나는?

- ① SELECT DNAME, JOB, COUNT(*) AS CNT, SUM(SAL) AS TOTAL_SAL
FROM SCOTT.EMP A, SCOTT.DEPT B
WHERE A.DEPTNO = B.DEPTNO
GROUP BY ROLLUP(DNAME, JOB)
ORDER BY DNAME, JOB;
- ② SELECT DNAME, JOB, COUNT(*) AS CNT, SUM(SAL) AS TOTAL_SAL
FROM SCOTT.EMP A, SCOTT.DEPT B
WHERE A.DEPTNO = B.DEPTNO
GROUP BY GROUPING SETS((DNAME, JOB), DNAME, NULL)
ORDER BY 1, 2;
- ③ SELECT DNAME, JOB, COUNT(*) AS CNT, SUM(SAL) AS TOTAL_SAL
FROM SCOTT.EMP A, SCOTT.DEPT B
WHERE A.DEPTNO = B.DEPTNO
GROUP BY CUBE(DNAME, JOB)
ORDER BY 1, 2;
- ④ SELECT DNAME, JOB, COUNT(*) AS CNT, SUM(SAL) AS TOTAL_SAL
FROM SCOTT.EMP A, SCOTT.DEPT B
WHERE A.DEPTNO = B.DEPTNO
GROUP BY DNAME, JOB
UNION ALL
SELECT DNAME, '' AS JOB, COUNT(*) AS CNT, SUM(SAL) AS TOTAL_SAL
FROM SCOTT.EMP A, SCOTT.DEPT B
WHERE A.DEPTNO = B.DEPTNO
GROUP BY DNAME
UNION ALL
SELECT '' AS DNAME, '' AS JOB, COUNT(*) AS CNT, SUM(SAL) AS TOTAL_SAL
FROM SCOTT.EMP A, SCOTT.DEPT B
WHERE A.DEPTNO = B.DEPTNO
ORDER BY 1, 2;

38. 다음 SQL 실행 결과로 가장 알맞은 것은?

<주문내역>

주문번호	일자	주문금액	지점
1	2024.01.01	1000	A
2	2024.01.02	1000	A
3	2024.01.02	2000	A
4	2024.01.03	3000	A
5	2024.01.04	3000	B
6	2024.01.05	4000	B

```
SELECT 주문번호, 일자,
       SUM(주문금액) OVER(ORDER BY 일자) AS 주문금액,
       DENSE_RANK() OVER(PARTITION BY 지점 ORDER BY 주문금액) AS 순위
FROM 주문내역;
```

①

주문번호	일자	주문금액	순위
1	2024.01.01	1000	1
2	2024.01.02	2000	1
3	2024.01.02	4000	2
4	2024.01.03	7000	3
5	2024.01.04	10000	1
6	2024.01.05	14000	2

②

주문번호	일자	주문금액	순위
1	2024.01.01	1000	1
2	2024.01.02	2000	1
3	2024.01.02	4000	3
4	2024.01.03	7000	4
5	2024.01.04	10000	1
6	2024.01.05	14000	2

③

주문번호	일자	주문금액	순위
1	2024.01.01	1000	1
2	2024.01.02	4000	1
3	2024.01.02	4000	2
4	2024.01.03	7000	3
5	2024.01.04	10000	1
6	2024.01.05	14000	2

④

주문번호	일자	주문금액	순위
1	2024.01.01	1000	1
2	2024.01.02	4000	1
3	2024.01.02	4000	3
4	2024.01.03	7000	4
5	2024.01.04	10000	1
6	2024.01.05	14000	2

39. 다음 SQL 실행 결과로 가장 알맞은 것은?

<TAB1>

NAME	SAL	DNAME
홍길동	300	아시아지부
박길동	400	아시아지부
최길동	500	아시아지부
김길동	450	남유럽지부
이길동	550	남유럽지부

```
SELECT DNAME,
       FIRST_VALUE(NAME) OVER(PARTITION BY DNAME ORDER BY SAL) AS VALUE1,
       LAST_VALUE(NAME) OVER(PARTITION BY DNAME ORDER BY SAL) AS VALUE2
FROM TAB1;
```

①

DNAME	VALUE1	VALUE2
아시아지부	홍길동	최길동
아시아지부	박길동	최길동
아시아지부	최길동	최길동
남유럽지부	김길동	이길동
남유럽지부	이길동	이길동

②

DNAME	VALUE1	VALUE2
아시아지부	홍길동	홍길동
아시아지부	박길동	박길동
아시아지부	최길동	최길동
남유럽지부	김길동	김길동
남유럽지부	이길동	이길동

③

DNAME	VALUE1	VALUE2
아시아지부	홍길동	최길동
아시아지부	홍길동	최길동
아시아지부	홍길동	최길동
남유럽지부	김길동	이길동
남유럽지부	김길동	이길동

④

DNAME	VALUE1	VALUE2
아시아지부	홍길동	홍길동
아시아지부	홍길동	박길동
아시아지부	홍길동	최길동
남유럽지부	김길동	김길동
남유럽지부	김길동	이길동

40. 다음 SQL 실행 결과 중 다른 하나는?

<EXAM>	
이름	성적
홍길동	98
박길동	60
최길동	72
김길동	80
이길동	80

- ① SELECT 성적
FROM EXAM
ORDER BY 성적 DESC
FETCH FIRST 3 ROWS ONLY;
- ② SELECT TOP(2) WITH TIES 성적
FROM EXAM
ORDER BY 성적 DESC;
- ③ SELECT 성적
FROM (SELECT 성적, RANK() OVER(ORDER BY 성적 DESC) AS RN
FROM EXAM)
WHERE RN <= 2;
- ④ SELECT 성적
FROM (SELECT 성적, ROWNUM AS RN
FROM EXAM
ORDER BY 성적 DESC)
WHERE RN <= 3;

41. 아래의 계층형 SQL 에서 리프 데이터이면 1, 그렇지 않으면 0 을 출력하고 싶을 때 사용하는 키워드로 알맞은 것은?

```
SELECT LEVEL,
       EMPNO,
       MGR,
       _____ AS 리프노드여부
FROM SCOTT.EMP
START WITH MGR IS NULL
CONNECT BY PRIOR EMPNO = MGR;
```

- ① CONNECT_BY_ISLEAF
- ② CONNECT_BY_ISCYCLE
- ③ SYS_CONNECT_BY_PATH
- ④ CONNECT_BY_ROOT

42. 아래 SQL 실행 결과로 가장 알맞은 것은?

<사원>

사원번호	이름	입사일자	매니저사원번호
10000	나사장	2010-01-01	
10001	박전무	2010-01-01	10000
10002	김전무	2010-12-01	10000
10003	홍상무	2011-12-01	10002
10004	김상무	2012-12-01	10002
10005	김사원	2013-01-01	10003
10006	홍사원	2014-12-01	10003
10007	박사원	2015-01-01	10004
10008	최사원	2016-12-01	10004
10009	박인턴	2017-01-01	10007
10010	강인턴	2018-12-01	10008

```
SELECT E.*, LEVEL
FROM 사원 E
START WITH 매니저사원번호 IS NULL
CONNECT BY PRIOR 사원번호 = 매니저사원번호 AND EXTRACT(MONTH FROM 입사일자) >= 7
ORDER SIBLINGS BY 이름;
```

①

사원번호	이름	입사일자	매니저사원번호	LEVEL
10000	나사장	2010-01-01 0:00		1
10002	김전무	2010-12-01 0:00	10000	2
10003	홍상무	2011-12-01 0:00	10002	3
10006	홍사원	2014-12-01 0:00	10003	4
10004	김상무	2012-12-01 0:00	10002	3
10008	최사원	2016-12-01 0:00	10004	4
10010	강인턴	2018-12-01 0:00	10008	5

②

사원번호	이름	입사일자	매니저사원번호	LEVEL
10000	나사장	2010-01-01 0:00		1
10002	김전무	2010-12-01 0:00	10000	2
10004	김상무	2012-12-01 0:00	10002	3
10008	최사원	2016-12-01 0:00	10004	4
10010	강인턴	2018-12-01 0:00	10008	5
10003	홍상무	2011-12-01 0:00	10002	3
10006	홍사원	2014-12-01 0:00	10003	4

③

사원번호	이름	입사일자	매니저사원번호	LEVEL
10002	김전무	2010-12-01 0:00	10000	2
10004	김상무	2012-12-01 0:00	10002	3
10008	최사원	2016-12-01 0:00	10004	4
10010	강인턴	2018-12-01 0:00	10008	5
10003	홍상무	2011-12-01 0:00	10002	3
10006	홍사원	2014-12-01 0:00	10003	4

④

사원번호	이름	입사일자	매니저사원번호	LEVEL
10002	김전무	2010-12-01 0:00	10000	2
10004	김상무	2012-12-01 0:00	10002	3
10008	최사원	2016-12-01 0:00	10004	4
10010	강인턴	2018-12-01 0:00	10008	5
10003	홍상무	2011-12-01 0:00	10002	3
10006	홍사원	2014-12-01 0:00	10003	4
10006	홍사원	2014-12-01 0:00	10003	4

43. 다음 SQL 구문 결과와 같은 결과를 갖는 SQL은?

```
SELECT COUNT(DECODE(DEPTNO,10,1)) AS "10",
       COUNT(DECODE(DEPTNO,20,1)) AS "20",
       COUNT(DECODE(DEPTNO,30,1)) AS "30"
FROM EMP;
```

- ① SELECT *
FROM (SELECT EMPNO, DEPTNO FROM EMP)
PIVOT (COUNT(EMPNO) FOR DEPTNO IN (10,20,30));
- ② SELECT *
FROM (SELECT EMPNO, JOB, DEPTNO FROM EMP)
PIVOT (COUNT(DEPTNO) FOR EMPNO IN (10,20,30));
- ③ SELECT *
FROM EMP
UNPIVOT (COUNT(DEPTNO) FOR DEPTNO IN (10,20,30));
- ④ SELECT *
FROM EMP
UNPIVOT (COUNT(*) FOR DEPTNO IN (10,20,30));

44. 다음 출력 결과로 가장 알맞은 것은?

```
SELECT REGEXP_COUNT('abc1004 zz1234', '\d{2}+') AS C1,
       REGEXP_COUNT('abc1004-zz1234-100', '\d{2,}+') AS C2
FROM DUAL;
```

- ① 2, 1
- ② 2, 3
- ③ 8, 1
- ④ 8, 3

45. 다음 출력 결과로 가장 알맞은 것은?

COL1
AAXYXY-Z
XXYYZ
XY-
XY-Z
XY-+?

```
SELECT COUNT(COL1)
FROM TAB1
WHERE REGEXP_LIKE(COL1, '[XY-]+Z?');
```

- ① 2
- ② 3
- ③ 4
- ④ 5

46. 다음 SQL 실행 결과로 알맞은 것은?

<EMPLOYEE>

ID	NAME	SAL	DNAME
0001	홍길동	1000	아시아지부
0002	박길동	2000	아시아지부
0003	최길동	3000	아시아지부
0004	이길동	4000	남유럽지부
0005	김길동	6000	남유럽지부
0006	김길동	8000	남유럽지부

```
UPDATE EMPLOYEE E1
  SET SAL = (SELECT MAX(SAL)
             FROM EMPLOYEE E2
            WHERE E1.DNAME = E2.DNAME)
WHERE SAL <= (SELECT AVG(SAL)
              FROM EMPLOYEE);
```

①

ID	NAME	SAL	DNAME
0001	홍길동	NULL	아시아지부
0002	박길동	NULL	아시아지부
0003	최길동	NULL	아시아지부
0004	이길동	NULL	남유럽지부
0005	김길동	6000	남유럽지부
0006	김길동	8000	남유럽지부

②

ID	NAME	SAL	DNAME
0001	홍길동	3000	아시아지부
0002	박길동	3000	아시아지부
0003	최길동	3000	아시아지부
0004	이길동	4000	남유럽지부
0005	김길동	6000	남유럽지부
0006	김길동	8000	남유럽지부

③

ID	NAME	SAL	DNAME
0001	홍길동	3000	아시아지부
0002	박길동	3000	아시아지부
0003	최길동	3000	아시아지부
0004	이길동	8000	남유럽지부
0005	김길동	6000	남유럽지부
0006	김길동	8000	남유럽지부

④

ID	NAME	SAL	DNAME
0001	홍길동	4000	아시아지부
0002	박길동	4000	아시아지부
0003	최길동	4000	아시아지부
0004	이길동	8000	남유럽지부
0005	김길동	6000	남유럽지부
0006	김길동	8000	남유럽지부

47. 아래 SQL 실행 결과로 가장 적절한 것은?

<TAB1>		
NO	COL1	COL2
1	A	1
2	B	2
3	C	2
4	D	3


```

INSERT INTO TAB1 VALUES(5, 'E', 3);
COMMIT;
UPDATE TAB1 SET COL2 = 3 WHERE NO = 2;
SAVEPOINT SAVE1;
INSERT INTO TAB1 VALUES(6, 'F', 5);
DELETE TAB1 WHERE NO = 4;
ROLLBACK TO SAVE1;
UPDATE TAB1 SET COL2 = 2 WHERE NO = 1;
ROLLBACK;
COMMIT;

SELECT SUM(COL2) FROM TAB1;

```

- ① 8
- ② 11
- ③ 12
- ④ 13

48. 비교연산자의 어느 한쪽이 VARCHAR 유형 타입인 경우 문자 유형 비교에 대한 설명 중 가장 알맞지 않은 것은?

- ① 서로 다른 문자가 나올 때까지 비교한다
- ② 길이가 다르다면 짧은 것이 끝날 때까지만 비교한 후에 길이가 긴 것이 크다고 판단한다
- ③ 길이가 같고 다른 것이 없다면 같다고 판단한다
- ④ 길이가 다르다면 작은 쪽에 SPACE 를 추가하여 길이를 같게 한 후에 비교한다

49. 다음 중 실행이 불가능한 SQL을 고르시오.

< TAB1 >	
COL1	NUMBER(4)
COL2	NUMBER(5,2)
COL3	VARCHAR2(10)
COL4	CHAR(10)

- ① INSERT INTO TAB1 VALUES(1000, 12.345, 100, 'ABC');
- ② INSERT INTO TAB1 VALUES(1000, 123.45, '100', 'ABC');
- ③ INSERT INTO TAB1 VALUES(1000, 123.456, '2024-01-01', NULL);
- ④ INSERT INTO TAB1 VALUES(1000, 1234.5, '100', '');

50. 유저와 권한 중 권한에 대한 설명 중 가장 옳바르지 않은 것은?

- ① 사용자가 실행하는 모든 DDL 문장은 그에 해당하는 적절한 권한이 있어야만 문장을 실행 할 수 있다.
- ② DBA 권한을 가진 유저만이 권한을 부여 할 수 있다
- ③ 테이블의 소유자는 해당 테이블의 DML 권한을 다른 유저에게 부여 할 수 있다.
- ④ 권한 부여를 편리하게 관리하기 위해 만들어진 권한의 집합인 ROLE 이 있다

2회차

1. 데이터 모델링을 할 때 유의해야 할 사항으로 가장 적절하지 않은 것은?
 - ① 같은 정보를 저장하지 않도록 하여 중복성을 최소화한다.
 - ② 데이터 간의 상호 연관관계를 명확하게 정의하여 일관성 있게 데이터가 유지되도록 한다.
 - ③ 데이터의 정의를 프로세스와 분리하여 유연성을 높인다.
 - ④ 사용자가 처리하는 프로세스에 따라 매핑이 될 수 있도록 프로그램과 테이블 간의 연계성을 높인다.
2. 엔터티 분류 중 발생시점에 따른 분류가 안된 것은?
 - ① 기본엔터티
 - ② 중심엔터티
 - ③ 사건엔터티
 - ④ 행위엔터티
3. 속성에 대한 설명으로 가장 적절하지 않은 것은?
 - ① 업무상 인스턴스로 관리하고자 하는 더 이상 분리되지 않는 최소 데이터 단위를 나타낸다.
 - ② 하나의 엔터티는 두 개 이상의 속성을 갖는다.
 - ③ 하나의 인스턴스에서 각각의 속성은 하나 이상의 속성값을 가질 수 있다.
 - ④ 속성은 정해진 주식별자에 함수적 종속성을 가져야 한다.
4. 데이터 모델링의 관계에 대한 설명으로 가장 적절하지 않은 것은?
 - ① 관계는 존재에 의한 관계와 행위에 의한 관계로 구분될 수 있으나 ERD에서는 존재와 행위를 구분하지 않는다.
 - ② 부서와 사원 엔터티 간의 '소속' 관계는 존재적 관계의 사례이다.
 - ③ 관계의 페어링은 하나의 엔터티와 다른 엔터티 간의 레코드 연결 방식을 나타낸다.
 - ④ 관계의 표기법은 관계명, 관계차수, 선택성 3가지 개념을 사용한다.
5. 데이터 모델링에서 식별자 관계에 대한 설명 중 가장 적절하지 않은 것은?
 - ① 식별 관계는 하나의 엔터티의 기본키를 다른 엔터티가 기본키의 하나로 공유하는 관계이다.
 - ② IE 표기법에서는 식별 관계는 실선으로, 비식별 관계는 점선으로 표시한다.
 - ③ 비식별 관계는 부모 엔터티가 소멸하면 자식 엔터티도 종속적으로 삭제되는 관계이다.
 - ④ 자식 엔터티의 식별자가 부모 엔터티의 주식별자를 상속받아 생성하는 것보다 별도의 주식별자를 생성하는 것이 더 유리하다고 판단되는 경우 비식별자 관계로 연결해야 한다.
6. 데이터 모델링의 정규화에 대한 설명으로 가장 적절하지 않은 것은?
 - ① 정규화는 모델의 일관성을 확보하고 중복을 제거하여 모델의 독립성을 확보하는 과정이다.
 - ② 개념 모델링 단계에서의 엔터티를 상세화 하는 과정이다.
 - ③ 제1정규형은 모든 인스턴스가 반드시 하나의 값을 가져야 함을 의미한다.
 - ④ 제3정규형을 만족하는 엔터티의 일반속성은 주식별자 전체에 종속적이다.

7. 관계(Relationship)와 조인(Join)에 대한 설명으로 가장 적절하지 않은 것은?

- ① 조인(Join)이란 식별자를 상속하고, 상속된 속성을 매핑키로 활용하여 데이터를 결합하는 것을 의미한다.
- ② 부모의 식별자를 자식의 일반속성으로 상속하면 식별 관계, 부모의 식별자를 자식의 식별자에 포함하면 비식별 관계라고 할 수 있다.
- ③ 엔터티 간의 관계를 통해 데이터의 중복을 피하고, 각 데이터 요소를 한 번만 저장하여 유지관리의 복잡성을 줄일 수 있다.
- ④ 관계는 엔터티간의 논리적 연관성을 의미한다.

8. 트랜잭션에 대한 설명 중 가장 적절하지 않은 것은?

- ① 트랜잭션에 의한 관계는 선택적인 관계 형태를 가진다.
- ② 원자성이란 하나의 트랜잭션의 작업이 모두 성공하거나 모두 취소되어야 하는 특징을 말한다.
- ③ 순차적으로 수행되는 작업 A와 B가 하나의 트랜잭션일 경우 A만 실행되고 시스템 장애가 발생했다면 A를 ROLLBACK해야 한다.
- ④ 하나의 트랜잭션으로 구성된 작업은 부분 COMMIT이 불가하다.

9. NULL에 대한 설명으로 가장 적절하지 않은 것은?

- ① 정해지지 않은 값을 의미한다.
- ② 논리모델 설계 시 각 컬럼별로 NULL을 허용할 지를 결정한다.
- ③ IE 표기법에서는 각 컬럼별 NULL 허용 여부를 알 수 없다.
- ④ 바커 표기법에서는 속성 앞에 별(*)을 사용하여 널 허용 속성을 표현한다.

10. 본질식별자와 인조식별자에 대한 설명으로 가장 적절하지 않은 것은?

- ① 인조식별자를 사용하면 불필요하게 발생하는 중복데이터를 막을 수 있다.
- ② 인조식별자를 사용하면 본질식별자를 사용할 때와 비교하여 추가적인 인덱스가 필요해진다.
- ③ 인조식별자는 대체로 본질식별자가 복잡한 구성을 가질 때 만들어진다.
- ④ 자동으로 증가하는 일련번호 같은 형태는 인조식별자에 해당한다.

11. 다음 SQL 중 항상 오류가 발생하는 구문으로 가장 적절한 것은?

- ①

```
SELECT T.COL1 C1, T.COL2 AS C2
FROM TABLE1 T
WHERE T.COL1 = 4;
```
- ②

```
SELECT TABLE1.COL1, SUM(TABLE1.COL2)
FROM TABLE1
GROUP BY TABLE1.COL1
ORDER BY COL1;
```
- ③

```
SELECT T.COL1 C1, TABLE1.COL2 AS C2
FROM TABLE1 T
WHERE TABLE1.COL2 = 'A'
ORDER BY 1, 2;
```
- ④

```
SELECT T.COL1 C1, T.COL2 AS "C1"
FROM TABLE1 T
WHERE T.COL2 IN ('A', 'B')
ORDER BY 1, "C1";
```

12. SELECT 문에 대한 설명으로 가장 적절하지 않은 것은?

- ① GROUP BY절에는 컬럼별칭을 사용할 수 없다.
- ② WHERE절에는 그룹함수를 사용한 조건 전달이 불가하다.
- ③ HAVING절에서는 그룹함수가 없는 일반 조건을 사용할 수 있다.
- ④ SELECT문의 6개 절 중에서 SELECT절이 가장 마지막에 실행된다.

13. SQL 문을 실행했을 때 오류가 발생하는 부분으로 가장 적절한 것은?

- ① SELECT T.COL1, COL2, SUM(COL3) AS "SUM VALUE"
- ② FROM TAB1 T
- ③ GROUP BY COL1, COL2
- ④ ORDER BY COL3;

14. 아래의 SQL 에 대해서 결과값이 다른 것은?

- ① SELECT CONCAT ('RDBMS', ' SQL') FROM DUAL;
- ② SELECT 'RDMBS' || ' SQL' FROM DUAL;
- ③ SELECT 'RDBMS' + ' SQL';
- ④ SELECT 'RDBMS' & ' SQL' FROM DUAL;

15. 아래 SQL에서 밑줄 친 자리에 쓰인 함수의 결과가 다른 하나는?

SELECT _____(5.47) FROM DUAL;

- ① TRUNC
- ② CEIL
- ③ FLOOR
- ④ ROUND

16. 다음 SQL의 수행 결과로 가장 적절한 것은?

<TAB1>

JUMIN

7510231111111

```
SELECT TO_CHAR(TO_DATE(SUBSTR(JUMIN, 1, 6), 'RRMMDD'), 'YYYY-MM-DD')
FROM TAB1;
```

- ① 1975-10-23
- ② 2075-10-23
- ③ 1975-10-23 00:00:00
- ④ 2075-10-23 00:00:00

17. 다음 SQL의 수행 결과로 알맞은 것은?

<TAB1>	
COL1	
1	
2	
3	
NULL	
SELECT ISNULL(COL1, 3) FROM TAB1;	

①	②	③	④
COL1	COL1	COL1	COL1
1	1	1	1
2	2	2	2
3	3	NULL	NULL
NULL	3	NULL	3

18. 아래 SQL의 실행 결과로 알맞은 것은?

(단, 문자타입인 경우 "를 붙여서 표현, 숫자타입인 경우 숫자만 전달)

SELECT LTRIM(' AB DE ') AS C1,		
INITCAP('ABCDE') AS C2,		
TO_CHAR('123', '999.99') AS C3		
FROM DUAL;		

①		
C1	C2	C3
'AB DE '	'Abcde'	'123.00'

②		
C1	C2	C3
' AB DE '	'Abcde'	123.00

③		
C1	C2	C3
'ABDE'	'abcde'	'999.99'

④		
C1	C2	C3
'AB DE '	'Abcde'	999.99

19. 다음 SQL 수행 결과로 가장 알맞은 것은?

<TAB1>		
COL1	COL2	COL3
NULL	10	20
10	NULL	30
20	30	NULL

SELECT COALESCE(COL1, COL2, COL3) RESULT
FROM TAB1;

①

RESULT
NULL
NULL
NULL

②

RESULT
10
10
20

③

RESULT
NULL
10
20

④

RESULT
10
30
NULL

20. 아래 SQL의 실행 결과로 알맞은 것은?

<TAB1>		<TAB2>	
NO	SAL	NO	SAL
1	100	1	100
2	200	3	200
3	300	3	300
4	400	NULL	300
NULL	100	5	400
		5	400

SELECT SUM(SAL)
FROM TAB1
WHERE NO NOT IN (SELECT NO
FROM TAB2
GROUP BY NO);

① 0

② NULL

③ 600

④ 700

21. 아래 수행 결과로 알맞은 것은?

<TAB1>	
COL1	COL2
A	10
B	NULL
B	20
NULL	30
NULL	40
C	20


```

SELECT COUNT(COL1) AS RESULT
  FROM TAB1
 WHERE COL2 >= 30
 GROUP BY COL1;
  
```

①

RESULT

②

RESULT
NULL

③

RESULT
0

④

RESULT
1

22. 다음 SQL 실행 결과로 가장 적절한 것은?

<TAB1>	
COL1	COL2
10	100
10	200
20	400
30	200
30	300
NULL	100
NULL	500


```

SELECT COL1 AS C1, SUM(COL2) AS C2
  FROM TAB1
 GROUP BY COL1
 HAVING SUM(COL2) >= 400;
  
```

①

C1	C2
10	300
20	400
30	500
NULL	600

②

C1	C2
20	400
30	500
NULL	600

③

C1	C2
20	400
30	500

④

C1	C2
20	400
NULL	500

23. 다음 중 틀린 설명은? (단, DBMS는 ORACLE)

- ① ORDER BY COMM 시 NULL이 마지막에 배치된다.
- ② ORDER BY COMM NULLS FIRST 시 NULL이 맨 앞에 배치된다.
- ③ ORDER BY COMM DESC 시 NULL이 맨 앞에 배치된다.
- ④ ORDER BY COMM DESC NULLS LAST 시 NULL이 맨 앞에 배치된다.

24. 아래 SQL 수행 결과로 가장 알맞은 것은?

<TAB1>	
COL1	COL2
SMITH	10
ALLEN	20
SCOTT	30
FORD	40


```

SELECT COL1
  FROM TAB1
 ORDER BY CASE WHEN MOD(COL2, 3) = 0 THEN 'A'
              ELSE 'B'
              END, COL1;

```

①	②	③	④
COL1	COL1	COL1	COL1
SMITH	SCOTT	SCOTT	SCOTT
ALLEN	ALLEN	SMITH	SMITH
SCOTT	FORD	ALLEN	FORD
FORD	SMITH	FORD	ALLEN

25. 다음 SQL 구문의 결과는?

<TAB1>		<TAB2>	
COL1	COL2	COL1	COL2
1	A	1	A
2	B	2	B
3		3	B
4	C	4	


```

SELECT COUNT(A.COL2)
  FROM TAB1 A JOIN TAB2 B
    ON A.COL2 = B.COL2;

```

- ① 0
- ② 1
- ③ 2
- ④ 3

26. 다음 SQL 구문의 결과는?

<TAB1>		<TAB2>	
COL1	COL2	COL1	COL2
1	A	1	A
2	B	2	B
3		3	B
4	C	4	


```

SELECT COUNT(TAB1.COL1)
  FROM TAB1, TAB2
 WHERE TAB1.COL2 = TAB2.COL2(+);

```

- ① 2
- ② 3
- ③ 4
- ④ 5

27. 다음 표준조인에 대한 설명 중 가장 적절하지 않은 것은?

- ① CROSS JOIN인 두 테이블의 조인 컬럼의 값과 상관없이 항상 모든 경우의 수를 출력한다.
- ② FULL OUTER JOIN은 LEFT OUTER JOIN 결과와 RIGHT OUTER JOIN 결과를 UNION한 것과 같다.
- ③ NATURAL JOIN시 같은 이름의 컬럼이 여러 개인 경우 USING절을 사용하여 원하는 컬럼을 선택할 수 있다.
- ④ INNER JOIN은 줄여서 JOIN으로 전달할 수 있다.

28. SQL의 실행 결과로 가장 적절한 것은?

<TAB1>		<TAB2>	
NO	CODE	NO	QTY
1	A	1	2
2	B	2	4
3		4	6
4	B	4	8
6	D	5	10
7	E	5	10
		5	10


```

SELECT COUNT(TAB1.NO) FROM TAB1 LEFT OUTER JOIN TAB2 ON TAB1.NO = TAB2.NO;
SELECT COUNT(DISTINCT TAB1.CODE) FROM TAB1 RIGHT OUTER JOIN TAB2 ON TAB1.NO = TAB2.NO;

```

- ① 7, 2
- ② 7, 3
- ③ 9, 2
- ④ 9, 3

29. 아래와 같은 테이블 데이터가 있다. SQL에 대한 결과로 가장 알맞은 것은?

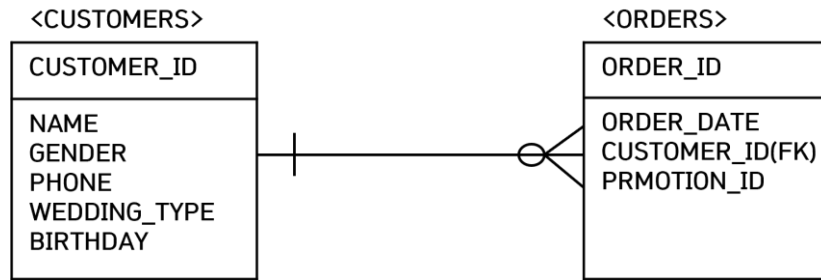
<TAB1>

COL1	COL2
A	10
A	20
A	30
A	30
B	30
B	40

```
SELECT SUM(T1.COL2)
  FROM TAB1 T1
 WHERE T1.COL2 = (SELECT MAX(COL2)
                  FROM TAB1 T2
                  WHERE T1.COL1 = T2.COL1);
```

- ① 에러
- ② NULL
- ③ 90
- ④ 100

30. 아래 ERD를 참고하여, 다음 SQL을 작성하였을 때 이들 중 실행 결과가 다른 하나는?



- ① SELECT DISTINCT C.CUSTOMER_ID
FROM CUSTOMERS C, ORDERS O
WHERE C.CUSTOMER_ID = O.CUSTOMER_ID
AND C.WEDDING_TYPE = 'Y'
AND C.GENDER = 'F'
AND TRUNC((SYSDATE - BIRTHDAY)/365) BETWEEN 40 AND 50;
- ② SELECT C.CUSTOMER_ID
FROM CUSTOMERS C, (SELECT O.CUSTOMER_ID, COUNT(O.PROMOTION_ID) AS ORD_CNT
FROM ORDERS O
GROUP BY O.CUSTOMER_ID
HAVING COUNT(O.PROMOTION_ID) >= 1) I
WHERE C.CUSTOMER_ID = I.CUSTOMER_ID
AND C.WEDDING_TYPE = 'Y'
AND C.GENDER = 'F'
AND TRUNC((SYSDATE - BIRTHDAY)/365) BETWEEN 40 AND 50;
- ③ SELECT C.CUSTOMER_ID
FROM CUSTOMERS C
WHERE C.WEDDING_TYPE = 'Y'
AND C.GENDER = 'F'
AND TRUNC((SYSDATE - BIRTHDAY)/365) BETWEEN 40 AND 50
AND C.CUSTOMER_ID IN (SELECT O.CUSTOMER_ID
FROM ORDERS O);
- ④ SELECT C.CUSTOMER_ID
FROM CUSTOMERS C
WHERE C.WEDDING_TYPE = 'Y'
AND C.GENDER = 'F'
AND TRUNC((SYSDATE - BIRTHDAY)/365) BETWEEN 40 AND 50
AND EXISTS (SELECT 'X'
FROM ORDERS O
WHERE C.CUSTOMER_ID = O.CUSTOMER_ID);

31. 다음 SQL 수행 결과로 알맞은 것은?

<EMP>

EMPNO	ENAME	HIREDATE
7369	SMITH	1980/12/17
7499	ALLEN	1981/02/20
7521	WARD	1981/02/22
7566	JONES	1981/04/02
7698	BLAKE	1981/05/01
7782	CLARK	1981/06/09

```
SELECT E1.EMPNO, E1.ENAME, E1.HIREDATE, COUNT(E2.EMPNO) AS CNT
FROM EMP E1 LEFT OUTER JOIN EMP E2
ON E1.HIREDATE > E2.HIREDATE
GROUP BY E1.EMPNO, E1.ENAME, E1.HIREDATE
ORDER BY E1.HIREDATE;
```

①

EMPNO	ENAME	HIREDATE	CNT
7499	ALLEN	1981/02/20	1
7521	WARD	1981/02/22	2
7566	JONES	1981/04/02	3
7698	BLAKE	1981/05/01	4
7782	CLARK	1981/06/09	5

②

EMPNO	ENAME	HIREDATE	CNT
7369	SMITH	1980/12/17	0
7499	ALLEN	1981/02/20	1
7521	WARD	1981/02/22	2
7566	JONES	1981/04/02	3
7698	BLAKE	1981/05/01	4
7782	CLARK	1981/06/09	5

③

EMPNO	ENAME	HIREDATE	CNT
7369	SMITH	1980/12/17	5
7499	ALLEN	1981/02/20	4
7521	WARD	1981/02/22	3
7566	JONES	1981/04/02	2
7698	BLAKE	1981/05/01	1

④

EMPNO	ENAME	HIREDATE	CNT
7369	SMITH	1980/12/17	5
7499	ALLEN	1981/02/20	4
7521	WARD	1981/02/22	3
7566	JONES	1981/04/02	2
7698	BLAKE	1981/05/01	1
7782	CLARK	1981/06/09	0

32. 서브쿼리 설명으로 가장 적절한 것은?

- ① 단일 행 서브쿼리는 서브쿼리의 실행 결과가 항상 한 건 이하인 서브쿼리로 IN, ALL 등의 비교 연산자를 사용하여 한다.
- ② 다중 행 서브쿼리 비교 연산자는 단일 행 서브쿼리의 비교 연산자로도 사용할 수 있다.
- ③ 연관 서브쿼리는 주로 메인쿼리에 값을 제공하기 위한 목적으로 사용한다.
- ④ 서브 쿼리는 항상 메인쿼리에서 읽힌 데이터에 대해 서브쿼리에서 해당 조건이 만족하는지를 확인하는 방식으로 수행된다.

33. 아래 결과를 출력하기 위해 빈칸에 들어갈 문장으로 적절한 것은?

<고객>			
고객번호	이름	포인트	
1000	홍길동	10000	
1001	박길동	9000	
1002	최길동	13000	
1003	이길동	2000	
<상품>			
상품번호	상품명	최소포인트	최대포인트
1	A	1	2000
2	B	2001	8000
3	C	8001	12000
4	D	12001	20000
고객번호	상품명		
1000	C		
1001	C		
1002	D		
1003	A		

- ① SELECT 고객.고객번호, 상품.상품명
FROM 고객 INNER JOIN 상품
ON 고객.포인트 >= 상품.최소포인트;

- ② SELECT 고객.고객번호, 상품.상품명
FROM 고객 INNER JOIN 상품
ON 고객.포인트 <= 상품.최대포인트;
- ③ SELECT 고객.고객번호, 상품.상품명
FROM 고객 JOIN 상품
ON 고객.포인트 BETWEEN 상품.최소포인트 AND 상품.최대포인트;
- ④ SELECT 고객.고객번호, 상품.상품명
FROM 고객, 상품
ON 고객.포인트 >= 상품.최소포인트(+);

34. 다음 SQL 실행 결과로 가장 알맞은 것은?

<TAB1>			<TAB2>		
COL1	COL2	COL3	COL1	COL2	COL3
1	A	10	6	A	10
2	A	15	7	A	30
3	B	10	8	B	10
4	B	30	9	B	30
5	B	20	10	A	20


```

SELECT SUM(COL1)
  FROM TAB1 T1
 WHERE COL3 >= (SELECT AVG(COL3)
                  FROM TAB2 T2
                 WHERE T2.COL2 = T1.COL2);

```

- ① NULL
- ② 0
- ③ 9
- ④ 23

35. 테이블이 아래와 같을 때, 다음 집합연산자 수행 결과로 가장 적절한 것은?

<TAB1>		<TAB2>		<TAB3>	
COL1	COL2	COL1	COL2	COL1	COL2
10	A	10	A	30	C
20	B	20	A	30	D
30	C	50	F	40	E
30	D	NULL	A	NULL	A
40	E				

```

SELECT *
  FROM (SELECT COL1, COL2
        FROM TAB1
        UNION
        SELECT COL1, COL2
        FROM TAB2)
MINUS
SELECT COL1, COL2
  FROM TAB3;

```

①

COL1	COL2
10	A
20	B
50	A

②

COL1	COL2
10	A
20	A
20	B
50	F

③

COL1	COL2
10	A
20	A
20	B
50	A
NULL	A

④

COL1	COL2
10	A
10	A
20	A
20	B
50	A
NULL	A

36. 아래 SQL의 결과로 가장 알맞은 것은?

<JUMUN>

JUMUN_NO	PRODUCT_NO	GOGAK_NO	QTY	JUMUN_DATE
1000	1	0001	3	2024/05/24 11:00:56
1001	2	0001	1	2024/05/25 12:01:56
1002	1	0002	2	2024/05/26 09:13:12
1003	1	0002	1	2024/05/27 10:24:30
1004	2	0002	2	2024/05/28 13:01:10

```
SELECT PRODUCT_NO, GOGAK_NO, SUM(QTY) AS TOTAL_QTY
  FROM JUMUN
 GROUP BY GROUPING SETS(PRODUCT_NO, GOGAK_NO, ());
```

①

PRODUCT_NO	GOGAK_NO	TOTAL_QTY
NULL	0001	4
NULL	0002	5
1	NULL	6
2	NULL	3
NULL	NULL	9

②

PRODUCT_NO	GOGAK_NO	TOTAL_QTY
1	NULL	6
2	NULL	3
1	0001	3
1	0002	3
2	0001	1
2	0002	2

③

PRODUCT_NO	GOGAK_NO	TOTAL_QTY
1	NULL	6
2	NULL	3
1	0001	3
1	0002	3
2	0001	1
2	0002	2
NULL	NULL	9

④

PRODUCT_NO	GOGAK_NO	TOTAL_QTY
NULL	0001	4
NULL	0002	5
1	NULL	6
2	NULL	3
1	0001	3
1	0002	3
2	0001	1
2	0002	2
NULL	NULL	9

37. 다음 중 RANK, DENSE_RANK, ROW_NUMBER 결과로 가장 적절한 것은?

<STUDENT>

NO	NAME	JUMSU
1	홍길동	100
2	박길동	90
3	최길동	90
4	이길동	80
5	구길동	70

```

SELECT NO,
       RANK() OVER(ORDER BY JUMSU DESC) AS RANK1,
       DENSE_RANK() OVER(ORDER BY JUMSU DESC) AS RANK2,
       ROW_NUMBER() OVER(ORDER BY JUMSU DESC) AS RANK3
FROM STUDENT;

```

①

NO	RANK1	RANK2	RANK3
1	1	1	1
2	2	2	2
3	2	2	3
4	3	3	4
5	4	4	5

②

NO	RANK1	RANK2	RANK3
1	1	1	1
2	2	2	2
3	2	2	3
4	4	3	4
5	5	4	5

③

NO	RANK1	RANK2	RANK3
1	1	1	1
2	2	2	2
3	2	2	2
4	3	4	3
5	4	5	4

④

NO	RANK1	RANK2	RANK3
1	1	1	1
2	2	2	2
3	2	3	2
4	4	4	3
5	5	5	4

38. 부서내에서의 급여의 비율을 출력하는 구문으로 가장 적절하지 않은 것은?

- ① SELECT ENAME, SAL, DEPTNO,
ROUND(SAL/(SELECT SUM(SAL)
FROM EMP E2
WHERE E1.DEPTNO = E2.DEPTNO) * 100, 2) AS SAL_RATIO
FROM EMP E1
ORDER BY DEPTNO, SAL DESC;
- ② SELECT ENAME, SAL, DEPTNO,
ROUND(RATIO_TO_REPORT(SAL) OVER(PARTITION BY DEPTNO) * 100, 2) AS SAL_RATIO
FROM EMP E1
ORDER BY DEPTNO, SAL DESC;
- ③ SELECT E1.ENAME, E1.SAL, E1.DEPTNO,
ROUND(E1.SAL/I.SUM_SAL * 100, 2) AS SAL_RATIO
FROM EMP E1, (SELECT DEPTNO, SUM(SAL) AS SUM_SAL
FROM EMP
GROUP BY DEPTNO) I
WHERE E1.DEPTNO = I.DEPTNO
ORDER BY DEPTNO, SAL DESC;
- ④ SELECT ENAME, SAL, DEPTNO,
ROUND(PERCENT_RANK() OVER(PARTITION BY DEPTNO ORDER BY SAL) * 100, 2) AS SAL_RATIO
FROM EMP E1
ORDER BY DEPTNO, SAL DESC;

39. 다음 SQL문의 실행 결과로 가장 알맞은 것은?

<고객>		
고객번호	이름	포인트
1	홍길동	100
2	박길동	110
3	최길동	200
4	이길동	220
5	구길동	80
6	안길동	150

SELECT *
FROM 고객
ORDER BY 포인트 DESC
OFFSET 2 ROWS
FETCH FIRST 2 ROWS ONLY;

①

고객번호	이름	포인트
6	안길동	150
2	박길동	110

②

고객번호	이름	포인트
4	이길동	220
3	최길동	200

③

고객번호	이름	포인트
4	이길동	220
3	최길동	200
6	안길동	150
2	박길동	110

④

고객번호	이름	포인트
1	홍길동	100
2	박길동	110

40. 아래 실행 결과를 출력하는 SQL로 가장 적절한 것은?

<사원>

사원번호	이름	상위관리자코드
1000	홍길동	NULL
1001	박길동	1000
1002	최길동	1001
1003	이길동	1001
1004	구길동	1002
1005	안길동	1003
1006	송길동	1000
1007	강길동	1006
1008	공길동	1006

```
SELECT 사원번호, 이름, LEVEL
FROM 사원
START WITH 사원번호 IN (1006, 1001)
CONNECT BY PRIOR 상위관리자코드 = 사원번호;
```

①

사원번호	이름	LEVEL
1001	박길동	1
1006	송길동	1

②

사원번호	이름	LEVEL
1001	박길동	1
1006	송길동	1
1000	홍길동	2

③

사원번호	이름	LEVEL
1001	박길동	1
1006	송길동	1
1000	홍길동	2
1000	홍길동	2

④

사원번호	이름	LEVEL
1001	박길동	1
1006	송길동	1
1002	최길동	2
1003	이길동	2
1007	강길동	2
1008	공길동	2

41. 계층형 질의에서 CONNECT BY 절에 사용되며, 현재 읽은 컬럼을 지정하는 구문은 무엇인가?

- ① START WITH
- ② PRIOR
- ③ NOCYCLE
- ④ ORDER SIBLINGS BY

42. 다음 빈칸에 들어갈 문장으로 가장 적절한 것은?

<판매>

지역	Q1	Q2
서울	10	20
경기	30	40

SELECT

FROM 판매

<결과>

지역	구분	판매량
서울	Q1	10
서울	Q2	20
경기	Q1	30
경기	Q2	40

- ① UNPIVOT (판매량 FOR 구분 IN (Q1, Q2))
- ② UNPIVOT (판매량 FOR 구분 FOR (Q1, Q2))
- ③ PIVOT (판매량 FOR 구분 IN (Q1, Q2))
- ④ PIVOT (판매량 FOR 구분 FOR (Q1, Q2))

43. 다음 SQL 실행 결과로 가장 알맞은 것은?

<TAB1>	
COL1	
AXXX.AYYY.AXAX.	
AAXYXY.XYYY.AXYAXY.	

SELECT REGEXP_REPLACE(COL1, 'A(X Y)+\..') FROM TAB1;
--

①	②	③	④
COL1	COL1	COL1	COL1
AX	AX	NULL	NULL
XYYY	AXYYY.AXY	XYYY	AXYYY.AXY

44. 다음 SQL 실행 결과로 가장 알맞은 것은?

SELECT REGEXP_SUBSTR('ORA-00600 Oracle SQL-Server 50', '[^0-9]+') "REGEXPR_SUBSTR" FROM DUAL;
--

- ① 50
- ② 00600 50
- ③ ORA-
- ④ ORA- Oracle SQL-Server

45. 다음 SQL 중 입력오류가 발생할 문장으로 가장 적절한 것은?

CREATE TABLE TAB1(COL1 VARCHAR(10) PRIMARY KEY, COL2 NUMBER NOT NULL, COL3 CHAR(10) NOT NULL, COL4 DATE NOT NULL);

- ① INSERT INTO TAB1 VALUES(1, 10, 'AAA', SYSDATE);
- ② INSERT INTO TAB1 VALUES('0001', '20', 'BBB', SYSTIMESTAMP);
- ③ INSERT INTO TAB1 VALUES('0002', '30', '1000', CURRENT_DATE);
- ④ INSERT INTO TAB1 VALUES('0003', 40, 'CCC', '2024/01/01');

46. 아래 SQL 실행 결과로 가장 적절한 것은?

```
CREATE TABLE TAB1(COL1 NUMBER, COL2 NUMBER);

INSERT INTO TAB1 VALUES(1,10);
INSERT INTO TAB1 VALUES(2,20);
INSERT INTO TAB1 VALUES(3,30);
COMMIT;

ALTER TABLE TAB1 ADD (COL3 NUMBER);

INSERT INTO TAB1 VALUES(4,40,100);
UPDATE TAB1 SET COL2 = 50 WHERE COL1 = 1;
DELETE TAB1 WHERE COL1 = 3;

ALTER TABLE TAB1 DROP COLUMN COL1;

ROLLBACK;

SELECT SUM(COL2 + COL3) FROM TAB1;
```

- ① 110
- ② 120
- ③ 130
- ④ 140

47. 다음 문장이 차례대로 수행된 이후의 데이터 값으로 가장 적절한 것은?

<TAB1>

COL1	NUMBER
COL2	VARCHAR2(10)
COL3	NUMBER

<TAB1>

COL1	COL2	COL3
1	A	10
2	B	20
3	C	30

```
ALTER TABLE TAB1 ADD COL4 CHAR(5);
ALTER TABLE TAB1 MODIFY COL4 DEFAULT 'AAA';
INSERT INTO TAB1 VALUES(4, 'D', 40, NULL);
INSERT INTO TAB1(COL1, COL2, COL3) VALUES(5, 'E', 50);
```

①

COL1	COL2	COL3	COL4
1	A	10	NULL
2	B	20	NULL
3	C	30	NULL
4	D	40	NULL
5	E	50	NULL

②

COL1	COL2	COL3	COL4
1	A	10	AAA
2	B	20	AAA
3	C	30	AAA
4	D	40	AAA
5	E	50	AAA

③

COL1	COL2	COL3	COL4
1	A	10	NULL
2	B	20	NULL
3	C	30	NULL
4	D	40	NULL
5	E	50	AAA

④

COL1	COL2	COL3	COL4
1	A	10	NULL
2	B	20	NULL
3	C	30	NULL
4	D	40	AAA
5	E	50	AAA

48. 다음 설명 중 가장 적절하지 않은 것은? (단, DBMS는 오라클)

- ① 데이터 타입을 변경할 경우에는 반드시 빈 컬럼이어야 한다.
- ② 컬럼 사이즈는 언제든지 늘릴 수 있다.
- ③ 컬럼 추가 시 DEFAULT값을 선언하면 기존 데이터의 새로운 컬럼 값은 DEFAULT값이 된다.
- ④ 컬럼은 동시에 여러 개를 삭제할 수 없다.

49. 제약조건에 대한 설명 중 가장 적절하지 않은 것은?

- ① PRIMARY KEY는 여러 컬럼으로 구성하여 생성할 수 있다.
- ② UNIQUE 제약조건에는 NULL값을 허용하지 않는다.
- ③ CREATE TABLE AS SELECT문으로 테이블 복제 시 NOT NULL속성은 복제된다.
- ④ FOREIGN KEY는 부모-자식 관계 중 자식 테이블에 생성한다.

50. 다음 중 사용자가 갖는 권한에 대한 설명으로 가장 적절한 것은?

SYSTEM) GRANT SELECT, INSERT ON SCOTT.EMP TO HR WITH GRANT OPTION;
SYSTEM) GRANT CREATE VIEW TO HR WITH ADMIN OPTION;
HR) GRANT SELECT, INSERT ON SCOTT.EMP TO HONG;
HR) GRANT CREATE VIEW TO HONG;
SYSTEM) REVOKE INSERT ON SCOTT.EMP FROM HR;
SYSTEM) REVOKE CREATE VIEW FROM HR;

- ① HR 계정에 부여된 SCOTT.EMP에 대한 SELECT 권한도 함께 회수된다.
- ② HR 유저에게 부여된 CREATE TABLE 권한 회수 시 HONG에게 부여된 CREATE TABLE 권한도 함께 회수되었다.
- ③ HR이 HONG에게 부여한 EMP 테이블의 조회 권한은 SYSTEM 계정에서 직접 회수가 가능하다.
- ④ HR 유저에게 부여된 EMP 테이블 입력 권한 회수 시 HONG에게 부여된 권한도 함께 회수되었다.

3회차

1. 다음 중 아래에서 설명하는 데이터모델의 개념으로 가장 적절한 것은?

학생이라는 엔터티에서 학년이라는 속성 값의 범위는 1~4 사이의 정수이며, 주민번호 속성은 13자리 이내 문자열로 정의할 수 있다.

- ① 도메인
- ② 릴레이션
- ③ 시스템카탈로그
- ④ 속성사전

2. 엔터티 - 인스턴스 - 속성 - 속성값에 대한 관계 설명 중 틀린 것을 고르시오.

- ① 한 개의 엔터티는 두 개 이상의 인스턴스의 집합이어야 한다.
- ② 한 개의 엔터티는 두 개 이상의 속성을 갖는다.
- ③ 하나의 속성은 하나 이상의 속성값을 가진다.
- ④ 하나의 엔터티의 인스턴스는 다른 엔터티의 인스턴스간의 관계인 Paring을 가진다.

3. 속성에 대한 설명으로 가장 적절하지 않은 것은?

- ① 하나의 속성에 여러 개의 값이 있는 다중값일 경우 별도의 엔터티를 이용하여 분리한다.
- ② 정해진 주식별자에 함수적 종속성을 가져야 한다.
- ③ 업무상 인스턴스로 관리하고자 하는 더 이상 분리되지 않는 최소의 데이터 단위를 말한다.
- ④ 엔터티에 속한 속성은 엔터티에 대한 추상적인 값을 갖는다.

4. 엔터티간 1:1, 1:M 과 같이 관계의 기수성을 나타내는 것을 무엇이라 하는가?

- ① 관계명
- ② 관계차수
- ③ 관계선택성
- ④ 관계정의

5. 다음 주식별자에 대한 설명 중 가장 적절하지 않은 것은?

- ① 주식별자에 의해 엔터티 내의 모든 인스턴스들이 유일하게 구분되어야 한다.
- ② 주식별자로 지정되더라도 속성 값으로 NULL이 들어갈 수 있다.
- ③ 주식별자를 구성하는 속성의 수는 유일성을 만족하는 최소의 수가 되어야 한다.
- ④ 지정된 주식별자의 값은 자주 변하지 않는 것이어야 한다.

6. 다음이 설명하는 정규화로 가장 적절한 것은?

테이블의 컬럼이 원자성(한 속성이 하나의 값을 갖는 특성)을 갖도록 테이블을 분해하는 단계

- ① 제 1 정규화
- ② 제 2 정규화
- ③ 제 3 정규화
- ④ 제 4 정규화

7. 다음이 설명하는 관계로 가장 적절한 것은?

두 엔터티나 두 속성 간에 동시에 발생할 수 없는 관계를 의미합니다.
즉, 하나의 엔터티나 속성이 특정한 경우 다른 엔터티나 속성은 해당 경우가 될 수 없음을 나타냅니다.

- ① 상호종속적
- ② 상호포괄적
- ③ 상호배타적
- ④ 상호일관적

8. 다음 중 트랜잭션에 대한 설명으로 가장 적절하지 않은 것은?

- ① 두 엔터티의 관계가 서로 필수적일 때 하나의 트랜잭션을 형성한다.
- ② 두 엔터티가 서로 독립적 수행이 가능하다면 선택적 관계로 표현한다.
- ③ 하나의 트랜잭션은 부분 COMMIT이 가능하다.
- ④ 하나의 트랜잭션에는 여러 SELECT, INSERT, DELETE, UPDATE 등이 포함될 수 있다.

9. NULL에 대한 설명으로 틀린 것은?

- ① 값이 존재하지 않거나 확정되지 않은 값을 의미한다.
- ② NULL과의 비교연산은 FALSE(거짓)를 리턴한다.
- ③ NULL과의 수치연산은 NULL 값을 리턴한다.
- ④ 공백과 같은 ASCII 값을 가진다.

10. 다음이 설명하는 식별자로 가장 적절한 것은?

다른 엔터티 참조 없이 엔터티 내부에서 스스로 생성되는 식별자

- ① 보조식별자
- ② 인조식별자
- ③ 본질식별자
- ④ 내부식별자

11. 다음 중 DBMS 특징이 아닌 것은?

- ① DBMS에 저장된 데이터는 다른 사용자에게 공유될 수 없다.
- ② 데이터 무결성을 유지 할 수 있다.
- ③ 실시간 접근, 자료의 계속적인 변화의 적용에 유리하다.
- ④ 인증된 사용자만이 참조 할 수 있는 보안기능이 제공된다.

12. 테이블 생성 시 주의 할 사항으로 적절하지 않은 것은?

- ① 컬럼 뒤에 데이터 유형은 꼭 지정되어야 한다.
- ② 테이블명과 컬럼명은 숫자로 시작해도 무관하다.
- ③ 테이블 생성시 대소문자 구분은 하지 않는다.
- ④ 소유자가 다를 경우 같은 이름의 테이블을 생성할 수 있다.

13. 다음 설명 중 틀린 하나는?

- ① ORDER BY 절에 SELECT 절에 정의되지 않은 컬럼을 사용할 수 있다.
- ② 테이블 별칭을 선언하면 컬럼 앞의 구분자는 반드시 테이블명 대신 테이블 별칭을 사용한다.
- ③ GROUP BY 절을 사용하는 경우 ORDER BY 절에는 GROUP BY절에 정의되지 않은 컬럼을 사용할 수 있다.
- ④ SELECT 문은 ORDER BY 절이 가장 나중에 실행된다.

14. 다음 중 DISTINCT에 대한 설명으로 가장 적절하지 않은 것은?

- ① DISTINCT 뒤에 나열되는 컬럼들의 중복값을 한 번만 출력하기 위해 사용한다.
- ② SELECT 문에서만 사용 가능하다.
- ③ DISTINCT 뒤에 나열되는 컬럼의 순서에 따라 결과 집합의 수가 달라진다.
- ④ DISTINCT 뒤에 * 를 사용할 수 있다.

15. 다음 SQL 수행 결과로 가장 적절한 것은?

```
SELECT ROUND(TO_DATE('2024-02-20 14:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'MONTH') AS D1,
       TRUNC(TO_DATE('2024-09-12 09:00:00', 'YYYY-MM-DD HH24:MI:SS')) AS D2
FROM DUAL;
```

①

D1	D2
2024-03-01 00:00:00	2024-09-01 00:00:00

②

D1	D2
2025-01-01 00:00:00	2024-09-01 00:00:00

③

D1	D2
2024-03-01 00:00:00	2024-09-12 00:00:00

④

D1	D2
2024-03-20 00:00:00	2024-09-12 00:00:00

16. 다음 함수 사용시 결과값이 올바르지 않은 것은?

- ① CEIL(-12.345) = -13
- ② FLOOR(-12.345) = -13
- ③ MOD(8,3) = 2
- ④ SIGN(0) = 0

17. 다음 함수의 결과로 가장 적절한 것은?

```
SELECT LTRIM('ORACLE', 'A') AS C1,
       SUBSTR('SQL-SERVER', 3, 3) AS C2,
       LENGTH(REPLACE('SQL-SERVER', 'E')) AS C3
FROM DUAL;
```

①

C1	C2	C3
ORCLE	L-S	8

②

C1	C2	C3
ORACLE	L	8

③

C1	C2	C3
ORACLE	L-S	8

④

C1	C2	C3
ORACLE	L-S	2

18. 다음 SQL중 실행 결과가 다른 하나는?

① SELECT DECODE(DEPTNO, 10, DECODE(JOB, 'CLERK', 'A', 'B'), 20, 'C', 'D')
FROM EMP;

② SELECT CASE WHEN (DEPTNO = 10 AND JOB = 'CLERK') THEN 'A' ELSE 'B'
WHEN DEPTNO = 20 THEN 'C' ELSE 'D'
END
FROM EMP;

③ SELECT CASE WHEN DEPTNO = 10 THEN CASE WHEN JOB = 'CLERK' THEN 'A' ELSE 'B' END
WHEN DEPTNO = 20 THEN 'C' ELSE 'D'
END
FROM EMP;

④ SELECT CASE DEPTNO WHEN 10 THEN CASE WHEN JOB = 'CLERK' THEN 'A' ELSE 'B' END
WHEN 20 THEN 'C' ELSE 'D'
END
FROM EMP;

19. 다음 중 정상적으로 실행되지 않는 문장은? (단, DBMS는 오라클)

① SELECT 100 + '1' FROM DUAL;

② SELECT TO_DATE('20240101', 'YYYYMMDD') - 10 FROM DUAL;

③ SELECT TO_DATE('11', 'DD') + 10 FROM DUAL;

④ SELECT NVL(100, 'NULL') FROM DUAL;

20. 다음 SQL 수행 결과로 가장 적절한 것은?

<TAB1>		
COL1	COL2	COL3
A	NULL	100
A	200	300
B	100	NULL
B	300	200
NULL	100	0

SELECT COUNT(COL1) RESULT FROM TAB1 WHERE COL3 < 100 GROUP BY COL1;
 SELECT COUNT(COL1) RESULT FROM TAB1 WHERE COL1 IS NOT NULL GROUP BY COL1
 HAVING SUM(COL2) > 500;

①

RESULT
NULL

②

RESULT
NULL

RESULT
0

RESULT
0

③

RESULT
0

④

RESULT

RESULT

RESULT

21. 아래 SQL의 실행 결과로 알맞은 것은?

<TAB1>		
COL1	COL2	COL3
A	1	NULL
B	2	10
B	NULL	20
NULL	3	30

SELECT SUM(COL2 + COL3) FROM TAB1 WHERE COL1 IS NOT NULL;

① 0

② NULL

③ 12

④ 33

22. 다음 SQL 문장 중 COL1 값이 널(NULL)이 아닌 경우를 찾아내는 문장으로 가장 적절한 것은?

① SELECT * FROM TAB1 WHERE COL1 IS NOT NULL;

② SELECT * FROM TAB1 WHERE COL1 <> NULL;

③ SELECT * FROM TAB1 WHERE COL1 != NULL;

④ SELECT * FROM TAB1 WHERE COL1 NOT NULL;

23. 다음 중 에러가 나지 않는 문장은? (단, DBMS는 오라클)

- ① SELECT COL1 컬럼1, AVG(COL2) 평 균
FROM TAB1;
- ② SELECT COL1 컬럼1, AVG(COL2) 평균
FROM TAB1
GROUP BY COL2;
- ③ SELECT COL1 AS 컬럼1, AVG(COL2) AS 평균
FROM TAB1
WHERE AVG(COL2) >= 100
GROUP BY 컬럼1;
- ④ SELECT COL1 AS 컬럼1, AVG(COL2) AS 평균
FROM TAB1
GROUP BY COL1
HAVING AVG(COL2) >= 100;

24. 다음 수행 결과로 가장 적절한 것은?

<EMP>

EMPNO	NUMBER
ENAME	VARCHAR2(10)
SAL	NUMBER
JUMIN	CHAR(13)
DEPTNO	NUMBER

EMPNO	ENAME	SAL	JUMIN	DEPTNO
1000	SMITH	2500	8012011234567	10
1001	ALLEN	2000	9011072212345	10
1002	FORD	3400	9506232221234	20
1003	SCOTT	3800	9801181112345	20
1004	KING	4000	9908091234432	30

```
SELECT EMPNO
FROM EMP
ORDER BY TO_CHAR(TO_NUMBER(SUBSTR(JUMIN, 3, 2)));
```

①

EMPNO
1003
1002
1004
1001
1000

②

EMPNO
1003
1001
1000
1002
1004

③

EMPNO
1001
1003
1000
1002
1004

④

EMPNO
1000
1001
1002
1003
1004

25. 아래 SQL 수행 결과로 가장 적절한 것은? (단, DBMS는 오라클)

<EMP>		
ENAME	DEPTNO	SAL
SMITH	10	2000
SCOTT	10	1800
FORD	10	3200
KING	20	4000
JAMES	20	5000
ADAMS	20	NULL


```

SELECT ENAME, DEPTNO, SAL
  FROM EMP
 ORDER BY DEPTNO, SAL DESC;

```

①

ENAME	DEPTNO	SAL
SCOTT	10	1800
SMITH	10	2000
FORD	10	3200
KING	20	4000
JAMES	20	5000
ADAMS	20	NULL

②

ENAME	DEPTNO	SAL	ENAME
SCOTT	10	1800	SCOTT
SMITH	10	2000	SMITH
FORD	10	3200	FORD
ADAMS	20	NULL	ADAMS
KING	20	4000	KING
JAMES	20	5000	JAMES

③

ENAME	DEPTNO	SAL
FORD	10	3200
SMITH	10	2000
SCOTT	10	1800
ADAMS	20	NULL
JAMES	20	5000
KING	20	4000

④

ENAME	DEPTNO	SAL	ENAME
JAMES	20	5000	JAMES
KING	20	4000	KING
ADAMS	20	NULL	ADAMS
FORD	10	3200	FORD
SMITH	10	2000	SMITH
SCOTT	10	1800	SCOTT

26. 아래 SQL 수행 결과로 가장 적절한 것은?

<TAB1>	
COL1	COL2
1	A
2	B
3	C
4	D

<TAB2>	
COL1	COL2
1	A
2	B
2	B
4	C
5	C


```

SELECT COUNT(TAB1.COL1) AS CNT
  FROM TAB1 LEFT OUTER JOIN TAB2
    ON TAB1.COL2 = TAB2.COL2
   AND TAB1.COL1 = TAB2.COL1;

```

- ① 3
- ② 4
- ③ 5
- ④ 6

27. 다음 FROM 절의 JOIN 형태에 대한 설명 중 옳바르지 못한 것은?

- ① INNER JOIN은 WHERE 절에서 사용하던 JOIN 조건을 FROM 절에서 정의하겠다는 표시이다.
- ② INNER JOIN 사용 시, USING 조건절이나 ON 조건절을 반드시 사용해야 한다.
- ③ RIGHT OUTER JOIN 결과와 LEFT OUTER JOIN 결과는 항상 다르다.
- ④ RIGHT OUTER JOIN, LEFT OUTER JOIN에서 OUTER는 생략 가능하다.

28. 다음 중 SELECT절에 사용하는 서브쿼리인 스칼라 서브쿼리에 대한 설명으로 가장 적절하지 않은 것은?

- ① 하나의 로우에 해당하는 스칼라 서브쿼리 결과 건수는 1건 이하여야 한다.
- ② 하나의 로우에 해당하는 스칼라 서브쿼리 결과가 0건이면 생략된다.
- ③ 스칼라 서브쿼리는 반드시 한 컬럼만 출력이 가능하다.
- ④ 메인쿼리와 스칼라 서브쿼리의 연결 조건이 필요하다면 반드시 스칼라 서브쿼리에 정의해야 한다.

29. 다음 서브쿼리 결과로 가장 적절한 것은?

<EMP>		
NAME	HIREDATE	SAL
TURNER	1981/09/08	1500
ADAMS	1987/05/23	1100
JAMES	1981/10/03	1000
FORD	1981/12/03	3000
MILLER	1982/01/23	1300


```

SELECT SUM(SAL)
  FROM EMP
 WHERE HIREDATE > (SELECT HIREDATE
                   FROM EMP
                   WHERE NAME = 'JAMES');

```

- ① 3800
- ② 4100
- ③ 5400
- ④ 6900

30. 다음 서브쿼리 결과로 가장 적절한 것은?

<EMPLOYEES>			
NAME	HIREDATE	SAL	DEPTNO
TURNER	1981/09/08	1500	10
ADAMS	1987/05/23	1100	10
JAMES	1981/10/03	1000	20
FORD	1981/12/03	3000	20
MILLER	1982/01/23	3000	20

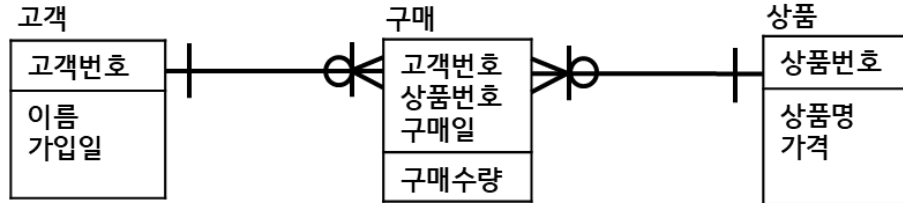

```

SELECT SUM(SAL)
  FROM EMPLOYEES
 WHERE (DEPTNO, SAL) IN (SELECT DEPTNO, MAX(SAL)
                        FROM EMPLOYEES
                        GROUP BY DEPTNO);

```

- ① 1500
- ② 3000
- ③ 4500
- ④ 7500

31. 다음 ERD를 보고 고객별로 가장 최근에 구매한 상품명(다수가능)을 출력하는 SQL로 가장 적절한 것은?



- ① SELECT G.고객번호, G.구매일, P.상품명
 FROM 구매 G, 상품 P, (SELECT 고객번호, MAX(구매일) 최근구매일
 FROM 구매
 GROUP BY 고객번호) I
 WHERE G.고객번호 = I.고객번호
 AND G.구매일 = I.최근구매일
 AND G.상품번호 = P.상품번호;
- ② SELECT G.고객번호, G.구매일, P.상품명
 FROM 고객 G, 상품 P, (SELECT 고객번호, MAX(구매일) 최근구매일
 FROM 구매
 GROUP BY 고객번호) I
 WHERE G.고객번호 = I.고객번호
 AND G.상품번호 = P.상품번호;
- ③ SELECT C.이름, G.구매일, P.상품명
 FROM 고객 C, 구매 G, 상품 P, (SELECT 고객번호, MIN(구매일) 최근구매일
 FROM 구매
 GROUP BY 고객번호) I
 WHERE G.고객번호 = I.고객번호
 AND G.구매일 = I.최근구매일
 AND G.상품번호 = P.상품번호
 AND C.고객번호 = G.고객번호;
- ④ SELECT C.이름, G.구매일, P.상품명
 FROM 고객 C, 구매 G, 상품 P
 WHERE G.상품번호 = P.상품번호
 AND C.고객번호 = G.고객번호;

32. 다음 쿼리의 수행 결과로 적절한 것은?

<TAB1>			<TAB2>	
NAME	CODE	FARE	CODE	STATUS
AAA	0001	500	0001	CLOSED
BBB	0001	100	0002	OPEN
CCC	0002	300	0003	CLOSED
DDD	0004	200	0004	OPEN


```

DELETE FROM TAB1
WHERE CODE IN (SELECT CODE
                FROM TAB2
                WHERE STATUS = 'OPEN');

SELECT SUM(FARE)
FROM TAB1;

```

- ① 100
- ② 400
- ③ 600
- ④ 800

33. 다음 수행 결과로 가장 적절한 것은?

<TAB1>		<TAB2>	
COL1	COL2	CODE	FARE
A	100	0001	250
B	200	0002	350
C	300		
D	400		


```

SELECT SUM(COL2)
FROM TAB1
WHERE COL2 < ANY(SELECT FARE
                  FROM TAB2);

```

- ① 100
- ② 300
- ③ 600
- ④ 1000

34. 다음 집합 연산자에 대한 설명 중 틀린 것은 무엇인가? (단, DBMS는 오라클)

- ① UNION 연산자는 조회 결과에 대한 합집합을 나타내며 정렬된 결과를 출력해준다.
- ② UNION ALL 연산자는 조회 결과를 정렬하고 중복되는 데이터를 한 번만 표현한다.
- ③ INTERSECT 연산자는 조회 결과에 대한 교집합을 의미한다.
- ④ MINUS 연산자는 조회 결과에 대한 차집합을 의미한다.

35. 아래 쿼리 결과와 같은 결과를 갖는 빈칸에 들어갈 문장으로 가장 적절한 것은?

```
SELECT DEPTNO, SUM(SAL) AS SUM_SAL
  FROM EMP
 GROUP BY DEPTNO
 UNION ALL
SELECT NULL DEPTNO, SUM(SAL) AS SUM_SAL
  FROM EMP;

SELECT DEPTNO, SUM(SAL) AS SUM_SAL
  FROM EMP
 GROUP BY _____;
```

- ① ROLLUP(DEPTNO)
- ② ROLLUP(SAL)
- ③ ROLLUP(DEPTNO, SAL)
- ④ ROLLUP(DEPTNO, ())

36. 순위관련 WINDOW 함수에 대한 설명 중 가장 적절하지 않은 것은?

- ① RANK함수는 동일한 값에 대해서는 동일한 순위를 부여한다.
- ② DENSE_RANK 함수는 RANK 함수처럼 동일한 값에 대해 동일한 순위를 부여하나, 동순위가 여러 존재하더라도 다음 순위가 이어진다.
- ③ PERCENT_RANK 함수는 각 값의 누적된 순위를 부여할 수 있다.
- ④ RANK 함수가 동일한 값에 대해서는 동일한 순위를 부여하는데 반해, ROW_NUMBER 함수는 고유한 순위를 부여한다.

37. 다음 출력 결과를 갖도록 하는 빈칸의 문장으로 가장 적절한 것은?

<EMP>

EMPNO	ENAME	DEPTNO	SAL
7934	MILLER	10	1300
7782	CLARK	10	2450
7839	KING	10	5000
7369	SMITH	20	800
7876	ADAMS	20	1100
7566	JONES	20	2975
7788	SCOTT	20	3000
7902	FORD	20	3000

<RESULT>

EMPNO	ENAME	DEPTNO	SAL	RESULT
7934	MILLER	10	1300	3750
7782	CLARK	10	2450	8750
7839	KING	10	5000	7450
7369	SMITH	20	800	1900
7876	ADAMS	20	1100	4875
7566	JONES	20	2975	7075
7788	SCOTT	20	3000	8975
7902	FORD	20	3000	6000

```
SELECT EMPNO, ENAME, DEPTNO, SAL,
       SUM(SAL) OVER(PARTITION BY DEPTNO ORDER BY SAL
                     _____) AS RESULT
FROM EMP;
```

- ① RANGE BETWEEN UNBOUNDED PRECEDING AND 1 FOLLOWING
- ② RANGE BETWEEN 1 PRECEDING AND 1 FOLLOWING
- ③ ROWS BETWEEN UNBOUNDED PRECEDING AND 1 FOLLOWING
- ④ ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

38. 다음 중 Top N Query에 대한 설명 중 틀린 것은?

- ① 윈도우 함수를 사용하여 상위 N개에 대한 값을 추출할 수 있으나 단일 Query로 표현 불가하다.
- ② ROWNUM을 사용한 방식은 ROWNUM 할당 전에 먼저 순서대로 데이터를 정렬한 뒤 ROWNUM을 부여 후 추출하는 것이 좋다.
- ③ FETCH 절을 사용하면 단일 Query로도 정렬 순서대로의 상위 N개에 대한 값을 추출할 수 있다.
- ④ SQL-Server의 TOP(N) 쿼리를 사용하면 정렬 순서대로 상위 N개 데이터만 출력 가능하다.

39. 아래 실행 결과를 출력하는 SQL로 가장 적절한 것은?

<사원>

사원번호	이름	상위관리자코드	지역
1000	홍길동	NULL	서울
1001	박길동	1000	경기
1002	최길동	1001	경기
1003	이길동	1001	인천
1004	구길동	1002	서울
1005	안길동	1003	서울
1006	송길동	1000	경기
1007	강길동	1006	경기
1008	공길동	1006	인천

```
SELECT 사원번호, 이름, LEVEL
      FROM 사원
     WHERE 지역 = '경기'
    START WITH 상위관리자코드 IS NULL
   CONNECT BY 상위관리자코드 = PRIOR 사원번호;
```

①

사원번호	이름	LEVEL
1001	박길동	2
1002	최길동	3
1006	송길동	2
1007	강길동	3

②

사원번호	이름	LEVEL
1000	홍길동	1
1001	박길동	2
1002	최길동	3
1006	송길동	2
1007	강길동	3

③ 공집합

④

사원번호	이름	LEVEL
1000	홍길동	1

40. 다음 수행 결과로 가장 적절한 것은?

<DEPARTMENT>

DEPTNO	DNAME	PART	BUILD
101	컴퓨터공학과	100	정보관
102	멀티미디어공학과	100	멀티미디어관
103	소프트웨어공학과	100	소프트웨어관
201	전자공학과	200	전자제어관
202	기계공학과	200	기계실험관
203	화학공학과	200	화학실습관
301	문헌정보학과	300	인문관
100	컴퓨터정보학부	10	
200	메카트로닉스학부	10	
300	인문사회학부	20	
10	공과대학		
20	인문대학		

```
SELECT DEPTNO, DNAME, LEVEL, CONNECT_BY_ROOT(DNAME) AS ROOT
  FROM DEPARTMENT
 START WITH PART IS NULL
CONNECT BY PRIOR PART = DEPTNO;
```

① 공집합

②

DEPTNO	DNAME	LEVEL	ROOT
10	공과대학	1	공과대학
20	인문대학	1	인문대학

③

DEPTNO	DNAME	LEVEL	ROOT
10	공과대학	1	공과대학
100	컴퓨터정보학부	2	공과대학
101	컴퓨터공학과	3	공과대학
102	멀티미디어공학과	3	공과대학
103	소프트웨어공학과	3	공과대학
200	메카트로닉스학부	2	공과대학
201	전자공학과	3	공과대학
202	기계공학과	3	공과대학
203	화학공학과	3	공과대학

④

DEPTNO	DNAME	LEVEL	ROOT
10	공과대학	1	공과대학
100	컴퓨터정보학부	2	공과대학
101	컴퓨터공학과	3	공과대학
102	멀티미디어공학과	3	공과대학
103	소프트웨어공학과	3	공과대학
200	메카트로닉스학부	2	공과대학
201	전자공학과	3	공과대학
202	기계공학과	3	공과대학
203	화학공학과	3	공과대학
20	인문대학	1	인문대학
300	인문사회학부	2	인문대학
301	문헌정보학과	3	인문대학

41. 다음 SQL의 실행 결과를 얻기 위한 빈칸에 들어갈 값으로 가장 적절한 것은?

<TAB1>		
성별	2023	2024
남자	10	20
여자	30	40
<결과>		
성별	연도	판매량
남자	2023	10
남자	2024	20
여자	2023	30
여자	2024	40
SELECT * FROM TAB1 UNPIVOT (____ FOR ____ IN ("2023", "2024"));		

- ① 판매량, 연도
- ② 성별, 연도
- ③ 성별, 판매량
- ④ 판매량, 판매량

42. 다음 SQL 실행 결과로 알맞은 것은?

```
SELECT LENGTH(REGEXP_REPLACE('REGEXP \. ESCAPE CHARACTER, A|B : A OR B', '[A-z|0-9\.\ ]'))
FROM DUAL;
```

- ① 2
- ② 3
- ③ 4
- ④ 5

43. DML에 대한 설명으로 가장 적절한 것은?

- ① DELETE 사용 시 FROM 문구는 생략이 불가능하다.
- ② 일부 데이터 DELETE 시 WHERE 절은 반드시 붙이지 않아도 된다.
- ③ 반드시 COMMIT 또는 ROLLBACK을 수행하여 TRANSACTION을 종료해야 한다.
- ④ UPDATE 사용 시 동시에 여러 컬럼 수정은 불가능하다.

44. COMMIT 이후의 데이터 상태로 옳지 않은 것은?

- ① 데이터에 대한 변경 사항이 데이터베이스에 영구 저장된다.
- ② 변경된 행에 대한 잠금이 풀리고 다른 사용자들이 행을 조작할 수 있다.
- ③ 이전 데이터는 영원히 되돌릴 수 없다.
- ④ COMMIT을 수행한 사용자만 결과를 볼 수 있다.

45. 컬럼 변경 시 주의 사항으로 옳지 않은 것은?

- ① 컬럼의 크기를 늘릴 수는 있지만 줄일 수는 없다.
- ② 컬럼이 NULL 값만 가지고 있으면 데이터 유형을 변경할 수 있다.
- ③ 컬럼에 NULL 값이 없을 경우에만 NOT NULL 제약조건을 추가할 수 있다.
- ④ 컬럼의 DEFAULT 값을 바꾸면 변경 작업 이후 발생하는 행 삽입에만 영향을 미친다.

46. 제약조건의 설명 중 가장 적절하지 않은 것은?

- ① 기본키는 테이블에 저장된 행 데이터를 고유하게 식별하기 위한 키이다.
- ② 테이블에 저장된 행 데이터를 고유하게 식별하기 위해 고유키를 정의한다.
- ③ 기본키는 고유키와 외래키 제약을 합쳐놓은 것이다.
- ④ NOT NULL 은 NULL 값의 삽입을 금지한다.

47. 아래와 같이 테이블 및 데이터가 생성된 경우 추가 실행이 불가능한 문장은?

(단, 보기 순서대로 실행됨을 가정)

```
CREATE TABLE TAB1(COL1 NUMBER, COL2 NUMBER);
INSERT INTO TAB1 VALUES(100, 100);
COMMIT;
```

- ① ALTER TABLE TAB1 ADD (COL3 NUMBER, COL4 VARCHAR2(10));
- ② ALTER TABLE TAB1 ADD COL5 NUMBER NOT NULL;
- ③ ALTER TABLE TAB1 MODIFY COL2 DEFAULT 100 NOT NULL;
- ④ ALTER TABLE TAB1 DROP COLUMN COL4;

48. 다음 문장의 수행 후 TAB2의 조회 결과로 가장 적절한 것은?

<TAB1>		<TAB2>		
NO	NAME	NO	NAME	CLASS_NO
1	A	1000	SMITH	1
2	B	1001	ALLEN	2
3	C	1002	FORD	3
4	D	1004	SCOTT	3

ALTER TABLE TAB2 ADD FOREIGN KEY(CLASS_NO) REFERENCES TAB1(NO) ON DELETE SET NULL;
DELETE FROM TAB1 WHERE NAME = 'C';

① DELETE 실행 오류

②

NO	NAME	CLASS_NO
1000	SMITH	1
1001	ALLEN	2

③

NO	NAME	CLASS_NO
1000	SMITH	1
1001	ALLEN	2
1002	FORD	NULL
1004	SCOTT	NULL

④

NO	NAME	CLASS_NO
1000	SMITH	1
1001	ALLEN	2
1002	FORD	3
1004	SCOTT	3

49. 다음 설명 중 가장 적절하지 않은 것은?

- ① 외래키를 생성 한 경우 부모 테이블의 참조키 컬럼을 삭제할 수 없다.
- ② 제약 조건 추가 시 제약조건 이름을 명시하지 않을 수 있다.
- ③ 이미 존재하는 컬럼에 대해 NOT NULL 제약조건 추가 시 반드시 MODIFY로 처리한다.
- ④ NULL값이 삽입되어 있는 경우 UNIQUE 제약조건을 추가할 수 없다.

50. 권한에 대한 설명으로 가장 적절한 것은?

- ① 권한은 테이블 소유자만이 부여할 수 있다.
- ② 테이블에 대한 조회 권한 부여 시 즉시 반영되지 않고 재접속을 해야 조회가 가능하다.
- ③ 롤에 있는 권한을 회수한 이후 롤을 부여받은 유저는 해당 권한을 갖지 않게 된다.
- ④ WITH ADMIN OPTION을 통해 부여받은 테이블 조회 권한을 다른 유저에게 부여할 수 있다.

4회차

1. 다음이 설명하는 모델링의 특징으로 가장 적절한 것은?

누구나 이해하기 쉽게 하기 위해 대상에 대한 애매모호함을 제거하고 정확하게 현상을 기술하는 것을 의미한다.

- ① 명확화 ② 추상화 ③ 단순화 ④ 그룹화

2. 엔터티 분류 중 유형과 무형에 따른 분류가 아닌 것은?

- ① 유형엔터티
② 개념엔터티
③ 사건엔터티
④ 행위엔터티

3. 다음 중 속성에 대한 설명 중 가장 적절하지 않은 것은?

- ① 기본속성에 대한 예를 들자면 입사날짜이다.
② 설계속성은 업무상 필요한 데이터 외에 업무를 규칙화하기 위하여 기본속성을 변형하는 속성이다.
③ 파생속성은 다른 속성에 영향을 받아 발생하는 속성이다.
④ 파생속성은 가급적 많이 정의할수록 좋은 속성이다.

4. 다음 중 관계를 구성하는 요소가 아닌 것은?

- ① 관계명
② 차수(Cardinality)
③ 선택성(Optionality)
④ 관계정의

5. 다음 중 아래 개념이 설명하는 관계로 가장 적절한 것은?

부모 엔터티의 주식별자를 상속받아 자식 엔터티에서 외부식별자이면서 주식별자로 사용하는 관계

- ① 식별관계
② 비식별관계
③ 필수관계
④ 선택관계

6. 다음 엔터티는 어떤 정규화를 위배한 것인가? (단, 고객번호+상품명이 PK임)

<이용이력>			
고객번호	상품명	가격	이용날짜
1000	A	1000	2024/01/01
1000	B	1500	2024/01/31
1001	A	1000	2024/02/15
1001	C	2000	2024/03/20

- ① 제 1 정규화
- ② 제 2 정규화
- ③ 제 3 정규화
- ④ 제 4 정규화

7. 관계(Relationship)와 조인(Join)에 대한 설명으로 가장 적절하지 않은 것은?

- ① 관계란 서로 다른 엔터티의 인스턴스들 간의 존재하는 논리적인 연관성을 의미한다.
- ② 정규화를 거쳐 분리된 엔터티는 서로 관계를 맺지 않아도 된다.
- ③ 관계를 맺는 엔터티를 다시 연결하는 과정을 조인이라고 한다.
- ④ 행위 관계는 엔터티 간의 어떤 행위가 있는 것을 의미한다.

8. 트랜잭션의 특징 중 옳지 않은 것은?

- ① 일관성(consistency)
- ② 원자성(atomicity)
- ③ 지속성(durability)
- ④ 중복성(duplication)

9. NULL에 대한 설명으로 틀린 것은?

- ① COUNT는 NULL을 세지 않는다.
- ② NULL + 100은 NULL을 리턴한다.
- ③ NULL이 포함된 컬럼의 SUM 값은 항상 NULL이다.
- ④ NULL과의 비교 연산은 FALSE를 리턴한다.

10. 다음의 주식별자가 나타내는 특징을 가장 잘 설명한 것은?

주식별자를 구성하는 속성 중에서 유일성을 만족하는 최소한의 속성으로 구성한다.

- ① 유일성
- ② 최소성
- ③ 단일성
- ④ 존재성

11. 테이블에 대한 설명 중 가장 적절한 것은?

- ① 같은 테이블을 동시에 두 계정으로 소유할 수 있다.
- ② 테이블명은 중복될 수 없지만, 소유자가 다른 경우 같은 이름으로 생성 가능하다.
- ③ 하나의 행의 하나의 컬럼에는 둘 이상의 값이 삽입될 수 있다.
- ④ 테이블 생성 시 각 컬럼의 데이터 유형을 정의할 수 있고, 생성 이후에는 변경이 불가하다.

12. 관계형 데이터베이스에 대한 특징 중 가장 적절하지 않은 것은?

- ① 데이터의 무결성을 보장할 수 있다.
- ② 데이터를 분류, 정렬, 탐색하는 속도가 빠르다.
- ③ 기존의 작성된 스키마를 수정하기 어렵다.
- ④ 데이터베이스의 부하를 분석하기 쉽다.

13. 다음 중 SELECT문에 대한 설명으로 가장 적절하지 않은 것은? (단, DBMS는 오라클)

- ① 실행 순서는 FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY 순 이다.
- ② FROM 절은 생략 불가하다.
- ③ GROUP BY 절은 NULL 그룹을 출력하지 않는다.
- ④ 일반적으로 SELECT 절에서는 * 와 컬럼명을 동시에 사용할 수 없다.

14. 다음 중 오류가 발생하는 문장으로 가장 적절한 것은?

- ① SELECT COL1, T.COL2, T.COL3
FROM TAB1 T
WHERE COL1 >= 100;
- ② SELECT T.COL2, SUM(T.COL1)
FROM TAB1 T
WHERE COL1 >= 100
GROUP BY COL2;
- ③ SELECT T.COL2, SUM(T.COL1) AS SUM_VALUE
FROM TAB1 T
GROUP BY T.COL2
ORDER BY SUM_VALUE, T.COL2;
- ④ SELECT T.COL2, SUM(T.COL1), SUM(T.COL3)
FROM TAB1 T
GROUP BY T.COL2
ORDER BY T.COL3;

15. 다음 함수의 출력 결과로 가장 적절하지 않은 것은?

- ① DATE_FORMAT('2024-01-01', '%Y-%m-%d') : 2024-01-01
- ② FLOOR(3.5) = 3
- ③ POWER(3,3) = 9
- ④ LAST_DAY(TO_DATE('2024-01-01', 'YYYY-MM-DD')) : 2024-01-31

16. 다음 SQL문의 실행 결과로 가장 적절한 것은?

<EMP>

EMPNO	DEPTNO	JOB
0001	10	CLERK
0002	10	MANAGER
0003	20	CLERK
0004	30	ANALYST

```
SELECT DECODE(DEPTNO, 10, DECODE(JOB, 'CLERK', 'A', 'B'), 'C') AS RESULT
FROM EMP
ORDER BY EMPNO;
```

①

RESULT
A
B
C
C

②

RESULT
A
B
A
C

③

RESULT
A
A
B
B

④

RESULT
A
A
B
C

17. 아래 SQL의 결과로 가장 적절한 것은?

```
SELECT ROUND(TO_DATE('2024-08-24 12:00:00','YYYY-MM-DD HH24:MI:SS')) COL1,
       ROUND(TO_DATE('2024-08-24 12:00:01','YYYY-MM-DD HH:MI:SS')) COL2
FROM DUAL;
```

①

COL1	COL2
2024-08-24	2024-08-25

②

COL1	COL2
2024-08-25	2024-08-25

③

COL1	COL2
2024-08-25	2024-08-26

④

COL1	COL2
2024-08-26	2024-08-26

18. 다음 SQL의 실행 결과에 대한 해석으로 가장 적절한 것은?

```
SELECT NEXT_DAY(ADD_MONTHS(HIREDATE, 6), '월요일')
FROM EMP;
```

- ① 각 직원의 입사날짜로부터 6일 후 첫 번째 월요일에 해당하는 날짜
- ② 각 직원의 입사날짜로부터 6일 후 두 번째 월요일에 해당하는 날짜
- ③ 각 직원의 입사날짜로부터 6개월 후 첫 번째 월요일에 해당하는 날짜
- ④ 각 직원의 입사날짜로부터 6개월 후 두 번째 월요일에 해당하는 날짜

19. 다음 SQL 실행 결과로 가장 적절한 것은?

<TAB1>

COL1	COL2
NULL	100
100	200
200	100

```
SELECT SUM(NULLIF(COL1, 100) + COL2) FROM TAB1;
```

- ① 300
- ② 400
- ③ 500
- ④ 600

20. 아래 SQL 실행 결과로 가장 적절한 것은?

<TAB1>

COL1	COL2
A	1
B	2
C	3
D	

<TAB2>

COL1	COL2
A	2
B	3
C	

```
SELECT COUNT(COL1)
  FROM TAB1
 WHERE NOT EXISTS(SELECT 'X'
                   FROM TAB2
                   WHERE TAB1.COL2 = TAB2.COL2);
```

- ① 1
- ② 2
- ③ 3
- ④ 4

21. 아래 SQL 실행 결과로 가장 적절한 것은?

<TAB1>

COL1	COL2	COL3
A	1000	10
B	1200	10
C	3000	20
D	2000	30
E	4000	20
F	3600	10

```
SELECT COUNT(COL1)
  FROM TAB1
 WHERE COL2 NOT BETWEEN 2000 AND 3000
       AND COL3 NOT IN (10, 20);
```

- ① NULL
- ② 0
- ③ 1
- ④ 2

22. 아래 SQL 실행 결과로 가장 적절한 것은?

<TAB1>

COL1	COL2	COL3
100	A	10
200	A	20
300	A	30
0	B	10
400	NULL	20
500	B	30
300	B	40

```
SELECT COL2, SUM(COL3) AS C1, MIN(COL3) AS C2, MAX(COL3) AS C3
FROM TAB1
WHERE COL1 > 100
GROUP BY COL2;
```

①

COL2	C1	C2	C3
NULL	20	20	20
A	50	20	30
B	70	30	40

②

COL2	C1	C2	C3
NULL	NULL	NULL	NULL
A	60	10	30
B	80	10	40

③

COL2	C1	C2	C3
A	50	20	30
B	70	30	40

④

COL2	C1	C2	C3
A	60	10	30
B	80	10	40

23. 다음의 정렬 결과로 가장 적절한 것은?

<TAB1>

ID
AA
ABC
BI
B
BAA

```
SELECT ID  
  FROM TAB1  
 ORDER BY CASE WHEN ID > 'B' THEN 'A' ELSE ID END, ID;
```

①

ID
AA
ABC
B
BI
BAA

②

ID
B
AA
BI
ABC
BAA

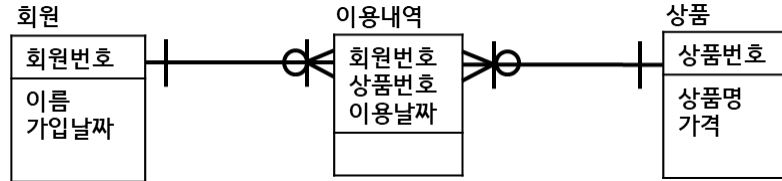
③

ID
BAA
BI
AA
ABC
B

④

ID
BI
BAA
AA
ABC
B

24. 다음 ERD를 참고하여 모든 회원의 상품 이용횟수와 총 이용가격을 출력하는 SQL로 가장 적절한 것은?



- ① SELECT 회원.회원번호, COUNT(상품.상품번호), SUM(상품.가격)
FROM 회원, 이용내역, 상품
WHERE 회원.회원번호 = 이용내역.회원번호
AND 이용내역.상품번호 = 상품.상품번호
GROUP BY 회원.회원번호;
- ② SELECT 회원.회원번호, COUNT(상품.상품번호), SUM(상품.가격)
FROM 회원 LEFT OUTER JOIN 이용내역
ON 회원.회원번호 = 이용내역.회원번호 JOIN 상품
ON 이용내역.상품번호 = 상품.상품번호
GROUP BY 회원.회원번호;
- ③ SELECT 회원.회원번호, COUNT(이용내역.상품번호), SUM(상품.가격)
FROM 회원 LEFT OUTER JOIN 이용내역
ON 회원.회원번호 = 이용내역.회원번호 LEFT OUTER JOIN 상품
ON 이용내역.상품번호 = 상품.상품번호
GROUP BY 회원.회원번호;
- ④ SELECT 회원.회원번호, COUNT(상품.상품번호), SUM(상품.가격)
FROM 회원, 이용내역, 상품
WHERE 회원.회원번호 = 이용내역.회원번호(+)
AND 이용내역.상품번호 = 상품.상품번호
GROUP BY 회원.회원번호;

25. 다음의 오라클 표준을 ANSI로 가장 잘 표현한 것은?

```

SELECT E.ENAME, E.SAL, E.DEPTNO, D.DNAME
FROM EMP E, DEPT D
WHERE E.DEPTNO = D.DEPTNO
AND E.SAL > 3000
ORDER BY E.ENAME;
  
```

- ① SELECT E.ENAME, E.SAL, E.DEPTNO, D.DNAME
FROM EMP E JOIN DEPT D
ON E.DEPTNO = D.DEPTNO
AND E.SAL > 3000
ORDER BY E.ENAME;
- ② SELECT E.ENAME, E.SAL, E.DEPTNO, D.DNAME
FROM EMP E INNER JOIN DEPT D
USING DEPTNO
AND E.SAL > 3000
ORDER BY E.ENAME;

- ③ SELECT E.ENAME, E.SAL, E.DEPTNO, D.DNAME
FROM EMP E INNER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO
WHERE E.SAL > 3000
ORDER BY E.ENAME;
- ④ SELECT E.ENAME, E.SAL, E.DEPTNO, D.DNAME
FROM EMP E LEFT JOIN DEPT D
ON E.DEPTNO = D.DEPTNO
WHERE E.SAL > 3000
ORDER BY E.ENAME;

26. 다음 중 실행 오류가 발생하는 조인 문법은?

- ① SELECT *
FROM TAB1 LEFT JOIN TAB2
ON TAB1.COL1 = TAB2.COL1;
- ② SELECT *
FROM TAB1 JOIN TAB2
ON (TAB1.COL1 = TAB2.COL1);
- ③ SELECT *
FROM TAB1 INNER JOIN TAB2
USING (COL1);
- ④ SELECT *
FROM TAB1 NATURAL JOIN TAB2
USING (COL1);

27. 서브쿼리에 대한 설명으로 가장 적절한 것은?

- ① FROM 절에 사용하는 서브쿼리를 스칼라 서브쿼리라고 한다.
- ② 단일행 서브쿼리는 비교 연산자 사용이 불가하다.
- ③ 서브쿼리가 메인쿼리 컬럼을 가지고 있을 경우 비연관 서브쿼리라고 한다.
- ④ 연관 서브쿼리는 메인쿼리가 먼저 수행된 후에 서브쿼리에서 조건이 맞는지 확인할 때 주로 사용한다.

28. 다음 출력 결과로 가장 알맞은 것은?

<TAB1>		<TAB2>		<TAB3>	
COL1	COL2	COL1	COL2	COL1	COL2
A	10	10	AAA	A	100
B	20	20	BBB	B	200
C	20	30	CCC	C	300
D	NULL			C	400


```

SELECT COL1, COL2,
       (SELECT COL2 FROM TAB2 WHERE TAB1.COL2 = TAB2.COL1) AS RESULT1,
       (SELECT MAX(COL2) FROM TAB3 WHERE TAB1.COL1 = TAB3.COL1) AS RESULT2
FROM TAB1;

```

①

COL1	COL2	RESULT1	RESULT2
A	10	AAA	100
B	20	BBB	200
C	20	BBB	400

②

COL1	COL2	RESULT1	RESULT2
A	10	AAA	100
B	20	BBB	200
C	20	BBB	400
D	NULL	0	0

③

COL1	COL2	RESULT1	RESULT2
A	10	AAA	100
B	20	BBB	200
C	20	BBB	400
D	NULL	NULL	NULL

④

COL1	COL2	RESULT1	RESULT2
A	10	AAA	100
B	20	BBB	200
C	20	BBB	400
D	NULL	NULL	0

29. 아래 SQL 중 결과가 다른 하나는?

- ① SELECT P.CODE, P.NAME, P.PRICE
FROM PRODUCT P, (SELECT CODE, MAX(PRICE) AS MAX_PRICE
FROM PRODUCT
GROUP BY CODE) I
WHERE P.CODE = I.CODE
AND P.PRICE = I.MAX_PRICE;
- ② SELECT CODE, NAME, PRICE
FROM PRODUCT
WHERE (CODE, PRICE) IN (SELECT CODE, MAX(PRICE)
FROM PRODUCT
GROUP BY CODE);
- ③ SELECT CODE, NAME, PRICE
FROM PRODUCT P1
WHERE PRICE = (SELECT MAX(PRICE)
FROM PRODUCT P2
WHERE P1.CODE = P2.CODE);
- ④ SELECT CODE, NAME, (SELECT PRICE
FROM PRODUCT
WHERE P1.CODE = P2.CODE)
FROM PRODUCT P1;

30. 아래 SQL에 대한 실행 결과로 가장 알맞은 것은?

<TAB1>		<TAB2>	
COL1	COL2	COL1	COL2
A	10	10	100
B	20	10	200
C	20	20	300
D	40		


```

SELECT SUM(TAB2.COL2)
  FROM TAB1, TAB2
 WHERE TAB1.COL2 = TAB2.COL1;

```

- ① 600
② 700
③ 800
④ 900

31. 다음 SQL 수행 결과로 가장 적절한 것은?

<EMP>		
ENAME	HIREDATE	SAL
CLARK	1981/06/09	1000
TURNER	1981/09/08	2000
KING	1981/11/17	3000
SCOTT	1987/04/19	4000


```

SELECT SUM(E2.SAL) AS RESULT
  FROM EMP E1, EMP E2
 WHERE E1.HIREDATE >= E2.HIREDATE
 GROUP BY E1.ENAME;

```

①	②	③	④
RESULT	RESULT	RESULT	RESULT
1000	3000	1000	10000
2000	6000	3000	6000
3000	10000	6000	3000
4000		10000	1000

32. 다음 수행 결과로 가장 적절한 것은?

<PRODUCT>		<PROMOTION>	
NAME	CODE	CODE	FARE
A	0001	0001	100
B	0002	0002	150
C	0003	0003	200
D	NULL	0004	250


```

SELECT NAME, NVL((SELECT FARE
                   FROM PROMOTION P2
                  WHERE P1.CODE = P2.CODE), 100) PRO_FARE
  FROM PRODUCT P1;

```

①	②
NAME	NAME
PRO_FARE	PRO_FARE
A	A
100	100
B	B
150	150
C	C
200	200
	D
	NULL

③

NAME	PRO_FARE
A	100
B	150
C	200
D	100

④

NAME	PRO_FARE
A	100
B	150
C	200
D	0

33. 집합 연산자 사용시 주의사항에 대한 설명으로 가장 적절하지 않은 것은?

- ① 두 집합의 컬럼 수가 일치해야 한다.
- ② 두 집합의 각 컬럼의 데이터 유형이 일치해야 한다.
- ③ 두 집합 중 위의 집합의 컬럼명을 전체 집합의 컬럼명으로 가져간다.
- ④ 위 집합의 컬럼의 사이즈보다 아래 집합의 컬럼 사이즈가 큰 경우 연산이 불가하다.

34. 아래 쿼리 결과와 같은 결과를 갖는 빈칸에 들어갈 문장으로 가장 적절한 것은?

```
SELECT DEPTNO, NULL AS JOB, SUM(SAL) AS SUM_SAL
  FROM EMP
 GROUP BY DEPTNO
 UNION ALL
SELECT NULL, JOB, SUM(SAL) AS SUM_SAL
  FROM EMP
 GROUP BY JOB
 UNION ALL
SELECT NULL, NULL, SUM(SAL) AS SUM_SAL
  FROM EMP;

SELECT DEPTNO, JOB, SUM(SAL) AS SUM_SAL
  FROM EMP
 GROUP BY _____;
```

- ① ROLLUP(DEPTNO, JOB)
- ② GROUPING SETS(DEPTNO, JOB)
- ③ GROUPING SETS(DEPTNO, JOB, NULL)
- ④ CUBE(DEPTNO, JOB)

35. 아래 SQL 문장 실행 결과로 가장 적절한 것은?

<EMP>

EMPNO	ENAME	DEPTNO	SAL
7934	MILLER	10	1300
7782	CLARK	10	2450
7839	KING	10	5000
7369	SMITH	20	800
7876	ADAMS	20	1100
7566	JONES	20	2975
7788	SCOTT	20	3000
7902	FORD	20	3000

```
SELECT EMPNO, ENAME, DEPTNO, SAL,
       LAG(SAL, 2, 0) OVER(PARTITION BY DEPTNO ORDER BY SAL) LAG_VALUE
FROM EMP;
```

①

EMPNO	ENAME	DEPTNO	SAL	LAG_VALUE
7934	MILLER	10	1300	NULL
7782	CLARK	10	2450	NULL
7839	KING	10	5000	1300
7369	SMITH	20	800	NULL
7876	ADAMS	20	1100	NULL
7566	JONES	20	2975	800
7788	SCOTT	20	3000	1100
7902	FORD	20	3000	2975

②

EMPNO	ENAME	DEPTNO	SAL	LAG_VALUE
7934	MILLER	10	1300	0
7782	CLARK	10	2450	0
7839	KING	10	5000	1300
7369	SMITH	20	800	0
7876	ADAMS	20	1100	0
7566	JONES	20	2975	800
7788	SCOTT	20	3000	1100
7902	FORD	20	3000	2975

③

EMPNO	ENAME	DEPTNO	SAL	LAG_VALUE
7934	MILLER	10	1300	NULL
7782	CLARK	10	2450	1300
7839	KING	10	5000	2450
7369	SMITH	20	800	NULL
7876	ADAMS	20	1100	800
7566	JONES	20	2975	1100
7788	SCOTT	20	3000	2975
7902	FORD	20	3000	3000

④

EMPNO	ENAME	DEPTNO	SAL	LAG_VALUE
7934	MILLER	10	1300	0
7782	CLARK	10	2450	1300
7839	KING	10	5000	2450
7369	SMITH	20	800	0
7876	ADAMS	20	1100	800
7566	JONES	20	2975	1100
7788	SCOTT	20	3000	2975
7902	FORD	20	3000	3000

36. MIN 함수와 동일하게 사용할 수 있는 WINDOW 함수는?

- ① LAG
- ② RATIO_TO_REPORT
- ③ LEAD
- ④ FIRST_VALUE

37. 다음 수행 결과로 가장 적절한 것은?

<TAB1>

COL1	COL2
A	10
B	20
C	30
D	40

```
SELECT RATIO_TO_REPORT(COL2) OVER() AS C1,
       CUME_DIST() OVER(ORDER BY COL2) AS C2,
       ROUND(PERCENT_RANK() OVER(ORDER BY COL2),2) AS C3
FROM TAB1;
```

①

C1	C2	C3
0.1	0.25	0
0.2	0.5	0.33
0.3	0.75	0.67
0.4	1	1

②

C1	C2	C3
0.1	0.1	0
0.2	0.3	0.33
0.3	0.6	0.67
0.4	1	1

③

C1	C2	C3
0.1	0.1	0.1
0.2	0.3	0.3
0.3	0.6	0.6
0.4	1	1

④

C1	C2	C3
0.1	0.25	0.1
0.2	0.5	0.3
0.3	0.75	0.6
0.4	1	1

38. 다음 수행 결과로 가장 적절한 것은?

<EMP>

EMPNO	ENAME	DEPTNO	SAL
7934	MILLER	10	1300
7782	CLARK	10	2450
7839	KING	10	5000
7369	SMITH	20	800
7876	ADAMS	20	1100
7566	JONES	20	2975
7788	SCOTT	20	3000
7902	FORD	20	3000

SELECT SUM(SAL)

FROM (SELECT ENAME, DEPTNO, SAL,

NTILE(2) OVER(PARTITION BY DEPTNO ORDER BY SAL) AS GN

FROM EMP)

WHERE (DEPTNO = 10 AND GN = 1) OR (DEPTNO = 20 AND GN = 2);

- ① 7300
- ② 8625
- ③ 9750
- ④ 10275

39. 다음 결과를 얻기 위한 SQL 문장으로 적절하지 않은 것은?

<EXAM>	
이름	성적
홍길동	98
박길동	80
최길동	80
김길동	75
이길동	70

<결과>	
이름	성적
박길동	80
최길동	80
김길동	75

- ① SELECT 이름, 성적
FROM EXAM
ORDER BY 성적 DESC
OFFSET 1 ROW
FETCH FIRST 3 ROWS ONLY;
- ② SELECT TOP(3) WITH TIES 이름, 성적
FROM EXAM
ORDER BY 성적 DESC;
- ③ SELECT 이름, 성적
FROM (SELECT 성적, ROW_NUMBER() OVER(ORDER BY 성적 DESC) AS RN
FROM EXAM)
WHERE RN BETWEEN 2 AND 4;
- ④ SELECT 이름, 성적
FROM (SELECT 성적, DENSE_RANK() OVER(ORDER BY 성적 DESC) AS RN
FROM EXAM)
WHERE RN BETWEEN 2 AND 3;

40. 다음과 같은 데이터 상황에서 계층형 질의절을 완성하기 위해 필요한 표현식은?

<EMP>		
EMPNO	NAME	MGR
0001	SMITH	0002
0002	ALLEN	0001


```

SELECT *
  FROM EMP
  START WITH EMPNO = '0001'
 CONNECT BY MGR = PRIOR EMPNO;

```

- ① CONNECT_BY_ROOT
- ② SYS_CONNECT_BY_PATH
- ③ NOCYCLE
- ④ ORDER SIBLINGS BY

41. 다음 수행 결과로 가장 적절한 것은?

<DEPARTMENT>			
DEPTNO	DNAME	PART	BUILD
101	컴퓨터공학과		정보관
102	멀티미디어공학과	100	멀티미디어관
103	소프트웨어공학과	100	소프트웨어관
201	문헌정보학과	200	인문관
100	컴퓨터정보학부		
200	인문사회학부		


```

SELECT SYS_CONNECT_BY_PATH(DNAME, '-')
  FROM DEPARTMENT
 WHERE DEPTNO = 103
  START WITH PART IS NULL
 CONNECT BY PART = PRIOR DEPTNO ;

```

- ① -소프트웨어공학과
- ② -컴퓨터정보학부-소프트웨어공학과
- ③ -소프트웨어공학과-컴퓨터정보학부
- ④ -컴퓨터정보학부

42. PIVOT과 UNPIVOT에 대한 설명으로 가장 적절한 것은?

- ① PIVOT은 교차표 형태의 데이터를 TIDY 데이터로 변경하는 문법이다.
- ② UNPIVOT은 LONG 데이터를 WIDE 데이터로 변환하는 기법이다.
- ③ UNPIVOT시 쌓을 컬럼을 지정할 수 없다.
- ④ PIVOT 시 FOR 앞에는 반드시 집계함수(SUM, AVG 등)의 형태여야 한다.

43. 다음 SQL 실행 결과로 가장 적절한 것은?

```
SELECT REGEXP_REPLACE('031-234-4567', '\d+', 'XXX', 1, 2) FROM DUAL;
```

- ① 031-234-4567
- ② XX1-234-4567
- ③ 031-XXX-4567
- ④ 031-234-XXX

44. 다음 SQL 실행 결과로 가장 적절한 것은?

```
SELECT REGEXP_SUBSTR('123-234-4545-233', '((\d+)-(\d+))-((\d+)-(\d+))', 1, 1, NULL, 4) FROM DUAL;
```

- ① 233
- ② 234
- ③ 4545-233
- ④ 4545

45. 다음 SQL 중 수행이 불가능 한 것은?

<TAB1>		<TAB2>	
NO	NUMBER	NO	NUMBER
NAME	VARCHAR2(10)	NAME	VARCHAR2(10)
JUMIN	CHAR(13)	JUMIN	CHAR(13)
DEPTNO	NUMBER		

- ① INSERT INTO TAB2 SELECT NO, NAME, JUMIN FROM TAB1;
- ② INSERT INTO TAB1(NO, NAME, JUMIN) AS SELECT * FROM TAB2;
- ③ INSERT INTO TAB1 SELECT NO, NAME, JUMIN, NULL FROM TAB2;
- ④ INSERT INTO TAB2 (SELECT NO, NAME, JUMIN FROM TAB1);

46. 다음 설명 중 가장 적절하지 않은 것은?

- ① COMMIT을 한 이후에는 ROLLBACK을 수행해도 이전 값으로 돌아갈 수 없다.
- ② INSERT 한 이후 컬럼 추가 시 INSERT 값은 자동 저장되어 ROLLBACK 할 수 없다.
- ③ ROLLBACK을 한 명령을 다시 ROLLBACK으로 취소할 수 없다.
- ④ SAVEPOINT 지점이 COMMIT 이전일때 해당 SAVEPOINT까지 ROLLBACK 시도 시 COMMIT이후
까지만 ROLLBACK 된다.

47. 테이블 생성 시 주의 사항으로 옳지 않은 것은?

- ① 테이블 생성시 대소문자 구분은 하지 않아도 된다.
- ② 문자 데이터 유형은 별도로 크기를 지정하지 않아도 된다.
- ③ 날짜 유형은 별도로 크기를 지정하지 않아도 된다.
- ④ 컬럼에 대한 제약조건을 추가하는 경우 CONSTRAINT를 사용한다.

48. 테이블 복제에 대한 설명 중 가장 적절하지 않은 것은?

- ① CREATE TABLE 테이블명 AS SELECT 문으로 테이블 복제가 가능하다.
- ② 테이블 복제 시 컬럼 순서 및 데이터 유형도 복제된다.
- ③ 테이블에 생성한 PRIMARY KEY 도 함께 복제된다.
- ④ PRIMARY KEY 나 UNIQUE 설정 없이 부여된 NOT NULL 속성은 함께 복제된다.

49. 외래키에 대한 설명으로 가장 적절하지 않은 것은?

- ① 외래키 생성 시 참조 테이블의 참조키에 반드시 기본키 또는 고유키가 생성되어 있어야 한다.
- ② 외래키 생성 후 부모 테이블은 자식 데이터가 있을 경우 삭제 불가하다.
- ③ ON DELETE CASCADE 옵션으로 외래키 생성 시 부모 데이터만 삭제되고 자식데이터는 그대로 남는다.
- ④ 자식 테이블은 UPDATE, INSERT 시 제약을 받는다.

50. 뷰에 대한 설명 중 가장 적절하지 않은 것은?

- ① 뷰는 가상의 테이블이기에 물리적으로 구현되어 있지 않으며 저장공간을 차지하지 않는다.
- ② 원본 테이블이 노출되지 않으므로 데이터를 안전하게 보호할 수 있다.
- ③ 뷰를 참조하는 또 다른 뷰는 생성 불가하다.
- ④ 기본 테이블이 삭제되면 그 테이블을 참조하여 만든 뷰는 삭제되지 않는다.

5회차

1. 다음이 설명하는 모델링 개념으로 가장 적절한 것은?

사용자 관점의 데이터베이스 스키마를 통합하여 데이터베이스의 전체 논리적 구조를 정의하는 단계로 전체 데이터베이스의 개체, 속성, 관계, 데이터 타입 등을 정의한다.

- ① 외부 스키마
- ② 개념 스키마
- ③ 논리 스키마
- ④ 내부 스키마

2. 엔터티에 대한 설명으로 가장 적절한 것은?

- ① 누락된 프로세스의 경우 이후 추가하거나 수정할 수 없다.
- ② 사용되지 않는 고립 엔터티는 제거를 고려한다.
- ③ 다른 엔터티와 관계를 꼭 갖지 않아도 된다.
- ④ 주로 약어를 사용하여 엔터티 이름을 정한다.

3. 다음 중 단일 속성이 아닌 것은?

- ① 이름
- ② 성별
- ③ 핸드폰번호
- ④ 주소

4. 두 개의 엔터티 사이의 관계 도출 시 고려사항이 아닌 것은?

- ① 두 개의 엔터티 사이에 정보의 조합이 발생되는가?
- ② 두 개의 엔터티 사이에 관련 있는 연관규칙이 존재하는가?
- ③ 업무기술서, 장표에 관계연결에 대한 규칙이 서술되어 있는가?
- ④ 업무기술서, 장표에 관계연결을 가능하게 하는 명사가 있는가?

5. 다음 중 주식별자의 특징이 아닌 것은?

- ① 유일성
- ② 최소성
- ③ 불변성
- ④ 종속성

6. 다음 정규화에 대한 설명으로 가장 적절한 것은?

이행적 종속을 없애도록 테이블을 분리하는 단계

- ① 제 1 정규화
- ② 제 2 정규화
- ③ 제 3 정규화
- ④ 제 4 정규화

7. 관계에 대한 설명으로 가장 적절하지 않은 것은?

- ① 주문항목은 반드시 주문이 있어야만 존재할 수 있으므로 존재 관계를 갖는다.
- ② 학생과 수업 간의 관계는 등록이라는 행위를 통해 형성되므로 행위 관계를 갖는다.
- ③ 관계를 맺는다는 의미는 자식의 식별자를 부모에 상속하는 일이다.
- ④ 관계는 존재에 의한 관계와 행위에 의한 관계로 분류한다.

8. 트랜잭션의 특성 그 의미와 맞는 것은?

- ① 일관성: 정의된 연산들은 모두 성공적으로 실행되든지 아니면 전혀 실행되지 않은 상태로 남아 있어야 한다.
- ② 원자성: 트랜잭션이 실행되기 전 데이터의 내용이 잘못되면 실행 이후에도 내용이 잘못 되어 있지 않다.
- ③ 고립성: 트랜잭션이 실행되는 도중 다른 트랜잭션의 영향을 받아 잘못된 결과를 만들어서는 안 된다
- ④ 연관성: 트랜잭션이 성공적으로 수행되면 그 트랜잭션이 갱신한 데이터베이스의 내용은 영구적으로 저장된다.

9. 다음 중 NULL만으로 구성된 컬럼을 계산하여 NULL이 리턴되지 않는 함수는?

- ① COUNT
- ② SUM
- ③ AVG
- ④ MIN

10. 다음 식별자에 대한 설명으로 가장 적절하지 않은 것은?

- ① 업무에 의해 만들어지는 식별자를 본질식별자라고 한다.
- ② 꼭 필요하지 않지만 관리의 편의성 등의 이유로 인위적으로 만들어지는 식별자를 인조식별자라 한다.
- ③ 본질식별자가 단순한 구성을 가질 때 주로 인조식별자를 생성한다.
- ④ 인조식별자를 사용하면 중복 데이터 발생 가능성이 있어 데이터 품질이 저하된다.

11. 다음이 설명하는 데이터 무결성의 종류는?

테이블의 기본키를 구성하는 컬럼은 NULL 값이나 중복값을 가질 수 없다.

- ① 개체 무결성
- ② 참조 무결성
- ③ 도메인 무결성
- ④ NULL 무결성

12. 다음이 설명하는 용어로 가장 적절한 것은?

데이터베이스의 속성(ATTRIBUTE, 또는 필드)이 가질 수 있는 값들의 범위 또는 집합을 의미한다.

- ① 릴레이션
- ② 도메인
- ③ 튜플
- ④ 키

13. 다음 중 SQL 분류로 적절하지 않은 것은?

- ① DDL - ALTER
- ② TCL - ROLLBACK
- ③ DCL - COMMIT
- ④ DML - UPDATE

14. 다음의 컬럼 별칭 정의가 가장 적절하지 않은 것은?

- ① SELECT ENAME 이름, SAL AS Salary
- ② SELECT ENAME, SAL, DEPTNO AS DEPT NUMBER
- ③ SELECT ENAME, SAL, JOB AS "JOB**"
- ④ SELECT ENAME, SAL, COMM AS SAL_COMM

15. 다음 날짜 연산의 결과로 가장 적절한 것은?

```
SELECT ADD_MONTHS(TO_DATE('2024/08/24 10:00:00', 'YYYY/MM/DD HH24:MI:SS') - 10, -3) + 10/24/60
FROM DUAL;
```

- ① 2024/05/14 10:10:00
- ② 2024/05/14 11:00:00
- ③ 2024/05/24 10:00:10
- ④ 2024/05/24 10:10:00

16. 다음의 연산 결과가 다른 하나는?

- ① SELECT TRUNC(TO_DATE('2024/05/14', 'YYYY/MM/DD'), 'MONTH') FROM DUAL;
- ② SELECT ROUND(TO_DATE('2024/05/14', 'YYYY/MM/DD'), 'MONTH') FROM DUAL;
- ③ SELECT TO_DATE('2024/05', 'YYYY/MM') FROM DUAL;
- ④ SELECT LAST_DAY(TO_DATE('2024/05/14', 'YYYY/MM/DD')) FROM DUAL;

17. 아래 함수 결과로 가장 적절하지 않은 것은?

- ① UPPER('Sql Developer') : SQL DEVELOPER
- ② SUBSTR('Sql Developer', -5, 2) : lo
- ③ TO_CHAR(1000, '9,999') : 1,000
- ④ RTRIM('ABCAA', 'A') : BC

18. 다음 함수의 결과가 다른 것은?

COMM
0
NULL
100

- ① NVL(COMM, 100)
- ② NVL2(COMM, COMM, 100)
- ③ NULLIF(COMM, 100)
- ④ ISNULL(COMM, 100)

19. 다음 결과를 출력하는 SQL 문으로 가장 적절한 것은?

<TAB1>	
COL1	COL2
A	10
B	10
C	10
D	20
E	20

<결과>	
CNT10	CNT20
3	2

- ① SELECT SUM(DECODE(COL2, 10, 1, 0)) AS CNT10,
SUM(DECODE(COL2, 20, 1, 0)) AS CNT20
FROM TAB1;
- ② SELECT COUNT(DECODE(COL2, 10, 1, 0)) AS CNT10,
COUNT(DECODE(COL2, 20, 1, 0)) AS CNT20
FROM TAB1;
- ③ SELECT SUM(DECODE(COL2, 10, 'O', 'X')) AS CNT10,
SUM(DECODE(COL2, 20, 'O', 'X')) AS CNT20
FROM TAB1;
- ④ SELECT COUNT(DECODE(COL2, 10, 'O', 'X')) AS CNT10,
COUNT(DECODE(COL2, 20, 'O', 'X')) AS CNT20
FROM TAB1;

20. 아래 SQL 수행 결과로 가장 적절한 것은?

<EMP>	
ENAME	SAL
SMITH	800
ALLEN	2000
KING	5000
BLAKE	3500
CLARK	1500

SELECT COUNT(ENAME) FROM EMP WHERE ENAME NOT LIKE '_L%E%';

- ① 1
- ② 2
- ③ 3
- ④ 4

21. 아래 SQL 중 출력 결과가 다른 것은?

- ① SELECT *
FROM EMP
WHERE DEPTNO IN (10,20)
AND JOB = 'CLERK'
OR SAL > 3000;
- ② SELECT *
FROM EMP
WHERE DEPTNO = 10 OR DEPTNO = 20
AND JOB = 'CLERK'
OR SAL > 3000;
- ③ SELECT *
FROM EMP
WHERE (DEPTNO = 10 AND JOB = 'CLERK')
OR (DEPTNO = 20 AND JOB = 'CLERK')
OR SAL > 3000;
- ④ SELECT *
FROM EMP
WHERE (DEPTNO = 10 OR DEPTNO = 20)
AND JOB = 'CLERK'
OR SAL > 3000;

22. 다음 GROUP BY에 대한 설명 중 가장 적절하지 않은 것은?

- ① GROUP BY 절에는 컬럼 별칭을 사용할 수 없다.
- ② GROUP BY 뒤의 컬럼 순서에 따라 출력되는 그룹의 수가 달라진다.
- ③ GROUP BY 뒤의 첫 번째 값이 UNIQUE한 경우 뒤에 어떤 컬럼을 추가적으로 명시해도 그룹의 수는 변화없다.
- ④ GROUP BY 절에 사용하지 않은 컬럼은 SUM과 같은 집계함수와 함께 SELECT절에 사용 가능하다.

23. ORDER BY에 대한 설명으로 가장 적절한 것은?

- ① ORDER BY 절을 사용하지 않으면 입력된 데이터의 순서대로 출력된다.
- ② ORDER BY 절의 기본 정렬 순서는 내림차순이다.
- ③ ORDER BY 절에는 컬럼 별칭을 사용할 수 없다.
- ④ ORDER BY 절에는 컬럼명과 숫자를 동시에 사용할 수 없다.

24. 다음 출력 결과를 얻기 위한 SQL 문장으로 가장 적절한 것은?

<EMP>			
EMPNO	ENAME	MGR	DEPTNO
7369	SMITH	7566	20
7499	ALLEN	7654	30
7521	WARD	7566	30
7566	JONES	7654	20
7654	MARTIN		30

<결과>		
EMPNO	ENAME	MANAGER_NAME
7369	SMITH	JONES
7499	ALLEN	MARTIN
7521	WARD	JONES
7566	JONES	MARTIN
7654	MARTIN	

- ① SELECT E1.EMPNO, E1.ENAME, E2.ENAME AS MANAGER_NAME
FROM EMP E1, EMP E2
WHERE E1.MGR = E2.EMPNO
ORDER BY E1.EMPNO;
- ② SELECT E1.EMPNO, E1.ENAME, E2.ENAME AS MANAGER_NAME
FROM EMP E1, EMP E2
WHERE E1.MGR = E2.EMPNO(+)
ORDER BY E1.EMPNO;
- ③ SELECT E1.EMPNO, E1.ENAME, E2.ENAME AS MANAGER_NAME
FROM EMP E1, EMP E2
WHERE E1.EMPNO = E2.MGR
ORDER BY E1.EMPNO;
- ④ SELECT E1.EMPNO, E1.ENAME, E2.ENAME AS MANAGER_NAME
FROM EMP E1, EMP E2
WHERE E1.EMPNO = E2.MGR(+)
ORDER BY E1.EMPNO;

25. 다음 두 테이블의 NATURAL JOIN을 수행한 결과의 출력 건수는?

<TAB1>		<TAB2>	
COL1	COL2	COL2	COL3
A	10	10	1
B	20	20	2
C	30	20	3
D	10	NULL	4
E	NULL		

- ① 2
- ② 3
- ③ 4
- ④ 5

26. 다음 ANSI 문법을 오라클 문법으로 바꾼 것으로 가장 적절한 것은?

```
SELECT *
FROM TAB1 JOIN TAB2
  ON TAB1.COL1 = TAB2.COL1
  LEFT OUTER JOIN TAB3
    ON TAB1.COL2 = TAB3.COL2;
```

- ① SELECT *
FROM TAB1, TAB2, TAB3
WHERE TAB1.COL1 = TAB2.COL1
AND TAB1.COL2 = TAB3.COL2;
- ② SELECT *
FROM TAB1, TAB2, TAB3
WHERE TAB1.COL1 = TAB2.COL1(+)
AND TAB1.COL2 = TAB3.COL2;
- ③ SELECT *
FROM TAB1, TAB2, TAB3
WHERE TAB1.COL1 = TAB2.COL1
AND TAB1.COL2 = TAB3.COL2(+);
- ④ SELECT *
FROM TAB1, TAB2, TAB3
WHERE TAB1.COL1 = TAB2.COL1(+)
AND TAB1.COL2 = TAB3.COL2(+);

27. 학생과 교수 테이블을 사용하여 지도교수가 없는 학생 정보도 출력을 하고자 할 때, 아래 빈칸으로 가장 적절한 것은?

```
SELECT 학생.이름 학생명, 교수.이름 교수명
FROM 학생 _____ 교수
ON 학생.교수번호 = 교수.교수번호;
```

- ① LEFT OUTER JOIN
- ② RIGHT OUTER JOIN
- ③ FULL OUTER JOIN
- ④ INNER JOIN

28. 5개의 테이블 조인 시 조인 조건의 최소 개수는?

- ① 1
- ② 2
- ③ 3
- ④ 4

29. 다음 수행 결과를 차례대로 나열한 것은?

<TAB1>

COL1	COL2	COL3
NULL	1	1
1	1	NULL
1	NULL	1

```
SELECT COUNT(*) C1, COUNT(COL1) C2, SUM(COL2) C3 , SUM(COL1+COL2+COL3) C4
FROM TAB1;
```

- ① 0, 2, 2, 0
- ② 3, 2, 2, 0
- ③ 3, 2, 2, NULL
- ④ 3, 2, NULL, NULL

30. 다음 중 순수 관계 연산자로 적절하지 않은 것은?

- ① SELECT
- ② DIVISION
- ③ JOIN
- ④ PRODUCT

31. 다음 SQL의 수행 결과로 가장 적절한 것은?

<TAB1>		<TAB2>	
COL1	COL2	COL1	COL2
A	10	A	10
B	20	C	20
C	30	NULL	30
D	40		


```

SELECT SUM(COL2)
  FROM TAB1
 WHERE COL1 NOT IN (SELECT COL1
                    FROM TAB2
                    WHERE COL1 IS NOT NULL);

```

- ① 0
- ② 40
- ③ 50
- ④ 60

32. 다음 SQL의 실행 결과로 가장 적절한 것은?

<TAB1>		<TAB2>	
COL1		COL1	
10		35	
20		45	
30			
40			


```

SELECT SUM(COL1)
  FROM TAB1
 WHERE COL1 <= ALL(SELECT COL1
                   FROM TAB2);

```

- ① 10
- ② 30
- ③ 60
- ④ 100

33. 서브쿼리에 대한 설명 중 가장 적절하지 않은 것은?

- ① < ALL(10, 20, 30)은 10보다 작다와 같다.
- ② 다중행 서브쿼리는 동등비교, 대소비교 연산자를 사용할 수 없다.
- ③ 스칼라 서브쿼리는 반드시 각 행마다 단 하나의 행이 리턴되어야 한다.
- ④ 다중컬럼 서브쿼리의 SELECT절에 사용된 컬럼은 메인쿼리 SELECT절에 전달할 수 있다.

34. 다음 결과를 얻기 위한 아래 빈칸으로 가장 적절한 것은?

<EMP>		
EMPNO	ENAME	MGR
0001	SMITH	
0002	ALLEN	0001
0003	FORD	0002
0004	SCOTT	0003

<결과>	
사원명	상위관리자명
SMITH	SMITH
ALLEN	SMITH
FORD	ALLEN
SCOTT	FORD

SELECT ENAME 사원명,
 _____ 상위관리자명
 FROM EMP E1;

- ① (SELECT ENAME FROM EMP E2)
 ② (SELECT NVL(E2.ENAME,E1.ENAME) FROM EMP E2)
 ③ NVL((SELECT E2.ENAME FROM EMP E2 WHERE E1.EMPNO = E2.MGR), E1.ENAME)
 ④ NVL((SELECT E2.ENAME FROM EMP E2 WHERE E1.MGR = E2.EMPNO), E1.ENAME)

35. 아래 쿼리와 중 결과가 다른 하나는?

<TAB1>		<TAB2>	
COL1	COL2	COL1	COL2
A	10	A	10
B	20	E	NULL
C	30		
D	40		
E	NULL		

- ① SELECT * FROM TAB1 MINUS SELECT * FROM TAB2;
- ② SELECT *
FROM TAB1, TAB2
WHERE TAB1.COL1 != TAB2.COL1;
- ③ SELECT *
FROM TAB1
WHERE TAB1.COL1 NOT IN (SELECT COL1
FROM TAB2);
- ④ SELECT *
FROM TAB1
WHERE NOT EXISTS (SELECT COL1
FROM TAB2
WHERE TAB1.COL1 = TAB2.COL1);

36. 아래 쿼리 결과와 같은 결과를 갖는 빈칸에 들어갈 문장으로 가장 적절한 것은?

```
SELECT DEPTNO, NULL AS JOB, SUM(SAL) AS SUM_SAL
  FROM EMP
 GROUP BY DEPTNO
 UNION ALL
SELECT DEPTNO, JOB, SUM(SAL) AS SUM_SAL
  FROM EMP
 GROUP BY DEPTNO, JOB
 UNION ALL
SELECT NULL, NULL, SUM(SAL) AS SUM_SAL
  FROM EMP;

SELECT DEPTNO, JOB, SUM(SAL) AS SUM_SAL
  FROM EMP
 GROUP BY _____;
```

- ① ROLLUP(DEPTNO, JOB)
- ② GROUPING SETS(DEPTNO, JOB)
- ③ GROUPING SETS(DEPTNO, JOB, NULL)
- ④ CUBE(DEPTNO, JOB)

37. 다음 수행 결과로 가장 적절한 것은?

<TAB1>

COL1	COL2
A	10
A	20
A	20
B	30
B	40
B	40

```
SELECT SUM(COL2) OVER(PARTITION BY COL1 ORDER BY COL2 ASC) AS RESULT
FROM TAB1;
```

①

RESULT
10
30
50
80
120
160

②

RESULT
10
30
50
30
70
110

③

RESULT
10
50
50
30
110
110

④

RESULT
10
50
70
80
110
150

38. 다음 수행 결과로 가장 적절한 것은?

<EMP>

EMPNO	ENAME	DEPTNO	SAL
7934	MILLER	10	2000
7782	CLARK	10	2000
7839	KING	10	5000
7369	SMITH	20	800
7876	ADAMS	20	2500
7566	JONES	20	2500
7788	SCOTT	20	3000
7902	FORD	20	3000

```
SELECT SUM(SAL)
FROM (SELECT ENAME, DEPTNO, SAL,
            RANK() OVER(PARTITION BY DEPTNO ORDER BY SAL DESC) AS RANK_VALUE
FROM EMP)
WHERE RANK_VALUE = 2;
```

① 2000

② 4000

③ 9000

④ 10000

39. 다음 SQL 결과로 가장 적절한 것은?

<EXAM>

이름	성적
홍길동	98
박길동	80
최길동	80
김길동	75
이길동	75
황길동	70

```
SELECT TOP(3) WITH TIES 이름, 성적
FROM EXAM
ORDER BY 성적 DESC;
```

①

이름	성적
홍길동	98
박길동	80

②

이름	성적
홍길동	98
박길동	80
최길동	80

③

이름	성적
홍길동	98
박길동	80
최길동	80
김길동	75

④

이름	성적
홍길동	98
박길동	80
최길동	80
김길동	75
이길동	75

40. 다음 SQL의 수행 결과로 가장 적절한 것은?

<EMP>			
EMPNO	ENAME	MGR	SAL
7839	KING		5000
7369	SMITH	7839	800
7499	ALLEN	7369	1600
7521	WARD	7499	1250
7566	JONES	7499	2975
7654	MARTIN	7839	1250


```

SELECT SUM(E2.SAL)
  FROM EMP E1 INNER JOIN EMP E2
    ON E1.MGR = E2.EMPNO
   AND E1.SAL > E2.SAL;

```

- ① 1600
- ② 1975
- ③ 2400
- ④ 4575

41. 계층형 질의에 대한 설명 중 가장 적절하지 않은 것은?

- ① 행끼리 연결관계가 있을 경우 연결관계를 표현하는 기법이다.
- ② 행의 연결관계는 CONNECT BY에 전달한다.
- ③ LEVEL은 항상 가장 먼저 시작하는 행이 1을 갖는다.
- ④ 같은 레벨일 경우 추가 정렬을 원한다면 ORDER BY에 컬럼명을 나열하면 된다.

42. 다음 출력 결과를 얻기 위한 SQL로 가장 적절하지 않은 것은?

<CUSTOMERS>		
CUSTOMER_ID	CUST_FIRST_NAME	CUST_JOB_NAME
C26306	Kathy	농업인
C31762	Kathy	농업인
C15506	Sachin	종합소매업
C32830	Farrah	금융업
C32291	Harry Mean	금융업

<결과>		
CNT1	CNT2	CNT3
2	2	1

- ① SELECT "'금융업'" AS CNT1, "'농업인'" AS CNT2, "'종합소매업'" AS CNT3
FROM (SELECT CUSTOMER_ID, CUST_FIRST_NAME, CUST_JOB_NAME FROM CUSTOMERS)
PIVOT (COUNT(CUST_FIRST_NAME) FOR CUST_JOB_NAME IN ('금융업','농업인','종합소매업'));
- ② SELECT "'금융업'" AS CNT1, "'농업인'" AS CNT2, "'종합소매업'" AS CNT3
FROM (SELECT CUST_JOB_NAME FROM CUSTOMERS)
PIVOT (COUNT(CUST_JOB_NAME) FOR CUST_JOB_NAME IN ('금융업','농업인','종합소매업'));
- ③ SELECT "'금융업'" AS CNT1, "'농업인'" AS CNT2, "'종합소매업'" AS CNT3
FROM (SELECT CUSTOMER_ID, CUST_JOB_NAME FROM CUSTOMERS)
PIVOT (COUNT(CUSTOMER_ID) FOR CUST_JOB_NAME IN ('금융업','농업인','종합소매업'));
- ④ SELECT "'금융업'" AS CNT1, "'농업인'" AS CNT2, "'종합소매업'" AS CNT3
FROM (SELECT CUST_FIRST_NAME, CUST_JOB_NAME FROM CUSTOMERS)
PIVOT (COUNT(CUST_FIRST_NAME) FOR CUST_JOB_NAME IN ('금융업','농업인','종합소매업'));

43. 다음 SQL 실행 결과로 가장 적절한 것은?

```
SELECT REGEXP_REPLACE('ABCCAC BC BBBC', 'AB|C+') FROM DUAL;
```

- ① AC B BBB
② A B BBB
③ CCC BB
④ AC BC BBBC

44. 이메일 주소에서 이메일 아이디만 추출하기 위한 빈칸 표현으로 가장 적절하지 않은 것은?

```
SELECT REGEXP_SUBSTR('HDATA LAB_1@NAVER.COM', '_____') FROM DUAL;
```

- ① (\w|_)+
② [A-z_0-9]+
③ ([[:alnum:]]|_)+
④ \w|\d+

45. 아래 SQL 수행 결과로 가장 적절한 것은?

<TAB1>			<TAB2>		
NO	NAME	PRICE	NO	NAME	PRICE
1	아메리카노	1000	1	아메리카노	1500
2	카페라떼	2000	2	카페라떼	2500
			3	카페모카	3000


```

MERGE INTO TAB1
USING TAB2
  ON (TAB1.NO = TAB2.NO)
 WHEN MATCHED THEN
   UPDATE
     SET TAB1.PRICE = TAB2.PRICE
   DELETE
     WHERE TAB1.PRICE <= 1000
 WHEN NOT MATCHED THEN
   INSERT VALUES(TAB2.NO, TAB2.NAME, TAB2.PRICE);

SELECT SUM(PRICE) FROM TAB1;

```

- ① 3000
- ② 5000
- ③ 5500
- ④ 7000

46. 아래 문장들을 순서대로 수행했을 경우 최종 데이터 형태로 가장 적절한 것은?
(단, 에러가 나는 문장은 무시한다.)

```

CREATE TABLE TAB1(NO NUMBER, NAME VARCHAR2(10));
INSERT INTO TAB1 VALUES(1, 'A');
INSERT INTO TAB1 VALUES(2, 'B');
COMMIT;
UPDATE TAB1 SET NAME = 'C' WHERE NO = 2;
SAVEPOINT SAVE1;
INSERT INTO TAB1 VALUES(3, 'C');
COMMIT;
DELETE FROM TAB1 WHERE NO = 1;
ROLLBACK TO SAVE1;
ROLLBACK;

```

①

NO	NAME
2	C

②

NO	NAME
2	C
3	C

③

NO	NAME
1	A
2	B
3	C

④

NO	NAME
1	A
2	C
3	C

47. 다음 설명 중 가장 적절하지 않은 것은?

- ① TRUNCATE는 데이터 삭제 시 즉시 반영되는 DML이다.
- ② DELETE는 COMMIT을 하기 전까지는 삭제된 데이터를 삭제 전으로 다시 되돌릴 수 있다.
- ③ DROP은 테이블의 구조를 삭제하기 때문에 삭제된 이후에 테이블 조회가 불가능하다.
- ④ TRUNCATE는 일부 데이터만을 삭제할 수 없다.

48. 다음 제약조건 생성 명령어로 가장 적절하지 않은 것은?

- ① CREATE TABLE TAB1(NO NUMBER PRIMARY KEY, NAME VARCHAR2(10) NOT NULL);
- ② CREATE TABLE TAB1(NO NUMBER, NAME VARCHAR2(10), JDATE DATE, PRIMARY KEY(NO, NAME));
- ③ CREATE TABLE TAB1(NO NUMBER, NAME VARCHAR2(10) NOT NULL);
ALTER TABLE TAB1 ADD PRIMARY KEY(NO);
- ④ CREATE TABLE TAB1(NO NUMBER PRIMARY KEY, NAME VARCHAR2(10) NOT NULL);
ALTER TABLE TAB1 ADD JDATE DATE NOT NULL DEFAULT SYSDATE ;

49. 테이블 구조 변경에 대한 설명 중 가장 적절하지 않은 것은?

- ① 컬럼을 동시에 여러 개 추가 시 반드시 괄호로 묶어 전달해야 한다.
- ② 데이터의 존재여부와 상관없이 컬럼 삭제는 항상 가능하다.
- ③ 데이터가 존재할 경우 데이터 유형 수정은 항상 불가능하다.
- ④ 기본키 생성 시 기본키 컬럼에 인덱스가 자동 생성된다.

50. 뷰에 대한 설명 중 가장 적절한 것은?

- ① 뷰 생성 후 뷰에 컬럼 정의를 변경할 수 있다.
- ② 뷰를 통해 원본 테이블에 데이터를 변경할 수 없다.
- ③ 뷰에 직접 인덱스를 생성할 수 있다.
- ④ 서브쿼리나 조인을 사용한 복잡한 쿼리에 대한 뷰를 생성할 수 있다.